

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2018

Huấn luyện viên: Zhang Ruizhe

China Computer Federation, tháng 4 năm 2018

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2018

IOI China candidate team papers / National training team 2018 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2018. Khác với một số tập trước, lớp văn bản của tệp này trích xuất được khá tốt bằng pdftotext: phần bìa, mục lục và các trang mở đầu của từng bài đều đọc được. Bản dịch này chuyển ngữ phần đầu sách và mục lục, đồng thời ghi lại các chủ đề chính có thể xác nhận từ mục lục và phần mở đầu của các bài. Bản hiện hiện tại đã bổ sung bản dịch đầy đủ mười ba bài đầu tiên: bài về hàm sinh trong các bài toán gieo xúc xắc, báo cáo ra đề *Số nút của cây hậu tố*, bài về hồi quy bảo toàn thứ tự, báo cáo ra đề *Fim 4*, và bài về các kỹ thuật xử lý khối liên thông trên cây, báo cáo ra đề *Jellyfish* và khảo cứu mở rộng, cùng bài về *Leafy Tree* và cây cân bằng có trọng số, và báo cáo ra đề *Tiểu H thích tô màu*, bài về các bài toán tổng hàm số học đặc biệt, cùng bài về ứng dụng DFT trong thi tin học, và báo cáo ra đề *Hàng đợi hoàn hảo*, và bài về một số mở rộng của matroid cùng ứng dụng, cùng bài về các tính chất và ứng dụng của Splay và Treap.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2018*. Huấn luyện viên ghi trên bìa là Zhang Ruizhe. Đơn vị xuất bản là China Computer Federation; thời điểm ghi trên bìa là tháng 4 năm 2018.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả / trường
1	Bàn sơ lược về ứng dụng hàm sinh trong bài toán gieo xúc xắc	Yang Maolong, Trường Trung học Changjun, Trường Sa
11	Báo cáo ra đề <i>Số nút của cây hậu tố</i> và tối ưu hóa một lớp bài toán đoạn	Chen Jianglun, Trường Trung học Changjun, Trường Sa
23	Bàn sơ lược về bài toán hồi quy bảo toàn thứ tự	Gao Ruiquan, Trường Ngoại ngữ Nam Kinh
34	Báo cáo ra đề <i>Fim 4</i>	Wu Jinzhao, Trường Trung học số 2 Hàng Châu, Chiết Giang
45	Một số kỹ thuật và công cụ giải bài toán khối liên thông trên cây	Ren Xuandi, Trường Trung học số 1 Thiệu Hưng
58	Báo cáo ra đề <i>Jellyfish</i> và khảo cứu mở rộng	Liang Yancheng, Trường Trung học Trần Hải, Ninh Ba
75	<i>Leafy Tree</i> và cây cân bằng có trọng số được hiện thực bằng nó	Wang Siqi, Trường Trung học số 7 Thành Đô
87	Báo cáo ra đề <i>Tiểu H thích tô màu</i>	Chen Jiale, Trường Trung học số 2 Hàng Châu, Chiết Giang
92	Một số bài toán tổng hàm số học đặc biệt	Zhu Zhenting, THPT trực thuộc Đại học Sư phạm An Huy
113	Bàn sơ lược về ứng dụng DFT trong thi tin học	Liu Chengao, THPT trực thuộc Đại học Công nghiệp Tây Bắc
132	Báo cáo ra đề <i>Hàng đợi hoàn hảo</i>	Lin Xuheng, THPT trực thuộc Đại học Sư phạm Phúc Kiến
143	Bàn sơ lược về một số mở rộng của matroid và ứng dụng	Yang Qianlan, Trường Trung học Hoài Âm, Giang Tô
164	Bàn sơ lược về tính chất của Splay và Treap cùng ứng dụng	Dong Weijun, Trường Trung học số 6 Quảng Châu
180	Báo cáo ra đề <i>Cây khung phương sai nhỏ nhất</i>	He Zhongtian, Trường Trung học số 80 Bắc Kinh
192	Khảo cứu các bài toán sinh và đếm liên quan tới đồ thị Euler	Chen Tong, Trường Thực nghiệm trực thuộc Đại học Sư phạm Bắc Kinh

3 Bài 1: Bàn sơ lược về ứng dụng hàm sinh trong bài toán gieo xúc xắc

Yang Maolong, Trường Trung học Changjun, Trường Sa

Tóm tắt

Bài toán gieo xúc xắc là một lớp bài toán thường gặp trong thi thuật toán. Sau khi nghiên cứu kỹ lớp bài toán này, tác giả nhận thấy chúng đều có thể được giải bằng hàm sinh. Vì vậy, bài viết này xoay quanh chuỗi bài toán gieo xúc xắc, trình bày cách dùng hàm sinh để từng bước giải quyết vấn đề, cũng như ưu thế của cách giải bằng hàm sinh so với phương pháp truyền thống.

Từ khóa: hàm sinh, bài toán gieo xúc xắc, xác suất và kỳ vọng.

1 Dẫn nhập

Bài toán gieo xúc xắc là một lớp bài toán thường gặp trong thi thuật toán, và đã nhiều lần xuất hiện trong các kỳ thi những năm gần đây.

Mặt khác, hàm sinh là một công cụ hữu hiệu để giải lớp bài toán này. So với phương pháp truyền thống, nó có các ưu điểm như dễ tính toán và khả năng mở rộng mạnh. Tuy nhiên, hiện nay trong giới OI vẫn có rất ít nghiên cứu về phương pháp này.

Mục 2 quy ước một số ký hiệu. Mục 3 giới thiệu định nghĩa của hàm sinh xác suất và một số tính chất. Mục 4 giới thiệu ứng dụng cơ bản của thuật toán này. Mục 5 và 6 kết hợp với các đề bài để giới thiệu một số ứng dụng phức tạp hơn.

2 Quy ước ký hiệu

1. A_i biểu thị số thứ i của dãy A .
2. $A[l, r]$ biểu thị dãy gồm các số thứ l đến thứ r của dãy A .
3. $f^{(k)}(x)$ là đạo hàm cấp k của $f(x)$.
4. $[P]$ là ngoặc Iverson: nếu điều kiện trong ngoặc vuông đúng thì giá trị bằng 1, nếu không thì bằng 0.

3 Kiến thức chuẩn bị

3.1 Hàm sinh thông thường

Hàm sinh thông thường của dãy $a_0, a_1, a_2, \dots$ là

$$A(x) = \sum_{i=0}^{\infty} a_i x^i.$$

3.2 Hàm sinh xác suất

Nếu với dãy $a_0, a_1, a_2, \dots$ tồn tại một biến ngẫu nhiên rời rạc X nào đó thỏa $\Pr(X = i) = a_i$, thì hàm sinh thông thường của a_n ($n \in \mathbb{N}$) được gọi là hàm sinh xác suất của X .

Nói cách khác, nếu X là biến ngẫu nhiên rời rạc trên tập số nguyên không âm $\mathbb{N}$, thì hàm sinh xác suất của X là

$$F(z) = \mathbb{E}(z^X) = \sum_{i=0}^{\infty} \Pr(X = i) z^i.$$

3.2.1 Tính chất của hàm sinh xác suất

Vì X là biến ngẫu nhiên rời rạc trên $\mathbb{N}$, ta nhất định có

$$F(1) = \sum_{i=0}^{\infty} \Pr(X = i) = 1.$$

Lấy đạo hàm $F(z)$, ta được

$$F'(z) = \sum_{i=0}^{\infty} i \Pr(X = i) \cdot z^{i-1}.$$

Do đó kỳ vọng của X là

$$\mathbb{E}(X) = F'(1) = \sum_{i=0}^{\infty} i \Pr(X = i).$$

Suy luận thêm có thể nhận được

$$\mathbb{E}(X^k) = F^{(k)}(1), \quad k \neq 0.$$

Vì vậy phương sai của X là

$$\text{Var}(X) = F''(1) + F'(1) - (F'(1))^2.$$

3.3 Border

Với một dãy A có độ dài L , nếu

$$A[1, i] = A[L - i + 1, L],$$

thì gọi $A[1, i]$ là một *border* của A .

4 Dạng bài cơ bản

Ví dụ 1. Vương quốc ca hát Nguồn: CTSC2006.

Cho một dãy A có độ dài L . Sau đó, mỗi lần gieo một con xúc xắc công bằng ghi các số từ 1 đến m , rồi đưa số gieo được vào cuối dãy B ban đầu rỗng. Nếu trong dãy B đã xuất hiện dãy A cho trước, tức A là một dãy con liên tiếp của B , thì dừng lại. Hãy tính độ dài kỳ vọng của dãy B . $L \leq 10^5$.

4.1 Phân tích

Dạng cơ bản của bài toán gieo xúc xắc là: cho một dãy và một con xúc xắc, sau đó liên tục gieo xúc xắc để sinh ra một dãy ngẫu nhiên, cho đến khi dãy đã cho xuất hiện trong dãy ngẫu nhiên thì dừng, và cần tính số lần gieo kỳ vọng.

4.2 Một phương pháp không dùng hàm sinh

Đặt E_i là số lần gieo xúc xắc kỳ vọng từ trạng thái hiện tại thỏa $A[1, i]$ là hậu tố của B cho đến lần đầu tiên thỏa $A[1, i + 1]$ là hậu tố của B . Khi đó đại lượng cần tìm là

$$\sum_{i=0}^{n-1} E_i.$$

Định nghĩa $f_{i,j}$ là độ dài border dài nhất không phải chính nó của dãy mới thu được bằng cách thêm số j vào sau $A[1, i]$. Khi đó có phương trình

$$E_i = 1 + \frac{1}{m} \sum_{j=1}^m [j \neq A_{i+1}] \sum_{k=f_{i,j}}^i E_k.$$

Giải phương trình có thể nhận được

$$E_i = m + (m - 1) \sum_{k=1}^{i-1} E_k - \sum_{j=1}^m [j \neq A_{i+1}] \sum_{k=1}^{f_{i,j}-1} E_k.$$

Độ phức tạp thời gian như vậy là $O(nm)$. Vấn đề chính nằm ở việc tính

$$\sum_{j=1}^m [j \neq A_{i+1}] \sum_{k=1}^{f_{i,j}-1} E_k,$$

và phần này có thể tối ưu. Dựa trên công thức so sánh $A[1, i]$ với border dài nhất không phải chính nó, có thể thấy giữa hai công thức chỉ có $O(1)$ giá trị $f_{i,j}$ khác nhau, nên mỗi lần chỉ cần bảo trì lại $O(1)$ giá trị này. Khi đó độ phức tạp thời gian giảm xuống $O(n)$.

Có thể thấy phương pháp này rất phức tạp và khả năng mở rộng thấp. Phương pháp hàm sinh được giới thiệu dưới đây ưu việt hơn phương pháp này khá nhiều.

4.3 Phương pháp hàm sinh

Đặt a_i biểu thị $A[1, i]$ có phải là một border của A hay không: $a_i = 1$ nghĩa là có, còn $a_i = 0$ nghĩa là không.

Đặt f_i là xác suất để khi kết thúc, độ dài dãy ngẫu nhiên bằng i ; hàm sinh xác suất của nó là $F(x)$.
Đặt dãy phụ trợ g_i là xác suất để độ dài dãy ngẫu nhiên đạt đến i mà vẫn chưa kết thúc; hàm sinh thông thường của nó là $G(x)$.

Đại lượng cần tìm là $F'(1)$. Qua phân tích, ta nhận được các đẳng thức sau:

$$F(x) + G(x) = 1 + G(x) \cdot x, \quad (1)$$

$$G(x) \cdot \left(\frac{1}{m}x\right)^L = \sum_{i=1}^L a_i \cdot F(x) \cdot \left(\frac{1}{m}x\right)^{L-i}. \quad (2)$$

Ý nghĩa của (1) là: sau khi thêm một số vào một dãy chưa kết thúc, dãy có thể kết thúc hoặc vẫn chưa kết thúc; 1 là trường hợp ban đầu khi dãy ngẫu nhiên rỗng.

Ý nghĩa của (2) là: sau khi thêm dãy đã cho vào sau một dãy chưa kết thúc, chắc chắn sẽ kết thúc, nhưng có thể đã kết thúc trước khi thêm đủ L số; khi đó dãy đã thêm nhất định phải là một border của dãy đã cho.

Lấy đạo hàm (1) rồi thay $x = 1$, ta được

$$F'(x) + G'(x) = G'(x) \cdot x + G(x), \quad (1)$$

$$F'(1) = G(1). \quad (3)$$

Tiếp tục thay $x = 1$ vào (2), ta được

$$G(1) \cdot \left(\frac{1}{m}\right)^L = \sum_{i=1}^L a_i \cdot F(1) \cdot \left(\frac{1}{m}\right)^{L-i}, \quad (2)$$

$$G(1) = \sum_{i=1}^L a_i \cdot m^i. \quad (4)$$

Từ (3)(4), suy ra

$$F'(1) = \sum_{i=1}^L a_i \cdot m^i.$$

Dùng thuật toán KMP hoặc hash là có thể tìm a_i và tính $F'(1)$ trong độ phức tạp thời gian $O(L)$.

4.4 Suy nghĩ thêm

Ta có thể tiến thêm một bước để nhận được phương pháp tính phương sai.

Theo

$$\text{Var}(X) = F''(1) + F'(1) - (F'(1))^2,$$

có thể thấy chỉ cần biết $F'(1)$ và $F''(1)$. $F'(1)$ đã biết, vì vậy cần tính $F''(1)$.

Lấy đạo hàm cấp hai của (1) rồi thay $x = 1$, ta được

$$F''(x) + G''(x) = G'(x) + G''(x) \cdot x + G'(x), \quad (3)$$

$$F''(1) = 2G'(1). \quad (4)$$

Tiếp tục lấy đạo hàm (2) rồi thay $x = 1$, ta được

$$G(x) = \sum_{i=1}^L a_i \cdot m^i \cdot F(x) \cdot x^{-i}, \quad (5)$$

$$G'(x) = \sum_{i=1}^L a_i \cdot m^i \cdot (F'(x) \cdot x^{-i} - i \cdot F(x) \cdot x^{-i-1}), \quad (6)$$

$$G'(1) = \sum_{i=1}^L a_i \cdot m^i \cdot (F'(1) - i). \quad (7)$$

Vì vậy

$$F''(1) = 2 \cdot \sum_{i=1}^L a_i \cdot m^i \cdot (F'(1) - i).$$

Nếu tiếp tục suy luận và quy nạp, có thể nhận được

$$F^{(k)}(1) = k \cdot \sum_{i=1}^L a_i \cdot m^i \sum_{j=0}^{k-1} (-1)^j \binom{k-1}{j} \frac{(i+j-1)!}{(i-1)!} F^{(k-1-j)}(1).$$

4.5 Tiểu kết

Phương pháp dùng hàm sinh để giải lớp bài toán này thường bắt đầu bằng cách định nghĩa một hàm sinh xác suất $F(x)$ và một hàm sinh phụ trợ $G(x)$. Sau đó xét việc thêm một số hoặc một dãy cho trước vào trạng thái chưa kết thúc, rồi dựa vào tình huống thực tế để lập phương trình. Cuối cùng, thông qua thay giá trị và lấy đạo hàm, ta giải ra $F'(1)$ cần tìm.

5 Dạng bài phức tạp

Ví dụ 2. Trò chơi gieo xúc xắc Cho n dãy A_i có độ dài lần lượt là L_i . Lại cho một con xúc xắc ghi các số từ 1 đến m , trong đó xác suất gieo ra i là P_i . Sau đó, mỗi lần gieo xúc xắc một lần và đưa số trên xúc xắc vào cuối dãy B ban đầu rỗng. Nếu cả n dãy đã cho đều là dãy con liên tiếp của B , thì dừng lại. Hãy tính độ dài kỳ vọng của dãy B . Bảo đảm $n \leq 15$, $L \leq 20000$, $m \leq 10^5$.

5.1 Phân tích

Về bản chất, bài toán này vẫn là bài toán gieo xúc xắc, nhưng đã biến thành trường hợp cần n dãy đều xuất hiện trong dãy ngẫu nhiên.

5.2 Lời giải

Đặt T_i là độ dài của dãy ngẫu nhiên khi A_i xuất hiện lần đầu. Khi đó đại lượng cần tìm là

$$\mathbb{E} \left(\max_{i \in \{1, 2, \dots, n\}} T_i \right).$$

Nếu một dãy A_i là dãy con liên tiếp của một dãy khác A_j , thì nhất định có $T_i \leq T_j$, nên có thể loại bỏ A_i .

Giá trị lớn nhất không dễ tính, nên dùng bao hàm–loại trừ Min–Max:

$$\max_{i=1}^n T_i = \sum_{S \subseteq \{1, 2, \dots, n\}} (-1)^{|S|+1} \min_{i \in S} T_i.$$

Theo tính tuyến tính của kỳ vọng, bài toán có thể chuyển thành tính $\mathbb{E}(\min_{i \in S} T_i)$, tức hể một dãy xuất hiện thì kết thúc. Không mất tính tổng quát, giả sử $S = \{1, 2, \dots, n\}$.

Định nghĩa

$$P(A) = \prod_{i \in A} P_i.$$

Đặt

$$a_{i,j,k} = [A_i[1, k] = A_j[L_j - k + 1, L_j]].$$

Đặt $f_{i,j}$ là xác suất để dãy xuất hiện đầu tiên là A_i và độ dài dãy ngẫu nhiên bằng j ; hàm sinh thông thường của nó là $F_i(x)$. Đặt dãy phụ trợ g_i là xác suất để độ dài dãy ngẫu nhiên đạt đến i mà vẫn chưa kết thúc; hàm sinh thông thường của nó là $G(x)$.

Ta có các đẳng thức sau:

$$\sum_{i=1}^n F_i(x) + G(x) = 1 + G(x) \cdot x, \quad (5)$$

$$G(x) \cdot P(A_i)x^{L_i} = \sum_{j=1}^n \sum_{k=1}^{L_i} a_{i,j,k} \cdot F_j(x) \cdot P(A_i[k+1, L_i]) \cdot x^{L_i-k}, \quad \forall i \in S. \quad (6)$$

Đại lượng cần tính là

$$\sum_{i=1}^n F'_i(1).$$

Lấy đạo hàm (5) rồi thay $x = 1$, ta được

$$\sum_{i=1}^n F'_i(1) = G(1).$$

Thay $x = 1$ vào (6), ta được

$$G(1) = \sum_{j=1}^n \left(\sum_{k=1}^{L_i} a_{i,j,k} \cdot \frac{1}{P(A_i[1, k])} \right) \cdot F_j(1), \quad \forall i \in S.$$

Hiện có $n+1$ ẩn, nhưng chỉ có n phương trình. Có thể thấy $\sum_{i=1}^n F_i(x)$ là hàm sinh xác suất của độ dài dãy ngẫu nhiên, nên

$$\sum_{i=1}^n F_i(1) = 1.$$

Như vậy có thể dùng khử Gauss để tìm $G(1)$. Đồng thời cũng có thể nhận được $F_i(1)$, tức xác suất để dãy xuất hiện đầu tiên là dãy thứ i ; chẳng hạn bài [SDOI2017] Trò chơi đồng xu yêu cầu tính $F_i(1)$.

Dùng hash, có thể tìm mảng a trong độ phức tạp thời gian $O(n^2L)$, và với mỗi cặp (i, j) tính

$$\sum_{k=1}^{L_i} a_{i,j,k} \cdot \frac{1}{P(A_i[1, k])}.$$

Với mỗi S , có thể dùng khử Gauss trong $O(n^3)$ để giải $G(1)$. Vì vậy tổng độ phức tạp thời gian là $O(n^2L + 2^n n^3)$.

6 Một số ứng dụng mới

6.1 Khớp mơ hồ

Ví dụ 3. Dice Nguồn: 2013 Multi-University Training Contest 5.

Với một con xúc xắc công bằng m mặt, hãy tính số lần gieo kỳ vọng để dừng khi n kết quả cuối cùng giống nhau, hoặc để dừng khi n kết quả cuối cùng đôi một khác nhau. Bảo đảm $n, m \leq 10^6$, và với câu hỏi thứ hai bảo đảm $n \leq m$.

6.1.1 Phương pháp sai phân truyền thống

Câu hỏi thứ nhất Đặt f_i là số lần gieo xúc xắc kỳ vọng từ trạng thái hiện tại đến khi kết thúc, trong đó trạng thái hiện tại thỏa i lần cuối cùng giống nhau. Khi đó đại lượng cần tìm là f_0 .

Có thể lập công thức chuyển:

$$f_i = \frac{1}{m}f_{i+1} + \frac{m-1}{m}f_1 + 1.$$

Trước hết sai phân, ta được

$$f_{i-1} - f_i = \frac{1}{m}(f_i - f_{i+1}).$$

Vì $f_0 - f_1 = 1$, nên $f_{i-1} - f_i = m^{i-1}$. Đồng thời $f_n = 0$, vì vậy

$$f_0 = \sum_{i=0}^{n-1} m^i = \frac{m^n - 1}{m - 1}.$$

Câu hỏi thứ hai Đặt g_i là số lần gieo xúc xắc kỳ vọng từ trạng thái hiện tại đến khi kết thúc, trong đó trạng thái hiện tại thỏa i lần cuối cùng đôi một khác nhau. Khi đó đại lượng cần tìm là g_0 .

Công thức chuyển là

$$g_i = \frac{m-i}{m}g_{i+1} + \frac{1}{m}\sum_{j=1}^i g_j + 1.$$

Trước hết sai phân:

$$g_{i-1} - g_i = \frac{m-i}{m}(g_i - g_{i+1}).$$

Tương tự có $g_0 - g_1 = 1$, $g_n = 0$, nên cũng có thể tìm g_0 trong thời gian $O(n)$.

6.1.2 Cách làm bằng hàm sinh

Phân tích Tuy có thể xem đơn giản bài này là một bài toán gieo xúc xắc nhiều dãy, số lượng dãy hợp lệ lại quá lớn. Nhưng tất cả chúng đều thỏa cùng một điều kiện, vì vậy có thể gom chúng lại với nhau.

Câu hỏi thứ nhất Đặt f_i là xác suất để kết thúc ở lần thứ i ; hàm sinh xác suất của nó là $F(x)$. Đặt g_i là xác suất để đã gieo i lần mà vẫn chưa kết thúc; hàm sinh thông thường của nó là $G(x)$.

Khi đó nhận được

$$F(x) + G(x) = G(x) \cdot x + 1, \tag{7}$$

$$G(x) \cdot \left(\frac{1}{m}x\right)^{n-1} = \sum_{i=1}^n F(x) \cdot \left(\frac{1}{m}x\right)^{n-i}. \tag{8}$$

Áp dụng cách giải tương tự phần trước, ta có thể giải ra

$$F'(1) = \frac{m^n - 1}{m - 1}.$$

Câu hỏi thứ hai Đặt f_i là xác suất để kết thúc ở lần thứ i ; hàm sinh xác suất của nó là $F(x)$. Đặt g_i là xác suất để đã gieo i lần mà vẫn chưa kết thúc; hàm sinh thông thường của nó là $G(x)$.

Khi đó nhận được

$$F(x) + G(x) = G(x) \cdot x + 1, \quad (9)$$

$$G(x) \cdot \left(\frac{1}{m}x\right)^n \cdot \frac{m!}{(m-n)!} = \sum_{i=1}^n F(x) \cdot \left(\frac{1}{m}x\right)^{n-i} \cdot \frac{(m-i)!}{(m-n)!}. \quad (10)$$

Có thể giải ra

$$F'(1) = \sum_{i=1}^n m^i \frac{(m-i)!}{m!}.$$

6.1.3 So sánh

Tuy hai phương pháp đều có thể cho cùng kết quả, cách làm bằng hàm sinh rõ ràng đơn giản hơn về mặt tính toán so với phương pháp truyền thống, và phương pháp cũng dễ hiểu hơn. Đồng thời, với các đề bài khác nhau, phần cần sửa đổi là rất ít.

6.2 Trường hợp phức tạp hơn

Ví dụ 4. Một bài xác suất nhỏ thú vị Nguồn: <http://roosephu.github.io/2017/12/31/condexp/>.

Một con xúc xắc n mặt, mỗi mặt được đánh số từ 1 đến n . Có một bộ đếm ban đầu bằng 0. Mỗi lần tung xúc xắc, nếu kết quả là số lẻ thì bộ đếm bị xóa về 0; nếu không thì bộ đếm tăng thêm 1, và nếu kết quả là n thì kết thúc. Hỏi kỳ vọng của bộ đếm khi kết thúc. Bảo đảm n là số chẵn.

6.2.1 Lời giải

Nhìn qua thì đáp án là $\frac{n}{2}$, nhưng thật ra không phải.

Tương tự, đặt f_i là xác suất để kết thúc khi bộ đếm bằng i ; hàm sinh xác suất của nó là $F(x)$. Nhưng định nghĩa của mảng phụ trợ cần đổi một chút: đặt g_i là số lần kỳ vọng để bộ đếm bằng i ; hàm sinh thông thường của nó là $G(x)$.

Ta có thể nhận được các phương trình

$$F(x) + G(x) = G(x) \cdot \frac{x}{2} + \frac{G(1)}{2} + 1, \quad (11)$$

$$F(x) = G(x) \cdot \frac{x}{n}. \quad (12)$$

Ý nghĩa của phương trình thứ nhất là: với mỗi trạng thái của G , có xác suất $\frac{1}{2}$ để bộ đếm tăng thêm 1, và có xác suất $\frac{1}{2}$ để bộ đếm bị xóa về 0.

Vì $F(1) = 1$, nên $G(1) = n$. Phương trình (11) có thể biến thành

$$F(x) + G(x) = G(x) \cdot \frac{x}{2} + \frac{n}{2} + 1. \quad (13)$$

Thay (12) vào (13), ta được

$$G(x) \cdot \left(1 + \frac{x}{n} - \frac{x}{2}\right) = \frac{n}{2} + 1, \quad (8)$$

$$G(x) = \frac{\frac{n}{2} + 1}{1 + \frac{x}{n} - \frac{x}{2}}, \quad (9)$$

$$G(x) = \frac{n^2 + 2n}{2n - (n-2)x}. \quad (14)$$

Lại thay (14) ngược vào (12), ta biết

$$F(x) = \frac{(n+2)x}{2n - (n-2)x}, \quad (10)$$

$$F'(x) = \frac{2n \cdot (n+2)}{[2n - (n-2)x]^2}, \quad (11)$$

$$F'(1) = \frac{2n}{n+2}. \quad (12)$$

7 Tổng kết

Với lớp bài toán gieo xúc xắc, ta có thể xây dựng hàm sinh và sử dụng thông tin trong đề bài để lập quan hệ giữa các hàm sinh, rồi giải ra giá trị mong muốn. Dùng phương pháp này có thể tránh phương pháp khử Gauss truyền thống.

Lớp bài toán được nhắc đến trong bài viết này vẫn đáng được nghiên cứu sâu hơn. Vì vậy tác giả hy vọng bài viết có thể đóng vai trò gợi mở, và hy vọng các độc giả quan tâm có thể nghiên cứu tiếp, mở rộng cách làm của tác giả, từ đó nhận được nhiều cách làm và ứng dụng thú vị hơn.

Lời cảm ơn

- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
- Cảm ơn sự hướng dẫn của huấn luyện viên đội tuyển quốc gia Zhang Ruizhe.
- Cảm ơn thầy Xie Qiufeng đã quan tâm và hướng dẫn.
- Cảm ơn bạn Chen Jianglun và đàn anh Guo Xiaoxu đã cung cấp ý tưởng cho tôi.
- Cảm ơn bạn Chen Jianglun, học trưởng Luo Yuping, đàn anh Guo Xiaoxu và bạn Wang Xiuhua đã đọc duyệt bài viết này.
- Cảm ơn gia đình đã quan tâm và ủng hộ tôi.

Tài liệu tham khảo

1. Wikipedia, *Probability-generating function*.
2. Ronald L. Graham, Donald E. Knuth, Oren Patashnik. *Concrete Mathematics*.
3. Jin Ce, *Phép toán trên hàm sinh và bài toán đếm tổ hợp*, luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015.
4. Hu Yuanming, *Phân tích cơ sở và ứng dụng của xác suất trong thi tin học*, luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2013.
5. Wei Taiyun, *Các cách giải khác nhau và so sánh cho một bài toán tung đồng xu*.

4 Bài 2: Báo cáo ra đề Số nút của cây hậu tố và tối ưu hóa một lớp bài toán đoạn

Chen Jianglun, Trường Trung học Changjun, Trường Sa

Tóm tắt

Cấu trúc dữ liệu hậu tố là một hệ thống hiện đang được ứng dụng rộng rãi trong thi thuật toán. Nó bao gồm các nội dung quen thuộc như mảng hậu tố, automaton hậu tố, cây hậu tố, v.v. Những cấu trúc dữ liệu hậu tố này có thể xử lý hiệu quả rất nhiều bài toán.

Tác giả đã nghiên cứu các thuật toán liên quan tới cấu trúc dữ liệu hậu tố xuất hiện trong những năm gần đây, rồi ra đề *Số nút của cây hậu tố*. Dựa vào tính chất của cây hậu tố và automaton hậu tố, tác giả nhận được một cách làm sơ bộ. Trong quá trình trao đổi ở bài tập đội tuyển và trại mùa đông, tác giả đã thảo luận cách làm này với các bạn khác, rồi cùng họ nghiên cứu ra các thuật toán hiệu quả và ngắn gọn hơn.

Khi nghiên cứu thuật toán cho bài này, tác giả nhận thấy rằng với lớp bài toán chuỗi dùng cây động để bảo trì cây hậu tố, có thể dùng hợp nhất theo heuristic để thay cây động, từ đó làm cho mô hình này được cài đặt đơn giản hơn. Đồng thời, phương pháp này còn có thể mở rộng sang nhiều bài toán đoạn hơn.

Bài viết này trình bày chi tiết nguồn gốc ý tưởng ra đề và quá trình tối ưu từng bước, đồng thời đưa ra một phương án tối ưu cho một lớp bài toán đoạn. Tác giả hy vọng bài viết có thể giúp nhiều người hiểu sâu hơn các tính chất liên quan của cấu trúc dữ liệu hậu tố, cũng như học được phương pháp tối ưu cho một lớp bài toán đoạn.

1 Dẫn nhập

Trong một kỳ ACM-ICPC khu vực năm 2016, đã xuất hiện một mô hình mới mẽ dùng cây động để bảo trì cây hậu tố. Mô hình này có thể giải hiệu quả một lớp bài toán như nhiều lần hỏi số xâu con phân biệt về bản chất của một xâu con trong một xâu, hoặc độ dài xâu con dài nhất xuất hiện ít nhất hai lần trong một xâu con. Điều đó khiến mọi người chú ý tới hướng này.

Trong kỳ tự kiểm tra đội tuyển quốc gia năm 2017, phiên bản tăng cường của bài toán này xuất hiện: đếm số xâu con phân biệt về bản chất trong một xâu con, với điều kiện bắt đầu bằng một ký tự cố định và kết thúc bằng một ký tự cố định.

Đồng thời, những năm gần đây cũng thường xuất hiện một dạng bài: đã biết một thuật toán giải một bài toán nào đó, hãy tính một số thông tin trong quá trình chạy thuật toán ấy. Chẳng hạn trong ZJOI2017 Day1 T2, đã biết một thuật toán hỗ trợ cộng điểm đơn và hỏi tổng đoạn; yêu cầu của đề không phải là đáp án mà thuật toán đó trả về, mà là xác suất thuật toán tính đúng kết quả theo đúng quá trình của nó.

Cây hậu tố là một công cụ mạnh để giải bài toán chuỗi. Từ đó tác giả nghĩ rằng: nếu không yêu cầu dùng cây hậu tố để giải bài toán chuỗi nói chung, mà hỏi số nút nhận được sau khi dựng cây hậu tố cho một xâu thì sao?

Từ đó tác giả bắt đầu nghiên cứu vấn đề này và ra đề bài này.

Cấu trúc bài viết như sau:

- Mục 2 đến 4 là một số định nghĩa cơ bản, mô tả thông tin đề bài và tình hình điểm số.
- Mục 5 đến 10 giới thiệu 3 thuật toán ăn điểm từng phần và 3 thuật toán đạt điểm tối đa.
- Mục 11 đề xuất phương án dùng hợp nhất theo heuristic để thay cây động, qua đó tối ưu độ phức tạp mã nguồn của lớp bài toán đoạn như dùng cây động để bảo trì cây hậu tố, đồng thời trình bày một số ứng dụng của phương án này.

2 Mô tả đề bài

Cho một xâu P độ dài n . Có m truy vấn; mỗi truy vấn cho hai tham số l, r , hỏi số nút của cây hậu tố tạo bởi xâu con $P[l, r]$.

2.1 Phạm vi dữ liệu

Trong bài này dùng các số để biểu thị xâu. Các số khác nhau đại diện cho các ký tự khác nhau, số giống nhau đại diện cho cùng một ký tự; giá trị các số nằm trong đoạn $[0, n]$.

- Với 20% dữ liệu: $n \leq 500$, $m \leq 500$.
- Với 40% dữ liệu: $n \leq 3000$, $m \leq 3000$.
- Với 50% dữ liệu: $n \times m \leq 3 \times 10^8$.
- Với 10% dữ liệu khác: xâu toàn là 0.
- Với 20% dữ liệu khác: mỗi ký tự là một số nguyên không âm ngẫu nhiên trong $[0, n]$, và l, r của mỗi truy vấn đều được sinh ngẫu nhiên.
- Với 100% dữ liệu: $n \leq 10^5$, $m \leq 3 \times 10^5$, mỗi số trong xâu đều không vượt quá n .

2.2 Giới hạn tài nguyên

Giới hạn thời gian 3 giây, giới hạn bộ nhớ 1 GB.

2.3 Định dạng nhập

Dòng đầu gồm hai số nguyên dương n, m , biểu thị độ dài xâu và số truy vấn.

Dòng thứ hai gồm n số nguyên không âm cách nhau bởi dấu cách, biểu thị xâu đó.

Tiếp theo là m dòng, mỗi dòng gồm hai số nguyên dương l, r ($l \leq r$), biểu thị truy vấn xâu con $[l, r]$.

2.4 Định dạng xuất

In m dòng, mỗi dòng một số nguyên dương biểu thị đáp án.

2.5 Ví dụ nhập

```
7 1
2 1 3 1 3 1 4
1 7
```

2.6 Ví dụ xuất

```
10
```

Khi tính bài này, bỏ qua nút gốc.

2.7 Giải thích ví dụ

Cho số 1 biểu thị ký tự a , số 2 biểu thị ký tự b , số 3 biểu thị ký tự n , số 4 biểu thị ký tự s . Khi đó cây hậu tố của xâu trong ví dụ chính là cây hậu tố của bananas. Hình trong bản gốc vẽ cây này với các cạnh lần lượt gắn các nhãn như bananas, a, na, s, na, s, nas, v.v., và tính đáp án sau khi bỏ qua gốc.

3 Phân bố điểm

Có 59 người tham gia lần huấn luyện này. Trong đội tuyển và nhóm bồi dưỡng tinh anh, có 4 người đạt điểm tối đa, 1 người đạt 60 điểm, 1 người đạt 50 điểm, 2 người đạt 40 điểm, 2 người đạt 30 điểm, và 1 người đạt 20 điểm.

4 Các khái niệm liên quan trong bài viết

Với một xâu S , dùng $S[i]$ hoặc S_i biểu thị ký tự thứ i của S , dùng $S[l, r]$ biểu thị xâu con của S nằm trong đoạn $[l, r]$, và dùng $|S|$ biểu thị độ dài xâu S .

Định nghĩa ký tự kế tiếp của một xâu con $S[a, b]$ của S là $S[b + 1]$. Đặc biệt, nếu $S[b]$ là cuối xâu thì ký tự kế tiếp của $S[a, b]$ không được định nghĩa.

4.1 Hash

Hash là một công cụ thường dùng để xử lý bài toán chuỗi; nó thực hiện một ánh xạ nén. Trong bài viết này sẽ dùng hash để ánh xạ mỗi xâu vào một số nguyên, biến bài toán xét hai xâu có bằng nhau hay không thành bài toán xét hai số nguyên có bằng nhau hay không.

4.2 Cây trie hậu tố

Cây trie hậu tố của một xâu S có $|S|$ ký tự là một cây có hướng chứa nút gốc. Mỗi nút đại diện cho một xâu; các nút khác nhau đại diện cho các xâu khác nhau; mỗi nút không phải gốc đại diện cho một xâu con của S ; nút gốc biểu thị xâu rỗng. Trên cây, xâu mà một tổ tiên đại diện là một tiền tố của xâu mà nút đang xét đại diện.

4.3 Cây hậu tố

Cây hậu tố là một dạng nén của cây trie hậu tố.

Trong cây trie hậu tố của S , nếu xâu mà một nút đại diện là một hậu tố của S , thì gọi nút đó là nút hậu tố.

Trên cơ sở cây trie hậu tố, sau khi gộp mỗi nút không phải nút hậu tố và có đúng một con với nút con của nó, ta nhận được cây hậu tố.

Trong cây hậu tố, dùng fa_k biểu thị cha của nút k trên cây. Dùng $last_k$ biểu thị vị trí xuất hiện cuối cùng của xâu mà nút k đại diện.

4.4 Automaton hậu tố

Automaton hậu tố của một xâu S là một automaton hữu hạn có thể chấp nhận mọi hậu tố của S .

Định nghĩa và phương pháp xây dựng automaton hậu tố đã rất phổ biến trong thi thuật toán, nên bài viết này không trình bày lại. Độc giả chưa quen có thể xem tài liệu tham khảo [1].

Cấu trúc cây tạo bởi tất cả các nút của automaton hậu tố được gọi là cây *parent*.

4.5 Cây động

Cây động là một cấu trúc dữ liệu thường dùng trong thi thuật toán hiện nay, dùng để bảo trì động phân rã đường của một cây.

Nội dung cơ bản của cây động đã rất phổ biến; độc giả chưa quen có thể xem tài liệu tham khảo [6].

5 Thuật toán một

Dựa vào cây trie hậu tố để xây dựng cây hậu tố: trước hết dùng mọi hậu tố của một xâu để xây dựng một cây trie, sau đó gộp mọi nút không phải nút hậu tố và có đúng một con với nút con của nó.

Độ phức tạp thời gian của thuật toán này là $O(n^2m)$, độ phức tạp không gian là $O(n^3)$. Lý do là kích thước bảng chữ cái bằng n , còn số nút trong cây trie của mọi hậu tố là $O(n^2)$.

Điểm kỳ vọng: 20 điểm.

6 Thuật toán hai

Với dữ liệu $n, m \leq 3000$, có thể độc lập tính đáp án của từng xâu trong mỗi truy vấn với độ phức tạp tương đối tốt.

Bổ đề 6.1. Cây *parent* của automaton hậu tố là cây hậu tố của xâu đảo.

Tính chất này thường được dùng trong bài toán chuỗi và cũng được nhắc tới trong tài liệu tham khảo [2].

Dựa vào tính chất này, có thể đảo xâu rồi xây dựng automaton hậu tố. Việc xây dựng automaton hậu tố có thể dùng phương pháp tăng dần tuyến tính.

Thuật toán này có độ phức tạp thời gian $O(nm \log n)$, độ phức tạp không gian $O(n)$. Vì kích thước bảng chữ cái của bài này rất lớn, cần dùng cấu trúc dữ liệu, chẳng hạn map trong C++, để bảo trì các cạnh đi ra của mỗi điểm; do đó độ phức tạp thời gian có thêm một nhân tử $\log$.

Điểm kỳ vọng: 40 điểm.

7 Thuật toán ba

Thực ra việc lưu các cạnh chuyển của automaton hậu tố có thể dùng hash để tối ưu cấu trúc dữ liệu của thuật toán hai. Khi đó đạt được độ phức tạp thời gian $O(nm)$, còn độ phức tạp không gian vẫn là $O(n)$.

Điểm kỳ vọng: 50 điểm.

8 Thuật toán bốn

8.1 Phân tích sơ bộ

Ký hiệu số nút cây hậu tố của xâu cần xét S là $A(S)$.

Các nút của cây hậu tố có thể chia làm hai loại: nút hậu tố và nút không phải hậu tố. Số nút hậu tố đúng bằng $|S|$; phần này chỉ liên quan tới độ dài xâu.

Vì nút lá của cây trie hậu tố nhất định là nút hậu tố, nên một nút không phải hậu tố nhất định có ít nhất một con. Nếu một nút không phải hậu tố có đúng một con thì trong cây hậu tố nút này sẽ bị gộp với con của nó. Do đó, những nút cần được tính vào đáp án phải thỏa có ít nhất hai con.

Một nút trên cây trie hậu tố đại diện cho một xâu con của S . Nếu nút k có ít nhất hai con, điều đó có nghĩa là tồn tại ít nhất hai ký tự khác nhau c_1, c_2 , sao cho xâu con t mà nút k đại diện thỏa: tc_1 và tc_2 cũng đều là xâu con của S .

8.2 Cách làm phức tạp

Sau khi phát hiện tính chất trên, tác giả nhận được một cách làm có độ phức tạp thời gian $O(n \log^3 n + m \log^2 n)$. Tuy nhiên, cách làm này quá phức tạp, hiệu suất cũng thấp, rất khó hiểu. Đồng thời phương pháp này không ảnh hưởng tới việc hiểu các cách làm phía sau, nên bài viết này không trình bày lại. Độ giả quan tâm có thể tìm phần giới thiệu chi tiết về cách làm này trong tài liệu tham khảo [5].

Điểm kỳ vọng: 100 điểm.

9 Thuật toán năm

9.1 Ứng dụng mô hình cây động bảo trì cây hậu tố

Điểm khó của bài này nằm ở nhiều nhóm truy vấn, mỗi nhóm hỏi một xâu con $P[l, r]$ của xâu nhập P . Xét việc liệt kê đầu trái l của truy vấn từ phải sang trái; với mỗi r , ta bảo trì động số nút cây hậu tố của xâu $P[l, r]$.

Khi l dịch sang trái một vị trí, với mỗi xâu $P[l, r]$, xét các thay đổi giữa $P[l, r]$ và $P[l + 1, r]$.

Dựa vào tính chất vừa phát hiện, xét một tiền tố $P[l, v]$ của $P[l, r]$, gọi xâu này là t . Nếu trong $[l, r]$ tồn tại hai ký tự c_1, c_2 sao cho tc_1 và tc_2 đều là xâu con của $P[l, r]$, còn xâu $P[l + 1, r]$ không thỏa điều kiện này, thì xâu t sẽ tạo thêm đóng góp 1 cho đáp án của $P[l, r]$. Sau khi đầu trái chuyển từ $l + 1$ sang l , cần lập tức tìm các v thỏa yêu cầu này.

Vì tc_1 và tc_2 đều là xâu con của $P[l, r]$, nên xâu t chắc chắn đã xuất hiện trong $P[l + 1, r]$. Do trong $P[l + 1, r]$, t chưa tạo đóng góp cho đáp án, nên với mọi vị trí xuất hiện $P[a, b]$ của t trong $P[l + 1, r]$ ($b - a + 1 = |t|$, $P[a, b] = t$), hoặc $b = r$, hoặc các ký tự kế tiếp của những xâu này, tức $P[b + 1]$, đều giống nhau.

Hình trong bản gốc minh họa trường hợp trên bằng các lần xuất hiện của t trong đoạn $[l, r]$, kèm các ký tự kế tiếp c_1, c_2, c_2 .

Mọi tiền tố $P[l, v]$ ($l \leq v \leq r$) của xâu $P[l, r]$ tương ứng trên cây trie hậu tố với một đường đi từ gốc tới một nút nào đó. Giả sử các nút trên đường này lần lượt là $x_1, x_2, \dots, x_{r-l+1}$. Cần thống kê có bao nhiêu nút x_i thỏa x_i tồn tại một con khác x_{i+1} . Nếu tồn tại, vì x_{i+1} là con của x_i , nên x_i đã có ít nhất hai con.

Mỗi lần thêm mọi tiền tố bắt đầu ở l hiện tại, ta đánh dấu đã thăm cho tất cả $x_1, x_2, \dots, x_{r-l+1}$. Định nghĩa *con thực* của một nút là nút con được thăm cuối cùng trong tất cả các con của nó. Nếu sau một thao tác, con thực của một nút k không thay đổi, điều đó cho thấy ký tự kế tiếp của xâu t mà nút k đại diện giống với ký tự kế tiếp trong lần xuất hiện trước của xâu này. Trong hình của bản gốc, ký tự kế tiếp của xâu hiện tại $t = P[l, l + |t| - 1]$ và ký tự kế tiếp của lần xuất hiện trước của t đều là c_1 .

Chú ý rằng điều kiện để xét một xâu có tạo đóng góp mới cho đáp án hay không là: hiện tại đã xuất hiện ít nhất hai loại ký tự kế tiếp, còn trước đó chỉ xuất hiện một loại ký tự kế tiếp. Vì vậy, nếu ký tự kế tiếp của một tiền tố của $P[l, r]$ bằng ký tự kế tiếp của lần xuất hiện cuối cùng, thì nó không thể tạo đóng góp mới cho đáp án.

Nói cách khác, chỉ những nút có con thực thay đổi mới có khả năng tạo đóng góp mới cho đáp án. Chú ý rằng định nghĩa con thực ở đây giống con thực trong cây động, và mỗi lần đánh dấu tương đương với thao tác access của cây động. Theo phân tích độ phức tạp của cây động, số lần chuyển con thực là $O(\log n)$ khấu hao, vì vậy điều này liên hệ với mô hình cổ điển dùng cây động để bảo trì cây hậu tố.

Cụ thể, dùng $last_t$ biểu thị vị trí xuất hiện trước của xâu t , và dùng $diff_t$ biểu thị vị trí xuất hiện trước của xâu t với ký tự kế tiếp khác. Hình trong bản gốc minh họa $last[t]$ và $diff[t]$ bằng bốn lần xuất hiện của t có các ký tự kế tiếp c_1, c_2, c_2, c_3 .

Với xâu con $P[l + 1, r]$, khi $r \geq diff_t + |t|$, t tạo đóng góp cho đáp án của $P[l + 1, r]$; khi $r < diff_t + |t|$, t chưa tạo đóng góp cho đáp án.

Với xâu con $P[l, r]$, khi $r \geq last_t + |t|$, t tạo đóng góp cho đáp án của $P[l, r]$; khi $r < last_t + |t|$, t chưa tạo đóng góp cho đáp án.

Do đó, khi chuyển từ $P[l + 1, r]$ sang $P[l, r]$, nếu r nằm trong đoạn $[last_t + |t|, diff_t + |t|)$, đáp án sẽ mới tăng thêm 1. Đây là một thao tác cộng đoạn.

Như vậy, có thể dùng cây động và cây đoạn để hỗ trợ access, tìm các nút có con thực thay đổi, và thực hiện thao tác cộng đoạn.

9.2 Tiểu kết

Nội dung trên chính là ứng dụng của một mô hình cổ điển trong bài này.

Nhưng đó chỉ là một phần của bài. Tiếp theo còn phải đối mặt với một vấn đề khác.

9.3 Xử lý đáp án bị tính lặp

Khi tính số nút cây hậu tố, ta chia toàn bộ nút thành hai loại: nút hậu tố và nút không phải hậu tố. Phần trước đã thống kê “nút hậu tố + nút trên cây trie hậu tố có ít nhất hai con”. Trong đó, các nút vừa là nút hậu tố vừa có hai con đã bị tính hai lần. Theo nguyên lý bù trừ, chỉ cần trừ số nút đồng thời thỏa hai điều kiện này là nhận được đáp án cuối cùng.

Khi hỏi xâu $P[l, r]$, nếu tồn tại một v sao cho xâu $P[v, r]$ xuất hiện trong $P[l, r]$ với ít nhất hai ký tự kế tiếp khác nhau, thì xâu này sẽ bị tính lặp.

9.4 Tính chất then chốt

Sau khi phân tích, có thể phát hiện một tính chất then chốt: nếu một xâu t có hai ký tự kế tiếp khác nhau trong $P[l, r]$, thì mọi hậu tố của t đều có ít nhất hai ký tự kế tiếp khác nhau.

Dựa vào tính chất này, độ dài các xâu bị tính lặp thỏa tính đơn điệu, nên có thể dùng chặt nhị phân để tính độ dài lớn nhất của xâu bị tính lặp.

Sau khi chặt nhị phân, bài toán chuyển thành: cho một xâu, hãy xét xâu này có hai ký tự kế tiếp khác nhau trong một đoạn hay không.

9.5 Hợp nhất cây đoạn

Có hai ký tự kế tiếp khác nhau nghĩa là trên cây trie hậu tố có hai con khác nhau. Vì vậy cần hỏi một nút có tồn tại hai cây con của hai con chứa nút hậu tố nằm trong $[l, r]$ hay không.

Vì con thực là nút con được thăm cuối cùng trong các con của một nút, nếu xâu mà con thực đại diện không xuất hiện trong đoạn $[l, r]$, điều đó cho thấy xâu mà nút hiện tại đại diện không có bất kỳ ký tự kế tiếp nào trong $[l, r]$, chắc chắn không thỏa điều kiện. Nếu xâu mà con thực đại diện xuất hiện trong đoạn $[l, r]$, tiếp theo cần xét trong cây con của các con khác có tồn tại một nút hậu tố nằm trong $[l, r]$ hay không; vấn đề này có thể giải bằng hợp nhất cây đoạn.

Dùng hợp nhất cây đoạn để bảo trì tập vị trí bắt đầu của các xâu do nút hậu tố trong mỗi cây con đại diện, ta có thể truy vấn trong $O(\log n)$ số nút hậu tố của một cây con nằm trong đoạn $[l, r]$. Xét ngoài con thực ra còn có cây con của con nào khác tồn tại nút hậu tố nằm trong $[l, r]$ hay không, tương đương với truy vấn xem hiệu giữa số nút hậu tố trong cây con của nút hiện tại nằm trong $[l, r]$ và số nút hậu tố trong cây con của con thực nằm trong $[l, r]$ có bằng 0 hay không.

Trên đây là toàn bộ nội dung của thuật toán năm. Thuật toán dùng cây động + cây đoạn và chặt nhị phân + hợp nhất cây đoạn; tổng độ phức tạp là $O(n \log^2 n + m \log^2 n)$.

Điểm kỳ vọng: 100 điểm.

10 Thuật toán sáu

Bài này thực ra còn có thể tối ưu thêm. Chú ý rằng quá trình xử lý là liệt kê đầu trái l của truy vấn, rồi tính đáp án cho mọi truy vấn có đầu trái hiện tại. Khi l cố định, với một xâu t , chỉ cần tìm một r nhỏ nhất sao cho t xuất hiện trong $[l, r]$ với ký tự kế tiếp khác nhau. Giá trị r này chính là $\text{diff}_{\text{node}(t)} + |t|$ đã được bảo trì trước đó, trong đó $\text{node}(t)$ là nút tương ứng với xâu t trên cây hậu tố.

Sau khi chèn nhị phân một xâu, có thể dùng bảng hash để truy vấn trong $O(1)$ vị trí của một xâu trên cây hậu tố. Như vậy có thể phán đoán trong $O(1)$ xem một xâu có xuất hiện trong một đoạn với ký tự kế tiếp khác nhau hay không.

Nhờ đó độ phức tạp thời gian được tối ưu thành $O(n \log^2 n + m \log n)$.

Điểm kỳ vọng: 100 điểm.

11 Tối ưu hóa một lớp bài toán đoạn

Khi nghiên cứu mô hình cây động bảo trì cây hậu tố, tác giả phát hiện bản chất của việc chuyển con thực là: với một xâu t , tìm mọi vị trí xuất hiện của nó và sắp xếp thành một dãy $\text{pos}_1, \text{pos}_2, \dots, \text{pos}_v$ theo thứ tự tăng dần của đầu phải vị trí xuất hiện. Với hai phần tử kế nhau bất kỳ pos_i và pos_{i+1} của dãy này, nếu ký tự kế tiếp của hai vị trí này khác nhau, điều đó cho thấy các nút lá hậu tố tương ứng với hai xâu này nằm trong cây con của hai con khác nhau của nút tương ứng với t trên cây hậu tố. Khi đầu trái truy vấn được liệt kê từ pos_{i+1} tới pos_i , con thực trên cây hậu tố sẽ chuyển một lần.

Một khi đã bảo trì dãy như vậy, ta cũng có thể tính được last và diff mà không cần cây động.

Dùng hợp nhất theo heuristic để tính dãy pos của mỗi cây con. Đồng thời, hợp nhất theo heuristic cũng có thể chứng minh độ phức tạp số lần chuyển con thực trong bài này. Độ phức tạp chính là số lần xuất hiện tình huống pos_i và pos_{i+1} có ký tự kế tiếp khác nhau.

Định nghĩa con nặng là con có số nút hậu tố trong cây con lớn nhất trong tất cả các con, các con khác là con nhẹ. Nếu trước hết chỉ xét con nặng, ký tự kế tiếp của pos_1 tới pos_v đều giống nhau. Sau đó thêm từng nút hậu tố của con nhẹ vào; mỗi lần thêm một nút vào dãy pos , nhiều nhất tạo ra 2 điểm đứt mới với hai phần tử kế trước và sau. Vì mỗi số nhiều nhất được chen $O(\log n)$ lần với tư cách thuộc con nhẹ, nên tổng độ phức tạp là $O(n \log n)$ nhân với độ phức tạp của phép cộng đoạn.

Do ngay từ bước đầu tác giả đã nghĩ tới việc dùng hợp nhất theo heuristic để thay cây động, nên đã không nhận ra rằng phần sau có thể trực tiếp dùng phương pháp trong thuật toán sáu để phán đoán. Dĩ nhiên, vì hợp nhất theo heuristic cũng có thể tính được diff và last tương ứng, thuật toán sáu vẫn có thể được cài đặt bằng hợp nhất theo heuristic.

Phương pháp như vậy không chỉ dùng được cho bài này, mà còn có thể dùng trong nhiều bài toán khác.

11.1 Ví dụ một

11.1.1 Mô tả đề bài và phạm vi dữ liệu Cho một xâu $P[1, n]$, có m truy vấn; mỗi truy vấn hỏi số xâu con phân biệt về bản chất trong một xâu con $[l, r]$.

$$n, m \leq 10^5.$$

11.1.2 Phân tích Dựa vào ý tưởng được cung cấp trong bài viết này, liệt kê đầu trái l của truy vấn từ phải sang trái, rồi bảo trì đáp án cho mỗi đầu phải.

Vấn xét lượng thay đổi của đáp án khi đầu trái truy vấn chuyển từ $l + 1$ thành l . Với một xâu $P[l, v]$, nếu đầu phải của vị trí xuất hiện trước của xâu này lớn hơn r , còn đầu phải v của vị trí xuất hiện hiện tại không vượt quá r , thì nó tạo đóng góp mới 1 cho đáp án. Tương tự, dùng $last_t$ biểu thị vị trí xuất hiện cuối cùng của xâu t . Khi đó với mọi $v \in [l, n]$, đáp án của các truy vấn có đầu phải nằm trong $[v, last_{P[l, v]} - 1]$ cần tăng thêm 1.

Tiếp theo, nếu xâu $P[l, v]$ và xâu $P[l, v + 1]$ có $last$ giống nhau, thì có thể xử lý chung, tương đương với cộng một cấp số cộng có công sai 1 lên một đoạn.

Theo tính chất của cây động, mọi điểm nằm trên cùng một đường thực có thời điểm được thăm cuối cùng giống nhau; tức là $last$ của các xâu mà những nút này đại diện giống nhau. Do đó, các điểm trên cùng một đường thực có thể được xử lý cùng lúc. Mỗi lần access, số đường thực được thăm là $O(\log n)$ khẩu hao, nên tổng độ phức tạp thời gian là $O(n \log^2 n)$, độ phức tạp không gian $O(n)$.

Khi access, ta sẽ tìm được mọi đường thực từ một nút tới gốc. Điểm đổi giữa hai đường thực chính là nơi xảy ra một lần chuyển con thực. Phần trước đã nói rằng hợp nhất theo heuristic có thể truy vấn mọi sự kiện chuyển con thực, nên có thể dùng hợp nhất theo heuristic để thay cây động.

11.2 Tiểu kết

Các bài toán chuỗi dùng được tối ưu này thường có các đặc điểm sau:

1. Có một xâu mẫu lớn được nhập vào, mỗi truy vấn hỏi thông tin của một xâu con của xâu này.
2. Có thể biết toàn bộ xâu mẫu lớn trước khi trả lời truy vấn, không phải bắt buộc trực tuyến thêm ký tự vào cuối xâu mẫu.
3. Đáp án cần tìm liên quan tới mỗi xâu con phân biệt về bản chất và tiền nhiệm, hậu nhiệm của nó trong xâu mẫu.

11.3 Mở rộng

Khi nghiên cứu sâu hơn quá trình này, tác giả phát hiện phương pháp này có thể mở rộng sang nhiều bài toán đoạn hơn.

11.4 Ví dụ hai

11.4.1 Mô tả đề bài và phạm vi dữ liệu Cho một cây n nút, có m nhóm truy vấn. Mỗi lần hỏi số nút sau khi xây dựng cây ảo trên tất cả các nút có số hiệu nằm trong $[l, r]$.

$$n, m \leq 10^5.$$

11.4.2 Phân tích Với truy vấn $[l, r]$, xét xem mỗi điểm k có tạo đóng góp cho đáp án hay không. k nằm trong cây ảo nếu $l \leq k \leq r$, hoặc nếu trong cây con của ít nhất hai con của k đều tồn tại một nút nằm trong đoạn $[l, r]$.

Vẫn dùng ý tưởng tương tự: liệt kê đầu trái l từ phải sang trái, rồi bảo trì động đáp án cho mỗi đầu phải r . Mỗi lần thực hiện một thao tác access với đầu trái hiện tại l ; việc chuyển cạnh ảo-thực tương ứng với việc đáp án của các truy vấn có đầu phải r trong một đoạn sẽ mới tăng thêm 1.

Cũng có thể dùng hợp nhất theo heuristic: sắp xếp mọi nút trong cây con của k theo số hiệu. Khi $[l, r]$ chứa hai nút đến từ cây con của hai con khác nhau, k sẽ nằm trong cây ảo. Ở đầu mục này đã chứng minh tổng số đoạn là $O(n \log n)$. Vì vậy bài này có thể được giải trong thời gian $O(n \log^2 n) + O(m \log n)$.

12 Tổng kết

Bài này là một bài toán lấy số nút cây hậu tố làm mô hình, khảo sát năng lực phân tích tính chất cây hậu tố của thí sinh và năng lực vận dụng mô hình cổ điển. Các cấu trúc liên quan gồm automaton hậu tố, cây hậu tố, cây đoạn, cây động và hợp nhất theo heuristic, đều thuộc phạm vi kiến thức của thí sinh NOI.

Tuy nhiên, bài này chú trọng kiểm tra khả năng phân tích tính chất của cấu trúc dữ liệu hậu tố. Chẳng hạn so với cây hậu tố, mỗi nút không bị nén trên cây trie hậu tố tương ứng với một xâu con trong xâu gốc; xâu con đó hoặc là một hậu tố, hoặc có hai ký tự kế tiếp khác nhau.

Bài này gồm hai phần, mỗi phần đều cần suy nghĩ sâu mới nhận được phương án giải hiệu quả; đối với thí sinh, độ khó là rất lớn.

Ban đầu bài này được tạo ra khi nghiên cứu tính chất của automaton hậu tố. Lúc đầu tác giả đặt trực tiếp vấn đề lên automaton hậu tố và nhận được thuật toán bốn rất phức tạp. Đưa bài này vào bài tập đội tuyển là để thử tìm cách làm đơn giản hơn.

Sau đó, bạn Zhu Zhenting từ THPT trực thuộc Đại học Sư phạm An Huy phát hiện một tính chất quan trọng, tức tính chất được nhắc tới trong bài viết này: độ dài các xâu bị tính lặp thỏa tính đơn điệu. Qua chắt nhai phân, cách làm được đơn giản hóa rất nhiều. Trong buổi trao đổi của trại mùa đông năm nay, bạn ấy cùng bạn Wang Xiuhuan từ Trường Trung học số 7 Thành Đô đã chia sẻ ý tưởng này. Đó chính là thuật toán nằm trong bài viết này.

Sau buổi trao đổi, tác giả và bạn Zhang Yubo từ Trường Trung học số 80 Bắc Kinh đã thảo luận sâu hơn, phát hiện chỉ cần dùng *last*, *diff* và bảng hash là có thể phán đoán trong $O(1)$ xem một xâu có từng xuất hiện trong một đoạn với ký tự kế tiếp khác nhau hay không. Đó chính là thuật toán sáu trong bài viết này.

Trong quá trình nghiên cứu các thuật toán này, tác giả phát hiện bản chất của cây động bảo trì cây hậu tố là tìm mọi vị trí xuất hiện của một xâu, rồi chia chúng theo ký tự kế tiếp: các xâu kế nhau và có ký tự kế tiếp giống nhau được chia vào cùng một đoạn; giữa các đoạn khác nhau sẽ phát hiện thay đổi của *last* và *diff*. Vì vậy có thể dùng hợp nhất theo heuristic để thay cây động, làm đơn giản độ phức tạp mã nguồn. Đồng thời tác giả phát hiện phương pháp này có thể mở rộng sang một số bài toán đoạn khác.

Hy vọng thông qua bài này, mọi người có thể hiểu sâu hơn về tính chất của automaton hậu tố và cây hậu tố, đồng thời thành thạo cách giải tinh tế cho một lớp bài toán đoạn.

13 Lời cảm ơn

- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
- Cảm ơn sự hướng dẫn của huấn luyện viên đội tuyển quốc gia Zhang Ruizhe.
- Cảm ơn thầy Xie Qiufeng của Trường Trung học Changjun đã quan tâm và hướng dẫn.
- Cảm ơn Wang Xiuhuan, Zhu Zhenting và Zhang Yubo đã giúp đỡ về ý tưởng tối ưu của bài này.
- Cảm ơn Luo Yuping, Guo Xiaoxu, Yang Maolong, Liu Chengao, Zhang Qingchuan và Wu Hang đã đọc duyệt bài viết này.
- Cảm ơn gia đình đã quan tâm và ủng hộ tôi.

Tài liệu tham khảo

1. Chen Lijie, *Automaton hậu tố*, trao đổi trại mùa đông năm 2012.
2. Zhang Tianyang, *Automaton hậu tố và ứng dụng*, tập luận văn ứng viên đội tuyển quốc gia Trung Quốc năm 2015.
3. Hong Huadun, *Báo cáo ra đề Tái cấu trúc hệ gene*, tập luận văn ứng viên đội tuyển quốc gia Trung Quốc năm 2017.
4. *String*, ACM/ICPC Asia Regional Qingdao Online 2016.
5. Chen Jianglun, *Báo cáo lời giải đề tự chọn trong bài tập đội tuyển quốc gia IOI2018*.
6. Huang Zhi'ao, *Bàn sơ lược về các vấn đề liên quan tới cây động và một số mở rộng đơn giản*, tập luận văn ứng viên đội tuyển quốc gia Trung Quốc năm 2014.

5 Bài 3: Bàn sơ lược về bài toán hồi quy bảo toàn thứ tự

Gao Ruiquan, Trường Ngoại ngữ Nam Kinh

Tóm tắt

Bài viết này là luận văn đội tuyển quốc gia Tin học Trung Quốc năm 2018 của tác giả. Bài viết chủ yếu giới thiệu bài toán hồi quy bảo toàn thứ tự. Trước hết, thông qua các trường hợp đặc biệt, bài viết giới thiệu những cách làm thường gặp trong thi tin học cho bài toán L_p ($1 \leq p < \infty$); tiếp đó giới thiệu một ý tưởng mới dùng chia đôi toàn cục để giải bài toán tổng quát và các tối ưu của nó; cuối cùng giới thiệu sơ lược cách giải và mở rộng của bài toán L_∞ .

1 Dẫn nhập

Bài toán hồi quy bảo toàn thứ tự là một lớp bài toán ít được thảo luận trong thi tin học, có độ khó cao và có thể xây dựng lời giải bằng nhiều hướng tư duy khác nhau. Trong đời sống thực tế, bài toán hồi quy bảo toàn thứ tự thường xuất hiện trong các bài toán thống kê như dự đoán ngộ độc thuốc, thiết lập cỡ quần áo, v.v.

Bài toán hồi quy bảo toàn thứ tự được thảo luận trong bài này phụ thuộc vào những kết luận toán học khá mạnh. Một loại kết luận được suy ra bằng cách kết hợp hình học và đại số; một loại khác được giải quyết bằng cách xây dựng bài toán mới rồi dùng chia đôi toàn cục. Hướng tư duy đó vượt khỏi cách nghĩ thông thường cho các bài toán giá trị tối ưu trong thi lập trình, và có thể đem lại nhiều gợi mở cho thí sinh.

Mục 2 giới thiệu các khái niệm cơ bản của bài toán hồi quy bảo toàn thứ tự (L_p). Các mục 3 đến 5 trình bày chi tiết cách làm cho L_p ($1 \leq p < \infty$): mục 3 giới thiệu một số lời giải trong các tình huống đặc biệt từ góc nhìn tham lam và bảo trì đường gấp khúc; mục 4 trình bày chi tiết cách dùng chia đôi toàn cục để giải bài toán hồi quy bảo toàn thứ tự tổng quát; mục 5 giới thiệu các tối ưu của cách làm trên những cấu trúc thứ tự bộ phận đặc biệt. Mục 6 giới thiệu sơ lược cách giải và mở rộng của L_∞ .

2 Bài toán hồi quy bảo toàn thứ tự

2.1 Quan hệ thứ tự bộ phận

Định nghĩa 2.1. Giả sử R là một quan hệ hai ngôi trên tập S . Nếu R thỏa:

- tính phản xạ: $\forall x \in S$, có xRx ;
- tính phản đối xứng: $\forall x, y \in S$, nếu xRy và yRx thì $x = y$;
- tính bắc cầu: $\forall x, y, z \in S$, nếu xRy và yRz thì xRz ,

thì gọi R là một quan hệ thứ tự bộ phận không nghiêm ngặt trên S , ký hiệu là $\leq$.

2.2 Mô tả bài toán

Cho số nguyên dương p , một đồ thị có hướng không chu trình G với tập đỉnh $V = \{v_1, v_2, \dots, v_n\}$, tập cạnh E ($|E| = m$), và hàm chi phí (y, w) ($\forall i, w_i > 0$). Nếu trong G có một đường đi có hướng từ v_i đến v_j , thì ta có quan hệ thứ tự bộ phận $v_i \leq v_j$.

Cần tìm dãy giá trị đỉnh f , thỏa với mọi $v_i \leq v_j$ ($i, j \in [1, n]$) đều có $f_i \leq f_j$, sao cho chi phí hồi quy là nhỏ nhất:

$$\sum_{i=1}^n w_i |f_i - y_i|^p, \quad 1 \leq p < \infty,$$

$$\max_{i=1}^n w_i |f_i - y_i|, \quad p = \infty.$$

Với cùng một p , lớp bài toán này được gọi chung là bài toán L_p .

2.3 Một số quy ước

Định nghĩa 2.2. Với dãy z , thao tác biến các phần tử không vượt quá a thành a , và biến các phần tử không nhỏ hơn b thành b , được gọi là làm tròn dãy z về tập $S = \{a, b\}$.

Định nghĩa 2.3. Giá trị trung bình L_p của tập đỉnh U là giá trị k làm nhỏ nhất

$$\sum_{v_i \in U} w_i |y_i - k|^p \quad (1 \leq p < \infty)$$

hoặc

$$\max_{v_i \in U} w_i |y_i - k| \quad (p = \infty).$$

3 Thuật toán trong các trường hợp đặc biệt

3.1 Một thuật toán tham lam

Ví dụ 1. Approximation Nguồn đề: phỏng theo ASC38 Problem A; trong đề gốc mọi w_i đều bằng 1.

Cho các dãy số nguyên dương y, w . Hãy tìm dãy số thực đơn điệu không giảm f , sao cho

$$\sum_{i=1}^n w_i (f_i - y_i)^2$$

nhỏ nhất. $n \leq 200000$.

Bổ đề 3.1. Giá trị trung bình L_2 của tập đỉnh U là trung bình có trọng số

$$\frac{\sum_{v_i \in U} w_i y_i}{\sum_{v_i \in U} w_i}.$$

Chứng minh. Có thể chứng minh đơn giản bằng đạo hàm. ■

Bổ đề 3.2. Với mọi $1 \leq i < n$, nếu $y_i > y_{i+1}$, thì trong nghiệm tối ưu nhất định có $f_i = f_{i+1}$.

Chứng minh. Phản chứng rằng trong nghiệm tối ưu $f_i < f_{i+1}$. Chọn một $\varepsilon > 0$ đủ nhỏ, khi đó đặt

$$f'_i = f_i + \varepsilon w_{i+1}, \quad f'_{i+1} = f_{i+1} - \varepsilon w_i$$

sẽ không ảnh hưởng đến quan hệ thứ tự bộ phận. Khi đó

$$\begin{aligned} & w_i(f'_i - y_i)^2 + w_{i+1}(f'_{i+1} - y_{i+1})^2 - w_i(f_i - y_i)^2 - w_{i+1}(f_{i+1} - y_{i+1})^2 \\ &= -2\varepsilon w_i w_{i+1}(f_{i+1} + y_i - f_i - y_{i+1}) + \varepsilon^2 w_i w_{i+1}(w_i + w_{i+1}) < 0. \end{aligned}$$

Điều này mâu thuẫn với tính tối ưu. ■

Theo Bổ đề 3.1 và Bổ đề 3.2, ta có thể dùng quy trình sau để giải:

1. Duy trì một ngăn xếp đơn điệu. Mỗi phần tử trong ngăn xếp là (S_i, y'_i, w'_i) , trong đó S_i là một tập, còn y'_i, w'_i là các số thực. Giả sử tại mỗi thời điểm kích thước ngăn xếp là m . Ban đầu ngăn xếp rỗng.
2. Xét các i từ trái sang phải, với mỗi i thêm phần tử $(\{i\}, y_i, w_i)$. Nếu tại một thời điểm nào đó $m > 1$ và $y'_{m-1} > y'_m$, thì lấy (S_m, y'_m, w'_m) và $(S_{m-1}, y'_{m-1}, w'_{m-1})$ khỏi ngăn xếp, rồi thêm phần tử

$$\left(S_{m-1} \cup S_m, \frac{y'_{m-1}w'_{m-1} + y'_m w'_m}{w'_{m-1} + w'_m}, w'_{m-1} + w'_m \right).$$

3. Đáp án f_i là y'_j ứng với tập S_j chứa i .

Độ phức tạp thời gian của thuật toán trên là $O(n)$. Thuật toán này có thể mở rộng sang bài toán $1 \leq p < \infty$: chỉ cần thay trung bình có trọng số sau khi gộp bằng giá trị trung bình L_p của tập mới.

Cách tham lam như vậy có thể mở rộng sang cấu trúc thứ tự bộ phận tổng quát hơn không? Câu trả lời là không. Lý do là kết luận “không ảnh hưởng đến quan hệ thứ tự bộ phận” trong Bổ đề 3.2 không còn đúng nữa, nên không thể dùng tiếp thuật toán phía sau.

3.2 Thuật toán bảo trì đường gấp khúc

Thuật toán bảo trì đường gấp khúc là một phương pháp quan trọng để tối ưu quy hoạch động, và có thể mở rộng sang bài toán hồi quy bảo toàn thứ tự khi G tạo thành một cây.

Ví dụ 2. Đồ thị có hướng Bài tập đội tuyển năm 2018, đề liên khảo day 2 T3, phần điểm 65.

Cho đồ thị có hướng liên thông yếu $G = (V, E)$ gồm n đỉnh và $n - 1$ cạnh. Mỗi đỉnh có trọng số đỉnh d_i và thời gian sửa đổi w_i . Với mỗi $i (1 \leq i \leq n)$, mỗi lần sửa đổi có thể tốn w_i thời gian để tăng hoặc giảm d_i đi 1. Hãy tìm tổng thời gian ít nhất để sau khi sửa đổi, với mọi $(u, v) \in E$ đều có $d_u \leq d_v$.

$$n \leq 3 \times 10^5, \quad 1 \leq d_i \leq 10^9, \quad 1 \leq w_i \leq 10^4.$$

Ta dễ nghĩ đến việc dùng quy hoạch động trên cây.

Dễ thấy mọi giá trị cuối cùng nhất định nằm trong tập $D = \{d_1, d_2, \dots, d_n\}$. Gọi D_j là phần tử nhỏ thứ j trong D , và $g_{i,j}$ là chi phí sửa đổi nhỏ nhất trong cây con của đỉnh i khi đỉnh i được chỉnh về D_j . Chuyển trạng thái quy hoạch động có thể được tối ưu bằng giá trị nhỏ nhất tiền tố $L_{i,j}$ và giá trị nhỏ nhất hậu tố $R_{i,j}$.

Với đỉnh lá i , hiển nhiên g_i, L_i, R_i đều là các đường gấp khúc có hệ số góc đơn điệu không giảm.

Với đỉnh bất kỳ i , g_i có thể nhận được bằng cách cộng các L hoặc R của mọi con của nó với đường gấp khúc $y = w_i|x - d_i|$. Còn L_i là đường nhận được bằng cách biến phần có hệ số góc dương của g_i thành $y = \min\{g_{i,j}\}$; R_i là đường nhận được bằng cách biến phần có hệ số góc âm của g_i thành $y = \min\{g_{i,j}\}$. Vì vậy, bằng quy nạp có thể chứng minh rằng mọi g_i, L_i, R_i đều là các đường gấp khúc có hệ số góc đơn điệu không giảm.

Do đó có thể dùng cây đoạn để duy trì biểu thức hàm của từng đoạn trong các đường gấp khúc tạo bởi g, L, R (hiển nhiên $x = D_j \sim D_{j+1}$ là một đoạn): trong tính toán g_i , việc cộng các L và R của con có thể thực hiện bằng gộp cây đoạn¹; việc cộng đường gấp khúc $y = w_i|x - d_i|$ là một thao tác cộng đoạn.

Vì L_i và R_i tương ứng với g_i sẽ không đồng thời ảnh hưởng đến g của tổ tiên, nên có thể tính L_i, R_i bằng cách chặt nhị phân và sửa trực tiếp trên cây đoạn của g_i .

Độ phức tạp thời gian của thuật toán trên là $O(n \log n)$.

Có thể thấy phương pháp bảo trì đường gấp khúc vẫn không thể mở rộng sang cấu trúc thứ tự bộ phận tổng quát. Nút thắt nằm ở chỗ nó cần cấu trúc thứ tự bộ phận của toàn bộ đồ thị hỗ trợ quá trình quy hoạch động từ dưới lên.

4 Thuật toán cho bài toán tổng quát

4.1 Một hướng tư duy khác: bắt đầu từ chia đôi toàn cục

Trong thi tin học, ta thường gặp những bài cần trả lời nhiều truy vấn mà mỗi truy vấn đều có thể trả lời bằng chặt nhị phân. Nếu chặt nhị phân riêng cho từng truy vấn, thường sẽ xuất hiện rất nhiều tính toán lặp. Vì vậy lúc này có thể dùng tư tưởng toàn cục: nhờ cùng một phép tiền xử lý, nhanh chóng truy vấn mọi câu hỏi có đáp án rơi vào một khoảng, rồi chia mỗi truy vấn sang một khoảng đáp án mới. Tư tưởng này cũng áp dụng được cho các bài tương tác mà ta chỉ biết số lượng đáp án trong khoảng.

Vì bài viết này không liên quan đến các cách làm chia đôi toàn cục phức tạp hơn, các mở rộng cụ thể có thể xem ở tài liệu tham khảo [1]. Dưới đây là một ví dụ giúp độc giả hiểu chia đôi toàn cục tốt hơn.

Ví dụ 3. Library Nguồn đề: JOI2018 spring camp day 4 T2.

Trên bàn có n quyển sách xếp thành một hàng từ trái sang phải. Ta chỉ biết các quyển được đánh số $1 \sim n$, nhưng không biết thứ tự của các số. Mỗi lần, người quản lý có thể lấy đi một số quyển sách liên tiếp. Mỗi lần bạn có thể hỏi thư viện tương tác một tập số hiệu S ; thư viện trả về số lần ít nhất để người quản lý lấy đi mọi quyển sách có số hiệu trong S . Yêu cầu dùng các truy vấn để tìm ra thứ tự số hiệu của n quyển sách. Vì không thể phân biệt thứ tự trái-phải qua truy vấn, chỉ cần tìm được dãy số hiệu từ trái sang phải hoặc từ phải sang trái.

$$n \leq 1000, \quad \text{số lần hỏi giới hạn là } 20000.$$

Gọi $\text{ask}(S)$ là đáp án khi hỏi thư viện tương tác với tập S .

Ta có thể tìm thứ tự cuối cùng bằng cách hỏi số hiệu kề với từng quyển sách. Dễ thấy quyển sách có số hiệu x kề với

$$\text{ask}(S) - \text{ask}(S \cup \{x\}) + 1$$

quyển trong tập S ($x \notin S$).

Vì chỉ cần tìm $n - 1$ quan hệ kề nhau, ta có thể xét việc hỏi một quyển sách x kề với những quyển nào có số hiệu lớn hơn nó. Dùng chia đôi toàn cục để giải: khi chia đến khoảng $[l, r]$, duy trì số lượng quyển

¹Với gộp cây đoạn có lazy tag, chỉ cần cộng trực tiếp các tag và bảo đảm thông tin của mỗi nút trong cây đoạn là đúng nếu không xét lazy tag của tổ tiên.

trong tập $[l, r]$ kề với x ; bằng cách hỏi số quyền kề với x trong tập $[l, \text{mid}]$, với $\text{mid} = (l + r)/2$ (dưới đây cũng dùng ký hiệu này), ta đệ quy tính các khoảng $[l, \text{mid}]$ và $[\text{mid} + 1, r]$ có chứa quyền kề với x .

Có thể thấy mỗi ràng buộc kề nhau chỉ dùng $\lfloor \log n \rfloor$ lần hỏi, và nhiều nhất chỉ có $\lfloor n/2 \rfloor$ quyền làm lãng phí 2 lần hỏi. Vì vậy tối đa khoảng $2n \lfloor \log n \rfloor + n \approx 19000$ lần hỏi là đủ để nhận được đáp án.

4.2 Xây dựng bài toán mới

Xét việc xây dựng bài toán $S = \{a, b\}$ ($a < b$) mới trên cơ sở bài toán L_p : ngoài việc thỏa mọi ràng buộc của bài toán gốc, còn cần thỏa $a \leq f_i \leq b$, và phải cực tiểu hóa chi phí hồi quy.

4.3 Trường hợp $p = 1$

Bổ đề 4.1. Trong bài toán L_1 , nếu mọi y_i đều không nằm trong khoảng (a, b) , và tồn tại một dãy nghiệm tối ưu z sao cho mọi phần tử z_i cũng không nằm trong khoảng (a, b) , thì với một nghiệm tối ưu z_S của bài toán S , nhất định tồn tại một nghiệm tối ưu z của bài toán gốc sao cho làm tròn z về S sẽ được z_S .

Chứng minh. Dùng phản chứng. Giả sử nghiệm tối ưu của bài toán gốc là z^0 , trong đó phải có $z_i^0 \leq a, z_i^S = b$ hoặc $z_i^0 \geq b, z_i^S = a$. Vì hai loại tình huống này tương ứng với các đỉnh hiển nhiên không thể có quan hệ thứ tự bộ phận, nên không mất tính tổng quát ta chứng minh không tồn tại $z_i^0 \leq a, z_i^S = b$.

Đặt

$$y = \max\{z_i^0 \mid z_i^0 \leq a, z_i^S = b\},$$

$$U = \{v_i \mid z_i^0 = y, z_i^S = b\},$$

và giả sử khoảng các giá trị trung bình L_1 của U là $[l, r]^2$. Dễ thấy trên khoảng $(-\infty, l]$, chi phí hồi quy của tập U là hàm đơn điệu giảm; trên khoảng $[r, +\infty)$, nó là hàm đơn điệu tăng.

Nếu $l > a$, thì chỉnh các z_i^0 trong U thành $\min\{l, b\}$ sẽ cho bài toán gốc một nghiệm tốt hơn: vì y là giá trị lớn nhất trong số các vị trí có $z_i^S = b$, nên không tồn tại i, j ($v_i \in U, v_j \notin U, z_j^0 \in [y, a]$) sao cho $v_i \leq v_j$. Vì vậy nghiệm sau khi chỉnh vẫn hợp lệ. Điều này mâu thuẫn với giả thiết nghiệm tối ưu của bài toán gốc.

Nếu $l \leq a, r \geq b$, tương tự trường hợp $l > a$, chỉnh các z_i^0 trong U thành b nhất định sẽ nhận được một nghiệm khả thi của bài toán gốc; và vì $[a, b]$ nằm trong khoảng tối ưu, nghiệm khả thi này không kém hơn z^0 . Sau lần chỉnh này, số vị trí mà kết quả làm tròn của z^0 về S khác z_S giảm đi. Vì vậy sau hữu hạn thao tác, nhất định có thể chỉnh z^0 thành một dãy làm tròn về z_S . Điều này mâu thuẫn với giả thiết không tìm được nghiệm tối ưu z tương ứng.

Nếu $l \leq a, r < b$, đặt

$$U' = \{v_i \mid z_i^0 \leq a, z_i^S = b\}.$$

Vì $z_i^S = b$, nên không tồn tại i, j ($v_i \in U', v_j \notin U', z_j^0 \in [-\infty, a]$) sao cho $v_i \leq v_j$. Nếu $U = U'$, chỉnh các z_i^S trong U' thành a hiển nhiên sẽ cho một dãy tốt hơn z_S , mâu thuẫn với giả thiết z_S là nghiệm tối ưu của bài toán S . Tiếp theo giả sử khoảng các giá trị trung bình L_1 của $U' \setminus U$ là $[l', r']$. Nếu $r' \leq y$, thì sau khi chỉnh các phần tử trong $U' \setminus U$ theo cùng cách, cũng có thể nhận được một nghiệm tốt hơn cho bài toán S , mâu thuẫn với tính tối ưu của z_S . Nếu $r' \geq y$, có thể chỉnh các z_i^0 của $U' \setminus U$ thành y , nhận được một nghiệm khả thi của bài toán gốc không kém hơn z^0 . Nếu nghiệm này tốt hơn, ta mâu thuẫn với giả thiết z^0 là tối ưu; ngược lại, nghiệm tối ưu này nhất định thỏa $U' = U$, và lặp lại quá trình chứng minh trên sẽ tìm được mâu thuẫn. ■

²Giá trị trung bình L_1 có thể không duy nhất, nhưng mọi giá trị đó nhất định tạo thành một khoảng.

Theo Bổ đề 4.1, thông qua một nghiệm tối ưu của bài toán S , ta có thể biết quan hệ lớn nhỏ giữa z_i và a, b trong một nghiệm tối ưu nào đó của bài toán gốc. Với bài toán L_1 , vì z_i trong nghiệm tối ưu nhất định nằm trong tập $\{Y_1, Y_2, \dots, Y_m\}$ tạo bởi các y_i ($\forall i < m, Y_i < Y_{i+1}$), ta chỉ cần thực hiện chia đôi toàn cục trên tập này. Khi chia đến khoảng $Y_{[l,r]}$, chỉ cần tính nghiệm tối ưu của bài toán $S = \{Y_{\text{mid}}, Y_{\text{mid}+1}\}$ tạo bởi các v_i rơi vào khoảng này, rồi dựa vào z_i^S để chia các v_i đó sang các khoảng chọn giá trị mới $Y_{[l,\text{mid}]}$ và $Y_{[\text{mid}+1,r]}$. Như vậy với mỗi đỉnh v_i , chỉ cần qua $O(\log n)$ tầng tính toán là xác định được giá trị được chọn.

4.4 Trường hợp $1 < p < \infty$

Bổ đề 4.2. Khi $1 < p < \infty$, giá trị trung bình L_p của một tập U bất kỳ là duy nhất.

Chứng minh. Đặt

$$g(x) = \sum_{v_i \in U} |x - y_i|^p.$$

Khi đó

$$g(x) = \sum_{v_i \in U, y_i \leq x} (x - y_i)^p + \sum_{v_i \in U, y_i > x} (y_i - x)^p,$$

$$g'(x) = p \left(\sum_{v_i \in U, y_i \leq x} (x - y_i)^{p-1} - \sum_{v_i \in U, y_i > x} (y_i - x)^{p-1} \right).$$

Dễ thấy khi x tăng, tổng

$$\sum_{v_i \in U, y_i \leq x} (x - y_i)^{p-1}$$

đơn điệu tăng, còn tổng

$$\sum_{v_i \in U, y_i > x} (y_i - x)^{p-1}$$

đơn điệu giảm, nên phương trình $g'(x) = 0$ có nhiều nhất một nghiệm. Lại vì khi $x > \max\{y_i\}$ thì $g'(x) > 0$, còn khi $x < \min\{y_i\}$ thì $g'(x) < 0$, nên $g'(x)$ có ít nhất một điểm không. ■

Bổ đề 4.3. Nếu giá trị trung bình L_p của mọi tập con không rỗng của V đều không nằm trong khoảng (a, b) ($a < b$), và tồn tại một nghiệm tối ưu z sao cho mọi phần tử z_i cũng không nằm trong (a, b) . Gọi $\tilde{z}$ là một nghiệm tối ưu của bài toán L_1 trên cùng tập có thứ tự bộ phận, với hàm chi phí (y', w') :

$$(y'_i, w'_i) = \begin{cases} (0, w_i((b - y_i)^p - (a - y_i)^p)), & y_i \leq a, \\ (1, w_i((y_i - a)^p - (y_i - b)^p)), & y_i > a. \end{cases}$$

Khi đó tồn tại một nghiệm tối ưu z của bài toán gốc thỏa $\tilde{z}_i = 0$ khi và chỉ khi $z_i \leq a$.

Chứng minh. Vì đã bảo đảm giá trị trung bình L_p của mọi tập con sẽ không rơi vào (a, b) , nên nghiệm tối ưu của bài toán S ứng với bài toán L_p sẽ không có phần tử nào trong khoảng (a, b) . Do đó bài toán S tương đương với bài toán L_1 ở trên. Phần chứng minh còn lại giống Bổ đề 4.1, xin lược bỏ. ■

Theo Bổ đề 4.2, hợp của các phần tử trong nghiệm tối ưu và mọi giá trị trung bình L_p là một tập hữu hạn. Vì vậy với a bất kỳ, nhất định tồn tại $\varepsilon > 0$ sao cho khoảng $(a, a + \varepsilon)$ không chứa các phần tử trên; đồng thời có thể đổi w'_i trong bài toán L_1 ở Bổ đề 4.3 thành đạo hàm tại vị trí a . Vì trong thi tin học chỉ cần tìm nghiệm tối ưu đến một độ chính xác nhất định, ta vẫn có thể áp dụng cách chia đôi toàn cục ở mục 4.3, chỉ thay việc chia đôi trên tập bằng chia đôi trên số thực.

4.5 Xây dựng mô hình luồng mạng

Với bài toán L_1 trên tập có thứ tự bộ phận tổng quát, bài toán S tương ứng có thể xem là một bài toán quyết định 0/1 đơn giản. Ràng buộc $f_i \leq f_j$ có thể xem là: nếu f_i chọn 1, thì f_j không được chọn 0. Như vậy bài toán trở thành bài toán đồ thị con đóng có trọng số nhỏ nhất kinh điển, có thể giải bằng luồng mạng.

4.6 Bài tập ví dụ

Ví dụ 4. *ExtremeSpanningTrees* Nguồn đề: *Single Round Match 720 Round 1 - Division I, Level Three*.

Cho một đồ thị vô hướng liên thông $G = (V, E)$ có n đỉnh, m cạnh, và hai tập cạnh E_1, E_2 ($E_1, E_2 \subseteq E$). Mỗi cạnh có trọng số ban đầu d_i . Mỗi thao tác có thể tăng hoặc giảm trọng số của một cạnh đi 1. Hỏi cần ít nhất bao nhiêu thao tác để thỏa:

- đồ thị con tạo bởi tập cạnh E_1 là cây khung nhỏ nhất của toàn đồ thị;
- đồ thị con tạo bởi tập cạnh E_2 là cây khung lớn nhất của toàn đồ thị.

$$n \leq 50, \quad m \leq 1000.$$

Ta có thể xây dựng quan hệ thứ tự bộ phận trên m cạnh: tính các quan hệ thứ tự bộ phận giữa cạnh không thuộc cây i và cạnh thuộc đường đi tương ứng trên cây j . Sau đó bài toán trở thành một bài toán L_1 trên tập có thứ tự bộ phận tổng quát.

5 Tối ưu trên cấu trúc thứ tự bộ phận đặc biệt

5.1 Bài toán trên cây

Với bài toán hồi quy bảo toàn thứ tự trên cây ở mục 3.2, ta cũng có thể dựa trên chia đôi toàn cục rồi lồng cùng kiểu quy hoạch động trên cây để giải bài toán S .

Đặc biệt, thuật toán này có thể mở rộng sang xương rồng. Khi quy hoạch động trên mỗi chu trình, chỉ cần liệt kê giá trị 0/1 được chọn của một đỉnh để phá chu trình thành đường đi.

Dù là phiên bản trên cây hay trên xương rồng, độ phức tạp thời gian đều là $O(n \log n)$. Ở đây bỏ qua độ phức tạp tính lũy thừa $p - 1$ trong chi phí; dưới đây cũng vậy.

5.2 Thứ tự bộ phận nhiều chiều

Định nghĩa 5.1. Thứ tự bộ phận d chiều: với hai điểm d chiều $(a_1, a_2, \dots, a_d)$ và $(b_1, b_2, \dots, b_d)$, nếu $\forall i (1 \leq i \leq d), a_i \leq b_i$, thì có

$$(a_1, a_2, \dots, a_d) \leq (b_1, b_2, \dots, b_d).$$

Ví dụ 5. Cho tờ giấy ô vuông kích thước $r \times c$. Với điểm có tọa độ (i, j) , chi phí sửa đổi để tăng hoặc giảm trọng số đi 1 là $w_{i,j}$, trọng số ban đầu là $d_{i,j}$. Hãy tìm tổng chi phí sửa đổi nhỏ nhất sao cho với mọi $(i_1, j_1) \leq (i_2, j_2)$, đều có $d_{i_1, j_1} \leq d_{i_2, j_2}$.

Có thể tiếp tục dùng tư tưởng chia đôi toàn cục. Xét một số tính chất của bài toán S ở mỗi tầng chia đôi:

1. Tọa độ y ứng với mỗi cột i (tọa độ x) là một đoạn liên tiếp $[l_i, r_i]$.
2. l_i, r_i đều là các dãy đơn điệu không tăng.

3. Nếu cột thứ i không có phần tử (tức $l_i > r_i$), thì $\forall j, k (j < i < k)$, có $l_j > r_k$.

Theo các tính chất trên, có thể thấy khi chia đôi ở mỗi tầng, ta chỉ cần xét ảnh hưởng giữa hai cột kề nhau. Có thể thiết kế trực tiếp trạng thái quy hoạch động: $g_{i,j}$ biểu thị tổng chi phí nhỏ nhất của i cột đầu trong trường hợp tọa độ y nhỏ nhất được chọn b ở cột thứ i là j .

Chuyển trạng thái là

$$g_{i,j} = \left(\min_{k=j}^{r_{i-1}} g_{i-1,k} \right) + \text{cost}_{i,j},$$

trong đó $\text{cost}_{i,j}$ là chi phí của quyết định này trên cột thứ i . Dễ thấy có thể dùng hậu tố nhỏ nhất để tối ưu, độ phức tạp thời gian là $O(rc \log r)$.

Ví dụ 6. Cho n điểm trên mặt phẳng, trong đó điểm thứ i , tức v_i , có tọa độ (x_i, y_i) . Chi phí sửa đổi để tăng hoặc giảm trọng số đi 1 là w_i , trọng số ban đầu là d_i . Hãy tìm tổng chi phí sửa đổi nhỏ nhất sao cho với mọi $v_i \leq v_j$, đều có $d_i \leq d_j$.

Khác với bài trước, trong bài toán S ở mỗi tầng không thể bỏ qua ảnh hưởng chi phí giữa các cột không kề nhau. Trạng thái cần duy trì chi phí nhỏ nhất $g_{i,j}$ khi trong i cột đầu, tọa độ y nhỏ nhất được chọn b là j .

Có thể thấy $g_{i,j}$ có hai loại chuyển trạng thái sau:

1. Ở cột thứ i , chọn vị trí đầu tiên $\geq j$, ký hiệu next_j , làm vị trí đầu tiên được chọn b . Khi đó

$$g_{i,j} = g_{i-1,j} + \text{cost}_{i,\text{next}_j}.$$

2. Ở cột thứ i , chọn vị trí j . Khi đó

$$g_{i,j} = \left(\min_{k \geq j} g_{i-1,k} \right) + \text{cost}_{i,j}.$$

Dễ thấy loại thứ nhất tương đương cộng đoạn, loại thứ hai tương đương truy vấn nhỏ nhất trên đoạn và sửa điểm; cả hai đều có thể duy trì bằng cây đoạn. Vì quá trình chia chọn giá trị của chia đôi toàn cục cần dựa vào phương án nghiệm tối ưu của bài toán S , nên cần dùng cây đoạn bền vững để hiện thực. Độ phức tạp thời gian là $O(n \log^2 n)$.

6 Thuật toán và mở rộng của bài toán L_∞

6.1 Chặt nhị phân đơn giản

Vì bài toán L_∞ yêu cầu cực tiểu hóa giá trị lớn nhất, ta có thể nghĩ đến phương pháp chặt nhị phân. Sau mỗi lần chặt, có thể xác định khoảng giá trị được chọn $[a_i, b_i]$ cho mỗi đỉnh v_i . Sau đó làm quy hoạch động trên DAG, tính giá trị nhỏ nhất của f_i tại mỗi đỉnh v_i nếu chỉ thỏa mọi điều kiện $f_j \leq f_i$ ($v_j \leq v_i$), rồi kiểm tra tính khả thi. Như vậy ta nhận được một cách làm tốt cho bài toán L_∞ tổng quát với độ phức tạp $O(m \log \max\{y_i\})$.

6.2 Mở rộng bài toán

Ví dụ 7. Cho tập đỉnh $V = \{v_1, v_2, \dots, v_n\}$ kích thước n và hàm chi phí (y, w) . Với mọi $i, j (1 \leq i < j \leq n)$, đều có $v_i \leq v_j$.

Yêu cầu quyết định một dãy giá trị đỉnh f , thỏa với mọi $v_i \leq v_j$ ($i, j \in [1, n]$), đều có $f_i \leq f_j$.

Yêu cầu với mỗi $k (1 \leq k \leq n)$, tính giá trị nhỏ nhất của

$$\max_{i=1}^k w_i |f_i - y_i|.$$

Nếu áp dụng cách chặt nhị phân trong mục 6.1, ta cần xử lý nhiều truy vấn; vì không thể tiền xử lý nhanh, cũng không thể dùng chia đôi toàn cục để tối ưu. Do đó ta cần tìm một số tính chất.

Định nghĩa 6.1. $\text{err}(v_i, r)$ biểu thị chi phí hồi quy khi dùng giá trị đỉnh r tại v_i :

$$\text{err}(v_i, r) = w_i |r - y_i|.$$

$\text{mean}(v_i, v_j)$ biểu thị trung bình có trọng số của v_i, v_j :

$$\text{mean}(v_i, v_j) = \frac{w_i y_i + w_j y_j}{w_i + w_j}.$$

Khi $v_i \leq v_j$ và $y_i > y_j$, $\text{mean_err}(v_i, v_j)$ là giá trị lớn hơn trong hai giá trị

$$\text{err}(v_i, \text{mean}(v_i, v_j)), \quad \text{err}(v_j, \text{mean}(v_i, v_j));$$

ngược lại, $\text{mean_err}(v_i, v_j) = 0$.

Bổ đề 6.1. $\text{mean_err}(v_i, v_j)$ là chi phí hồi quy nhỏ nhất khi chỉ xét v_i và v_j .

Chứng minh. Khi $v_i \leq v_j$ hoặc $y_i \leq y_j$, hiển nhiên chọn $f_i = y_i, f_j = y_j$ cho chi phí hồi quy bằng 0.

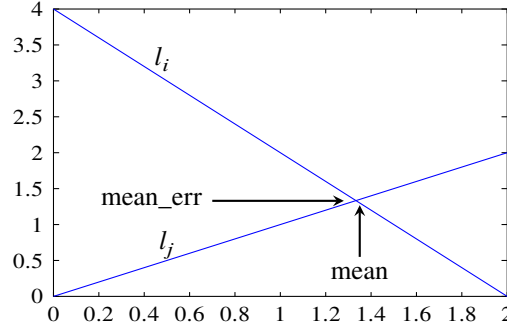


Figure 1: $y_i = 2, w_i = 2, y_j = 0, w_j = 1$.

Khi $v_i \leq v_j$ và $y_i > y_j$, theo Hình 1 (trong đó l_i là nhánh trái của hàm $y = \text{err}(v_i, x)$, còn l_j là nhánh phải của hàm $y = \text{err}(v_j, x)$), có thể thấy $\text{mean}(v_i, v_j)$ là giá trị trung bình L_∞ của tập $\{v_i, v_j\}$, và $\text{mean_err}(v_i, v_j)$ chính là chi phí hồi quy nhỏ nhất. ■

Bổ đề 6.2. Với bài toán L_∞ bất kỳ, chi phí hồi quy nhỏ nhất của toàn đồ thị là

$$\max\{\text{mean_err}(v_i, v_j)\}.$$

Chứng minh. Theo Bổ đề 6.1, dễ thấy chi phí hồi quy nhỏ nhất không nhỏ hơn $\max\{\text{mean_err}(v_i, v_j)\}$.

Ta chứng minh chi phí hồi quy nhỏ nhất không vượt quá $\max\{\text{mean_err}(v_i, v_j)\}$.

Với đỉnh v_k , có thể chọn giá trị đỉnh f_k là $\text{mean}(v_i, v_j)$ của một cặp v_i, v_j làm $\text{mean_err}(v_i, v_j)$ lớn nhất trong số các cặp thỏa $v_i \leq v_k \leq v_j$. Dưới đây chứng minh tính đúng đắn của cách chọn này theo ba bước.

Trước hết, nếu còn có một cặp v'_i, v'_j ($v'_i \leq v_k \leq v'_j$) thỏa $\text{mean_err}(v'_i, v'_j) = \text{mean_err}(v_i, v_j)$, thì nhất định

$$\text{mean}(v'_i, v'_j) = \text{mean}(v_i, v_j).$$

Nếu không, giả sử $\text{mean}(v'_i, v'_j)$ lớn hơn. Theo tính chất của đồ thị hàm, nếu $\text{mean_err}(v'_i, v'_j)$ không lớn hơn $\text{mean_err}(v_i, v_j)$, thì hệ số góc của l'_i phải không âm, mâu thuẫn với việc l'_i là nhánh trái của hàm err .

Tiếp theo, ta chứng minh $\text{err}(v_k, \text{mean}(v_i, v_j))$ không vượt quá $\text{mean_err}(v_i, v_j)$: nếu đồ thị $y = \text{err}(v_k, x)$ đi qua phần phía trên giao điểm của l_i, l_j , thì mean_err của v_k với v_i hoặc v_j sẽ lớn hơn $\text{mean_err}(v_i, v_j)$, mâu thuẫn với việc trước đó ta chọn một cặp lớn nhất.

Cuối cùng, ta chứng minh cách chọn giá trị này thỏa ràng buộc thứ tự bộ phận. Với quan hệ $v_a \leq v_b$, gọi C_a, D_a lần lượt là tập các tia tạo bởi nhánh trái của hàm err của mọi v_e thỏa $v_e \leq v_a$, và tập các tia tạo bởi nhánh phải của hàm err của mọi v_e thỏa $v_e \geq v_a$. Định nghĩa C_b, D_b tương tự. Để thấy C_b nhận được bằng cách thêm một số tia vào C_a , còn D_b nhận được bằng cách xóa một số tia khỏi D_a . Dùng quy nạp, có thể chuyển bài toán thành chứng minh rằng sau khi thêm một tia vào C_a hoặc xóa một tia khỏi D_a , trong mọi giao điểm, tọa độ x của điểm có tọa độ y lớn nhất sẽ không nhỏ đi. Xét bao lồi dưới tạo bởi C_a , giao điểm có tọa độ y lớn nhất nhất định nằm trên bao lồi; xóa một tia khỏi D_a chỉ làm tọa độ y lớn nhất giảm đi, và tọa độ x tương ứng dịch sang phải. Tương tự có thể chứng minh việc thêm một tia vào C_a cũng có tính chất như vậy. ■

Trong Ví dụ 7, ta có thể đặt

$$\text{pre}(v_i) = \max\{\text{mean_err}(v_j, v_i) \mid v_j \leq v_i\}.$$

Để thấy chỉ cần tính được mọi $\text{pre}(v_i)$ là có thể nhận được đáp án cho mọi k . Như vậy ta có một thuật toán $O(n^2)$.

Tiếp tục xét tối ưu quá trình tính $\text{pre}(v_i)$. Có thể theo tư duy tương tự phần chứng minh: duy trì bao lồi dưới tạo bởi nhánh trái của mọi hàm err ứng với $v_1 \sim v_i$, rồi truy vấn giao điểm giữa nhánh phải của hàm err của v_i và bao lồi.

Một cách duy trì khá đơn giản là xem đường thẳng dạng $y = kx + b$ như điểm (k, b) trên mặt phẳng, và duy trì bao lồi trên tạo bởi các điểm này. Khi đó mỗi lần chèn trở thành chèn nhị phân vị trí rồi xóa bỏ các điểm không còn nằm trên bao lồi; truy vấn trở thành chèn nhị phân để so sánh giao điểm giữa đường thẳng truy vấn với đường thẳng trong bao lồi và đoạn gấp khúc tương ứng trên bao lồi. Tất cả đều có thể duy trì đơn giản bằng set trong C++. Vì vậy ta nhận được cách làm tốt với độ phức tạp $O(n \log n)$.

Bài toán L_∞ còn có nhiều mở rộng phức tạp hơn; ở đây không trình bày từng cái. Nếu quan tâm, có thể xem tài liệu tham khảo [5].

7 Tổng kết

Bắt đầu từ mô tả bài toán hồi quy bảo toàn thứ tự L_p , bài viết lần lượt giới thiệu các cách làm cho $1 \leq p < \infty$ và $p = \infty$.

Trong phần $1 \leq p < \infty$, trước hết bài viết giới thiệu lời giải cho các tình huống đặc biệt từ góc nhìn tham lam và bảo trì đường gấp khúc; sau đó dùng tư tưởng chia đôi toàn cục hoàn toàn mới để giới thiệu cách làm cho bài toán tổng quát. Ba phương pháp đều có ưu và nhược điểm. Cách chia đôi toàn cục có thể giải bài toán tổng quát, đồng thời có thể tối ưu trên các cấu trúc thứ tự bộ phận đặc biệt, nhưng rất khó trả lời nhiều nhóm truy vấn khác nhau; còn cách tham lam có thể đạt độ phức tạp tuyến tính trên bài toán L_2 đặc biệt, và bảo trì đường gấp khúc có hằng số tốt hơn chia đôi toàn cục. Hơn nữa, cả hai cách này đều có thể đồng thời tính được đáp án của một số bài toán con, nhưng chúng chỉ giải được các trường hợp đặc biệt tương ứng.

Trong phần $p = \infty$, trước hết bài viết giới thiệu cách chặt nhị phân đơn giản; sau đó thông qua suy luận kết hợp hình học và đại số, rút ra kết luận tổng quát cho bài toán L_∞ , rồi vận dụng nó vào bài toán trên cấu trúc thứ tự bộ phận tuyến tính. Hai phương pháp cũng có ưu và nhược điểm riêng. Chặt nhị phân có thể giải bài toán trên cấu trúc thứ tự bộ phận bất kỳ, nhưng không thể đồng thời hỗ trợ giải nhiều bài

toán; còn vận dụng kết luận tổng quát tuy có thể đồng thời giải nhiều bài toán, nhưng rất khó tính nhanh trên thứ tự bộ phận tổng quát. Điều này nhắc ta phải chọn phương pháp phù hợp với đề bài.

Bài toán hồi quy bảo toàn thứ tự được giới thiệu trong bài này vẫn đáng được nghiên cứu sâu hơn. Hy vọng bài viết có thể đóng vai trò gợi mở, giúp độc giả tiếp tục nghiên cứu và tìm ra nhiều cách làm cũng như ứng dụng thú vị hơn.

Lời cảm ơn

- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
- Cảm ơn thầy Li Shu của Trường Ngoại ngữ Nam Kinh đã quan tâm và hướng dẫn.
- Cảm ơn các huấn luyện viên đội tuyển quốc gia Zhang Ruizhe và Yu Linyun đã hướng dẫn.
- Cảm ơn Yang Jinzhao, Yang Qianlan và Xu Haike đã thảo luận các thuật toán này với tôi.
- Cảm ơn Wang Xiuhua và Xu Haike đã đọc duyệt bài viết này.
- Cảm ơn cha mẹ đã quan tâm và chăm sóc tôi.

Tài liệu tham khảo

1. Xu Haoran, *Bàn sơ lược về một số lời giải phi kinh điển cho bài toán cấu trúc dữ liệu*, tập luận văn đội tuyển quốc gia năm 2013.
2. Zhang Hengjie, *Một số kỹ thuật tối ưu DP*, tập luận văn đội tuyển quốc gia năm 2015.
3. Wikipedia, https://en.wikipedia.org/wiki/Isotonic_regression.
4. Quentin F. Stout, “Isotonic Regression via Partitioning”, *Algorithmica* 66 (2013), pp. 93–112.
5. Stout, Q. F., “Algorithms for L_∞ isotonic regression”, submitted (2011).

6 Bài 4: Báo cáo ra đề *Fim 4*

Wu Jinzhao, Trường Trung học số 2 Hàng Châu, Chiết Giang

Tóm tắt

Bài viết này giới thiệu đề *Fim 4*, một bài toán lấy bối cảnh khớp xương do tác giả ra trong đợt thi đối kháng của đội tuyển tập huấn. Đề khai thác sâu các tính chất liên quan tới chu kỳ của xương, đồng thời kết hợp tính chất đó với thuật toán KMP quen thuộc. Bài này đòi hỏi thí sinh hiểu khá sâu về xương, đồng thời cần năng lực cài đặt nhất định; đây là một đề tốt để kiểm tra chiều sâu tư duy của thí sinh.

Ngoài ra, cách chia điểm của đề hợp lý, dữ liệu mạnh, độ phân hóa cao, và có thể dẫn dắt thí sinh suy nghĩ tới lời giải chuẩn. Dù là thí sinh chỉ viết vét cạn, thí sinh đã suy nghĩ thêm một chút, hay thí sinh nghiên cứu sâu và tìm ra mọi tính chất, đều có thể nhận được phần điểm phù hợp.

Bài viết cũng cung cấp một hướng ra đề: với một thuật toán đã được biết rộng rãi, ta suy nghĩ đầy đủ về bản chất của nó, tìm những tính chất ít được dùng trong thuật toán gốc, rồi mở rộng thuật toán đó sang một bài toán tổng quát hơn.

1 Đại ý đề bài và phạm vi dữ liệu

1.1 Đề gốc

1.1.1 Đại ý đề gốc. Cho n xâu s_i và m số nguyên a_i nằm trong khoảng $1 \sim n$. Gọi xâu mẹ là

$$s_{a_1} + s_{a_2} + \dots + s_{a_m}.$$

Cần trả lời q truy vấn; mỗi truy vấn cho một xâu t_i , hỏi số lần xuất hiện của xâu này trong xâu mẹ.

Bảo đảm s_i và t_i đều chỉ gồm hai chữ cái a, b.

1.1.2 Phạm vi dữ liệu của đề gốc. Với mọi dữ liệu:

$$1 \leq n, m, q \leq 5 \times 10^4, \quad \sum_{1 \leq i \leq n} |s_i|, \quad \sum_{1 \leq i \leq q} |t_i| \leq 10^5.$$

- Subtask 1 (20 điểm):

$$\sum_{1 \leq i \leq m} |s_{a_i}| \leq 10^6.$$

- Subtask 2 (20 điểm):

$$\max |t_i| \leq \min |s_i|.$$

- Subtask 3 (30 điểm):

$$\max |t_i| \leq 200.$$

- Subtask 4 (30 điểm): không có tính chất đặc biệt.

Trên thực tế, lời giải tác giả đưa ra có thể giải một phiên bản mạnh hơn đề gốc. Đại ý của đề tăng cường được nêu dưới đây. Không khó thấy các truy vấn của đề gốc là một tập con thật sự của đề tăng cường, vì vậy chỉ cần giải đề tăng cường là đồng thời giải được đề gốc. Bài viết này cũng chủ yếu giới thiệu cách làm cho đề tăng cường.

1.2 Đề tăng cường

1.2.1 Đại ý đề tăng cường. Cho một cây Trie có n nút. Gọi xâu tạo bởi đường đi từ gốc đến nút thứ i là s_i .

Lại cho m số nguyên a_i nằm trong khoảng $1 \sim n$.

Cần trả lời q truy vấn; mỗi truy vấn cho một xâu t_i và hai số nguyên dương L_i, R_i . Đặt

$$S_i = s_{a_{L_i}} + s_{a_{L_i+1}} + \dots + s_{a_{R_i}},$$

hỏi số lần xuất hiện của t_i trong S_i .

Kích thước bảng chữ cái không vượt quá 10.

1.2.2 Phạm vi dữ liệu của đề tăng cường. Với mọi dữ liệu:

$$n \leq 10^6, \quad 1 \leq m, q \leq 5 \times 10^4, \quad \sum_{1 \leq i \leq q} |t_i| \leq 10^5.$$

- Subtask 1 (20 điểm):

$$\sum_{1 \leq i \leq m} |s_{a_i}| \leq 10^5.$$

- Subtask 2 (30 điểm):

$$\max |t_i| \leq 200.$$

- Subtask 3 (20 điểm):

$$\max |t_i| \leq \min |s_{a_i}|.$$

- Subtask 4 (30 điểm): không có tính chất đặc biệt.

2 Kiến thức chuẩn bị

Phần này giới thiệu sơ lược một số định nghĩa và thuật toán có thể cần dùng để giải bài này.

2.1 Khái niệm cơ bản về xâu

Xét một xâu $s_1 s_2 \dots s_n$ có độ dài n . Với $1 \leq i \leq j \leq n$, gọi $s_i s_{i+1} \dots s_j$ là một xâu con của s , ký hiệu $s[i : j]$. Ký hiệu $|s|$ là độ dài của s , tức $|s| = n$.

Với $1 \leq i \leq n$, gọi $s[i : n]$ là một hậu tố của s , ký hiệu $s[i :]$. Tương tự, với $1 \leq i \leq n$, gọi $s[1 : i]$ là một tiền tố của s , ký hiệu $s[: i]$.

Với $1 \leq i < n$, nếu $s[: i] = s[n - i + 1 :]$, thì gọi $s[: i]$ là một *border* của s . Đặc biệt, xâu rỗng cũng là một border của s . Border dài nhất của s được gọi là $\text{Lborder}(s)$.

Nếu $2|\text{Lborder}(s)| \geq n$, thì gọi s là xâu tuần hoàn; độ dài chu kỳ là $|s| - |\text{Lborder}(s)|$.

2.2 Một số ký hiệu

Quy ước xâu mẹ $S = s_{a_1} + s_{a_2} + \dots + s_{a_m}$. Ký hiệu lp_i là vị trí đầu trái của s_{a_i} trong S , và rp_i là vị trí đầu phải của s_{a_i} trong S . Gọi L là độ dài lớn nhất của các xâu truy vấn, và Σ là bảng chữ cái.

Định nghĩa $[cond]$ bằng 1 khi $cond$ đúng, và bằng 0 khi $cond$ sai.

Định nghĩa $c(T, S)$ là số lần xâu T xuất hiện trong xâu S .

Định nghĩa $\text{suf}(S)$ là tập mọi hậu tố của S , và $\text{pre}(S)$ là tập mọi tiền tố của S .

2.3 Automaton hậu tố

Automaton hậu tố của một xâu S là một automaton hữu hạn tất định (DFA) có thể và chỉ có thể chấp nhận mọi hậu tố của S , đồng thời có số trạng thái và số chuyển ít nhất. Dưới đây gọi tắt là SAM.

Với một xâu S , ta xây dựng SAM bằng phương pháp tăng dần trong thời gian $O(|S|)$. Cách xây dựng cụ thể và chứng minh độ phức tạp không liên quan đến chủ đề chính của bài viết nên không trình bày ở đây; có thể xem tài liệu tham khảo [5].

2.4 SAM on Trie

Với một cây Trie, định nghĩa tập xâu con của nó là tập các hậu tố của những xâu tạo bởi đường đi từ gốc đến một nút bất kỳ.

Ta xây dựng cho cây Trie một automaton hữu hạn có thể nhận biết mọi xâu con trên cây Trie, đồng thời cố gắng giảm số trạng thái và số chuyển của automaton; automaton đó được gọi là SAM on Trie.

Tính chất của SAM on Trie gần như hoàn toàn giống tính chất của SAM trên một dãy. Ví dụ: SAM on Trie có thể nhận biết mọi xâu con trong một Trie; cây parent của SAM on Trie là một cây hậu tố đảo

chiều, trong đó cha của mỗi nút là nút biểu diễn hậu tố dài nhất khác nhau về bản chất của xâu mà nút đó biểu diễn. “Khác nhau về bản chất” ở đây nghĩa là có số lần xuất hiện khác nhau trong Trie này.

Cách xây dựng SAM on Trie tương tự như trên đây. Mỗi lần mở rộng một nút, ta dùng nút tương ứng với cha của nó trên SAM làm last để tiếp tục mở rộng. Nếu xây dựng theo thứ tự FIFO, độ phức tạp thời gian là $O(n|\Sigma|)$, trong đó n là số nút của Trie và Σ là bảng chữ cái. Chứng minh độ phức tạp không liên quan đến chủ đề chính của bài viết nên không trình bày ở đây; có thể xem tài liệu tham khảo [1].

2.5 KMP

Dùng thuật toán KMP có thể tính số lần xâu t xuất hiện trong xâu s trong thời gian $O(|s| + |t|)$. Ngoài ra, nó còn có thể tính mọi $Lborder(t[: i])$ với $1 \leq i \leq n$. Quy trình cụ thể như sau:

Thuật toán 1. Thuật toán KMP.

```
j <- 0
np <- 0
for i <- 2 to |t| do
  while t[i] != t[j+1] and j != 0 do
    j <- p[j]
  end while
  if t[i] = t[j+1] then
    j <- j + 1
  end if
  p[i] <- j
end for
j <- 0
for i <- 1 to |s| do
  while s[i] != t[j+1] and j != 0 do
    j <- p[j]
  end while
  if s[i] = t[j+1] then
    j <- j + 1
  end if
  if j = |t| then
    pos[np] <- i
    np <- np + 1
    j <- p[j]
  end if
end for
```

2.6 Tìm nhanh tiền-hậu tố chung dài nhất của hai xâu

Ta cần giải bài toán sau: cho một Trie và một xâu T , cần trả lời nhanh truy vấn: với một nút nào đó trên Trie, tìm xâu dài nhất T' sao cho T' là tiền tố của T , đồng thời T' là hậu tố của xâu tạo bởi đường đi từ gốc của Trie đến nút này.

Trước hết xây dựng SAM của Trie đó. Xét từng tiền tố của T ; tại nút tương ứng với tiền tố này trên Trie, ta gán một thẻ có trọng số bằng độ dài tiền tố hiện tại.

Khi truy vấn, ta chỉ cần tìm tổ tiên trên cây parent có thể trọng số lớn nhất trong số các tổ tiên của nút SAM ứng với xâu tạo bởi đường đi từ gốc Trie đến nút đang xét. Chỉ cần dùng một cây LCT để duy trì cây parent là có thể hỗ trợ thao tác này trong thời gian $O(\log n)$.

3 Giới thiệu thuật toán

3.1 Thuật toán một

Với Subtask 1, chú ý rằng tổng độ dài xâu mẹ nằm trong phạm vi 10^5 , nên có thể trực tiếp dựng xâu mẹ.

Trước hết dựng SAM của xâu mẹ.

Sau đó dùng gộp cây đoạn bền vững để tiền xử lý tập Right của mỗi nút trên cây parent.

Với mỗi truy vấn, ta chỉ cần tìm nút tương ứng với xâu truy vấn t trên cây parent, rồi hỏi trong tập Right của nó có bao nhiêu phần tử nằm trong khoảng $[L + |t| - 1, R]$. Vì trước đó ta đã dùng cây đoạn bền vững để tiền xử lý tập Right của mỗi nút, chỉ cần truy vấn tổng trên khoảng này trong cây đoạn tương ứng với nút đó.

Độ phức tạp thời gian là

$$O\left(\sum |s_{a_i}| \log n + \sum |t_i| + q \log n\right).$$

Điểm kỳ vọng là 20 điểm.

3.2 Thuật toán hai

Xét Subtask 2; chú ý độ dài xâu truy vấn dài nhất chỉ không quá 200.

Khi một xâu truy vấn khớp trong xâu mẹ, nó hoặc xuất hiện bên trong một xâu từ điển đơn lẻ, hoặc bắc qua một số xâu từ điển. Ta xử lý riêng hai phần này.

3.2.1 Phần 1. Với mỗi xâu truy vấn t , cần tính

$$\sum_{i=L}^R c(t, S_i).$$

Đây là một bài toán khá thường gặp. Ta xây SAM on Trie cho Trie đã cho.

Tương tự thuật toán một, với mỗi nút trên SAM, ta dùng gộp cây đoạn bền vững để tiền xử lý số lần xuất hiện của xâu mà mỗi nút trên cây parent biểu diễn trong s_{a_i} .

Khi truy vấn, tìm nút tương ứng với xâu truy vấn t trên cây parent, rồi hỏi tổng trên khoảng $[L, R]$ trong cây đoạn tương ứng với nút này.

Độ phức tạp của phần này là

$$O\left(n|\Sigma| + n \log n + \sum |t_i|\right).$$

3.2.2 Phần 2. Nói một cách hình thức, tính số lần xuất hiện bắc qua nhiều xâu từ điển tức là cần tính

$$\sum_{i=L}^{R-1} c\left(t, S[\max(lp_i, rp_i - |t| + 2) : \min(rp_R, rp_i + |t| - 1)]\right).$$

Ta lấy mọi xâu

$$S[\max(lp_i, rp_i - L + 2) : rp_i + L - 1]$$

ra để xây Trie, rồi xây SAM cho Trie này. Với tiền tố

$$S[\max(lp_i, rp_i - L + 2) : rp_i + k - 1],$$

ta gán nhãn cho nó là k , và đặt chỉ số của nó là $rp_i + k - 1$.

Khi đó, việc thống kê

$$\sum_{i=L}^{R-1} c(t, S[\max(lp_i, rp_i - |t| + 2) : \min(rp_R, rp_i + |t| - 1)])$$

tương đương với thống kê trong cây con của nút biểu diễn t trên cây parent có bao nhiêu nút có nhãn nằm trong $[2, |t| - 1]$ và chỉ số nằm trong $[lp_L + |t| - 1, rp_R]$. Sau khi lấy thứ tự DFS trên cây parent, bài toán chuyển thành đếm điểm ba chiều.

Ta chú ý rằng chiều “nhân” của bài toán đếm điểm ba chiều này chỉ cỡ $O(L)$. Tổng số điểm cỡ $O(mL)$, số truy vấn cỡ $O(q)$. Ta có thể dễ dàng xử lý offline với chi phí thêm điểm $O(\log n)$ và truy vấn $O(L \log n)$. Vì vậy, tổng độ phức tạp của phần này là $O(mL \log n)$.

Tóm lại, ta có thể giải Subtask này trong độ phức tạp

$$O(n \log n + \sum |t_i| + mL \log n),$$

điểm kỳ vọng là 30 điểm.

3.3 Thuật toán ba

Với Subtask 3, chú ý rằng độ dài xâu truy vấn dài nhất cũng không vượt quá độ dài xâu từ điển ngắn nhất.

Điều này đem lại tính chất: một xâu truy vấn nhiều nhất chỉ bắc qua hai xâu từ điển.

Ta tiếp tục dùng ý tưởng của thuật toán hai: vẫn thống kê riêng phần nằm trong xâu từ điển và phần bắc qua xâu từ điển.

Đặt một tham số B . Với các truy vấn có độ dài không quá B , ta dùng thuật toán đã giới thiệu trong thuật toán hai; với truy vấn có độ dài lớn hơn B , ta dùng một thuật toán khác.

Vẫn theo thuật toán hai, xét các truy vấn có độ dài lớn hơn B . Định nghĩa

$$\tau(s, t) = \text{suf}(s) \cap \text{pre}(t).$$

Định lý 3.1. Gọi $\tau(s, t) = \{s[p_i :]\}$, trong đó p là một dãy tăng đơn điệu. Khi đó p có thể được biểu diễn thành $O(\log n)$ cấp số cộng nối tiếp nhau.

Chứng minh. $s[p_{i+1} :]$ là tiền tố của $s[p_i :]$, nên chỉ có hai trường hợp:

1. $|s[p_{i+1} :]| \leq |s[p_i :]|/2$: độ dài giảm một nửa. Tổng cộng chỉ có $O(\log n)$ lần giảm một nửa như vậy, không cần xử lý thêm.
2. $|s[p_{i+1} :]| > |s[p_i :]|/2$: khi đó $s[p_i :]$ nhất định là xâu tuần hoàn. Ta có thể nén các vị trí có dạng $p_k = p_i + (k - i)l$ thành một cấp số cộng, trong đó l là độ dài chu kỳ. Sau khi xử lý, $|s[p_j :]| < |s[p_i :]|/2$, với $j > i$ là chỉ số đầu tiên không còn nằm trong cấp số cộng. Trường hợp này cũng chỉ có $O(\log n)$ lần giảm một nửa.

■

Muốn tính số lần xuất hiện của mỗi xâu giữa s_{a_i} và $s_{a_{i+1}}$, ta có thể tính trước $\tau(s_{a_i}, t)$ và $\tau(t, s_{a_{i+1}})$, rồi gộp kết quả của hai phần. Chú ý miền giá trị của $O(\log n)$ đoạn cấp số cộng này nhất định đôi một không giao nhau, nên có thể gộp bằng cách tương tự merge sort; mỗi lần gộp tốn $O(\log^2 n)$.

Vấn đề còn lại là cách tính $\tau(s_{a_i}, t)$. Việc tính $\tau(t, s_{a_{i+1}})$ là đối xứng, dùng cùng cách làm.

Ta xây Trie cho các xâu từ điển, rồi xây SAM cho Trie này. Trong $O(m \log n)$ thời gian, với mỗi i tính

$$\max\{\text{LCP}(s_{a_i}[j:], t)\}.$$

Cách tính cụ thể đã được giới thiệu ở phần kiến thức chuẩn bị. Đây chính là p_1 cần tính. Sau đó dùng KMP tiền xử lý mảng next của t . Chú ý $s_{a_i}[p_i:]$ nhất định là một border của $s_{a_i}[p_1:]$, nên ta có thể dùng mảng next này để dựng toàn bộ mảng p .

Khi chọn $B = \sqrt{m} \log^{1.5} n$, độ phức tạp thời gian của toàn bộ thuật toán là nhỏ nhất, bằng

$$O(n|\Sigma| + n \log n + m\sqrt{m} \log^{1.5} n).$$

Điểm kỳ vọng là 20 điểm; kết hợp thuật toán một và hai có thể đạt 70 điểm.

3.4 Thuật toán bốn

Ta tiếp tục dùng thuật toán hai, và xét các truy vấn có độ dài lớn hơn B .

Trước hết giới thiệu hai bổ đề.

Bổ đề 3.1. Nếu p, q đều là chu kỳ của xâu S , và $p + q \leq |S|$, thì $\gcd(p, q)$ cũng là một chu kỳ của xâu S .

Chứng minh. Đặt $d = q - p$ ($p < q$). Khi đó từ $i - p > 0$ hoặc $i + q \leq |S|$ đều có thể suy ra $S_i = S_{i+d}$. ■

Bổ đề 3.2. Gọi l là độ dài chu kỳ. Với $i > 2l$, ta có $\text{next}_i = i - l$.

Chứng minh. Điều này tương đương với việc tiền tố độ dài $i > 2l$ có chu kỳ ngắn nhất là l . Dùng phản chứng: nếu tồn tại chu kỳ T ngắn hơn l , thì $T + l < i$. Theo Bổ đề 3.1, $\gcd(T, l)$ cũng là một chu kỳ của xâu này. Chú ý $\gcd(T, l) \mid l$, mâu thuẫn với việc l là chu kỳ ngắn nhất của toàn xâu. ■

Xét việc tối ưu thuật toán KMP. Ta đặt xâu truy vấn lên xâu mẹ và chạy KMP. Chú ý ta chỉ cần tính các khớp giữa các xâu.

Gọi i là con trở hiện tại của KMP trở vào xâu từ điển, j là con trở hiện tại của KMP trở vào xâu truy vấn, và L là độ dài xâu truy vấn.

Nếu tồn tại một k sao cho $i \in [lp_k, rp_k]$ và $i - j + 1 \in [lp_k, rp_k]$, thì phần đang khớp hiện tại nằm hoàn toàn trong một xâu, không cần được tính ở đây. Vì vậy ta trực tiếp đặt j bằng độ dài tiền tố dài nhất của xâu truy vấn sao cho tiền tố này là một hậu tố của s_{a_k} , rồi đặt i thành vị trí bắt đầu của hậu tố đó và tiếp tục chạy KMP.

Vì có thể tìm nhanh LCP của hai hậu tố, ta dễ dàng tính được vị trí mất khớp tiếp theo. Do đó, chỉ cần tối ưu quá trình nhảy theo next.

Sau khi mất khớp, nếu $\text{next}_j < j/2$, ta trực tiếp thực hiện $j := \text{next}_j$. Ngược lại:

- Nếu sau khi thực hiện $j := \text{next}_j$ vẫn mất khớp, theo Bổ đề 3.2, ta có thể nhảy thẳng đến vị trí $l + j \bmod l$, trong đó l là độ dài chu kỳ. Đây là Case (1).
- Nếu không, ta thống kê tiếp theo $S[i + 1:]$ còn khớp được bao nhiêu chu kỳ của $t[:j]$, rồi tìm vị trí mất khớp. Khi đó có hai khả năng: khả năng thứ nhất là $S[i + 1] = t_j$, tức Case (2), và ta tiếp tục khớp xuống dưới; khả năng thứ hai xử lý giống Case (1).

Còn một tình huống đặc biệt là đã khớp đến cuối xâu. Khi đó ta có thể tính $S[i + 1:]$ còn khớp được bao nhiêu chu kỳ của $t[:j]$; số chu kỳ khớp được chính là đóng góp vào đáp án. Sau đó xử lý giống trường hợp mất khớp.

Định lý 3.2. Thuật toán nêu trên có độ phức tạp thời gian $O(m \log n)$.

Chứng minh. Với Case (1), xét giá trị $j - (i - lp_k)$. Hiển nhiên khi $j - (i - lp_k) \leq 0$, ta sẽ đặt lại giá trị của i và j .

Trước hết, khi tìm LCP, giá trị $j - (i - lp_k)$ không đổi.

Trong quá trình nhảy next, nếu $\text{next}_j \leq j/2$, thực hiện $j := \text{next}_j$ sẽ làm j giảm ít nhất một nửa. Nếu $\text{next}_j > j/2$, khi thực hiện $j := l + j \bmod l$, ta có

$$l + j \bmod l \leq \left\lfloor \frac{2l}{3} \right\rfloor,$$

nên j giảm ít nhất một phần ba. Vì vậy mỗi thao tác Case (1) đều làm đại lượng đang xét giảm theo một tỉ lệ hằng số, và tổng độ phức tạp của Case (1) là $O(m \log n)$.

Với Case (2), trước hết đưa vào một định nghĩa về tầng. Đặt

$$b_i = [\text{next}_i \geq i/2],$$

và gọi đoạn liên tiếp toàn 1 thứ i từ trái sang phải trong dãy b là tầng thứ i .

Ta chứng minh thêm một tính chất về tầng.

Định lý 3.3. Gọi chu kỳ của tiền tố ở tầng thứ i là per_i . Khi đó

$$\frac{3}{2}|\text{per}_i| \leq |\text{per}_{i+1}|.$$

Chứng minh. Gọi pos_i là vị trí ngoài cùng bên phải của tầng thứ i . Hiển nhiên per_i và per_{i+1} đều là chu kỳ của $S[: \text{pos}_i]$.

Nếu $|\text{per}_i| + |\text{per}_{i+1}| \leq \text{pos}_i$, thì theo Bổ đề 3.2, $\text{gcd}(|\text{per}_i|, |\text{per}_{i+1}|)$ cũng là một chu kỳ của $S[: \text{pos}_i]$. Chú ý $\text{gcd}(|\text{per}_i|, |\text{per}_{i+1}|)$ là một ước của $|\text{per}_{i+1}|$, mâu thuẫn với việc per_{i+1} là chu kỳ ngắn nhất của tầng thứ $i + 1$. Do đó

$$|\text{per}_i| + |\text{per}_{i+1}| > \text{pos}_i.$$

Lại có $\text{pos}_i \geq 2|\text{per}_i|$; kết hợp hai bất đẳng thức thu được $\frac{3}{2}|\text{per}_i| \leq |\text{per}_{i+1}|$. ■

Với Case (2), theo Định lý 3.3 hiển nhiên tổng cộng chỉ có $O(\log n)$ tầng. Chú ý chỉ có hai loại thao tác: thao tác Case (1) mỗi lần chỉ lùi một tầng, còn thao tác Case (2) mỗi lần chỉ tăng một tầng. Vì vậy số thao tác Case (2) nhiều nhất chỉ là $m \log n$ lần.

Do đó tổng số lần mất khớp là $O(m \log n)$. Với phần tìm LCP, sau khi xây SAM on Trie của các xâu từ điển, chỉ cần hỏi LCA của các nút tương ứng trên SAM để nhận được LCP của chúng. Chú ý bài toán LCA có thể chuyển thành bài toán RMQ trên thứ tự Euler, nên có thể trả lời $O(1)$. Với phần đặt lại i, j , phần kiến thức chuẩn bị đã giới thiệu cách tìm tiền-hậu tố chung dài nhất của hai xâu trong $O(\log n)$ thời gian. Vì vậy tổng độ phức tạp thời gian cũng là $O(m \log n)$. ■

Tóm lại, chọn $B = \sqrt{m} \log n$ cho độ phức tạp thấp nhất:

$$O((q + m) \sqrt{m} \log n).$$

4 Cách sinh dữ liệu

Vì đây là bài toán liên quan đến khớp xâu, nếu chỉ sinh ngẫu nhiên trực tiếp thì gần như toàn bộ xâu sẽ không có chu kỳ, khiến vết cạn dễ dàng qua được. Do đó cần một cách xây dựng dữ liệu tinh tế hơn.

4.1 Cách xây dựng xâu từ điển

Ý tưởng cơ bản là trước hết sinh ngẫu nhiên một số xâu khá nhỏ sao cho chu kỳ của chúng càng nhỏ càng tốt. Sau đó lặp xâu nhỏ này nhiều lần để nhận được một xâu lớn. Tiếp theo cắt xâu lớn này thành nhiều xâu từ điển. Khi cắt cần bảo đảm xâu từ điển dài nhất không quá nhỏ, đồng thời số lượng xâu từ điển cũng không quá ít. Sau đó lại sinh thêm một số xâu từ điển là bội số của cùng một chu kỳ, và ngẫu nhiên thêm một số nhiều. Cuối cùng đưa các xâu đã sinh vào một Trie.

4.2 Cách xây dựng mảng a

Khi xây dựng mảng a , có thể dùng một xác suất nào đó để quyết định đoạn tiếp theo gồm các xâu có nhiều hay không có nhiều. Nhờ vậy vừa tránh được việc một xâu truy vấn chỉ bắc qua vài xâu kề nhau, vừa tránh được tình huống một xâu khớp thẳng đến cuối.

Ngoài ra, cũng phải bảo đảm $\sum_{i=1}^m |s_{a_i}|$ đủ lớn.

4.3 Cách xây dựng xâu truy vấn

Với xâu truy vấn, ta cũng đặt chu kỳ của nó bằng một trong các xâu nhỏ được sinh ngẫu nhiên lúc đầu, rồi thêm nhiều với xác suất khá thấp.

Về độ dài của xâu truy vấn, cần bảo đảm cả truy vấn dài lẫn truy vấn ngắn đều không quá ít. Tương tự, có ba cách sinh ngẫu nhiên: cách thứ nhất là toàn bộ đều là xâu khá dài, cách thứ hai là toàn bộ đều là xâu khá ngắn, cách thứ ba là độ dài tương đối đồng đều và cố gắng có càng nhiều độ dài khác nhau càng tốt.

4.4 Các cách xây dựng khác

Để bao phủ tốt hơn một số corner case, còn có một số dữ liệu dựng tay, chẳng hạn xâu toàn A, xâu ngẫu nhiên, v.v.

5 Duyên cớ và quá trình ra đề

Bài này do tôi và bạn Zhang Yubo cùng thảo luận và ra đề.

Bạn Zhang Yubo là người đầu tiên nghĩ ra đề bài ban đầu. Chúng tôi cho rằng một đề có ý nghĩa ngắn gọn như vậy chắc cũng sẽ có một lời giải đẹp và đơn giản. Trong quá trình suy nghĩ ban đầu, tôi và Zhang Yubo trước hết nghĩ tới thuật toán ba. Nhưng về sau chúng tôi phát hiện rằng nếu không có tính chất mỗi xâu truy vấn chỉ bắc qua hai xâu được khớp, thì chúng tôi không biết cách tính $\tau(t, S[i :])$ trong độ phức tạp tốt.

Sau đó chúng tôi nhìn lại các thuật toán khớp xâu đã học, hy vọng tìm được gợi ý từ đó. Cuối cùng chúng tôi phát hiện tính chất tốt rằng mảng next trong KMP và chu kỳ của xâu có quan hệ bổ sung cho nhau; điều này có thể tăng tốc quá trình khớp rất mạnh. Vì vậy chúng tôi tối ưu trên nền thuật toán KMP, thêm một số xử lý bổ sung, rồi nhận được thuật toán bốn.

Về cách đặt subtask, chúng tôi cũng cố ý đặt điểm cho các bộ phận của lời giải chuẩn và cho những ngã rẽ mà chính chúng tôi đã đi qua trong quá trình suy nghĩ bài này, nhằm dẫn dắt thí sinh suy nghĩ tới lời giải chuẩn.

Trong đợt thi đối kháng của đội tuyển tập huấn, do cân nhắc độ phức tạp cài đặt, chúng tôi đã giảm độ khó đề gốc qua ba phiên bản rồi mới đưa vào kỳ thi.

Với thuật toán ba, tôi và Zhang Yubo đều chỉ biết giải bài này dưới tính chất mỗi xâu truy vấn chỉ bắc qua hai xâu được khớp. Chúng tôi chưa có kết luận tốt về việc lời giải này có thể mở rộng tốt hơn hay không; hoan nghênh mọi người thảo luận với tôi.

6 Tổng kết

Đề bài khai thác sâu các tính chất liên quan đến chu kỳ của xâu, đồng thời áp dụng tính chất đó vào khớp xâu.

Trong bốn subtask của đề, Subtask 1 là thuật toán thô, độ khó về kiến thức, tư duy và mã nguồn đều rất thấp. Subtask 2 cần dùng kiến thức liên quan tới SAM on Trie, nhưng sau khi phân tích đề một chút cũng không khó nhận ra lời giải này. Subtask 3 đòi hỏi thí sinh hiểu tính chất của border, nên độ khó tăng thêm một chút. Subtask 4 ngoài việc nắm vững thuật toán truyền thống, còn yêu cầu thí sinh có năng lực đổi mới trên thuật toán truyền thống sau khi quan sát tính chất; đây là một kiểm tra khá cao đối với trình độ xử lý xâu của thí sinh. Dù là thí sinh chỉ viết vét cạn, thí sinh đã suy nghĩ thêm một chút, hay thí sinh nghiên cứu sâu và tìm ra mọi tính chất, đều có thể nhận được phần điểm phù hợp.

Trong kỳ thi thực tế, có 4 bạn đạt 20 điểm và 1 bạn qua được bài này. Sau kỳ thi, tôi trao đổi với một số bạn về suy nghĩ của họ đối với bài này, và phát hiện đa số bạn nghĩ tới thuật toán hai, một vài bạn nghĩ tới thuật toán ba, nhưng vì nhiều lý do chỉ viết được vét cạn cơ bản nhất. Có thể thấy đề này có độ phân hóa tốt.

Nhìn chung, đây là một bài vừa yêu cầu thí sinh nắm được một số tính chất thường gặp trong xâu, vừa yêu cầu họ thuần thục các thuật toán xâu cơ bản. Đồng thời, bài này cũng có độ khó cài đặt khá lớn; độ khó tổng thể gần với CTSC những năm gần đây.

Về cá nhân tôi, tôi rất thích ý tưởng liên quan tới thuật toán bốn. KMP là một thuật toán kinh điển. Sau khi đào sâu tính chất của KMP, chúng tôi áp dụng thuật toán KMP vào một bối cảnh mới như vậy, qua đó kiểm tra việc nắm vững và hiểu thuật toán cơ bản này của mọi người. Điều này nhắc ta rằng với một thuật toán quen thuộc, có lẽ ta vẫn có thể tiếp tục khai thác tính chất của nó, từ đó ra được một số đề thú vị.

7 Lời cảm ơn

- Cảm ơn bạn Zhang Yubo đã giúp đỡ rất nhiều cho việc viết bài này, đồng thời cảm ơn bạn đã luôn giúp đỡ và đồng hành cùng tôi trên con đường OI.
- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
- Cảm ơn huấn luyện viên đội tuyển quốc gia Zhang Ruizhe đã hướng dẫn.
- Cảm ơn thầy Li Jian cùng cha mẹ đã quan tâm và chỉ dẫn trong nhiều năm.
- Cảm ơn Wang Xiuhuan, Wu Yue, Dong Weijun và Li Ningyuan đã đọc duyệt bài viết này.

Tài liệu tham khảo

1. Liu Yanyi, *Mở rộng automaton hậu tố trên cây Trie*, tập luận văn đội tuyển quốc gia năm 2015.
2. Wikipedia, *Knuth–Morris–Pratt algorithm*, https://en.wikipedia.org/wiki/KMP_algorithm.
3. Jin Ce, *Chuyên đề thuật toán xâu*, trại mùa đông NOI năm 2017.
4. Wang Jianhao, *Bàn sơ lược về một số phương pháp khớp xâu*, tập luận văn đội tuyển quốc gia năm 2015.

5. Zhang Tianyang, *Automaton hậu tố và ứng dụng*, tập luận văn đội tuyển quốc gia năm 2015.

7 Bài 5: Một số kỹ thuật và công cụ giải bài toán khối liên thông trên cây

Ren Xuandi, Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Bài toán khối liên thông trên cây là một lớp bài toán được ưa chuộng trong thi tin học. Bài viết này giới thiệu một số kỹ thuật và công cụ thường dùng để giải lớp bài toán đó. Với các bài toán DP trên khối liên thông của cây, bài viết giới thiệu những kỹ thuật quen thuộc như chuyển trạng thái theo thứ tự DFS và phân trị theo điểm, “số đỉnh trừ số cạnh”, DP toàn cục bằng gộp cây đoạn, và bảo trì DP động bằng phân trị theo chuỗi, cùng các ứng dụng tương ứng. Với bài toán bảo trì khối liên thông cùng màu trên cây, bài viết khảo sát tư duy chung và giới thiệu một cấu trúc dữ liệu tổng quát dựa trên LCT, hỗ trợ đổi màu cả đường đi và bảo trì thông tin trên một thứ tự DFS bất kỳ.

1 Dẫn nhập

Cây là một cấu trúc rất đặc biệt. Mọi đồ thị con liên thông của một cây, tức một khối liên thông trên cây, cũng là một cây; giao của một số khối liên thông trên cây cũng là một cây. Trong quá trình học tập, tác giả gặp nhiều bài toán liên quan tới khối liên thông trên cây, và đại thể chia chúng thành hai nhóm lớn: bài toán DP đếm hoặc tìm giá trị tối ưu trên khối liên thông của cây, và bài toán bảo trì thông tin của các khối liên thông cùng màu trên cây.

Trong những năm gần đây, nhiều bài toán liên quan tới khối liên thông trên cây có rất nhiều cách giải, lại thay đổi theo từng đề, nên thường khiến người học khó biết bắt đầu từ đâu. Bài viết này sắp xếp và tổng kết một số phương pháp, tư duy thường gặp khi giải bài toán khối liên thông trên cây. Riêng với bài toán bảo trì thông tin khối liên thông cùng màu trên cây, bài viết giới thiệu một cấu trúc dữ liệu tổng quát hỗ trợ đổi màu theo đường đi và bảo trì thông tin trên một thứ tự DFS bất kỳ, nhằm giúp độc giả có một lộ trình rõ hơn khi gặp lớp bài toán này.

Bài viết chủ yếu gồm hai phần. Phần thứ nhất đi từ vài ví dụ, lần lượt giới thiệu ứng dụng của chuyển trạng thái theo thứ tự DFS và phân trị theo điểm, “số đỉnh trừ số cạnh”, DP toàn cục bằng gộp cây đoạn, và bảo trì DP động bằng phân trị theo chuỗi trong các bài toán DP trên khối liên thông của cây. Phần thứ hai xoay quanh bài toán khối liên thông cùng màu trên cây: thông qua hai ví dụ kinh điển, bài viết trình bày tư duy chung của lớp bài toán này, rồi giới thiệu cấu trúc dữ liệu Link Cut Memphis, có thể đổi màu theo đường đi và bảo trì thông tin trên thứ tự DFS.

2 Định nghĩa và quy ước liên quan

Một đồ thị vô hướng $G = \{V, E\}$ được gọi là cây khi và chỉ khi giữa hai đỉnh bất kỳ có đúng một đường đi đơn. Mọi đồ thị vô hướng liên thông và không có chu trình đều là cây.

Một khối liên thông trên cây $T = \{V, E\}$ là một cây $T' = \{V', E'\}$ thỏa $V' \subseteq V$, $E' \subseteq E$ và T' liên thông. Dễ thấy cấu trúc của T' cũng là một cây.

Các bài toán DP trên khối liên thông của cây trong bài viết này thường có một trong hai dạng sau:

- Bài toán đếm: cho một cây T , hỏi có bao nhiêu khối liên thông trên cây thỏa tính chất A .
- Bài toán tối ưu: cho một cây T và một hàm giá trị, hỏi giá trị tốt nhất trong mọi khối liên thông trên cây T thỏa một tính chất nào đó.

Các bài toán bảo trì thông tin khối liên thông cùng màu trên cây trong bài viết này thường có dạng: cho một cây T , mỗi đỉnh có hai màu đen hoặc trắng. Định nghĩa một khối liên thông trên cây là hợp lệ khi và chỉ khi mọi đỉnh trong đó có cùng màu. Truy vấn hỏi thông tin của khối liên thông hợp lệ cực đại chứa đỉnh x .

Ngoài hai dạng này còn có thể có một số bài toán ít gặp hơn liên quan tới khối liên thông trên cây; chúng không nằm trong phạm vi thảo luận của bài viết này.

3 Bài toán DP trên khối liên thông của cây

3.1 Thuật toán thô

Với một bài toán DP trên khối liên thông của cây nói chung, cách thô phổ biến là đặt dp_x là số phương án hoặc đáp án tối ưu khi chọn một khối liên thông trong cây con của x và khối đó chứa x . Giá trị dp_x có thể thu được bằng cách gộp các giá trị dp của con x , tức duyệt mọi con và quyết định trong cây con đó bỏ trống hay chọn một khối liên thông chứa gốc.

Nếu gộp thông tin của hai con tốn $O(1)$, chẳng hạn đếm số khối liên thông khác nhau về bản chất trên cây, độ phức tạp tổng là $O(n)$. Nếu gộp hai con tốn $O(size_a \cdot size_b)$, chẳng hạn đếm số khối liên thông khác nhau về bản chất gồm đúng k đỉnh, thì hai đỉnh bất kỳ sẽ phát sinh độ phức tạp 1 tại LCA của chúng; độ phức tạp tổng là $O(n^2)$.

Thuật toán thô này đều dùng được cho bài toán đếm và bài toán tối ưu.

3.2 Chuyển theo thứ tự DFS và phân trị theo điểm

Với một lớp bài toán DP trên khối liên thông của cây, nếu việc gộp thông tin không đủ hiệu quả nhưng thêm thông tin của một đỉnh đơn lại khá hiệu quả, ta thường có thể chuyển trạng thái theo thứ tự DFS, biến thao tác gộp cây con thành thao tác thêm một đỉnh. Mô hình thường gặp là ba lô: giả sử giới hạn thể tích là m , gộp hai ba lô tốn $O(m^2)$, còn thêm một vật phẩm tốn $O(m)$.

Trước hết giả sử khối liên thông được chọn bắt buộc chứa gốc. Lấy thứ tự DFS của toàn cây; cấu trúc của một khối liên thông chứa gốc nhất định có dạng toàn cây sau khi xóa đi một số cây con đôi một không giao nhau. Trong thứ tự DFS, những phần bị xóa chính là một số đoạn liên tiếp.

Đặt dp_i là thông tin sau khi đã xét i đỉnh đầu tiên trong thứ tự DFS. Nếu chọn đỉnh thứ i , ta chuyển sang dp_{i+1} . Nếu không chọn, ta chuyển sang dp_j , trong đó j là đầu phải của đoạn DFS ứng với cây con của đỉnh thứ i , cộng thêm 1; điều này biểu thị việc xóa cả cây con đó.

Như vậy mỗi bước chỉ cần thêm một vật phẩm vào ba lô, thay vì gộp hai ba lô. Nếu thêm một đỉnh tốn $O(m)$, độ phức tạp tổng là $O(nm)$.

Với bài toán mà khối liên thông được chọn không nhất thiết chứa gốc, chỉ cần dùng phân trị theo điểm: mỗi lần chọn trọng tâm làm gốc, tính số phương án bắt buộc chứa trọng tâm, rồi xóa trọng tâm và đệ quy trên từng khối liên thông còn lại. Độ phức tạp tổng là $O(n \log n \cdot m)$.

Ví dụ 1. Nguồn: *Tsinghua Training 2016*, bài con Cây con liên thông.

Đề bài. Cho một cây, mỗi đỉnh có một màu. Cho ba màu u, v, w , hỏi có bao nhiêu khối liên thông trên cây sao cho số đỉnh mang màu u, v, w lần lượt là a, b, c .

Cách làm. Cách thô: DP từ dưới lên, dùng $dp_{x,i,j,k}$ biểu thị số phương án sau khi xử lý cây con gốc x , trong đó số lượng ba màu lần lượt là i, j, k . Khi chuyển trạng thái và gộp hai cây con, ta liệt kê số đỉnh

của ba màu trong từng cây con; độ phức tạp là $O(na^2b^2c^2)$.

Chú ý việc gộp hai cây con ở đây là một phép tích chập ba chiều. Nếu dùng FFT ba chiều để tối ưu, độ phức tạp có thể giảm thành $O(nabc \log abc)$, nhưng hằng số và lượng mã đều khá lớn.

Xét dùng thuật toán phân trị theo điểm nói trên, kết hợp với chuyển theo thứ tự DFS. Độ phức tạp là $O(nabc \log n)$, hằng số rất nhỏ, và cũng có thể mở rộng thuận tiện sang bài toán tối ưu.

3.3 “Số đỉnh trừ số cạnh”

Với một cây không rỗng, số đỉnh trừ số cạnh luôn bằng 1. Ta có thể dùng đẳng thức này để thống kê số khối liên thông trên cây thỏa một tính chất nào đó: liệt kê từng đỉnh hoặc từng cạnh, tính số khối liên thông hợp lệ chứa đỉnh hoặc cạnh đó, rồi lấy đáp án theo đỉnh trừ đáp án theo cạnh. Khi đó mỗi khối liên thông hợp lệ và không rỗng được tính đúng một lần.

Kỹ thuật này thường dùng trong trường hợp cần xét giao của một số khối liên thông trên cây, vì giao của một số khối liên thông trên cây vẫn là một cây, có thể rỗng. Số phương án để giao của nhiều khối liên thông không rỗng không dễ tính trực tiếp; nhưng nếu chỉ xét một đỉnh hoặc một cạnh, điều kiện trở thành mỗi khối liên thông đều phải chứa đỉnh hoặc cạnh đó. Các khối liên thông độc lập với nhau, nên bài toán trở nên tương đối dễ tính.

Ví dụ 2: Tập hoàn hảo. *Nguồn: thi đối kháng đội tuyển tập huấn 2018, Liang Yancheng.*

Đề bài. Cho một cây; mỗi đỉnh có trọng lượng và giá trị, mỗi cạnh có trọng số. Xét việc chọn một tập đỉnh S sao cho tổng trọng lượng các đỉnh không vượt quá M và các đỉnh đó tạo thành một khối liên thông trên cây. Những tập S có tổng giá trị lớn nhất được gọi là tập hoàn hảo.

Nếu hai đỉnh x, y thỏa $\text{dist}(x, y) \cdot v_y \leq \text{Max}$, thì x có thể kiểm tra y . Hỏi có bao nhiêu cách chọn k tập trong tất cả các tập hoàn hảo sao cho trong giao của chúng tồn tại một đỉnh x có thể kiểm tra mọi đỉnh thuộc các tập đó.

Đáp án lấy modulo 523. Giới hạn:

$$n \leq 60, \quad m \leq 10000, \quad k \leq 10^9.$$

Cách làm. Tập các đỉnh có thể kiểm tra mọi đỉnh trong một tập là một khối liên thông trên cây. Giao của một số khối liên thông trên cây vẫn là một khối liên thông trên cây. Xét dùng kỹ thuật “số đỉnh trừ số cạnh”: thống kê số tập hoàn hảo có thể được một đỉnh x nào đó kiểm tra, rồi trừ đi số tập hoàn hảo có thể được đồng thời hai đầu mút của một cạnh kiểm tra.

Xét cách tính số tập hoàn hảo có thể được đỉnh x kiểm tra. Những tập này chắc chắn không được chứa bất kỳ đỉnh nào có khoảng cách tới x không hợp lệ. Sau khi xóa các đỉnh không hợp lệ, ta làm DP trên cây còn lại; bài toán tương đương hỏi có bao nhiêu khối liên thông trên cây có tổng trọng lượng không vượt quá M và tổng giá trị lớn nhất. Đây là bài toán ba lô tối ưu kinh điển: chỉ cần vừa ghi giá trị tốt nhất trong ba lô, vừa ghi số phương án đạt giá trị đó. Lấy x làm gốc và chuyển theo thứ tự DFS, độ phức tạp là $O(nm)$.

Cuối cùng còn phải xử lý tổ hợp modulo, không liên quan nhiều đến chủ đề bài viết nên không triển khai. Độ phức tạp tổng là $O(n^2m + \text{độ phức tạp tính tổ hợp modulo})$.

3.4 DP toàn cục bằng gộp cây đoạn

Có một lớp bài toán đếm khối liên thông trên cây như sau: mỗi đỉnh cần bảo trì một mảng DP kích thước m ; khi gộp hai cây con, thao tác là gộp theo từng vị trí tương ứng trong m vị trí; khi thêm một đỉnh, chỉ

cần sửa một vị trí nào đó. Dù số trạng thái DP là $O(nm)$, phần lớn trạng thái chỉ lặp lại việc gộp từ trạng thái của con. Mặt khác, thao tác “thêm một đỉnh” có quy mô $O(n)$. Nếu ta có thể hoàn thành nhanh việc gộp theo từng vị trí, lớp bài toán này sẽ được giải quyết tốt.

Xét dùng gộp cây đoạn: với mỗi đỉnh, dùng một cây đoạn để bảo trì m giá trị DP của nó. Khi thêm một đỉnh, ta sửa trong cây đoạn của nó; khi gộp hai cây con, ta dùng gộp cây đoạn để hoàn thành chuyển trạng thái hàng loạt. Tổng số nút được chèn vào các cây đoạn là $O(n \log n)$, còn độ phức tạp của gộp cây đoạn không vượt quá tổng độ phức tạp chèn.

Ví dụ 3. Nguồn: THUWC 2018.

Đề bài. Cho một cây, mỗi đỉnh có một màu. Hỏi có bao nhiêu khối liên thông trên cây chứa không quá 2 màu.

Cách làm. Đặt $f(x, c)$ là số phương án chọn một khối liên thông chứa x trong cây con của x , sao cho hai màu là col_x và c .

Xét màu của một con y của x , không khó nhận được chuyển trạng thái: nếu y cùng màu với x , nhân trực tiếp mọi $f(y, \sim) + 1$ vào $f(x, \sim)$; ngược lại, nhân $f(y, \text{col}_x) + 1$ vào $f(x, \text{col}_y)$.

Loại chuyển thứ hai tổng cộng chỉ cần chuyển $O(n)$ trạng thái. Nút thất nằm ở loại chuyển thứ nhất: mỗi khi có một con cùng màu, ta phải tốn độ phức tạp bằng số màu trong cây con.

Quan sát rằng các màu khác nhau độc lập với nhau. Vì vậy có thể dùng cây đoạn để bảo trì mọi giá trị DP tại một đỉnh; khi gộp cây con, thực hiện gộp cây đoạn là được.

Độ phức tạp là $O(n \log n)$.

3.5 Bảo trì DP động bằng phân trị theo chuỗi

DP động là bài toán hỗ trợ sửa đầu vào của DP, và sau mỗi lần sửa phải nhanh chóng tính lại giá trị DP; nó thường xuất hiện trong bài toán đếm. Trong luận văn đội tuyển tập huấn năm 2017, Chen Junkun đề xuất một thuật toán có tính mở rộng để giải bài toán DP động trên cây: phân trị theo chuỗi. Ở đây ta xét cách áp dụng phân trị theo chuỗi vào bài toán DP động trên khối liên thông của cây.

Cụ thể, ta cần hỗ trợ sửa thông tin của đỉnh và sau mỗi lần sửa lại hỏi số khối liên thông trên cây.

Tư tưởng của phân trị theo chuỗi là phân rã nặng-nhẹ trên cây, rồi tính DP theo thứ tự các chuỗi nặng từ dưới lên. Với một chuỗi nặng, thông tin DP của các chuỗi nặng con rẽ ra từ nó đã được tính xong; ta có thể gắn các thông tin đó lên chuỗi nặng hiện tại, khi đó bài toán trở thành DP động trên một dãy. Trên dãy, ta thường có thể dùng cấu trúc dữ liệu để hỗ trợ sửa thông tin của một vị trí và nhanh chóng bảo trì giá trị DP.

Khi sửa thông tin của một đỉnh, theo cấu trúc phân trị này chỉ có $O(\log n)$ chuỗi nặng trên tổ tiên bị ảnh hưởng. Ta lần lượt đi từ dưới lên, sửa và bảo trì giá trị DP trong cấu trúc dữ liệu của từng chuỗi nặng.

Nói chung, đặt $G(x)$ là đáp án khi chọn một khối liên thông chứa gốc trong cây con của x . Với một chuỗi nặng, giả sử các đỉnh trên đó theo thứ tự độ sâu tăng dần là $x_1, x_2, \dots, x_k$. Với mỗi đỉnh x_i , ta đã tính được các giá trị $G()$ của mọi con nhẹ; gộp các thông tin này với thông tin của bản thân x_i sẽ thu được đóng góp khi chọn đỉnh x_i trên chuỗi nặng. Gọi giá trị này là $F(x_i)$.

Nếu khối liên thông được chọn giao với chuỗi nặng này, giao đó chắc chắn là một đoạn, và đóng góp là tích các $F(x)$ trong đoạn. Do đó cần tính tổng tích $F(x)$ trên mọi đoạn của chuỗi nặng. Chỉ cần dùng cây đoạn hoặc cây cân bằng để bảo trì tích thông tin trong đoạn, tổng tích của mọi tiền tố và hậu tố, là có thể thuận tiện hỗ trợ sửa một vị trí và tính lại DP trong thời gian $O(\log n)$.

Đóng góp của chuỗi nặng này cho chuỗi nặng cha chính là số phương án chọn một tiền tố trên chuỗi nặng. Chỉ cần dùng thông tin tiền tố do cây đoạn hoặc cây cân bằng bảo trì để cập nhật giá trị $G()$ của đỉnh cha.

Đến đây khung thuật toán đã khá rõ: phân rã nặng-nhẹ trên cây, tính DP theo thứ tự các chuỗi nặng từ dưới lên. Khi sửa thông tin của một đỉnh, với mỗi chuỗi nặng trên tổ tiên của nó, tính lại bằng cây đoạn hoặc cây cân bằng số phương án chọn một tiền tố, rồi dùng số phương án này cập nhật thông tin của cha chuỗi nặng.

Nếu thông tin cần bảo trì có phép trừ, sau khi sửa thông tin của một chuỗi nặng con, chỉ cần trừ thông tin cũ và cộng thông tin mới. Nếu không, với mỗi đỉnh còn cần mở một cây đoạn hoặc cây cân bằng kích thước $O(\text{số con nhẹ})$ để nhanh chóng bảo trì thông tin của các con nhẹ. Tuy vậy độ phức tạp không đổi.

Ví dụ 4: Trò chơi cắt cây. Nguồn: SDOI 2017 Round 2 Day 1.

Đề bài. Cho một cây, mỗi đỉnh có một trọng số. Có hai loại thao tác:

1. Change $x \ y$: đổi trọng số của đỉnh số x thành y .
2. Query k : hỏi có bao nhiêu cây con liên thông không rỗng có xor trọng số các đỉnh đúng bằng k .

In đáp án modulo 10007. Giới hạn:

$$n, Q \leq 30000, \quad 0 \leq A_i, y, k < 128.$$

Cách làm. Trước hết chuyển toàn bộ giá trị xor thành giá trị điểm của FWT; trong quá trình tính toán, mọi thao tác đều dùng giá trị điểm, cuối cùng mới FWT ngược lại. Đặt m là giá trị trọng số lớn nhất; độ phức tạp gộp thông tin là $O(m)$.

Xét áp dụng thuật toán phân trị theo chuỗi. Đặt $G(x, \sim)$ là giá trị điểm khi chọn một khối liên thông chứa gốc trong cây con của x . Với một đỉnh trên chuỗi nặng, tính tích các $G(y, \sim) + 1$ của mọi con nhẹ y , rồi dùng cấu trúc dữ liệu bảo trì số phương án chọn một đoạn; tiếp đó dùng số phương án chọn một tiền tố để cập nhật $G(fa, \sim)$ của cha chuỗi nặng.

Vì thông tin được bảo trì là tích, khi bỏ đóng góp cũ của một con nhẹ có thể gặp vấn đề chia cho 0. Một cách xử lý là bảo trì tích của các giá trị khác 0 modulo 10007 và số mũ của 10007.

Dùng phân rã chuỗi trên cây, độ phức tạp là $O(qm \log^2 n)$.

Nếu dùng LCT để bảo trì cấu trúc phân trị theo chuỗi, khi Access ta động cập nhật quan hệ chuỗi nặng-nhẹ và bảo trì đáp án. Độ phức tạp có thể đạt $O(qm \log n)$; hơn nữa, cách này truy vấn thông tin của một số cấu trúc con thuận tiện hơn, chẳng hạn đáp án trong cây con của một đỉnh, hoặc đáp án của các khối liên thông có giao với một đường đi $u \sim v$, đồng thời còn hỗ trợ các thao tác Link/Cut.

3.6 Tiểu kết

Bài toán DP trên khối liên thông của cây thường được chia thành bài toán đếm và bài toán tối ưu.

Nếu độ phức tạp gộp hai trạng thái DP cao, còn độ phức tạp thêm một đỉnh thấp, chẳng hạn phần lớn bài toán ba lô, ta thường có thể dùng chuyển trạng thái theo thứ tự DFS thay cho chuyển trạng thái từ dưới lên thô. Nếu khối liên thông cần tìm không nhất thiết chứa gốc, có thể trả thêm $O(\log n)$ bằng phân trị theo điểm: mỗi lần lấy trọng tâm làm gốc, tính thông tin của khối liên thông bắt buộc chứa gốc, rồi đệ quy trên từng khối liên thông.

Kỹ thuật “số đỉnh trừ số cạnh” thường dùng trong bài toán đếm khi chọn nhiều khối liên thông trên cây và yêu cầu giao không rỗng. Bằng cách làm $O(n)$ lần DP bắt buộc chứa một đỉnh hoặc một cạnh,

ta có thể khiến các khối liên thông khác nhau độc lập với nhau. Đặc biệt, nếu đề yêu cầu các khối liên thông đôi một có giao, điều này cũng tương đương với việc giao của tất cả chúng không rỗng.

Với những đề mà phần lớn thời gian nằm ở việc gộp thông tin tương ứng của hai cây con, có thể xét dùng cây đoạn để lưu thông tin DP, rồi dùng gộp cây đoạn để hoàn thành nhanh các chuyển trạng thái hàng loạt.

Với bài toán cần hỗ trợ sửa thông tin đỉnh rồi đếm lại khối liên thông trên cây, có thể dùng phân rã chuỗi trên cây hoặc LCT để bảo trì DP động. Bản chất là tận dụng điều kiện phép gộp thông tin thỏa tính kết hợp. Với mỗi chuỗi nặng, ta tính tổng đáp án khi chọn một đoạn trên chuỗi, rồi dùng tổng đáp án khi chọn một tiền tố để cập nhật đáp án của chuỗi nặng cha.

4 Bài toán bảo trì khối liên thông cùng màu trên cây

4.1 Tư duy chung

Xét một cây hai màu đen-trắng. Với mỗi khối liên thông cùng màu cực đại, đỉnh nông nhất của nó, tức LCA của toàn khối liên thông, là duy nhất. Đỉnh này có thể được tìm nhanh bằng phân rã chuỗi trên cây hoặc LCT: tìm đỉnh áp chót trên đường từ một đỉnh tới tổ tiên gần nhất có màu khác. Ta có thể bảo trì thông tin tại đỉnh nông nhất đó.

Khi sửa màu của đỉnh nông nhất, hình dạng khối liên thông có thể thay đổi rất lớn: các con vốn cùng màu với nó trở thành khác màu, còn các con vốn khác màu trở thành cùng màu. Nếu trực tiếp bảo trì “thông tin của khối liên thông trong cây con của x có cùng màu với x ”, ta phải tốn $O(\text{số con})$ thời gian.

Cách giải quyết là đồng thời ghi thông tin của cả hai màu tại mỗi đỉnh: nếu đỉnh này là đen hoặc trắng, thông tin của khối liên thông màu tương ứng lấy nó làm đỉnh nông nhất là gì. Khi sửa màu của đỉnh, thông tin của bản thân nó không cần sửa; chỉ cần thêm hoặc xóa thông tin của đỉnh này tại cha nó.

Nếu số màu không phải hai mà là một hằng số nhỏ k , ta có thể bảo trì k cấu trúc. Trong cấu trúc thứ i , coi màu thứ i là đen, còn các màu khác là trắng. Như vậy ta hỗ trợ được nhiều màu với chi phí nhân thêm k .

Ví dụ 5. Nguồn: *SPOJ QTREE6*.

Đề bài. Cho một cây, mỗi đỉnh có một trong hai màu đen-trắng. Cần hỗ trợ hai thao tác:

1. Hỏi kích thước khối liên thông cùng màu chứa đỉnh u .
2. Lật màu đỉnh u , tức đen thành trắng và trắng thành đen.

Giới hạn $n, q \leq 10^5$.

Cách làm. Xét bảo trì hai giá trị $B(x)$ và $W(x)$ tại mỗi đỉnh, lần lượt biểu thị: nếu đỉnh x là đen hoặc trắng, kích thước khối liên thông cùng màu có màu đó và lấy x làm đỉnh nông nhất.

Với thao tác hỏi, giả sử hỏi đỉnh u và u màu trắng. Chỉ cần tìm đỉnh nông nhất v của khối liên thông trắng chứa u , rồi xuất $W(v)$.

Xét thao tác lật màu u . Giả sử đổi từ trắng sang đen. Chỉ cần tìm đỉnh đen đầu tiên v trên đường từ u đi lên, trừ $W(u)$ khỏi mọi giá trị $W()$ trên đoạn $(u, v]$; rồi tìm đỉnh trắng đầu tiên v trên đường từ u đi lên, cộng $B(u)$ vào mọi giá trị $B()$ trên đoạn $(u, v]$. Khi đó bảo trì đã hoàn tất.

“Đỉnh trắng hoặc đen đầu tiên trên đường từ u đi lên” có thể tìm bằng cấu trúc dữ liệu bảo trì số đỉnh của hai màu trên đoạn đường. Nếu dùng phân rã chuỗi trên cây cộng cây đoạn, độ phức tạp là $O(n \log^2 n)$; nếu dùng LCT, độ phức tạp là $O(n \log n)$.

Ví dụ 6. Nguồn: *SPOJ QTREE7*.

Đề bài. Cho một cây, mỗi đỉnh có một trong hai màu đen-trắng và có một trọng số. Cần hỗ trợ ba thao tác:

1. Hỏi trọng số lớn nhất trong khối liên thông cùng màu chứa đỉnh u .
2. Lật màu đỉnh u .
3. Sửa trọng số đỉnh u .

Giới hạn $n, q \leq 10^5$.

Cách làm. Khác với bài trước, bài này cần bảo trì giá trị lớn nhất, nên không thể trực tiếp trừ đóng góp như ở bài trước.

Quan sát bản chất của thông tin được bảo trì trong bài trước: nếu một đỉnh là trắng, thông tin của nó được cộng vào $W()$ của cha; nếu là đen, thông tin của nó được cộng vào $B()$ của cha.

Điều này tương đương với việc có một cây đen và một cây trắng. Đỉnh đen nối với cha nó trong cây đen, đỉnh trắng nối với cha nó trong cây trắng. Trong mỗi cây, mỗi khối liên thông ngoài đỉnh nông nhất ra đều có màu tương ứng. Khi hỏi thông tin của khối liên thông cùng màu chứa một đỉnh, ta vẫn tìm tổ tiên cùng màu nông nhất, rồi hỏi thông tin của cả cây con của đỉnh đó.

Xét dùng LCT. Tại mỗi đỉnh bảo trì một *multiset*, ghi trọng số lớn nhất trong cây con của mọi con nhẹ. Trên splay, đồng thời bảo trì trọng số lớn nhất của chuỗi nặng.

Khi Access chuyển cạnh nhẹ-nặng, ta cập nhật thông tin trong *multiset*: chèn giá trị của con nặng cũ và xóa giá trị của con nặng mới. Khi sửa trọng số, chỉ cần Access trước là có thể bảo trì thuận tiện.

Sửa màu một đỉnh, chẳng hạn đổi u từ trắng sang đen: nếu u không phải gốc, chỉ cần cắt cạnh giữa u và cha trong cây trắng, rồi nối cạnh giữa u và cha trong cây đen.

Mỗi lần chuyển cạnh nhẹ-nặng có một độ phức tạp $O(\text{số con nhẹ})$ của *multiset*. Theo chứng minh độ phức tạp của Top Tree, cách làm này có độ phức tạp $O(n \log n)$.

4.2 Bảo trì thông tin bất kỳ trên thứ tự DFS

Với thông tin phức tạp hơn, chỉ cần thông tin đó thỏa tính kết hợp và có thể gộp nhanh, nói cách khác có thể được cấu trúc dữ liệu bảo trì trên thứ tự DFS, thì đều có thể áp dụng cách làm phía trên.

Quan sát hình thái của cây đen và cây trắng nói trên, ta thấy chúng hoàn toàn nhất quán với cây gốc, và trong toàn bộ quá trình không cần dùng thao tác đổi gốc. Nói cách khác, có thể bảo trì trực tiếp theo thứ tự DFS của cây gốc.

Xét dùng một cây cân bằng hỗ trợ tách đoạn, như Splay hoặc Treap. Khi Link/Cut trong LCT, ta lấy đoạn cây con ra khỏi cây cân bằng rồi ghép vào thứ tự DFS của cha, hoặc tách cây con khỏi cha.

Mỗi thao tác trên cây cân bằng tốn $O(\log n)$, nên độ phức tạp tổng là $O(n \log n)$.

4.3 Đổi màu theo đường đi

Với thao tác đổi màu theo đường đi, mục này giới thiệu một cấu trúc dữ liệu dựa trên LCT do Tao Yuanzheng đề xuất.

Nếu đổi toàn bộ đỉnh trên một đường đi thành đen hoặc trắng và coi thao tác phủ màu đường đi là một thao tác LCT, ta có thể chuyển bài toán thành $O(n \log n)$ lần đảo màu một chuỗi cùng màu.

4.3.1 Định nghĩa mới của cây đen/trắng. Phỏng theo cách làm phía trên, ta vẫn bảo trì cây đen và cây trắng, nhưng cần sửa nhẹ định nghĩa của chúng. Lấy một đỉnh trắng u làm ví dụ:

1. u và cha v của nó nối với nhau trong cây trắng.
2. Nếu u và v nối với nhau bằng cạnh nặng trong cây trắng, ta nối chúng trong cây đen.

Hình trong bản gốc minh họa một hình thái khả dĩ của cây đen và cây trắng bằng hai cây trên các đỉnh $1, \dots, 8$, trong đó cạnh liền nét biểu thị cạnh nặng và cạnh đứt nét biểu thị cạnh nhẹ.

Trong định nghĩa mới, một khối liên thông bất kỳ trong cây đen có thể xem là hợp bởi hai phần:

1. Một số cây con liên thông tạo bởi các khối liên thông màu đen; gọi phần này là *cây khối*.
2. Một chuỗi tạo bởi các đỉnh trắng tương ứng với một chuỗi nặng trong cây trắng; gọi phần này là *cây chuỗi*.

Có thể thấy dưới chuỗi trắng có thể treo một số cây khối đen, nhưng dưới cây khối đen chắc chắn không có đỉnh trắng, vì trong cây đen một đỉnh trắng không thể nối với cha của nó.

Trong định nghĩa mới, khối liên thông cùng màu trong cây có màu tương ứng vẫn là một đoạn liên tiếp trên thứ tự DFS, nên có thể dùng cây cân bằng để bảo trì. Khi truy vấn, ta vẫn tìm tổ tiên cùng màu nông nhất rồi hỏi thông tin cây con. Vấn đề còn lại là bảo trì hình thái cây đen và cây trắng khi đổi màu theo đường đi.

4.3.2 Thao tác Expose. Định nghĩa thao tác Expose là một biến thể của thao tác Access. Gọi v là tổ tiên cùng màu nông nhất của u . Hiệu quả của $\text{Expose}(u)$ là: đổi đường đi giữa u và v thành các cạnh nặng, đồng thời cắt cạnh nặng phía dưới u . Khi thay đổi cạnh nhẹ-nặng, ta thực hiện thao tác Link/Cut tương ứng trong cây khác màu.

Xét việc đảo màu chuỗi từ u đến tổ tiên cùng màu nông nhất v , giả sử u là đen. Trước hết $\text{Expose}(u)$; khi đó trong cây đen, chuỗi từ u tới đỉnh đầu chuỗi đã được nối thông, nên có thể trực tiếp coi phần chuỗi này là cây chuỗi trắng. Còn trong cây trắng, phần từ chuỗi $u \sim v$ trở lên là cây khối trắng; ta trực tiếp coi phần chuỗi này là cây khối và ghép vào dưới cây khối của cha. Đồng thời cần cắt cạnh giữa v và cha trong cây đen, rồi nối một cạnh nhẹ giữa v và cha trong cây trắng.

Hình trong bản gốc xét ví dụ đổi chuỗi đen $4 \sim 6$ thành trắng. Sau $\text{Expose}(6)$, chỉ cần cắt cạnh ảo $(2, 4)$ trong cây đen, nối cạnh ảo $(2, 4)$ trong cây trắng, rồi trực tiếp đổi phần này thành trắng.

4.3.3 Sắp xếp quy trình. Giả sử cần đổi màu đường đi $u \sim v$ thành màu c . Trước hết thảo luận trường hợp tại LCA để chuyển thành một số lần đổi màu đường đi tới gốc.

Bắt đầu từ u , liên tục tìm đoạn cùng màu đang cần đảo trên đường tới gốc. Gọi đỉnh nông nhất của đoạn hiện tại là v .

Dùng thao tác $\text{Expose}(u)$ để nối thông chuỗi $u \sim v$, đồng thời bảo trì hình thái tương ứng trong cây còn lại.

Nối hoặc cắt cạnh giữa v và cha của nó, rồi đặt u bằng cha của v .

Lặp quá trình này cho tới khi u leo tới gốc.

4.3.4 Phân tích độ phức tạp. Số đoạn cùng màu trên đường đi, tức số lần Expose chuyển cạnh nhẹ-nặng, giống thao tác Access của LCT: theo nghĩa khấu hao là $O(n \log n)$. Mỗi lần chuyển cạnh nhẹ-nặng đều phải thực hiện một thao tác Link/Cut trong cây còn lại; khi Link/Cut, cần dùng cây cân bằng để bảo trì tốt thứ tự DFS trong $O(\log n)$. Vì vậy độ phức tạp tổng là $O(n \log^2 n)$.

Ví dụ 7. Nguồn: BZOJ 3914.

Đề bài. Cho một cây vô căn gồm n đỉnh. Mỗi đỉnh có hai màu, ban đầu mọi đỉnh đều màu đen. Mỗi cạnh có trọng số dương.

Cần bảo trì m thao tác:

1. 1 u : hỏi đường kính của khối liên thông cùng màu chứa u .
2. 2 $u \ v \ c$: phủ màu c lên đường đi $u \sim v$.

Giới hạn $n, m \leq 10^5$.

Cách làm. Tìm đường kính trên cây có một cách làm $O(n)$ cho một lần: tùy ý chọn một đỉnh xuất phát, chạy BFS một lần để tìm một đỉnh xa nhất u , rồi từ u chạy BFS một lần để tìm một đỉnh xa nhất v . Khi đó $u \sim v$ là một đường kính. Đáng tiếc, cách này khó bảo trì hiệu quả.

Có một cách khác bảo trì được trên thứ tự DFS: thứ tự duyệt Euler. Tức trong quá trình DFS, mỗi lần đi xuôi hoặc đi ngược qua một cạnh đều ghi lại, lần lượt viết thành ngoặc trái và ngoặc phải. Khi đó độ dài của dãy ngoặc sau khi rút gọn giữa hai đỉnh chính là khoảng cách giữa hai đỉnh đó.

Chẳng hạn, với cây ví dụ trong bản gốc có các cạnh $1 - 2$, $1 - 5$, $2 - 3$ và $2 - 4$, thứ tự DFS là

[1 [2 [3] [4]] [5]].

Khi đó $\text{dist}(2, 4)$ bằng độ dài của dãy ngoặc giữa 2 và 4 sau khi rút gọn, tức 1; còn $\text{dist}(3, 4) = 2$.

Dùng cặp (A, B) để biểu diễn dãy ngoặc sau khi rút gọn:

$\underbrace{]]] \dots]}_A \underbrace{[[[\dots []}_B$.

Đường kính của cây chính là chọn một xâu con sao cho $A + B$ sau khi rút gọn là lớn nhất.

Khi gộp hai xâu, ở giữa sẽ sinh ra $\min(B_1, A_2)$ cặp ngoặc khớp nhau. Do đó

$$(A_1, B_1) + (A_2, B_2) = (A_1 + \max(A_2 - B_1, 0), B_2 + \max(B_1 - A_2, 0)),$$

và

$$A_0 + B_0 = \max(A_1 + B_1 + B_2 - A_2, A_1 - B_1 + A_2 + B_2).$$

Dùng splay để bảo trì trên đoạn thứ tự DFS các thông tin: giá trị lớn nhất của $A + B$, giá trị lớn nhất của $A + B$ trên hậu tố, giá trị lớn nhất của $A - B$ trên hậu tố, giá trị lớn nhất của $B - A$ trên tiền tố, giá trị lớn nhất của $A + B$ trên tiền tố, cùng với A, B của cả đoạn. Các thông tin này đều có thể tính qua lại.

Với thao tác đổi màu theo đường đi, áp dụng trực tiếp LCM là được. Độ phức tạp là $O(n \log^2 n)$.

5 Lời cuối

Sau khi thảo luận với bạn Zhao Yuyang từ THPT trực thuộc Đại học Sư phạm An Huy, tác giả được biết còn có một cách làm khác: từ góc nhìn Rake và Compress, bảo trì trên Top Tree các thông tin liên quan tới đỉnh biên của *cluster* và hai màu. Cách làm này cũng giải được bài toán bảo trì khối liên thông cùng màu trên cây có đổi màu theo đường đi, và độ phức tạp đạt $O(n \log n)$. Ở đây không giới thiệu thêm; bạn đọc quan tâm có thể tự tham khảo các bài viết liên quan.

6 Triển vọng

Bài viết đã nhắc tới khá nhiều phương pháp và kỹ thuật để giải bài toán khối liên thông trên cây, nhưng trong đại dương thuật toán rộng lớn, chúng vẫn chỉ là một góc nhỏ. Hy vọng trong tương lai sẽ có thêm nhiều thuật toán liên quan tới bài toán khối liên thông trên cây được phát hiện và sáng tạo, và cũng hy vọng sẽ có thêm nhiều bài toán khối liên thông trên cây thú vị xuất hiện trong các kỳ thi tin học.

7 Lời cảm ơn

- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
- Cảm ơn thầy Chen Heli và thầy Dong Yehua của Trường Trung học số 1 Thiệu Hưng đã quan tâm và hướng dẫn.
- Cảm ơn huấn luyện viên đội tuyển quốc gia Zhang Ruizhe đã hướng dẫn và giúp đỡ.
- Cảm ơn anh Chen Junkun của Đại học Thanh Hoa đã đề xuất nhiều thuật toán và kỹ thuật thực dụng cho bài toán quy hoạch động trên cây.
- Cảm ơn anh Tao Yuanzheng của Đại học Bắc Kinh và anh Ren Zhizhou của Đại học Thanh Hoa đã đóng góp vào nghiên cứu cấu trúc dữ liệu LCM.
- Cảm ơn bạn Zhao Yuyang từ THPT trực thuộc Đại học Sư phạm An Huy đã gợi mở cho tác giả về thuật toán Top Tree.
- Cảm ơn các anh Hong Huadun, Sun Yaofeng, Feng Zhe, Sun Yican, Ye Jianing của Đại học Bắc Kinh và các bạn Trường Trung học số 1 Thiệu Hưng đã giúp đỡ tác giả.
- Cảm ơn anh Sun Yican của Đại học Bắc Kinh đã đọc duyệt bài viết này.
- Cảm ơn các thầy cô và bạn học khác từng giúp đỡ, gợi mở cho tác giả.
- Cảm ơn cha mẹ đã quan tâm, ủng hộ và chăm sóc chu đáo.

8 Tài liệu tham khảo

1. Chen Junkun, *Cây tròn-vuông bình thường và quy hoạch động (động) kỳ diệu*, WC 2018.
2. Chen Junkun, *Báo cáo ra đề Đề thị con kỳ diệu và các mở rộng*, tập luận văn đội tuyển quốc gia năm 2017.
3. Tao Yuanzheng, “Mở rộng cây động”, linkcutmemphis.99293.html.
4. Ren Zhizhou, *Một thuật toán bảo trì khối liên thông cùng màu trên cây dựa trên LCT*, trao đổi học viên WC 2016.
5. Zhao Yuyang, *Bàn sơ lược về bảo trì thông tin trên cây động*.
6. Stephen Alstrup, Jacob Holm, Kristian de Lichtenberg, Mikkel Thorup, “Maintaining Information in Fully-Dynamic Trees with Top Trees”, 2003.
7. Robert E. Tarjan, Renato F. Werneck, “Self-Adjusting Top Trees”, 2005.

8 Bài 6: Báo cáo ra đề *Jellyfish* và khảo cứu mở rộng

Liang Yancheng, Trường Trung học Trần Hải, Ninh Ba

Tóm tắt

Jellyfish là một bài do tác giả ra trong đợt bài tập vòng một của đội tuyển, với tên gốc “Sửa? Sửa!”. Bài viết này trước hết giới thiệu thông tin đề bài và cách giải, sau đó dựa trên bài này để bàn một số bài toán tích chập đa thức và chuỗi lũy thừa tập hợp dưới góc nhìn phép nhân chia để trị.

1 Dẫn nhập

Trong những năm gần đây, các bài toán về đa thức xuất hiện ngày càng nhiều. Loại bài này linh hoạt, đa dạng, vừa có độ khó tư duy, vừa kiểm tra nền tảng toán học; một số bài còn kiểm nghiệm khả năng hiện thực của thí sinh. Vì vậy chúng ngày càng được người ra đề ưa chuộng.

Trong các bài toán đa thức, “tích chập” giữ một vị trí quan trọng. Khi giải bài toán tích chập, ta thường dùng tư tưởng biến đổi để tăng tốc. Các phép biến đổi quen thuộc như biến đổi Fourier nhanh (FFT), biến đổi Walsh nhanh (FWT) và biến đổi Mobius nhanh (FMT) đều là các ví dụ điển hình của cách dùng biến đổi để xử lý tích chập.

Phép nhân chia để trị là một phương pháp giải tích chập từ một góc nhìn khác. Thuật toán Karatsuba³ là phép nhân chia để trị kinh điển nhất; nó nhân hai đa thức bậc N trong thời gian xấp xỉ $O(N^{1.59})$. Vì độ phức tạp không bằng FFT nên nó không quá thường gặp, nhưng nó cung cấp một góc nhìn mới. Thực tế, khi dùng tư tưởng nhân chia để trị cho một số tích chập dưới các quy tắc phép toán bit, ta thường thu được hiệu quả khá tốt.

Mục 2 và 3 của bài viết giới thiệu bài *Jellyfish*, một bài dùng tư tưởng nhân chia để trị để giải tích chập, cùng lời giải của nó; đồng thời giới thiệu thuật toán Karatsuba và một cải tiến. Mục 4 đưa ra một số ví dụ ứng dụng của phép nhân chia để trị.

Để tiện trình bày, ta quy ước một số ký hiệu.

Ký hiệu 1.1. Với một biểu thức Boolean x ,

$$[x] = \begin{cases} 1, & x = \text{true}, \\ 0, & x = \text{false}. \end{cases}$$

Ký hiệu 1.2. Với một số thực x , $[x]$ biểu thị số nguyên lớn nhất không vượt quá x .

2 Thông tin đề bài

2.1 Tóm tắt đề

- Cho N điểm then chốt trong không gian 20 chiều. Với mỗi điểm, xem các tọa độ là một số cơ số d , rồi đổi sang thập phân và ký hiệu bằng a_i ; gọi a_i là mã vị trí của điểm đó. Giá trị tọa độ ở mỗi chiều của điểm then chốt thuộc tập $\{0, \dots, d-1\}$.
- Bây giờ chọn ngẫu nhiên một điểm then chốt làm điểm xuất phát, đi ngẫu nhiên K bước, mỗi bước đi tới một điểm kề với điểm hiện tại. Hai điểm kề nhau khi và chỉ khi chúng chỉ khác nhau đúng một chiều với độ lệch tọa độ bằng 1. Hãy tính kỳ vọng số điểm then chốt khác nhau đã đi qua.

2.2 Giới hạn dữ liệu, thời gian và bộ nhớ

$$1 \leq N \leq 200000, \quad 0 \leq K \leq 500, \quad d \in \{2, 4\}, \quad 0 \leq a_i \leq 250000.$$

³https://en.wikipedia.org/wiki/Karatsuba_algorithm

Có 25% dữ liệu thỏa $1 \leq N \leq 3000$. Ngoài ra có 15% dữ liệu thỏa $d = 2$. Ngoài ra có 20% dữ liệu thỏa $1 \leq K \leq 4$, trong đó 10% thỏa $d = 2$.

Giới hạn thời gian: 4 giây trên TUOJ.

Giới hạn bộ nhớ: 512 MB.

3 Lời giải đề bài

3.1 Phân tích ngắn

Gọi $e_{x,y}$ là xác suất xuất phát từ điểm x và đi qua điểm y trong K bước; ở đây x, y có thể trùng nhau. Khi đó đáp án cần tìm là

$$\sum_{i=1}^N \sum_{j=1}^N e_{i,j}.$$

Theo tính tuyến tính của kỳ vọng, ta có thể độc lập tính tất cả các $e_{x,y}$, rồi cộng lại để nhận đáp án.

Gọi $\text{ways}_{x,y}$ là số phương án xuất phát từ điểm x và đi qua điểm y trong K bước. Khi đó

$$e_{x,y} = \frac{\text{ways}_{x,y}}{N \times 40^N}.$$

Vì vậy ta chỉ cần tính $\text{ways}_{x,y}$ cho từng cặp x, y , cộng lại rồi chia cho $N \times 40^N$.

3.2 Lời giải vét cạn

3.2.1 Vét cạn đơn giản. Trước hết xét cách liệt kê một điểm x , rồi trực tiếp liệt kê ở từng bước trong K bước tiếp theo sẽ di chuyển theo chiều nào, chiều đó tăng 1 hay giảm 1. Khi lần đầu đi qua một điểm nào đó thì cập nhật đáp án.

Độ phức tạp thời gian là $O(N40^K)$.

3.2.2 Cải tiến 1. Chú ý độ phức tạp của thuật toán trên là $O(N40^K)$. Nếu có thể làm K nhỏ đi một chút thì thời gian sẽ cải thiện đáng kể.

Xét dùng tư tưởng tìm kiếm chia đôi. Giả sử tại bước thứ s ta đi từ x tới y . Có thể thay quá trình này bằng cách trước hết đi từ x trong $\lfloor s/2 \rfloor$ bước tới một điểm p , rồi đi ngược từ y trong $\lfloor s/2 \rfloor$ bước cũng tới điểm p .

Như vậy, trước hết ta liệt kê tổng số bước s , ghi lại mọi điểm có thể đạt được khi đi ngược $\lfloor s/2 \rfloor$ bước từ mọi y , sau đó mỗi lần liệt kê điểm xuất phát x và $\lfloor s/2 \rfloor$ bước tiếp theo để thống kê đáp án.

Cách thống kê này có thể bị đếm trùng, nhưng vì nó chỉ nhắm tới trường hợp K rất nhỏ, ta chỉ cần thảo luận đơn giản là có thể trừ phần trùng lặp.

Độ phức tạp thời gian giảm xuống $O(NK40^{K/2})$, đủ qua 20% dữ liệu.

3.2.3 Cải tiến 2. Chú ý rằng với cặp điểm (x, y) , ta không quan tâm vị trí cụ thể của x, y , mà chỉ quan tâm vị trí tương đối của chúng.

Vì vậy trước hết liệt kê $O(N^2)$ cặp điểm (x, y) , rồi xem x là gốc tọa độ và đưa vị trí tương ứng của y vào một đa tập S . Sau đó bài toán có thể xem là: xuất phát từ gốc tọa độ, với mỗi điểm trong đa tập S , tính số phương án đi K bước và đi qua điểm đó.

Nếu dùng băm để bảo trì S và truy vấn số lần xuất hiện của một điểm trong S , độ phức tạp thời gian có thể đạt $O(N^2 + 40^K)$.

3.3 Phần tiền xử lý

Cải tiến 2 của thuật toán vét cạn cho ta một hướng đi: nếu có thể nhanh chóng tiền xử lý đáp án cho mọi vị trí tương đối có thể có, thì ta có thể tính đáp án cuối cùng trong $O(N^2)$.

Nói cách khác, ta cần giải bài toán sau:

Cho một điểm then chốt, hỏi xuất phát từ gốc tọa độ $(0, 0, \dots, 0)$, đi K bước thì có bao nhiêu phương án đi qua điểm then chốt này.

Ta xét dùng quy hoạch động để tiền xử lý đáp án cho mọi trường hợp có thể. Để tiện, giả sử $d = 4$; trường hợp $d = 2$ thực ra có thể được bao hàm bởi $d = 4$.

Bổ đề 3.1. Sau khi hoán đổi hai chiều bất kỳ của một điểm then chốt, số phương án xuất phát từ gốc tọa độ và đi qua điểm then chốt đó trong K bước không đổi.

Chứng minh. Xét một điểm then chốt

$$P = (a_1, \dots, u, \dots, v, \dots, a_n)$$

và điểm tương ứng sau khi hoán đổi u, v ,

$$P' = (a_1, \dots, v, \dots, u, \dots, a_n).$$

Xây dựng một ánh xạ $A(x)$ thỏa

$$A(x) = \begin{cases} v, & x = u, \\ u, & x = v, \\ x, & x \neq u, x \neq v. \end{cases}$$

Ta dùng (t_i, d_i) để biểu thị bước thứ i đi theo chiều thứ t_i , và lượng thay đổi trên chiều đó là d_i . Nếu ta đi qua P bằng $(t_1, d_1), \dots, (t_K, d_K)$, thì có thể đi qua P' bằng $(A(t_1), d_1), \dots, (A(t_K), d_K)$. Chiều ngược lại cũng tương tự. Vì vậy các phương án đi qua P và đi qua P' tương ứng một-một, và số phương án bằng nhau. ■

Bổ đề 3.2. Sau khi lấy đối của một chiều nào đó của một điểm then chốt, số phương án xuất phát từ gốc tọa độ và đi qua điểm then chốt đó trong K bước không đổi.

Chứng minh. Giả sử lấy đối chiều thứ k . Chỉ cần định nghĩa hàm $B(x)$:

$$B(x) = \begin{cases} -1, & x = k, \\ 1, & x \neq k. \end{cases}$$

Sau đó thay (t_i, d_i) bằng $(t_i, B(t_i)d_i)$ là được; các phần còn lại dùng cùng phương pháp chứng minh như bổ đề trước. ■

Xây dựng một mảng phụ trợ b , trong đó b_i biểu thị điểm then chốt này có bao nhiêu chiều mà trị tuyệt đối của tọa độ bằng i . Theo Bổ đề 3.1 và Bổ đề 3.2, nếu hai điểm then chốt có cùng mảng b , đáp án của chúng bằng nhau.

Đồng thời, vì $b_0 + b_1 + b_2 + b_3 = 20$, ta có thể dùng dp_{b_1, b_2, b_3} để biểu thị số phương án xuất phát từ gốc tọa độ, đi K bước và đi qua một điểm then chốt có mảng

$$b = \{20 - b_1 - b_2 - b_3, b_1, b_2, b_3\}.$$

Trước khi tính mảng dp , ta cần một số chuẩn bị.

- Trước hết, cần một mảng $f_{i,j,k}$, biểu thị số phương án đi từ gốc tọa độ trong không gian j chiều, đi k bước và tới $(i, i, \dots, i)$.
- Khi đó có thể liệt kê trên chiều thứ j đã đi x lần theo hướng -1 và $x + i$ lần theo hướng $+1$, nhận được chuyển tiếp

$$f_{i,j,k} = \sum_x \binom{k}{x+x+i} \binom{x+x+i}{x} f_{i,j-1,k-x-(x+i)}.$$

Tiền xử lý tất cả các giá trị f mất $O(pdK^2)$, trong đó p là số chiều của không gian; ở bài này $p = 20$. Bây giờ xét một mảng b và cần tính mảng dp tương ứng.

- Trước hết tính một mảng h , trong đó $h_{i,j}$ biểu thị trong $b_0 + \dots + b_i$ chiều, số phương án xuất phát từ gốc tọa độ, đi j bước và tới điểm

$$(0, \dots, 0 \text{ (} b_0 \text{ số } 0), 1, \dots, 1 \text{ (} b_1 \text{ số } 1), \dots, i, \dots, i \text{ (} b_i \text{ số } i)).$$

- Liệt kê k là số bước đi trên các chiều ứng với b_i , ta có

$$h_{i,j} = \sum_k \binom{j}{k} h_{i-1,j-k} f_{i,b_i,k}.$$

- Gọi g_i là số phương án đi i bước và vừa đúng tới điểm then chốt này, tức lần đầu tới điểm then chốt là ở bước thứ i . Dùng bao hàm-loại trừ: lấy tổng số phương án $h_{d,i}$ trừ các phần không hợp lệ.
- Với phần không hợp lệ, chắc chắn trước đó đã đi qua điểm then chốt. Liệt kê lần đầu trước đó đi tới điểm then chốt là bước thứ j , nhận được

$$g_i = h_{d,i} - \sum_{j < i} g_j f_{0,20,i-j}.$$

- Khi đó

$$dp_{b_1,b_2,b_3} = \sum_i g_i \cdot 40^{K-i}.$$

Toàn bộ phần tiền xử lý có độ phức tạp thời gian $O(p^3 d K^2)$, nhưng hằng số rất nhỏ nên có thể tính trong thời gian ngắn.

Nếu sau khi tiền xử lý, ta liệt kê $O(N^2)$ cặp điểm và dùng mảng dp để tính đáp án, có thể qua 25% dữ liệu.

3.4 Phân tích thêm

Khi đã có mảng dp , ta chỉ cần tính một mảng lưu chênh lệch khoảng cách giữa mọi cặp trong N điểm, cộng số phương án của tất cả các cặp, rồi chia cho $N \times 40^N$.

Định nghĩa 3.1. Định nghĩa toán tử $\oplus$ là hàm chênh lệch khoảng cách của hai mã vị trí trong cơ số d . Cụ thể:

$$\overline{(a_1 b_1 c_1 \dots)_d} \oplus \overline{(a_2 b_2 c_2 \dots)_d} = \overline{(|a_1 - a_2| |b_1 - b_2| |c_1 - c_2| \dots)_d}.$$

Viết hình thức:

$$x \oplus y = \sum_{k=0}^{\infty} \left(\left\lfloor \frac{x}{d^k} \right\rfloor - \left\lfloor \frac{y}{d^k} \right\rfloor \right) \bmod d \cdot d^k.$$

Định nghĩa 3.2. Với hai dãy f, g có độ dài N , định nghĩa $f \oplus g = a$, trong đó dãy a thỏa

$$a_i = \sum_{j=0}^{N-1} \sum_{k=0}^{N-1} [j \oplus k = i] f_j g_k.$$

Có thể thấy, với mảng a ban đầu, nếu ta xây dựng mảng phụ trợ a' sao cho a'_i biểu thị trong mảng a có bao nhiêu mã vị trí bằng i , rồi tính $a' \oplus a'$, ta nhận được đúng “mảng chênh lệch khoảng cách” cần tìm.

Khi $d = 2$, $\oplus$ tương đương với xor, nên có thể dùng biến đổi Walsh nhanh để qua phần dữ liệu $d = 2$ trong giới hạn thời gian.

Khi $d = 4$, ta có thể xét mở rộng cơ số d thành cơ số $2d - 1$, để mỗi chiều có phạm vi $[-(d - 1), d - 1]$. Như vậy ta có thể bỏ trị tuyệt đối trong hàm chênh lệch khoảng cách, và trực tiếp đổi hàm chênh lệch khoảng cách thành phép trừ hai số.

Sau khi mở rộng cơ số d thành $2d - 1$, độ dài dãy mới mở rộng thành

$$M = (\max a_i)^{\log_d(2d-1)}.$$

Dùng biến đổi Fourier nhanh để thu được mảng chênh lệch khoảng cách có độ phức tạp thời gian

$$O(M \log M) = O((\max a_i)^{\log_d(2d-1)} \log \max a_i) \approx O((\max a_i)^{1.40} \log \max a_i).$$

Cách này khá khó qua bài trong giới hạn thời gian đã cho, nên ta xét dùng phép nhân chia để trị.

3.5 Bài toán nhân đa thức

Xét bài toán nhân đa thức kinh điển: cho hai dãy a, b có độ dài N , hãy tìm dãy c thỏa

$$c_i = \sum_{j=0}^i a_j b_{i-j}.$$

Đặt

$$A(x) = \sum a_k x^k, \quad B(x) = \sum b_k x^k, \quad C(x) = \sum c_k x^k.$$

Khi đó bài toán tương đương với tìm hệ số của đa thức $C(x)$ nhận được từ tích hai đa thức $A(x), B(x)$.

3.5.1 Thuật toán Karatsuba. Với bài toán nhân đa thức, thông thường ta dùng biến đổi Fourier nhanh để đổi a, b sang dạng giá trị tại điểm, lấy được giá trị tại điểm của c , rồi nội suy để thu được dãy c .

Thuật toán Karatsuba cung cấp một hướng khác: dùng chia để trị để giải nhân đa thức. Quy trình cơ bản như sau; để tiện, giả sử N là lũy thừa của 2.

- Tách $A(x)$ thành hai đa thức $A_0(x), A_1(x)$ thỏa $A(x) = A_0(x) + A_1(x)x^{N/2}$.
- Tương tự, tách $B(x)$ thành hai đa thức $B_0(x), B_1(x)$ thỏa $B(x) = B_0(x) + B_1(x)x^{N/2}$.
- Khi đó

$$C(x) = A(x)B(x) = A_0(x)B_0(x) + (A_0(x)B_1(x) + A_1(x)B_0(x))x^{N/2} + A_1(x)B_1(x)x^N.$$

- Tiếp theo cần tính nhanh hạng $A_0(x)B_1(x) + A_1(x)B_0(x)$.
- Chú ý tổng của ba hạng khá dễ tính:

$$A_0B_0 + A_0B_1 + A_1B_0 + A_1B_1 = (A_0 + A_1)(B_0 + B_1).$$

- Vì vậy chỉ cần tính $(A_0(x) + A_1(x))(B_0(x) + B_1(x))$, ta nhận được

$$A_0B_1 + A_1B_0 = (A_0 + A_1)(B_0 + B_1) - A_0B_0 - A_1B_1.$$

- Do đó chỉ cần giải ba bài toán con có kích thước bằng một nửa ban đầu: $A_0(x)B_0(x)$, $A_1(x)B_1(x)$, và $(A_0(x) + A_1(x))(B_0(x) + B_1(x))$.
- Từ đó có thể khôi phục $C(x)$.

Phân tích đơn giản độ phức tạp thời gian cho ta

$$T(N) = 3T\left(\frac{N}{2}\right) + O(N) = O(N^{\log_2 3}) \approx O(N^{1.59}).$$

Với $N = 100000$, bậc tối ưu $O2$, thuật toán này chạy trên UOJ khoảng 0.7 giây. Nếu xét nhân đa thức theo mô-đun, thời gian chạy xấp xỉ 1.5 ~ 2 lần thuật toán NTT ba mô-đun.⁴

3.5.2 Cải tiến thuật toán Karatsuba. Khi giải nhân đa thức, thuật toán Karatsuba tách đa thức thành hai phần để tăng tốc độ tính.

Vậy nếu ta tách đa thức thành $\gamma + 1$ phần, trong đó γ là hằng số, liệu có nhận được độ phức tạp thời gian tốt hơn không?

Xét hai đa thức bậc $(\gamma + 1)n - 1$ có hệ số lần lượt là $a_0, \dots, a_{(\gamma+1)n-1}$ và $b_0, \dots, b_{(\gamma+1)n-1}$. Ta cắt a, b thành $\gamma + 1$ đoạn độ dài n , ký hiệu $A_0, \dots, A_\gamma$ và $B_0, \dots, B_\gamma$. Đặt $p = x^n$, và

$$A(p) = \sum_{k=0}^{\gamma} A_k p^k, \quad B(p) = \sum_{k=0}^{\gamma} B_k p^k.$$

Khi đó

$$C(p) = A(p)B(p), \quad C_i = \sum_{j=0}^i A_j B_{i-j}.$$

Vì

$$C(p) = \sum_{k=0}^{2\gamma} C_k p^k$$

là một đa thức bậc 2γ theo p , ta xét lấy $2\gamma + 1$ giá trị tại điểm rồi khôi phục $C(p)$.

Khi lấy $p = p_0$ và thế vào, ta nhận được một dãy mới a' có độ dài n , trong đó

$$a'_i = \sum_{k=0}^{\gamma} a_{i+kn} p_0^k.$$

Tương tự thu được b' . Khi đó cần nhân hai đa thức có hệ số là a' và b' , nên kích thước bài toán giảm còn $\frac{1}{\gamma+1}$ ban đầu.

Ngoài phần đệ quy, mọi phần còn lại đều có thể hoàn thành trong $O(N)$. Do đó độ phức tạp là

$$T(N) = (2\gamma + 1)T\left(\frac{N}{\gamma + 1}\right) + O(N) = O\left(N^{\log_{\gamma+1}(2\gamma+1)}\right).$$

Từ đó ta chứng minh được định lý sau.⁵

⁴Thuật toán nhanh về mặt này có thể xem trong luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015 của Mao Xiao, *Xét lại biến đổi Fourier nhanh*.

⁵The Art of Computer Programming, tập 2, Mục 4.3.3, Định lý A.

Định lý 3.1. Với mọi $\varepsilon > 0$, tồn tại một hằng số $c(\varepsilon)$ không phụ thuộc vào N và một thuật toán nhân sao cho

$$T(N) < c(\varepsilon)N^{1+\varepsilon}.$$

Thực tế, thuật toán Karatsuba có thể xem là trường hợp đặc biệt $\gamma = 1$.

3.6 Lời giải cuối cùng

Quay lại bài toán ban đầu. Ta thử áp dụng tư tưởng của thuật toán Karatsuba vào bài này. Xét cách tính $a = f \oplus g$, trong đó f, g có độ dài N .

Trong thuật toán Karatsuba, ta tăng tốc bằng cách tách dãy thành hai phần. Vậy trong cơ sở d , ta xét tách dãy theo chữ số cao nhất ở cơ sở d . Dùng a_i để biểu thị phần của dãy a có chữ số cao nhất trong cơ sở d bằng i . Tách riêng a, f, g , ta thấy với trường hợp $d = 4$:

- $a_0 = f_0 \oplus g_0 + f_1 \oplus g_1 + f_2 \oplus g_2 + f_3 \oplus g_3$.
- $a_1 = f_0 \oplus g_1 + f_1 \oplus g_0 + f_1 \oplus g_2 + f_2 \oplus g_1 + f_2 \oplus g_3 + f_3 \oplus g_2$.
- $a_2 = f_0 \oplus g_2 + f_2 \oplus g_0 + f_1 \oplus g_3 + f_3 \oplus g_1$.
- $a_3 = f_0 \oplus g_3 + f_3 \oplus g_0$.

Để đơn giản hóa các công thức trên, trước hết tách các hạng lẻ và chẵn:

$$x' = (f_0 + f_1 + f_2 + f_3) \oplus (g_0 + g_1 + g_2 + g_3),$$

$$y' = (f_0 - f_1 + f_2 - f_3) \oplus (g_0 - g_1 + g_2 - g_3).$$

Sau đó đặt $x = (x' + y')/2$, $y = (x' - y')/2$. Khi đó

$$x = \sum_{i=0}^3 \sum_{j=0}^3 [i \equiv j \pmod{2}] f_i \oplus g_j, \quad y = \sum_{i=0}^3 \sum_{j=0}^3 [i \not\equiv j \pmod{2}] f_i \oplus g_j.$$

Nói cách khác, $x = a_0 + a_2$, $y = a_1 + a_3$.

Bây giờ xét trực tiếp tính a_3 . Từ

$$u' = (f_0 + f_3) \oplus (g_0 + g_3), \quad v' = (f_0 - f_3) \oplus (g_0 - g_3),$$

đặt $u = (u' + v')/2$, $v = (u' - v')/2$, ta có

$$u = f_0 \oplus g_0 + f_3 \oplus g_3, \quad v = f_0 \oplus g_3 + f_3 \oplus g_0.$$

Do đó $a_3 = v$, $a_1 = y - a_3$.

Bây giờ ta chỉ cần tính một trong hai giá trị a_0, a_2 . Quan sát u đã tính ở trên, ta có thể xét tính $f_1 \oplus g_1$ và $f_2 \oplus g_2$. Khi đó

$$a_0 = u + f_1 \oplus g_1 + f_2 \oplus g_2,$$

và $a_2 = x - a_0$.

Vì vậy ta tách một bài toán kích thước N thành 6 bài toán con kích thước $N/4$ để tính đệ quy, nhận được độ phức tạp thời gian

$$T(N) = 6T\left(\frac{N}{4}\right) + O(N) = O(N^{\log_4 6}) \approx O(N^{1.29}).$$

Trong bài toán gốc, $N = \max a_i$, nên có thể ước lượng độ phức tạp là

$$O((\max a_i)^{1.29}).$$

Tóm lại, ta đã giải trọn vẹn bài toán.

4 Phép nhân chia để trị và tích chập bit

Qua bài *Jellyfish*, có thể thấy khi áp dụng tư tưởng nhân chia để trị vào tích chập bit, ta thường nhận được hiệu quả tốt.

Tiếp theo tác giả dùng một số ví dụ để trình bày thêm các ứng dụng của phép nhân chia để trị trong tích chập bit.⁶

4.1 Tích chập dưới quy tắc and, or

Định nghĩa 4.1. Cho hai dãy a, b có chỉ số là các số nhị phân n bit, định nghĩa $a \cdot b = c$, trong đó dãy c thỏa

$$c_i = \sum_{j=0}^{2^n-1} \sum_{k=0}^{2^n-1} [j \text{ and } k = i] a_j b_k.$$

Ta gọi c là tích chập của hai dãy a, b dưới quy tắc and.

Bắt chước thuật toán Karatsuba, tách các dãy a, b theo bit cao nhất thành hai dãy a_0, a_1 và b_0, b_1 , đồng thời tách c thành c_0, c_1 . Khi đó:

- $c_0 = a_0 \cdot b_0 + a_0 \cdot b_1 + a_1 \cdot b_0$.
- $c_1 = a_1 \cdot b_1$.
- Từ $c_0 + c_1 = (a_0 + a_1) \cdot (b_0 + b_1)$, suy ra

$$c_0 = (a_0 + a_1) \cdot (b_0 + b_1) - c_1.$$

Như vậy chỉ cần tính đệ quy $(a_0 + a_1) \cdot (b_0 + b_1)$ và $a_1 \cdot b_1$, độ phức tạp thời gian là

$$T(n) = 2T(n-1) + O(2^n) = O(n2^n).$$

Tích chập dưới quy tắc or về bản chất giống and. Chỉ cần hoán đổi 0, 1 trong biểu diễn nhị phân của chỉ số trên cơ sở lời giải and là thu được lời giải cho quy tắc or.

4.2 Tích chập dưới quy tắc xor nhị phân

Định nghĩa 4.2. Cho hai dãy a, b có chỉ số là các số nhị phân n bit, định nghĩa $a \cdot b = c$, trong đó dãy c thỏa

$$c_i = \sum_{j=0}^{2^n-1} \sum_{k=0}^{2^n-1} [j \text{ xor } k = i] a_j b_k.$$

Ta gọi c là tích chập của hai dãy a, b dưới quy tắc xor.

Tương tự, tách ba dãy. Khi đó:

- $c_0 = a_0 \cdot b_0 + a_1 \cdot b_1$.
- $c_1 = a_0 \cdot b_1 + a_1 \cdot b_0$.
- Từ $c_0 + c_1 = (a_0 + a_1) \cdot (b_0 + b_1)$, suy ra

$$c_1 = (a_0 + a_1) \cdot (b_0 + b_1) - c_0.$$

⁶Nội dung Mục 4.1 và 4.2 cũng được thảo luận trong luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015 của Lu Kaifeng, *Tính chất, ứng dụng và thuật toán nhanh của chuỗi lũy thừa tập hợp*.

Nếu làm như vậy, cần tính đệ quy $a_0 \cdot b_0$, $a_1 \cdot b_1$, và $(a_0 + a_1) \cdot (b_0 + b_1)$, nên độ phức tạp thời gian là

$$T(n) = 3T(n-1) + O(2^n) = O(3^n).$$

Tất nhiên, có thể xây dựng các đa thức để đệ quy tính, từ đó nhận lời giải tốt hơn:

- $x = (a_0 + a_1) \cdot (b_0 + b_1)$.
- $y = (a_0 - a_1) \cdot (b_0 - b_1)$.
- Khi đó $c_0 = (x + y)/2$, $c_1 = (x - y)/2$.

Như vậy chỉ cần tính đệ quy $(a_0 + a_1) \cdot (b_0 + b_1)$ và $(a_0 - a_1) \cdot (b_0 - b_1)$, độ phức tạp thời gian là

$$T(n) = 2T(n-1) + O(2^n) = O(n2^n).$$

4.3 Tích chập dưới quy tắc xor cơ sở K

Định nghĩa 4.3. Định nghĩa phép xor trong cơ sở K , ký hiệu xor_K , là kết quả cộng từng chữ số trong cơ sở K mà không nhớ. Viết hình thức:

$$x \text{ xor}_K y = \sum_{i=0}^{\infty} \left(\left\lfloor \frac{x}{K^i} \right\rfloor + \left\lfloor \frac{y}{K^i} \right\rfloor \bmod K \right) K^i.$$

Định nghĩa 4.4. Cho hai dãy a, b có độ dài N , định nghĩa $a \cdot b = c$, trong đó dãy c thỏa

$$c_i = \sum_{j=0}^{N-1} \sum_{k=0}^{N-1} [j \text{ xor}_K k = i] a_j b_k.$$

Ta gọi c là tích chập của hai dãy a, b dưới quy tắc xor cơ sở K .

4.3.1 Cách giải. Thông thường, ta dùng biến đổi Fourier nhanh nhiều chiều, mỗi chiều có kích thước K , để giải bài toán này. Tương tự, ta cũng có thể dùng phép nhân chia để trị để giải bài toán nhân đa thức nhiều chiều với mỗi chiều có kích thước K .

Tách dãy a, b theo chữ số đầu tiên trong cơ sở K thành K dãy $(a_0, a_1, \dots, a_{K-1})$ và $(b_0, b_1, \dots, b_{K-1})$; tách dãy c tương tự. Khi đó

$$c_i = \sum_{j=0}^{K-1} \sum_{k=0}^{K-1} [j + k \equiv i \pmod{K}] a_j \cdot b_k.$$

Xây dựng $2K - 1$ dãy $d_0, d_1, \dots, d_{2K-2}$ thỏa

$$d_i = \sum_{j=0}^{K-1} \sum_{k=0}^{K-1} [j + k = i] a_j \cdot b_k.$$

Khi đó

$$c_i = \sum_j [j \equiv i \pmod{K}] d_j.$$

Vì vậy ta chuyển sang tìm dãy d . Xây dựng các đa thức

$$A(x) = \sum a_k x^k, \quad B(x) = \sum b_k x^k, \quad D(x) = \sum d_k x^k.$$

Khi đó chỉ cần tính $D(x) = A(x)B(x)$. Có thể dùng thuật toán Karatsuba để biến nó thành $t(K)K^{1.59}$ bài toán con và giải đệ quy, trong đó $t(K)$ là hệ số hằng.

Giả sử N là lũy thừa của K , độ phức tạp thời gian có thể ước lượng là

$$T(N) \approx t(K)K^{1.59}T\left(\frac{N}{K}\right) + t(K)K^{1.59}O\left(\frac{N}{K}\right) = O\left(N^{\log_K(t(K)K^{1.59})}\right) = O\left(N^{1.59+\log_K t(K)}\right).$$

Khi K tương đối lớn, $\log_K t(K)$ có thể bỏ qua; độ phức tạp xấp xỉ $O(N^{1.59})$.⁷

4.3.2 Một tối ưu. Vì thực ra ta chỉ cần tìm dãy c , không cần tìm toàn bộ dãy d , nên khi K là số chẵn ta có thể dùng một số tối ưu.

Chú ý trong lần tách đầu tiên của thuật toán Karatsuba, ta làm như sau:

$$D(x) = A_0(x)B_0(x) + (A_0(x)B_1(x) + A_1(x)B_0(x))x^{K/2} + A_1(x)B_1(x)x^K.$$

Xét các d có chỉ số đồng dư theo mô-đun K ; chúng được cộng vào cùng một c_i . Vì vậy ta có thể lấy $D(x)$ theo mô-đun x^K , tức là

$$D(x) = A_0(x)B_0(x) + A_1(x)B_1(x) + (A_0(x)B_1(x) + A_1(x)B_0(x))x^{K/2}.$$

Xây dựng hai đa thức $u(x), v(x)$ thỏa

$$u(x) = (A_0(x) + A_1(x))(B_0(x) + B_1(x)),$$

$$v(x) = (A_0(x) - A_1(x))(B_0(x) - B_1(x)).$$

Khi đó

$$D(x) = \frac{u(x) + v(x)}{2} + \frac{u(x) - v(x)}{2}x^{K/2}.$$

Như vậy ta đã giảm $t(K)$ xuống còn $2/3$ ban đầu; khi K nhỏ, hiệu quả khá rõ.

4.4 Tích chập dưới quy tắc bit phức tạp hơn

Ngay cả với các quy tắc bit phức tạp hơn, phép nhân chia để trị cũng thường cho ta một thuật toán khá tốt.

Xét ví dụ sau.

Định nghĩa 4.5. Định nghĩa toán tử $x * y$ là kết quả lấy lũy thừa từng chữ số của hai số thập phân x, y mà không nhớ. Viết hình thức:

$$x * y = \sum_{k=0}^{\infty} \left(\left\lfloor \frac{x}{10^k} \right\rfloor \left\lfloor \frac{y}{10^k} \right\rfloor \bmod 10 \right) 10^k.$$

Đặc biệt, đặt $0^0 = 0$.

Ta định nghĩa tích chập dưới quy tắc $*$ như sau.

Định nghĩa 4.6. Cho hai dãy a, b có độ dài N , định nghĩa $a \cdot b = c$, trong đó dãy c thỏa

$$c_i = \sum_{j=0}^{N-1} \sum_{k=0}^{N-1} [j * k = i] a_j b_k.$$

Ta gọi c là tích chập của a, b dưới quy tắc $*$.

⁷Ở đây cũng có thể dùng phương pháp nhắc tới trong Mục 3.5.2 để cải thiện độ phức tạp thời gian.

4.4.1 Cách giải. Có thể thấy toán tử $*$ hầu như không có tính chất tốt nào: giao hoán, kết hợp hay phân phối đều không thỏa.

Tuy nhiên ta vẫn có thể dùng tư tưởng nhân chia để trị để giảm độ phức tạp xuống dưới $O(N^2)$.

Xét hai số $x, y \in [0, 10)$. Ta có thể gộp một số cặp x, y có cùng giá trị $x^y \bmod 10$. Chẳng hạn khi $x = 0, 1, 5, 6$, với mọi số nguyên dương y , giá trị x^y đều không đổi. Dựa trên tư tưởng “gộp”, ta có thể thử giảm độ phức tạp thời gian.

Trước hết vẫn tách các dãy a, b, c theo chữ số cao nhất trong cơ số 10, mỗi dãy thành 10 dãy.

Để tiện mô tả, ta quy ước một số ký hiệu.

Ký hiệu 4.1. Đặt $U = \{0, 1, \dots, 9\}$, $V = \{1, 2, \dots, 9\}$.

Ký hiệu 4.2. Với hai tập $S, T \subseteq U$, ký hiệu

$$g_{S,T} = \sum_{i \in S} \sum_{j \in T} a_i \cdot b_j.$$

Có thể thấy

$$g_{S,T} = \left(\sum_{i \in S} a_i \right) \cdot \left(\sum_{j \in T} b_j \right).$$

Nghĩa là với S, T đã cho, chỉ cần làm đệ quy một bài toán con có kích thước bằng $1/10$ bài toán ban đầu để tính $g_{S,T}$.

Ký hiệu 4.3. Với $x, y \in V$, đặt

$$F(x, y) = \{i \mid i \in V, x^i \equiv y \pmod{10}\},$$

$$h_{x,y} = \sum_{i \in F(x,y)} a_x \cdot b_i.$$

Hiển nhiên $h_{x,y} = g_{\{x\}, F(x,y)}$, nên chỉ cần một lần đệ quy để tính $h_{x,y}$.

Chú ý rằng với một x cố định, không có quá nhiều y thỏa $F(x, y) \neq \emptyset$. Xét lần lượt từng $x \in V$:

- Khi $x = 1$, chỉ cần tính $h_{x,1}$.
- Khi $x = 2$ hoặc $x = 8$, chỉ cần tính $h_{x,2}, h_{x,4}, h_{x,6}, h_{x,8}$.
- Khi $x = 3$ hoặc $x = 7$, chỉ cần tính $h_{x,1}, h_{x,3}, h_{x,7}, h_{x,9}$.
- Khi $x = 4$, chỉ cần tính $h_{x,4}, h_{x,6}$.
- Khi $x = 5$, chỉ cần tính $h_{x,5}$.
- Khi $x = 6$, chỉ cần tính $h_{x,6}$.
- Khi $x = 9$, chỉ cần tính $h_{x,1}, h_{x,9}$.

Có thể thấy chỉ có 23 cặp có thứ tự $x, y \in V$ thỏa $F(x, y) \neq \emptyset$. Ta trực tiếp đệ quy vét cạn 23 bài toán con để tính các $h_{x,y}$ ở trên.

Bây giờ giả sử đã có $d_1, \dots, d_9$ thỏa

$$d_k = \sum_{x \in V} h_{x,k}.$$

Tiếp theo xét trường hợp $x = 0$ hoặc $y = 0$:

- Khi $x = 0$, $x^y \equiv 0 \pmod{10}$.

- Khi $x \neq 0$ và $y = 0$, $x^y \equiv 1 \pmod{10}$.

Thêm hai trường hợp trên vào dãy d , ta nhận được mảng c cuối cùng:

- $c_0 = g_{\{0\}, U}$.
- $c_1 = d_1 + g_{V, \{0\}}$.
- Với các c_k khác, $c_k = d_k$.

Như vậy chỉ cần đệ quy 25 bài toán con kích thước $1/10$ bài toán ban đầu là có thể thu được dãy c . Độ phức tạp thời gian là

$$T(N) = 25T\left(\frac{N}{10}\right) + O(N) = O(N^{\log_{10} 25}) \approx O(N^{1.40}).$$

4.4.2 Một số tối ưu đơn giản. Trong cách làm trên, ta tính $h_{x,y}$ đơn giản theo x , và thu được một thuật toán.

Thực ra khi tìm dãy d , ta cộng các $h_{x,y}$ theo y . Vì vậy một số $h_{x,y}$ có thể được gộp, chẳng hạn:

- $h_{2,4} + h_{8,4} = g_{\{2,8\}, \{2,6\}}$.
- $h_{2,6} + h_{8,6} = g_{\{2,8\}, \{4,8\}}$.
- $h_{3,9} + h_{7,9} = g_{\{3,7\}, \{2,6\}}$.
- $h_{3,1} + h_{7,1} = g_{\{3,7\}, \{4,8\}}$.

Như vậy giảm được 4 lần đệ quy, độ phức tạp thời gian trở thành

$$T(N) = 21T\left(\frac{N}{10}\right) + O(N) = O(N^{\log_{10} 21}) \approx O(N^{1.32}).$$

Ngoài ra, ta thấy

- $h_{2,2} + h_{8,2} + h_{2,8} + h_{8,8} = g_{\{2,8\}, \{1,3,5,7,9\}}$.
- $h_{2,2} + h_{8,2} - h_{2,8} - h_{8,8} = (a_2 - a_8) \cdot (b_1 - b_3 + b_5 - b_7 + b_9)$.

Vì vậy chỉ cần hai lần đệ quy là lần lượt tính được $h_{2,2} + h_{8,2}$ và $h_{2,8} + h_{8,8}$, giảm thêm hai lần đệ quy. Độ phức tạp thời gian tiếp tục giảm thành

$$T(N) = 19T\left(\frac{N}{10}\right) + O(N) = O(N^{\log_{10} 19}) \approx O(N^{1.28}).$$

4.5 Hàm có nhiều dãy làm biến

Tích chập thông thường lấy hai dãy làm biến. Nếu có ba dãy hoặc nhiều hơn làm biến, rất khó tăng tốc bằng một kiểu biến đổi nào đó. Nhưng dùng phép nhân chia để trị, ta vẫn có thể tìm ra một thuật toán tốt hơn vết cặn trực tiếp.

Xét ví dụ sau.

Định nghĩa 4.7. Cho một hàm $f(x, y, z)$ có miền xác định $\{(x, y, z) \mid x, y, z \in \{0, 1\}\}$, miền giá trị $\{0, 1\}$, và thỏa $f(0, 0, 0) = 0$.

Định nghĩa hàm $g(x, y, z)$ là kết quả áp dụng f theo từng bit của x, y, z . Viết hình thức:

$$g(x, y, z) = \sum_{k=0}^{\infty} 2^k f(x_k, y_k, z_k),$$

trong đó x_i biểu thị bit thứ i , tính từ thấp lên cao, trong biểu diễn nhị phân của x .

Định nghĩa 4.8. Cho ba dãy a, b, c có độ dài N , định nghĩa hàm $H(a, b, c)$. Đặt $d = H(a, b, c)$, khi đó

$$d_i = \sum_{j=0}^{N-1} \sum_{k=0}^{N-1} \sum_{l=0}^{N-1} [g(j, k, l) = i] a_j b_k c_l.$$

Đặt

$$S = \{(x, y, z) \mid f(x, y, z) = 0\}, \quad T = \{(x, y, z) \mid f(x, y, z) = 1\}.$$

Sau khi tách các dãy a, b, c, d theo bit cao nhất, hiển nhiên có

$$d_0 = \sum_{x, y, z, (x, y, z) \in S} H(a_x, b_y, c_z),$$

$$d_1 = \sum_{x, y, z, (x, y, z) \in T} H(a_x, b_y, c_z).$$

Ta biết $S \cap T = \emptyset$ và $|S| + |T| = 8$, nên chắc chắn $\min(|S|, |T|) \leq 4$. Vì vậy ta quyết định tính vết cận đệ quy d_0 hay d_1 theo kích thước của $|S|$ và $|T|$, rồi dùng

$$d_0 + d_1 = H(a_0 + a_1, b_0 + b_1, c_0 + c_1)$$

để thu được hạng còn lại.

Vì $\min(|S|, |T|) \leq 4$, nhiều nhất cần tính đệ quy $4 + 1 = 5$ lần. Độ phức tạp thời gian là

$$T(N) = 5T\left(\frac{N}{2}\right) + O(N) = O(N^{\log_2 5}) \approx O(N^{2.32}).$$

5 Tổng kết

Jellyfish là một bài toán khá tổng hợp, kết hợp toán học, quy hoạch động và đa thức; bài có độ khó tư duy và độ khó hiện thực nhất định. Thí sinh cần dùng kiến thức về kỳ vọng để đơn giản hóa bài toán, rồi dùng quy hoạch động và phép nhân chia để trị để tối ưu thuật toán.

Tác giả lần đầu tiếp xúc với tư tưởng dùng phép nhân chia để trị để giải một số bài tích chập dưới các quy tắc bit đặc biệt trong kỳ thi mô phỏng nội bộ của đàn anh Huang Zhihuan. Sau đó tác giả nghiên cứu thêm loại bài này và nhận thấy phép nhân chia để trị thường đạt hiệu quả tốt. Vì vậy tác giả chọn *Jellyfish* làm đề tự chọn trong đợt bài tập vòng một của đội tuyển để trao đổi, thảo luận với mọi người, hy vọng bài viết có thể gợi mở phần nào.

6 Lời cảm ơn

Cảm ơn CCF đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn cha mẹ đã nuôi dưỡng và giáo dục.

Cảm ơn nhà trường đã bồi dưỡng, thầy Ying Ping'an đã chỉ dạy, và các bạn học đã giúp đỡ.

Cảm ơn đàn anh Huang Zhihuan của Đại học Bắc Kinh đã hướng dẫn chúng tôi.

Cảm ơn Ren Xuandi của Trường Trung học số 1 Thiệu Hưng và Liu Chengao của THPT trực thuộc Đại học Công nghiệp Tây Bắc đã trao đổi, thảo luận, gợi mở cho tác giả, đồng thời đọc duyệt bài viết này.

Tài liệu tham khảo

1. Tư liệu về thuật toán Karatsuba trên Wikipedia tiếng Anh: https://en.wikipedia.org/wiki/Karatsuba_algorithm.

2. Lu Kaifeng, *Tính chất, ứng dụng và thuật toán nhanh của chuỗi lũy thừa tập hợp*, luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015.
3. Mao Xiao, *Xét lại biến đổi Fourier nhanh*, luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015.
4. Donald E. Knuth, *The Art of Computer Programming*, Nhà xuất bản Công nghiệp Quốc phòng.

9 *Leafy Tree* và cây cân bằng có trọng số được hiện thực bằng nó

Wang Siqi, Trường Trung học số 7 Thành Đô

Tóm tắt

Cây nhị phân và cây cân bằng là các cấu trúc dữ liệu quan trọng trong thi tin học. Chúng có công dụng rất lớn khi cần duy trì tập hợp hoặc dãy.

Bài viết này giới thiệu *Leafy Tree*, một cấu trúc dữ liệu có thể hiện thực phần lớn các cấu trúc dữ liệu dựa trên cây nhị phân. Bài viết mô tả cách dùng nó để hiện thực cây tìm kiếm nhị phân và cây cân bằng có trọng số, đồng thời so sánh hai cấu trúc dữ liệu này với các cấu trúc dữ liệu khác.

1 Lời nói đầu

Trong những năm gần đây, các bài toán cấu trúc dữ liệu trong thi tin học dần được khai thác nhiều hơn. Rất nhiều bài cần dùng cây nhị phân, chẳng hạn cây đoạn, cây cân bằng, cây lệch trái, v.v. để duy trì thông tin; trong đó cây cân bằng là một nội dung khảo sát quan trọng.

Các cây cân bằng thường gặp gồm Splay, Treap, v.v. Chúng ít nhiều đều có một số vấn đề, như hằng số lớn, lượng mã lớn, khó khả tri hóa. Để xử lý các vấn đề này, bài viết đưa vào một cấu trúc cây nhị phân mới, *Leafy Tree*, và một chiến lược cân bằng mới, cân bằng có trọng số. Khi kết hợp hai thứ này, phần lớn vấn đề của cây cân bằng có thể được khắc phục.

Mục 2 giới thiệu *Leafy Tree*. Mục 3 mô tả cách dùng *Leafy Tree* để hiện thực cây tìm kiếm nhị phân. Mục 4 đưa vào cây cân bằng có trọng số và mô tả cách dùng *Leafy Tree* để hiện thực nó.

2 *Leafy Tree* là gì

Leafy Tree là một cây nhị phân mà mỗi nút hoặc là lá, hoặc có đúng hai con. Toàn bộ thông tin được lưu trên các lá; thông tin ở mỗi nút không phải là kết quả gộp thông tin của các con. Cấu trúc của nó giống cây đoạn, đồng thời cũng có thể xem nó như cây tái cấu trúc Kruskal của một cây tìm kiếm nhị phân. Ở đây, cây tái cấu trúc Kruskal không phải là cây tái cấu trúc Kruskal của cây khung nhỏ nhất, mà là thao tác tương tự trên một cây tìm kiếm nhị phân: mỗi lần gộp hai nút và biến cạnh ở giữa chúng thành một nút trong đồ thị mới, cuối cùng thu được một cây.

Đây là một cấu trúc dữ liệu thuần hàm, có thể hiện thực các cấu trúc dữ liệu hàm khác một cách thuận tiện và hiệu quả, đồng thời cũng dễ hiện thực khả tri.

3 Dùng *Leafy Tree* hiện thực cây tìm kiếm nhị phân

3.1 Định nghĩa

Xét việc dùng một *Leafy Tree* để duy trì một tập hợp. Mỗi nút lá lưu một giá trị trong tập hợp, mỗi nút không phải lá lưu giá trị của con phải, tức giá trị lớn nhất trong cây con. Đồng thời, giá trị lớn nhất của con trái ở mỗi nút không phải lá không lớn hơn giá trị nhỏ nhất của con phải; nói cách khác, trong thứ tự duyệt giữa của cây này, các giá trị ở nút lá là có thứ tự. Ta gọi cây như vậy là cây tìm kiếm nhị phân được hiện thực bằng *Leafy Tree*.

3.2 Thao tác cơ bản

Để tiện mô tả, ta dùng $A.left$, $A.right$, $A.value$ để biểu thị con trái, con phải và giá trị của nút A . Với nút lá X , định nghĩa $X.left = X.right = null$. Đồng thời định nghĩa hàm $NEWNODE(x)$ để tạo một nút mới có giá trị x , đặt hai con của nút mới là $null$, và $DELETENODE(x)$ để thu hồi nút x .

3.2.1 Truy vấn. Quá trình tìm giá trị x trong cây con của nút T như sau.

Nếu T là nút lá, so sánh x với giá trị của T ; nếu hai giá trị bằng nhau thì tìm kiếm thành công, ngược lại thất bại. Nếu không, nếu x không lớn hơn giá trị của cây con trái của T , tìm trong cây con trái; ngược lại tìm trong cây con phải.

Thuật toán 1. Truy vấn trong cây tìm kiếm nhị phân bằng *Leafy Tree*.

```
Find(T, x):
  if T.left = null then
    if T.value = x then
      return True
    else
      return False
  end if
else
  if x <= T.left.value then
    return Find(T.left, x)
  else
    return Find(T.right, x)
  end if
end if
```

3.2.2 Chèn. Quá trình chèn giá trị x vào cây con của nút T như sau.

Nếu T là nút lá, so sánh x với giá trị của T . Nếu x lớn hơn giá trị của T , tạo hai nút mới A, B để lần lượt lưu giá trị của T và x , rồi đặt con trái và con phải của T là A và B ; khi x không lớn hơn giá trị của T , thao tác tương tự. Nếu không, nếu x không lớn hơn giá trị của cây con trái của T , chèn x vào cây con trái; ngược lại chèn vào cây con phải.

Thuật toán 2. Chèn trong cây tìm kiếm nhị phân bằng *Leafy Tree*.

```
Insert(T, x):
  if T.left = null then
```



```

A <- newNode(T.value)
B <- newNode(x)
if A.value > B.value then
    swap(A, B)
end if
T.left <- A
T.right <- B
T.value <- B.value
else
    if x <= T.left.value then
        Insert(T.left, x)
    else
        Insert(T.right, x)
    end if
    T.value <- T.right.value
end if

```

3.2.3 Xóa. Quá trình xóa giá trị x trong cây con của nút T như sau.

Nếu T là nút lá, so sánh x với giá trị của T . Nếu hai giá trị bằng nhau, việc xóa thành công: đặt mọi thuộc tính của cha T thành thuộc tính của nút anh em của T , rồi xóa nút anh em của T và chính T ; nếu không, việc xóa thất bại. Nếu T không phải lá, nếu x không lớn hơn giá trị của cây con trái của T , xóa trong cây con trái; ngược lại xóa trong cây con phải.

Thuật toán 3. Xóa trong cây tìm kiếm nhị phân bằng *Leafy Tree*. Hàm Remove nhận nút T và giá trị cần xóa x , trả về việc xóa có thành công hay không. Hàm này yêu cầu T không phải nút lá.

```

Remove(T, x):
    if x <= T.left.value then
        if T.left.size = 1 then
            if T.left.value = x then
                temp <- T.right
                T = T.right
                deleteNode(temp)
                deleteNode(T.left)
                return True
            else
                return False
            end if
        else
            return Remove(T.left, x)
        end if
    else
        if T.right.size = 1 then
            if T.right.value = x then
                temp <- T.left
                T = T.left
                deleteNode(temp)
                deleteNode(T.right)
            end if
        else
            return Remove(T.right, x)
        end if
    end if
end if

```

```

        return True
    else
        return False
    end if
else
    temp <- Remove(T.right, x)
    T.value <- T.right.value
    return temp
end if
end if

```

Tuy mã giả ở đây khá dài, trong hiện thực thực tế, nếu bảo đảm giá trị cần xóa tồn tại và dùng mảng để lưu hai con của mỗi nút, có thể tiết kiệm rất nhiều mã.

3.3 So sánh với cây tìm kiếm nhị phân

Có thể thấy, nhờ tính chất chỉ nút lá duy trì thông tin, thao tác chèn xóa trên *Leafy Tree* khá thuận tiện. Nút được chèn hoặc xóa mỗi lần thực chất đều là nút lá, giảm rất nhiều phần thảo luận rườm rà trong cây tìm kiếm nhị phân thông thường về kiểu của nút được chèn hoặc xóa: nút đó là lá, có một con, hay có hai con. Đồng thời, do mỗi nút không phải là duy trì thông tin đoạn và định vị bằng thông tin đoạn của con trái, về bản chất nó giống cách cây tìm kiếm nhị phân định vị bằng cách so sánh với giá trị ở gốc cây. Vì vậy, một hiện thực gọn hơn vẫn đạt cùng hiệu quả.

4 Dùng *Leafy Tree* hiện thực cây cân bằng có trọng số

Cây cân bằng có trọng số (*Weight Balanced Tree*, còn gọi là cây $BB[\alpha]$, hay cây cân bằng trọng lượng) là một cây tìm kiếm nhị phân lưu kích thước cây con. Nghĩa là một nút chứa các trường: giá trị (*value*), con trái (*left*), con phải (*right*) và kích thước cây con (*size*).

Trọng số *weight* của một nút phụ thuộc vào kích thước cây con của nó hoặc bằng kích thước cây con của nó. Cụ thể, cần thỏa

$$\text{weight}[x] = \text{weight}[x.\text{left}] + \text{weight}[x.\text{right}].$$

Nếu một nút x thỏa

$$\min(\text{weight}[x.\text{left}], \text{weight}[x.\text{right}]) \geq \alpha \cdot \text{weight}[x],$$

thì gọi nút đó là α -cân bằng theo trọng số; hiển nhiên $0 < \alpha \leq \frac{1}{2}$. Chiều cao h của một cây cân bằng có trọng số chứa n phần tử thỏa

$$h \leq \log_{\frac{1}{1-\alpha}} n = O(\log n).$$

4.1 Định nghĩa

Trong cây cân bằng có trọng số được hiện thực bằng *Leafy Tree*, kích thước cây con là số nút lá trong cây con, còn giá trị của một nút không phải lá là giá trị của con phải của nó. Tức là với một nút lá x , $x.\text{size} = 1$; với một nút không phải lá x ,

$$x.\text{size} = x.\text{left}.\text{size} + x.\text{right}.\text{size}, \quad x.\text{value} = x.\text{right}.\text{value}.$$

Trọng số của một nút chính là kích thước cây con của nó, tức là $\text{weight}[x] = x.\text{size}$.

4.2 Cách hiện thực

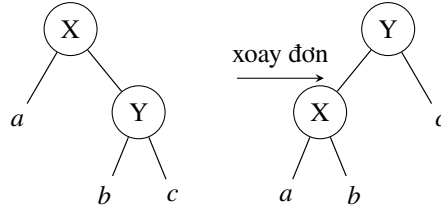
Cây cân bằng có trọng số có hai cách hiện thực: tái cấu trúc và xoay. Ở đây chỉ giới thiệu cân bằng bằng phép xoay.

Với một cây thỏa α -cân bằng theo trọng số, sau khi chèn hoặc xóa một nút, các nút không còn thỏa α -cân bằng chắc chắn nằm trên một dây chuyền. Ta xét xử lý lần lượt các nút trên dây chuyền này.

Giả sử trong một cây con T của cây này, mọi nút ngoài nút gốc đều thỏa α -cân bằng theo trọng số. Ta có thể dùng một phép xoay đơn hoặc xoay kép để làm T thỏa α -cân bằng theo trọng số.

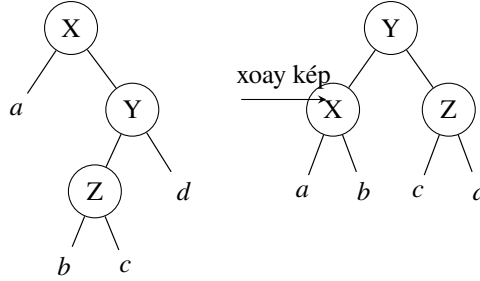
Định nghĩa độ cân bằng của x là

$$\frac{\text{weight}[x.\text{left}]}{\text{weight}[x]}.$$



Như hình trên, xét một phép xoay đơn. Gọi độ cân bằng của X, Y trước khi xoay là ρ_1, ρ_2 , sau khi xoay là γ_1, γ_2 . Ta có

$$\gamma_1 = \frac{\rho_1}{\rho_1 + (1 - \rho_1)\rho_2}, \quad \gamma_2 = \rho_1 + (1 - \rho_1)\rho_2.$$



Như hình trên, xét một phép xoay kép. Gọi độ cân bằng của X, Y, Z trước khi xoay là ρ_1, ρ_2, ρ_3 , sau khi xoay là $\gamma_1, \gamma_2, \gamma_3$. Ta có

$$\gamma_1 = \frac{\rho_1}{\rho_1 + (1 - \rho_1)\rho_2\rho_3}, \quad \gamma_2 = \rho_1 + (1 - \rho_1)\rho_2\rho_3, \quad \gamma_3 = \frac{\rho_2(1 - \rho_3)}{1 - \rho_2\rho_3}.$$

Giả sử $\rho_1 < \alpha$. Trong trường hợp chỉ chèn hoặc xóa một nút, ρ_1 nhỏ nhất là $\frac{\alpha}{2-\alpha}$, đạt được khi cây con a vốn có 2 nút và nay xóa một nút trong đó.

Trong các ràng buộc đã cho, có thể chứng minh rằng khi $\alpha \leq 1 - \frac{\sqrt{2}}{2}$, bằng hai phép nói trên, các nút có độ cân bằng bị thay đổi đều có độ cân bằng mới nằm trong $[\alpha, 1 - \alpha]$. Khi chứng minh có thể còn cần chặn thêm ρ_1 và phân loại trong phạm vi nhỏ. Do khuôn khổ có hạn, phần chứng minh cụ thể không trình bày ở đây.

Thao tác cụ thể là: khi

$$\rho_2 < \frac{1 - 2\alpha}{1 - \alpha},$$

thực hiện một lần xoay đơn; ngược lại thực hiện một lần xoay kép.

4.3 Thao tác cơ bản

Để tiện mô tả, ngoài *left* và *right*, ta còn dùng $A.child[0]$, $A.child[1]$ để biểu thị con trái và con phải của nút A . Với nút lá X , định nghĩa $X.child[0] = X.child[1] = \text{null}$. Đồng thời định nghĩa hàm $\text{NEWNODE}(x)$ để tạo nút mới có giá trị x , đặt hai con của nút mới là null , và $\text{DELETENODE}(x)$ để thu hồi nút x .

4.3.1 Gộp thông tin. Sau khi một nút trải qua thao tác, cần tính lại kích thước cây con và giá trị của nó.

Thuật toán 4. Gộp thông tin trong cây cân bằng có trọng số bằng *Leafy Tree*.

$\text{Pushup}(T)$:

```
if not T.left = null then
    T.size <- T.left.size + T.right.size
    T.value <- T.right.value
end if
```

4.3.2 Xoay đơn. Thao tác này xoay Y đơn lên vị trí của X . Ở đây, để không thay đổi con trở trở tới X , sau khi xoay ta hoán đổi vị trí của chúng. Trong thuật toán sau, $\oplus$ biểu thị xor.

Thuật toán 5. Xoay đơn trong cây cân bằng có trọng số bằng *Leafy Tree*. Hàm Rotate nhận nút T và biến bool d , biểu thị xoay $child[d]$ lên vị trí của T .

$\text{Rotate}(T, d)$:

```
temp <- T.child[d xor 1]
T.child[d xor 1] <- T.child[d]
T.child[d] <- T.child[d xor 1].child[d]
T.child[d xor 1].child[d] <- T.child[d xor 1].child[d xor 1]
T.child[d xor 1].child[d xor 1] <- temp
Pushup(T.child[d xor 1])
Pushup(T)
```

4.3.3 Duy trì cân bằng. Sau khi một nút trải qua thao tác, cần dùng một lần xoay đơn hoặc xoay kép để duy trì cân bằng.

Thuật toán 6. Duy trì cân bằng trong cây cân bằng có trọng số bằng *Leafy Tree*.

$\text{Maintain}(T)$:

```
if not T.left = null then
    if T.left.size < T.size * alpha then
        d <- 1
    else if T.right.size < T.size * alpha then
        d <- 0
    else
        return
    end if
    if T.child[d].child[d xor 1].size
        >= T.child[d].size * (1 - 2 * alpha) / (1 - alpha) then
        Rotate(T.child[d], d xor 1)
```

```

    end if
    Rotate(T, d)
end if

```

4.3.4 Chèn. Thao tác này tương tự cây tìm kiếm nhị phân được hiện thực bằng *Leafy Tree*.

Thuật toán 7. Chèn trong cây cân bằng có trọng số bằng *Leafy Tree*.

```

Insert(T, x):
    if T.size = 1 then
        T.left <- newNode(x)
        T.right <- newNode(T.value)
        if x > T.value then
            swap(T.left, T.right)
        end if
    else
        Insert(T.child[x > T.left.value], x)
    end if
    Pushup(T)
    Maintain(T)

```

4.3.5 Xóa. Thao tác này tương tự cây tìm kiếm nhị phân được hiện thực bằng *Leafy Tree*.

Thuật toán 8. Xóa trong cây cân bằng có trọng số bằng *Leafy Tree*. Hàm Remove nhận nút T và giá trị cần xóa x . Hàm này yêu cầu T không phải nút lá.

```

Remove(T, x):
    d <- x > T.left.value
    if T.child[d].size = 1 then
        if T.child[d].value = x then
            deleteNode(T.child[d])
            temp <- T.child[d xor 1]
            T.left = T.child[d xor 1].left
            T.right = T.child[d xor 1].right
            T.value = T.child[d xor 1].value
            deleteNode(temp)
        else
            return
        end if
    else
        Remove(T.child[d])
    end if
    Pushup(T)
    Maintain(T)

```

4.3.6 Truy vấn. Thao tác này giống cây tìm kiếm nhị phân được hiện thực bằng *Leafy Tree*. Mã giả xem thuật toán 1.

4.4 Tốc độ chạy

Ở đây tác giả chọn một số mã chạy nhanh cho bài LOJ #104, *Cây cân bằng thông thường*, để kiểm thử.

Do kết quả kiểm thử từ các bộ sinh dữ liệu khác nhau gần như giống nhau, ở đây chỉ trình bày kết quả trên dữ liệu ngẫu nhiên thuần túy.⁸ Kết quả này có thể có sai số nhất định, và mã cũng không bảo đảm có hằng số tốt nhất, nhưng có thể phản ánh đại khái tốc độ chạy của các loại cây cân bằng này.

Hình trong bản gốc so sánh tỉ lệ giữa thời gian chạy thực tế của từng cây cân bằng và thời gian chạy thực tế của cây cân bằng có trọng số, trên trục hoành $\log_{10}(n)$. Có thể thấy, tuy cây cân bằng có trọng số chậm hơn cây đỏ-đen và SBT, nó nhanh hơn Treap và Splay, đồng thời có tốc độ gần cây thay thế.

4.5 Thao tác khác

Ngoài các thao tác cơ bản như chèn và xóa, cây cân bằng có trọng số còn có thể hiện thực thao tác gộp, tách, v.v. với độ phức tạp tốt, cũng như hiện thực khả trì.

4.5.1 Gộp. Thao tác gộp là: cho hai cây A và B , bảo đảm giá trị lớn nhất trong A không lớn hơn giá trị nhỏ nhất trong B , cần gộp chúng thành một cây mới. Độ phức tạp của thao tác này là

$$O\left(\log \frac{\max(A.size, B.size)}{\min(A.size, B.size)}\right).$$

Trước hết, giả sử $A.size \geq B.size$. Nếu

$$B.size \geq \frac{\alpha}{1-\alpha} \cdot A.size,$$

ta có thể trực tiếp tạo một nút mới, với con trái là A và con phải là B .

Ngược lại, nếu

$$A.left.size \geq \alpha \cdot (A.size + B.size),$$

có thể đệ quy gộp con phải của A với B , rồi gộp kết quả với con trái của A .

Nếu không, có thể đệ quy gộp con trái của A với con trái của con phải của A , đồng thời gộp con phải của con phải của A với B , cuối cùng gộp hai kết quả này với nhau. Do khuôn khổ có hạn, phần chứng minh độ phức tạp không trình bày ở đây.

4.5.2 Tách. Thao tác tách là: cho một cây T , cần tách phần không lớn hơn k và phần lớn hơn k thành hai cây độc lập.

Thao tác này có thể được thực hiện trực tiếp bằng đệ quy, rồi gọi thao tác gộp nói trên với các kết quả trả về. Độ phức tạp là $O(\log T.size)$. Do khuôn khổ có hạn, phần chứng minh độ phức tạp không trình bày ở đây.

4.5.3 Khả trì hóa. Với mỗi thao tác, có thể sao chép toàn bộ các nút bị sửa đổi và lần lượt tạo mới chúng, tức path copying. Số nút bị sửa đổi trong một lần chèn hoặc xóa hiển nhiên là $O(\log size)$.

Trong cây cân bằng có trọng số được hiện thực bằng *Leafy Tree*, số nút bị sửa đổi trong một thao tác đơn lẻ là rất nhỏ theo kỳ vọng, và không cần tạo thêm các nút dư thừa. Vì vậy nó dùng ít bộ nhớ, hằng số cũng nhỏ, và có ưu thế lớn so với các cây cân bằng khác, chẳng hạn Treap không xoay.

⁸Mã kiểm thử và dữ liệu gốc có thể tải tại <https://mcfx.us/ctsc>.

5 Tổng kết

Bài viết này giới thiệu *Leafy Tree* và cây cân bằng có trọng số được hiện thực bằng nó.

So với cây tìm kiếm nhị phân, *Leafy Tree* có thao tác chèn xóa dễ hiện thực hơn, nhưng dùng gấp đôi số nút.

Cây cân bằng có trọng số có hằng số nhỏ, hiện thực tương đối đơn giản, đồng thời cũng có thể hiện thực khả tri một cách thuận tiện. So với cây đỏ-đen có hằng số nhỏ hơn, nó không cần ghi thêm thông tin phụ, vì kích thước nút có thể dùng để cắt đoạn hoặc tìm phần tử nhỏ thứ K , và dễ hiện thực hơn nhiều. So với Splay dễ hiện thực, lượng mã về cơ bản tương đương, nhưng hằng số nhỏ hơn và có thể khả tri hóa. So với Treap, nó không cần ghi thông tin phụ và có hằng số nhỏ hơn. So với cây thay thế, độ phức tạp của nó không phải dạng khẩu hao, và có thể hiện thực khả tri.

Leafy Tree cân bằng có trọng số, kết hợp hai ý tưởng trên, có thể hiện thực phần lớn chức năng của cây cân bằng bằng lượng mã khá ngắn, hằng số nhỏ, và có thể khả tri hóa.

Ngoài ra, *Leafy Tree* còn có thể hiện thực nhiều cấu trúc dữ liệu dựa trên cây nhị phân, như các cây cân bằng khác, cây đoạn, heap, v.v. Nhưng do khuôn khổ có hạn, bài viết không thể lần lượt giới thiệu tất cả.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn các thầy Zhang Junliang và Lin Yang của Trường Trung học số 7 Thành Đô đã quan tâm và chỉ dẫn trong nhiều năm.

Cảm ơn bạn Li Xinlong của Trường Trung học số 7 Thành Đô và bạn Zhao Yuyang của THPT trực thuộc Đại học Sư phạm An Huy đã cung cấp ý tưởng cho bài viết này.

Cảm ơn các thầy cô và bạn học khác đã giúp đỡ tôi.

Tài liệu tham khảo

1. Nievergelt, J. and Reingold, E. M. (1972), *Binary search trees of bounded balance*. In *Proceedings of the Fourth Annual ACM Symposium on Theory of Computing*. ACM, pp. 137–142.
2. Wikipedia, *Weight-balanced tree*.
3. Wikipedia, *Binary search tree*.
4. LOJ #104, *Cây cân bằng thông thường*, <https://loj.ac/problem/104>.

10 Bài 8: Báo cáo ra đề Tiểu H thích tô màu

Chen Jiale, Trường Trung học số 2 Hàng Châu, Chiết Giang

Tóm tắt

Bài viết này giới thiệu một bài do tác giả ra trong vòng thứ năm của kỳ tự kiểm tra đội tuyển tập huấn năm 2018. Bài này khảo sát cụ thể các kiến thức về tổ hợp, số Stirling, nghịch đảo nhị thức, đa thức và một ít kỹ thuật đếm, yêu cầu thí sinh có nền tảng cơ bản vững và năng lực tư duy nhất định.

Ngoài ra, phần điểm của bài được thiết kế hợp lý, có độ phân hóa cao, nhằm dẫn dắt thí sinh tiến gần tới lời giải đúng. Đồng thời, hướng giải bài này khá đa dạng; thí sinh có thể suy nghĩ sâu từ nhiều góc độ khác nhau và đều có thể đạt điểm cao.

1 Đề bài

1.1 Mô tả đề bài

Có n quả bóng xếp thành một hàng, được đánh số lần lượt từ 0 đến $n - 1$, ban đầu đều có màu trắng. Tiểu H sẽ lặp thao tác sau tổng cộng 2 lần: chọn ngẫu nhiên m quả bóng để nhuộm đen (có thể nhuộm đen lại một quả bóng đã đen). Tiểu H đặc biệt thích quả bóng đen có số hiệu nhỏ nhất. Cô muốn biết, nếu gọi A là số hiệu của nó, thì kỳ vọng của $F(A)$ là bao nhiêu. Trong đó, $F(x)$ là một đa thức bậc không vượt quá m , và $F(A)$ biểu thị giá trị của đa thức tại $x = A$.

1.2 Định dạng nhập

Dòng đầu gồm hai số nguyên n, m . Dòng thứ hai gồm $m + 1$ số nguyên, số nguyên thứ i là $F(i - 1)$.

1.3 Định dạng xuất

In ra một số nguyên trên một dòng. Nếu gọi E là kỳ vọng của $F(A)$, hãy in ra

$$E \times \binom{n}{m}^2$$

theo modulo 998244353.

1.4 Ví dụ nhập

```
8 5
45856608 386378255 106492167 28766400 272276589 93721672
```

1.5 Ví dụ xuất

```
321347828
```

1.6 Phạm vi dữ liệu và quy ước

- Với 10% dữ liệu, $n \leq 10, m \leq 5$.
- Với 20% dữ liệu, $n \leq 100, m \leq 100$.
- Với 30% dữ liệu, $n \leq 1000, m \leq 1000$.
- Với 5% dữ liệu khác, $m \leq 1000000$, và bảo đảm đa thức $F(x) = 1$.
- Với 5% dữ liệu khác, $m \leq 1000000$, và bảo đảm đa thức $F(x) = x$.
- Với 10% dữ liệu khác, $m \leq 5000$, và bảo đảm đa thức $F(x) = x^m$.
- Với 70% dữ liệu, $m \leq 5000$.
- Với 80% dữ liệu, $m \leq 20000$.
- Với 100% dữ liệu, $n \leq 998244353, m \leq 1000000$.

1.7 Giới hạn thời gian và bộ nhớ

Giới hạn thời gian: 2 giây.

Giới hạn bộ nhớ: 512 MB.

2 Giới thiệu thuật toán

2.1 Thuật toán 1

Trước hết nội suy để chuyển đa thức sang dạng biểu diễn bằng hệ số, rồi tính giá trị của đa thức tại các điểm $0, \dots, n-1$. Liệt kê mọi phương án tô màu có thể, tìm giá trị nhỏ nhất trong đó và cộng dồn đóng góp.

Độ phức tạp tiền xử lý là $O(nm)$. Độ phức tạp tổng thể là

$$O\left(\binom{n}{m}^2 \times \text{poly}(n)\right).$$

Điểm kỳ vọng là 10 điểm.

2.2 Thuật toán 2

Ký hiệu $f_{i,j,k}$ là số phương án khi tô màu từ sau ra trước đến quả bóng có số hiệu i , trong đó lần tô thứ nhất đã tô j quả và lần tô thứ hai đã tô k quả. Có bốn kiểu chuyển: quả bóng hiện tại không được tô, được tô đen trong lần thứ nhất, được tô đen trong lần thứ hai, hoặc được tô trong cả hai lần. Khi đó

$$f_{i,j,k} = f_{i+1,j,k} + [j > 0] \times f_{i+1,j-1,k} + [k > 0] \times f_{i+1,j,k-1} \\ + [j > 0] \times [k > 0] \times f_{i+1,j-1,k-1},$$

và

$$ans = \sum_{i=0}^{n-1} F(i) \times (f_{i,m,m} - f_{i+1,m,m}).$$

Độ phức tạp là $O(nm^2)$.

Điểm kỳ vọng là 20 điểm.

2.3 Thuật toán 3

Thực ra có thể tính trực tiếp $f_{i,m,m}$ mà không cần truy hồi.

Xét số lượng số lớn hơn hoặc bằng i , tức $n-i$. Vì vậy số phương án tô màu của mỗi màu thực chất đều là $\binom{n-i}{m}$. Tức là

$$f_{i,m,m} = \binom{n-i}{m}^2.$$

Dùng cùng cách để tính đóng góp, tổng độ phức tạp là $O(nm)$.

Điểm kỳ vọng là 30 điểm.

2.4 Thuật toán 4

Khi n đạt đến cỡ 10^9 , ta khó có thể liệt kê giá trị nhỏ nhất, nên thử đổi hướng suy nghĩ.

Khi thiết kế phần điểm, tác giả muốn tạo gợi ý để thí sinh xuất phát từ góc độ đa thức đóng góp $F(x)$.

2.4.1 $F(x) = 1$. Điều này tương đương với việc mỗi phương án tô màu khác nhau có đóng góp vào đáp án như nhau, đều bằng 1, nên đáp án chính là số phương án. Tức là

$$ans = \binom{n}{m}^2.$$

Do

$$\binom{n}{m} = \binom{n}{m-1} \times \frac{n-m+1}{m},$$

sau khi tiền xử lý nghịch đảo, có thể tính trong $O(m)$.

Điểm kỳ vọng là 5 điểm.

2.4.2 $F(x) = x$. Ta tìm một dạng tương đương của kỳ vọng: đó là kỳ vọng của số quả bóng có số hiệu nhỏ hơn A . Theo tính tuyến tính của kỳ vọng, thứ cần tính là tổng, trên mỗi quả bóng i , của xác suất để số hiệu của i nhỏ hơn A .

Ta cùng liệt kê quả bóng i và các quả bóng được tô đen. Gọi k là số lượng quả bóng cuối cùng bị tô đen. Vì hai lần mỗi lần tô m quả, nên $k \in [m, 2m]$. Xét số phương án tô màu. Lần thứ nhất chọn tùy ý, số phương án là $\binom{k}{m}$. Vì tất cả vị trí đã chọn đều phải bị tô màu, những vị trí chưa tô ở lần thứ nhất nhất định phải được tô đen ở lần thứ hai, còn phần dư ra chỉ có thể tô vào các quả bóng đã bị tô đen. Số phương án là $\binom{m}{2m-k}$. Vậy tổng số phương án là

$$H_k = \binom{k}{m} \times \binom{m}{2m-k}.$$

Sau khi tiền xử lý giai thừa và nghịch đảo giai thừa, có thể tính tuyến tính mảng H .

Khi thống kê đáp án, trực tiếp chọn $k+1$ quả bóng, số phương án là $\binom{n}{k+1}$. Vì số hiệu của quả bóng i nhỏ hơn A , nên các quả bị tô chính là k quả có số hiệu lớn hơn; hai cách đếm này tương ứng một-một.

Do đó

$$ans = \sum_{k=m}^{2m} \binom{n}{k+1} H_k.$$

Tổng độ phức tạp là $O(m)$.

Điểm kỳ vọng là 5 điểm.

2.4.3 $F(x) = x^c$. Tương tự như trên, dạng tương đương là: kỳ vọng của việc chọn lặp c quả bóng sao cho số hiệu của tất cả chúng đều nhỏ hơn A . Theo tính tuyến tính của kỳ vọng, ta cần tính tổng xác suất, trên mỗi phương án chọn lặp c quả, để số hiệu của cả c quả đều nhỏ hơn A .

Chú ý rằng các quả bóng được chọn có thể lặp lại. Ta xét số phương án G_q khi sau khi khử trùng, số lượng quả bóng được chọn là q ($q = 0, \dots, c$).

Xét nghịch đảo nhị thức. Tổng số lần chọn tùy ý là

$$q^c = \sum_{i=0}^q \binom{q}{i} G_i,$$

nên

$$G_q = \sum_{i=0}^q \binom{q}{i} (-1)^{q-i} i^c.$$

Dĩ nhiên, thực ra G_q chính là $S(c, q)q!$, trong đó S biểu thị số Stirling loại hai.

Mảng G có thể tính trong thời gian $O(m^2)$.

Gọi T_i là số phương án sao cho tổng số quả bóng đã bị tô và quả bóng được chọn là i . Vì số hiệu của quả bóng được chọn nhất định nhỏ hơn số hiệu của quả bóng bị tô, các phương án tương ứng một-một. Dễ thấy

$$T_i = \sum_{j=0}^i G_j H_{i-j}.$$

Mảng T có thể tính trong thời gian $O(m^2)$.

Khi đó

$$ans = \sum_{i=0}^{3m} \binom{n}{i} T_i.$$

Tổng độ phức tạp là $O(m^2)$.

Điểm kỳ vọng là 10 điểm.

2.4.4 $F(x) = \sum_{i=0}^m a_i x^i$. Chú ý rằng khi $F(x) = x^c$,

$$G_q = \sum_{i=0}^q \binom{q}{i} (-1)^{q-i} i^c.$$

Vậy sau khi lấy tổng, hiện nay

$$\begin{aligned} G_q &= \sum_{i=0}^q \binom{q}{i} (-1)^{q-i} \sum_{j=0}^m a_j i^j \\ &= \sum_{i=0}^q \binom{q}{i} (-1)^{q-i} F(i). \end{aligned}$$

Các phần còn lại giống như trước.

Tổng độ phức tạp là $O(m^2)$.

Điểm kỳ vọng là 70 điểm.

2.5 Thuật toán 5

Khi n rất lớn, có thể vẫn còn cách xử lý khác. Nếu đóng góp của vị trí nhỏ nhất A là một đa thức theo A , thì lấy tổng tiền tố của nó cũng sẽ nhận được một đa thức. Khi đó đáp án chính là giá trị thu được khi thay $n - 1$ vào đa thức ấy.

Xét cách tính trong thuật toán 3:

$$\binom{n-A}{m}^2$$

là một đa thức bậc $2m$ theo A . Còn đóng góp mà nó cần nhân, $F(x)$, là một đa thức bậc m . Vì vậy đóng góp cuối cùng của mỗi vị trí là một đa thức bậc $3m$, và tổng tiền tố của nó là một đa thức bậc $3m + 1$. Ta lấy các điểm $0, \dots, 3m + 1$ để tính giá trị điểm của đa thức ấy, rồi dùng nội suy Lagrange là có thể tuyến tính nhận được giá trị của đa thức tại $n - 1$.

Độ phức tạp là $O(m \log m)$, với hằng số khá lớn.

Điểm kỳ vọng là 80–100 điểm.

2.6 Thuật toán 6

Tối ưu trên cơ sở thuật toán 4.

Sau khi suy diễn công thức tính mảng G , có thể nhận được

$$\frac{G_q}{q!} = \sum_{i=0}^q \frac{(-1)^{q-i}}{(q-i)!} \times \frac{F(i)}{i!}.$$

Có thể dùng NTT để tính mảng G . Tương tự, dùng NTT để tính mảng T .

Tổng độ phức tạp là $O(m \log m)$, với hằng số nhỏ hơn.

Điểm kỳ vọng là 100 điểm.

3 Tổng kết

Đề bài này có phát biểu ngắn gọn, yêu cầu thí sinh hiểu sâu và khảo sát đầy đủ các tính chất của đa thức đóng góp, nắm vững nghịch đảo nhị thức hoặc số Stirling, cũng như các kỹ thuật đa thức. Trong khi khảo sát nền tảng cơ bản của thí sinh, bài cũng kiểm tra khả năng quan sát phân bố phần điểm, từ đó tìm ý tưởng và tổng kết. Đồng thời, cả hai hướng suy nghĩ của bài đều còn không gian để tối ưu, có thể đạt điểm cao, qua đó nâng cao tính khả thi của bài. Trong kỳ tự kiểm tra thực tế, có 3 thí sinh đạt điểm tối đa, 7 thí sinh đạt 70 điểm trở lên; phân bố điểm đồng đều và có độ phân hóa nhất định. Nhìn chung, đây là một bài có độ khó NOI.

4 Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn thầy Li Jian và cha mẹ đã quan tâm, chỉ dẫn trong nhiều năm.

Cảm ơn các bạn học ở Trường Trung học số 2 Hàng Châu đã đọc kiểm bài viết này.

Tài liệu tham khảo

1. Graham, Ronald L.; Knuth, Donald E.; Patashnik, Oren (February 1994). *Concrete Mathematics - A Foundation for Computer Science* (2nd ed.). Reading, MA, USA: Addison-Wesley Professional.
2. Peng Yuxiang, *Fast Fourier Transform*.
3. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Nhà xuất bản Đại học Thanh Hoa.
4. Liu Rujia, *Kinh điển nhập môn thi thuật toán*, Nhà xuất bản Đại học Thanh Hoa.

11 Bài 9: Một số bài toán tổng hàm số học đặc biệt

Zhu Zhenting, THPT trực thuộc Đại học Sư phạm An Huy

Tóm tắt

Tổng hàm số học là một lớp bài toán khá quan trọng trong thi tin học. Bài viết này chủ yếu chia thành ba phần: phần thứ nhất giới thiệu một phương pháp tính tổng hàm nhân tính từ góc nhìn của sàng Eratosthenes; phần thứ hai thảo luận bài toán tính tổng trên các số nguyên tố; phần thứ ba nghiên cứu mối liên hệ giữa tổng của một hàm số học đặc biệt và việc xấp xỉ một đường cong phẳng.

Dẫn nhập

Trong thi tin học, ta thường gặp bài toán tính tổng của một hàm có miền xác định là các số nguyên dương, tức một hàm số học. Khi nghiên cứu một số bài toán tổng hàm số học đặc biệt, ta thường phát hiện rằng tuy cách giải những bài toán này khá riêng, chúng lại có ý nghĩa gợi mở rất lớn. Một vài cách giải còn có thể mở rộng sang những bài toán nhiều hơn và phổ quát hơn, hoặc cung cấp một hướng nhìn mới cho nhiều bài toán. Bài viết này chủ yếu thảo luận ba bài toán trong số đó.

1 Kiến thức cơ sở

Định nghĩa 1.1. Một hàm có miền xác định là tập số nguyên dương và miền giá trị là trường số phức được gọi là một hàm số học.

Định nghĩa 1.2. Giả sử $f(x)$ là một hàm số học. Nếu với mọi cặp số nguyên dương nguyên tố cùng nhau a, b , ta có

$$f(ab) = f(a)f(b),$$

thì $f(x)$ được gọi là hàm nhân tính.

Bổ đề 1.3. Với số nguyên dương n cho trước và mọi số nguyên dương m , giá trị $[n/m]$ chỉ có $O(\sqrt{n})$ loại.

Chứng minh. Khi $m \leq \sqrt{n}$, số lượng m thỏa điều kiện chỉ là $O(\sqrt{n})$. Khi $m > \sqrt{n}$, ta có

$$0 \leq \left\lfloor \frac{n}{m} \right\rfloor < \sqrt{n},$$

nên các giá trị $[n/m]$ thỏa điều kiện cũng chỉ có $O(\sqrt{n})$ loại. Bổ đề được chứng minh. ■

Định lý 1.4 (định lý số nguyên tố). Gọi $\pi(x)$ là số lượng số nguyên tố không vượt quá x . Khi đó

$$\pi(x) = O\left(\frac{x}{\ln x}\right).$$

Có thể xem chứng minh định lý số nguyên tố trong tài liệu tham khảo 1 và các tài liệu liên quan.

2 Tính tổng hàm nhân tính

Là một loại hàm số học đặc biệt, bài toán tính tổng hàm nhân tính xuất hiện thường hơn trong thi tin học. Trong luận văn đội tuyển quốc gia năm 2016, bạn Ren Zhizhou đã thảo luận sâu về bài toán này và đưa ra một phương pháp tính tổng hàm nhân tính có phạm vi áp dụng khá rộng. Thuật toán đó trong giới OI còn được gọi là “sàng Zhounge”; về bản chất, nó là một mở rộng của sàng Eratosthenes thường dùng để sàng số nguyên tố.

Tuy nhiên, sàng Zhounge vẫn có những điểm chưa thuận tiện: thứ nhất, trong quá trình tối ưu từ $O(n/\log n)$ xuống

$$O\left(\frac{n^{3/4}}{\log n}\right),$$

nhiều phần tương đối khó hiểu, người mới học khó tiếp cận; thứ hai, hệ số hằng lớn và mã nguồn khá phức tạp. Nhưng mở rộng sàng Eratosthenes không chỉ có một cách hiện thực. Thực tế tồn tại một phương pháp dễ cài hơn, đồng thời có biểu hiện tốt hơn trên phạm vi dữ liệu nhỏ. Tư tưởng của nó gần với sàng

Zhouge, nhưng dễ hiểu hơn. Vì thuật toán này bắt đầu phổ biến sau khi Min 25 sử dụng, nó cũng được gọi là sàng Min 25.

Trong bài viết này ta thảo luận phương pháp thứ hai, tức sàng Min 25.

2.1 Dẫn nhập bài toán

Định nghĩa $\sigma_0(n)$ là số ước dương của n . Hãy tính

$$S(n, k) = \sum_{i=1}^n \sigma_0(i^k), \quad n, k \leq 10^{10}.$$

2.2 Phân tích

Đặt $f(x) = \sigma_0(x^k)$. Dễ thấy $f(x)$ là hàm nhân tính, nhưng không dễ tìm thêm tính chất tốt hơn, vì vậy cách nghĩ thông thường là dùng sàng Zhouge.

Ta xét một phương pháp hơi thô hơn một chút. Giả sử hàm nhân tính cần tính tổng là f , khi đó

$$\begin{aligned} \sum_{i=1}^n f(i) &= 1 + \sum_{\substack{2 \leq p \leq n \\ p \text{ là số nguyên tố}}} \sum_{\substack{2 \leq x \leq n \\ \text{small}(x)=p}} f(x) \\ &= 1 + \sum_{\substack{2 \leq p^e \leq n, e \geq 1 \\ p \text{ là số nguyên tố}}} f(p^e) \left(1 + \sum_{\substack{2 \leq x \leq \lfloor n/p^e \rfloor \\ x \text{ không chứa thừa số nguyên tố } \leq p}} f(x) \right). \end{aligned}$$

Nói cách khác, ta đang liệt kê thừa số nguyên tố nhỏ nhất của i .

Chú ý rằng với một hợp số n , thừa số nguyên tố nhỏ nhất của nó chắc chắn không vượt quá $\sqrt{n}$. Do đó

$$\begin{aligned} &\sum_{\substack{2 \leq p^e \leq n, e \geq 1 \\ p \text{ là số nguyên tố}}} f(p^e) \left(1 + \sum_{\substack{2 \leq x \leq \lfloor n/p^e \rfloor \\ x \text{ không chứa thừa số nguyên tố } \leq p}} f(x) \right) \\ &= \sum_{\substack{2 \leq p \leq \sqrt{n}, 2 \leq p^e \leq n, e \geq 1 \\ p \text{ là số nguyên tố}}} f(p^e) \left(1 + \sum_{\substack{2 \leq x \leq \lfloor n/p^e \rfloor \\ x \text{ không chứa thừa số nguyên tố } \leq p}} f(x) \right) + \sum_{\substack{\sqrt{n} < p \leq n \\ p \text{ là số nguyên tố}}} f(p). \end{aligned}$$

Đặt

$$g_{n,m} = \sum_{\substack{2 \leq x \leq n \\ x \text{ không chứa thừa số nguyên tố } \leq m}} f(x), \quad h_n = \sum_{\substack{2 \leq p \leq n \\ p \text{ là số nguyên tố}}} f(p).$$

Tương tự như trên, ta viết được truy hồi

$$g_{n,m} = \sum_{\substack{m < p \leq \sqrt{n}, p^e \leq n, e \geq 1 \\ p \text{ là số nguyên tố}}} f(p^e) ([e > 1] + g_{\lfloor n/p^e \rfloor, p}) + h_n - h_m.$$

Đáp án cần tính chính là $g_{n,0}$.

Nếu đã tính được mọi h_i , ta có hai lựa chọn: dùng trực tiếp công thức trên để tính; hoặc liệt kê m từ lớn xuống nhỏ để tính tất cả các giá trị g . Với cách thứ hai, chỉ khi $n > m^2$ thì giá trị chuyển mới khác 0, nên có thể tối ưu bằng tổng hậu tố theo đúng công thức chuyển.

Bây giờ xét cách tính h . Dễ thấy những h_i hữu dụng nhất định thỏa tồn tại số nguyên dương m sao cho $\lfloor n/m \rfloor = i$. Theo Bổ đề 1.3, số lượng i như vậy là $O(\sqrt{n})$. Nói chung, $f(p)$ là một đa thức bậc thấp theo p , có thể viết dưới dạng

$$f(p) = \sum_t a_t p^{b_t}.$$

Vì vậy, với mỗi p^{b_t} và mỗi i , ta chỉ cần cộng vào h_i a_t lần tổng lũy thừa bậc b_t của các số nguyên tố không vượt quá i .

Như vậy bài toán trở thành: cho n và s , với mọi $i = \lfloor n/m \rfloor$, tính

$$h_i = \sum_{\substack{p \leq i \\ p \text{ là số nguyên tố}}} p^s.$$

Ta có thể hiểu từ góc nhìn của sàng Eratosthenes: quá trình sàng là mỗi lần liệt kê một p , rồi loại bỏ mọi bội của p không nhỏ hơn p^2 . Đặt $h'_{i,j}$ là tổng lũy thừa bậc s của mọi số không vượt quá j còn lại sau khi sàng đã liệt kê i số nguyên tố đầu tiên, tức

$$h'_{i,j} = \sum_{\substack{1 \leq p \leq j \\ p=1, \text{ hoặc } p \text{ là số nguyên tố, hoặc } p \text{ không có thừa số nguyên tố } \leq p_i}} p^s.$$

Giả sử số nguyên tố thứ i là p_i . Xét quá trình sàng, với $j \geq p_i^2$ ta có

$$h'_{i,j} = h'_{i-1,j} - p_i^s (h'_{i-1,\lfloor j/p_i \rfloor} - h'_{i-1,p_i-1}).$$

Trong các trường hợp khác, giá trị không đổi. Vì vậy chỉ cần hiện thực trực tiếp. Thực chất, nửa sau của phần này hoàn toàn giống phần đầu trong sàng Zhouge.

2.3 Tính độ phức tạp thời gian

Với nửa sau của phép tính, chứng minh độ phức tạp hoàn toàn giống sàng Zhouge. Mỗi $\lfloor n/m \rfloor = i$ chỉ đóng góp khi liệt kê các số nguyên tố không vượt quá $\sqrt{i}$. Theo Định lý 1.4, có thể ước lượng độ phức tạp là

$$\begin{aligned} \sum_{i=1}^{\sqrt{n}} O\left(\frac{\sqrt{i}}{\log \sqrt{i}}\right) + \sum_{i=1}^{\sqrt{n}} O\left(\frac{\sqrt{n/i}}{\log \sqrt{n/i}}\right) &= O\left(\int_1^{\sqrt{n}} \frac{\sqrt{n/x}}{\log \sqrt{n/x}} dx\right) \\ &= O\left(\frac{n^{3/4}}{\log n}\right). \end{aligned}$$

Tiếp theo xét độ phức tạp của phần trước, tức việc tính $g_{n,0}$. Để tiện mô tả, trước hết đưa ra một định nghĩa.

Định nghĩa 2.1. Với số nguyên dương i , định nghĩa big_i là thừa số nguyên tố lớn nhất của i , small_i là thừa số nguyên tố nhỏ nhất của i , và cnt_i là số thừa số nguyên tố có tính bội của i . Khi $i = 1$, đặt

$$\text{big}_i = \text{small}_i = \infty, \quad \text{cnt}_i = 0.$$

Nếu dùng công thức chuyển của g để làm DP, độ phức tạp không gian là $O(\sqrt{n})$, còn chứng minh độ phức tạp thời gian tương tự phần sau, cũng là

$$O\left(\frac{n^{3/4}}{\log n}\right).$$

Nếu trực tiếp tính thô theo công thức ban đầu thì sao? Trong các thử nghiệm với dữ liệu nhỏ, thuật toán này trông còn tốt hơn

$$O\left(\frac{n^{3/4}}{\log n}\right),$$

vì vậy ta thử phân tích độ phức tạp thời gian của nó.

Xét quá trình đệ quy khi tính $g_{n,m}$ ở trên. Độ phức tạp thời gian chính là số lần đi vào đệ quy, tức số lần $h_n - h_m$ được cộng vào đáp án.

Với một lần đệ quy, giả sử đóng góp của số nguyên tố p được cộng trong lần đó thực chất là đóng góp của $F(l \times p)$ trong bài toán ban đầu. Ta tính đóng góp độ phức tạp của lần đệ quy này cho $l \times m$. Theo cách chuyển của ta, m hiển nhiên là thừa số nguyên tố lớn nhất của l . Vì mỗi giá trị hàm chỉ được thống kê đóng góp một lần, mỗi $l \times m$ chắc chắn khác nhau; đồng thời, mọi $l \times m$ không vượt quá n đều sẽ xuất hiện. Do đó độ phức tạp thực tế của thuật toán là số lượng i thỏa

$$i \times \text{big}_i \leq n,$$

trong đó big_i là thừa số nguyên tố lớn nhất của i .

Bổ đề 2.2. Với hằng số thực $0 < \alpha < 1$, đặt

$$Q(n) = \{i \leq n : \text{big}_i \leq i^\alpha\}.$$

Khi đó $|Q(n)| \sim n\rho(1/\alpha)$, trong đó ρ là hàm Dickman.

Bổ đề 2.3. Với mọi $x > 1$, $\rho(x) > 0$, giảm rất nhanh khi x tăng, và $\rho(x) \approx x^{-x}$.

Có thể xem hai bổ đề trên trong tài liệu tham khảo 3.

Định lý 2.4. Đặt

$$M(n) = \{i : i \times \text{big}_i \leq n\}.$$

Với mọi $0 < \alpha < 1$, ta có $|M(n)| = \Omega(n^\alpha)$.

Chứng minh. Lấy

$$P = \{i \leq n^\alpha : \text{big}_i \leq i^{1-\alpha}\}.$$

Theo Bổ đề 2.3, $|Q(n^\alpha)| = \Omega(n^\alpha)$. Mọi phần tử x trong tập P đều thỏa

$$x \times \text{big}_x \leq x^{2-\alpha} \leq n^{\alpha(2-\alpha)} \leq n,$$

và $P \subseteq M(n)$, nên $|M(n)| = \Omega(n^\alpha)$. ■

Định lý 2.5.

$$|M(n)| = \Theta(n^{1-\varepsilon}).$$

Chứng minh. Xét $\log \log n$ số nguyên tố đầu tiên, gọi p_i là số nguyên tố thứ i . Số lượng số có thừa số nguyên tố lớn nhất không vượt quá p_i không quá

$$(\log n)^{\log \log n} = O(n^\varepsilon),$$

còn các i khác thỏa $i \times \text{big}_i \leq n$ đều không vượt quá n/p_i . Kết hợp với Định lý 2.4 suy ra $|M(n)| = \Theta(n^{1-\varepsilon})$. ■

Cuối cùng thuật toán này được chứng minh là $\Theta(n^{1-\varepsilon})$, nhưng điều đó khác khá xa hiệu quả chạy thực tế. Giải thích hiện tượng này như thế nào?

Bổ đề 2.6.

$$\sum_{\substack{p \geq n \\ p \text{ là số nguyên tố}}} \frac{1}{p^2} = \Theta\left(\frac{1}{n \ln n}\right).$$

Chứng minh.

$$\begin{aligned} \sum_{\substack{p > n \\ p \text{ là số nguyên tố}}} \frac{1}{p^2} &= \int_n^\infty \frac{d\pi(x)}{x^2} \\ &= \Theta\left(\frac{\pi(x)}{x^2} \Big|_n^\infty - \int_n^\infty \pi(x) d\frac{1}{x^2}\right) \\ &= \Theta\left(-\frac{1}{n \ln n} + 2 \int_n^\infty \frac{dx}{x^2 \ln x}\right) \\ &= \Theta\left(-\frac{1}{n \ln n} - 2 \operatorname{li}\left(\frac{1}{n}\right)\right). \end{aligned}$$

Trong đó $\operatorname{li}(x)$ là hàm tích phân logarit. Tra cứu tài liệu có $\operatorname{li}(1/n) \sim -1/(n \ln n)$, nên mệnh đề được chứng minh. ■

Định lý 2.7. Khi $\operatorname{big}_i \geq n^{1/4}$, số lượng i thỏa điều kiện nhiều nhất là

$$O\left(\frac{n^{3/4}}{\log n}\right).$$

Chứng minh. Khi $\operatorname{big}_i \geq n^{1/4}$, ta liệt kê từng $p = \operatorname{big}_i$. Với mỗi p , có nhiều nhất $\lfloor n/p^2 \rfloor$ số thỏa điều kiện, nên phần này có độ phức tạp

$$O\left(\sum_{\substack{p \geq n^{1/4} \\ p \text{ là số nguyên tố}}} \frac{n}{p^2}\right) = O\left(\frac{n^{3/4}}{\log n}\right).$$

Theo bảng giá trị thực nghiệm, khi $n \leq 10^{13}$, với một số nguyên tố $p \leq n^{1/4}$, số lượng i thỏa $\operatorname{big}_i = p$ chỉ ở cấp $n^{1/3}$, còn tổng cộng chỉ có $n^{1/4}/\log n$ số nguyên tố như vậy, nên số phép tính ở cấp

$$\frac{n^{7/12}}{\log n}.$$

Hiện thực của thuật toán: <https://ideone.com/0l4aml>.

3 Tổng tiền tố lũy thừa bậc k của số nguyên tố

Thuật toán trước đó liên quan tới bài toán tính tổng tiền tố lũy thừa bậc k của số nguyên tố cho mọi $\lfloor n/i \rfloor$, và đã đưa ra một cách làm độ phức tạp $O(n^{3/4}/\log n)$. Nhưng đôi khi thứ ta cần chỉ là

$$\sum_{i=1}^n [i \text{ là số nguyên tố}] i^k.$$

Khi đó tồn tại cách làm có độ phức tạp tốt hơn. Trong phần tính độ phức tạp bên dưới, ta đều không xét ảnh hưởng của việc tính lũy thừa bậc k .

Định nghĩa 3.1. Với số nguyên dương n , định nghĩa $\pi_k(n)$ là hàm tổng tiền tố lũy thừa bậc k của số nguyên tố:

$$\pi_k(n) = \sum_{i=1}^n [i \text{ là số nguyên tố}] i^k.$$

Đặc biệt, π_0 chính là hàm đếm số nguyên tố, tức $\pi_0(n) = \pi(n)$.

Định nghĩa 3.2. Với số nguyên dương n và số nguyên dương a , định nghĩa hàm sàng bộ phận

$$\phi_k(n, a) = \sum_{i=1}^n [\text{small}_i > p_a] i^k,$$

trong đó p_a là số nguyên tố thứ a . Khi $a = 0$, $\phi_k(n, a) = \sum_{i=1}^n i^k$.

Định nghĩa 3.3. Với các số nguyên dương s, n, a , định nghĩa

$$P_{s,k}(n, a) = \sum_{i=1}^n [\text{small}_i > p_a, \text{cnt}_i = s] i^k.$$

Định lý 3.4 (phương pháp Meissel–Lehmer). Ta có

$$\phi_k(x, a) = \sum_{s \geq 0} P_{s,k}(x, a).$$

Khi $x^{1/3} \leq B = p_a \leq x^{1/2}$, ta có

$$\phi_k(x, a) = P_{0,k}(x, a) + P_{1,k}(x, a) + P_{2,k}(x, a) = 1 + \pi_k(x) - \pi_k(p_a) + P_{2,k}(x, a).$$

Đẳng thức trên suy trực tiếp từ định nghĩa, nên lược bỏ chứng minh. Chuyển về ta nhận được

$$\pi_k(x) = \phi_k(x, a) - P_{2,k}(x, a) + \pi_k(p_a) - 1.$$

Hiện cần tính $\pi_k(x)$, còn $\pi_k(p_a)$ có thể tính bằng sàng $O(B)$. Vì vậy chỉ cần xét cách tính $P_{2,k}(x, a)$ và $\phi_k(x, a)$.

3.1 Tính $P_{2,k}(x, a)$

Theo định nghĩa,

$$P_{2,k}(x, a) = \sum_{\substack{n \leq x, n=pq \\ p_a < p \leq q \leq x/p \\ p, q \text{ là số nguyên tố}}} n^k.$$

Đồng thời $B < q < x/B$, nên có thể liệt kê p , rồi tính đóng góp của mọi q khả thi. Dùng sàng tuyến tính để tính, độ phức tạp thời gian là

$$O\left(\frac{x}{B}\right).$$

3.2 Tính $\phi_k(x, a)$

Bổ đề 3.5.

$$\phi_k(x, a) = \phi_k(x, a-1) - p_a^k \phi_k(\lfloor x/p_a \rfloor, a-1).$$

Đẳng thức này hiển nhiên theo nguyên lý bao hàm - loại trừ.

Định lý 3.6.

$$\phi_k(x, a) = \sum_{\substack{\text{big}_n \leq p_a \\ \text{small}_n > p_b}} \mu(n) n^k \phi_k(\lfloor x/n \rfloor, b).$$

Chứng minh. Xét quá trình chuyển. Giả sử trong quá trình chuyển ta lần lượt loại bỏ các số nguyên tố $p_1, p_2, \dots, p_l$, và đặt $p_1 p_2 \dots p_l = n$. Vì số n có thể được phân tích duy nhất bởi các số nguyên tố từ thứ $b + 1$ đến thứ a , nên khi

$$\text{big}_n \leq p_a, \quad \text{small}_n > p_b$$

sẽ sinh ra đóng góp $\mu(n) n^k$, đúng như công thức. ■

Từ định lý trên, ta có

$$\phi_k(x, a) = \sum_{\text{big}_n \leq p_a} \mu(n) n^k \left\lfloor \frac{x}{n} \right\rfloor^k.$$

Trước hết xét cách thống kê thô đóng góp của $n \leq p_a = B$. Phần còn lại cần thống kê là

$$\sum_{\substack{n > B \\ \text{big}_n \leq p_a}} \mu(n) n^k \left\lfloor \frac{x}{n} \right\rfloor^k.$$

Định lý 3.7.

$$\sum_{\substack{n > B \\ \text{big}_n \leq p_a}} \mu(n) n^k \left\lfloor \frac{x}{n} \right\rfloor^k = \sum_{B < n \leq B \text{small}_n} \mu(n) n^k \phi_k\left(\left\lfloor \frac{x}{n} \right\rfloor, \pi_0(\text{small}_n) - 1\right).$$

Chứng minh. Khai triển vế phải bằng Định lý 3.6 là nhận được thông tin tương đương với vế trái. ■

Thực chất điều này tương đương với việc thêm nhánh cắt dừng đệ quy khi $n > B$ trong DFS tính $\phi_k(x, a)$.

Đặt $p = \text{small}_n$, $m = n / \text{small}_n$. Khi đó $\mu(m) = -\mu(n)$, và phần cần tính là

$$- \sum_{p \leq B} \sum_{\substack{\text{small}_m > p \\ m \leq B < mp}} \mu(m) m^k p^k \phi_k\left(\left\lfloor \frac{x}{mp} \right\rfloor, \pi_0(p) - 1\right).$$

Ta xét cách tính biểu thức này.

3.2.1 $p \leq \sqrt{B}$. Với phần này, ta có thể tính thô mọi giá trị ϕ , rồi liệt kê p và m để thống kê.

Theo Bổ đề 1.3, số trạng thái hữu dụng chỉ là

$$O\left(\frac{x}{B \log B}\right),$$

nên đóng góp phần này có thể tính trong

$$O\left(\frac{x}{B \log B}\right).$$

3.2.2 $p > x^{1/3}$. Khi $p > \sqrt{B}$, nếu m không phải số nguyên tố thì $m > p^2 > B$, do đó không tồn tại m khả thi. Điều này gợi ý rằng ta chỉ cần xét trường hợp m là số nguyên tố. Khi đó tổng trở thành

$$\sum_{x^{1/3} < p \leq B} \sum_{\substack{q > p \\ q \leq B < qp}} (pq)^k \phi_k \left(\left\lfloor \frac{x}{pq} \right\rfloor, \pi_0(p) - 1 \right),$$

trong đó p, q đều là số nguyên tố.

Khi $p > x^{1/3}$, ta có $\lfloor x/(pq) \rfloor < x^{1/3} < p$. Theo định nghĩa,

$$\phi_k \left(\left\lfloor \frac{x}{pq} \right\rfloor, \pi_0(p) - 1 \right) = 1,$$

nên tổng trở thành

$$\sum_{x^{1/3} < p \leq B} \sum_{\substack{q > p \\ q \leq B < qp}} (pq)^k.$$

Vì tổng lũy thừa bậc k của các số nguyên tố có thể được thống kê bằng thông tin tiền xử lý ở mục 3.1, chỉ cần liệt kê p là tính được mọi đóng góp. Độ phức tạp thời gian của phần này là $O(B)$.

3.2.3 $\sqrt{B} < p \leq x^{1/3}$. Trong phần này, m vẫn có tính chất là số nguyên tố, và hiển nhiên $p^2 > B$. Vì vậy bài toán là tính

$$\sum_{\sqrt{B} < p \leq x^{1/3}} \sum_{p < q \leq B} (pq)^k \phi_k \left(\left\lfloor \frac{x}{pq} \right\rfloor, \pi_0(p) - 1 \right).$$

Ta dùng định nghĩa để tính ϕ_k :

$$\begin{aligned} & \sum_{\sqrt{B} < p \leq x^{1/3}} \sum_{p < q \leq B} (pq)^k \phi_k \left(\left\lfloor \frac{x}{pq} \right\rfloor, \pi_0(p) - 1 \right) \\ &= \sum_{\sqrt{B} < p \leq x^{1/3}} \sum_{p < q \leq B} (pq)^k \sum_{\substack{pqr \leq x \\ \text{small}_r \geq p}} r^k \\ &= \sum_{r=1}^{\lfloor x/B \rfloor} r^k \sum_{\sqrt{B} < p \leq \min(x^{1/3}, \text{small}_r)} p^k \sum_{p < q \leq \min(\lfloor x/(pr) \rfloor, B)} q^k. \end{aligned}$$

Ta có thể liệt kê p , đồng thời dùng cây Fenwick để bảo trì mọi r thỏa $\text{small}_r \geq p$. Khi truy vấn, theo Bổ đề 1.3, giá trị $\lfloor x/(pr) \rfloor$ chỉ có $O(\sqrt{x/p})$ loại, vì vậy khi liệt kê $\lfloor x/(pr) \rfloor$, chỉ cần truy vấn cây Fenwick $O(\sqrt{x/p})$ lần.

Định lý 3.8. Với mọi $0 < \alpha < 1$, số lượng số không vượt quá n có đúng $k \geq 1$ thừa số nguyên tố và thừa số nguyên tố nhỏ nhất không nhỏ hơn n^α là

$$O\left(\frac{n}{\log^k n}\right).$$

Chứng minh. Xét quy nạp. Khi $k = 1$, mệnh đề chính là định lý số nguyên tố. Khi $k \geq 2$, theo giả thiết quy nạp, mệnh đề đúng với $k - 1$ thừa số nguyên tố; đồng thời

$$O\left(\int_{n^\alpha}^{\sqrt{n}} \frac{n/p}{\log^{k-1}(n/p)} d\pi(p)\right) = O\left(\frac{n}{\log^k n}\right).$$

Vì vậy số lượng số có đúng $k \geq 1$ thừa số nguyên tố và thừa số nguyên tố nhỏ nhất không nhỏ hơn n^α là $O(n/\log^k n)$. ■

Theo Định lý 3.8, số lần sửa đoạn trên cây Fenwick không vượt quá

$$O\left(\frac{x^{2/3}}{\log^2 x}\right).$$

Mỗi lần sửa tốn $O(\log n)$, nên độ phức tạp sửa của phần này là $O(x^{2/3})$.

Xét số lần truy vấn.

Định lý 3.9.

$$\sum_{p \leq m} \sqrt{\frac{x}{p}} = O\left(\frac{\sqrt{mx}}{\log m}\right).$$

Chứng minh.

$$\begin{aligned} \sum_{p \leq m} \sqrt{\frac{x}{p}} &= \sqrt{x} \int_1^m \frac{d\pi(p)}{\sqrt{p}} \\ &= \sqrt{x} \left(\frac{\pi(m)}{\sqrt{m}} - \int_1^m \pi(p) d\frac{1}{\sqrt{p}} \right) \\ &= O\left(\frac{\sqrt{mx}}{\log m}\right). \end{aligned}$$

■

Thay $m = x^{1/3}$ vào định lý trên, số lần truy vấn vẫn là

$$O\left(\frac{x^{2/3}}{\log x}\right),$$

nên độ phức tạp truy vấn cũng là $O(x^{2/3})$.

Tổng độ phức tạp là

$$O\left(x^{2/3} + \frac{x}{B} + \frac{x}{B \log B}\right).$$

Lấy $B = x^{1/3} \log^{2/3} x$ để cân bằng độ phức tạp phần sau, tổng độ phức tạp vẫn là $O(x^{2/3})$.

3.3 Tối ưu

Nút thắt độ phức tạp của cách làm trước không nằm ở phần chia khối, mà nằm ở đoạn $\sqrt{B} < p \leq x^{1/3}$.

Điều này gợi ý cho ta tối ưu thêm.

Vấn xét biểu thức cần tính:

$$\begin{aligned} &\sum_{\sqrt{B} < p \leq x^{1/3}} \sum_{p < q \leq B} (pq)^k \phi_k\left(\left\lfloor \frac{x}{pq} \right\rfloor, \pi_0(p) - 1\right) \\ &= \sum_{r=1}^{\lfloor x/B \rfloor} r^k \sum_{\sqrt{B} < p \leq \min(x^{1/3}, \text{small}_r)} p^k \sum_{p < q \leq \min(\lfloor x/(pr) \rfloor, B)} q^k. \end{aligned}$$

Thời gian khởi tạo cây Fenwick là $O(x/B)$. Sau đó khi sửa r trên cây Fenwick, điều đó nghĩa là thừa số nguyên tố nhỏ nhất của r nằm giữa $\sqrt{B}$ và $x^{1/3}$.

Khi r là số nguyên tố, nếu $r \geq x^{1/3}$, thì với mọi p thỏa điều kiện, r đều sinh đóng góp. Vì vậy ta có thể liệt kê p , rồi thống kê toàn bộ đóng góp khi r là số nguyên tố. Độ phức tạp thời gian là

$$O\left(\frac{x^{2/3}}{\log x}\right).$$

Khi r không phải số nguyên tố, theo Định lý 3.8, số lần sửa hữu dụng chỉ là

$$O\left(\frac{x^{2/3}}{\log^2 x}\right),$$

không ảnh hưởng tới độ phức tạp tổng thể. Đồng thời r không phải số nguyên tố cũng có nghĩa $r > p^2$. Điều kiện tồn tại q là $x/(pr) > p$, tức $p^2 r \leq x$, vì vậy $p \leq x^{1/4}$.

Theo Định lý 3.9, số lần truy vấn là

$$O\left(\frac{x^{5/8}}{\log x}\right),$$

cũng không ảnh hưởng tới độ phức tạp tổng thể. Như vậy phần này được tối ưu thành

$$O\left(\frac{x^{2/3}}{\log x}\right),$$

và tổng độ phức tạp được tối ưu thành

$$O\left(\left(\frac{x}{\log x}\right)^{2/3}\right).$$

4 Tổng tiền tố hàm ước

4.1 Dẫn nhập bài toán

Định nghĩa $\sigma(i)$ là số ước của i . Cho số nguyên dương $n < 2^{63}$, hãy tính

$$\sum_{i=1}^n \sigma(i).$$

Nguồn: spoj.com/problems/DIVCNT1/.

4.2 Cách làm ngây thơ

Với bài toán này, có một cách làm ngây thơ rất hiển nhiên:

$$\begin{aligned} \sum_{i=1}^n \sigma(i) &= \sum_{i=1}^n \sum_{d|i} 1 \\ &= \sum_{d=1}^n \left\lfloor \frac{n}{d} \right\rfloor. \end{aligned}$$

Dễ thấy công thức này thống kê số cặp (x, y) thỏa $x, y > 0$ và $xy \leq n$. Đường cong đối xứng qua $x = y$, nên

$$\sum_{d=1}^n \left\lfloor \frac{n}{d} \right\rfloor = 2 \sum_{d=1}^{\lfloor \sqrt{n} \rfloor} \left\lfloor \frac{n}{d} \right\rfloor - \lfloor \sqrt{n} \rfloor^2.$$

Vì vậy chỉ cần liệt kê d là tính được đáp án, độ phức tạp thời gian là $O(\sqrt{n})$. Nhưng trong bài này n có thể đạt tới cỡ 2^{63} , nên thuật toán ngây thơ sẽ quá thời gian.

4.3 Stern–Brocot Tree và dãy Farey

Trước khi giới thiệu thuật toán, ta giới thiệu một số công cụ được dùng về sau.

Định nghĩa 4.1. Sắp xếp từ nhỏ đến lớn mọi phân số tối giản nằm giữa 0 và 1 có mẫu không vượt quá n , ta nhận được một dãy; dãy này được gọi là dãy Farey bậc n . Nếu tồn tại một dãy Farey trong đó hai phân số liên tiếp xuất hiện, thì hai phân số ấy được gọi là *Farey neighbour*.

Bổ đề 4.2. Trong một dãy Farey bất kỳ, nếu hai hạng tử liên tiếp thỏa

$$\frac{a}{b} > \frac{c}{d},$$

thì $ad - bc = 1$, và chiều ngược lại cũng đúng.

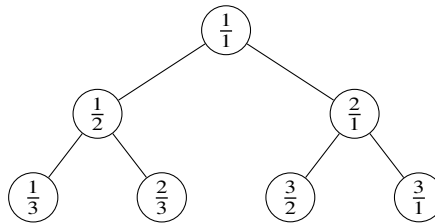
Chứng minh liên quan của bổ đề này có thể tra cứu trong tài liệu. Trong bài viết này, nếu $\frac{a}{b}$ và $\frac{c}{d}$ là Farey neighbour thì $\frac{a}{b}$ và $\frac{c}{d}$ cũng được gọi là Farey neighbour theo ngữ cảnh tương ứng; hiển nhiên trong trường hợp đó bổ đề vẫn đúng.

Định nghĩa 4.3 (cách dựng Stern–Brocot Tree). Ta xây dựng một cây nhị phân đầy đủ vô hạn. Giả sử đã có i tầng đầu của Stern–Brocot Tree. Với một nút ở tầng thứ i đại diện cho phân tử $\frac{x}{y}$, xét trong i tầng đầu phân tử lớn nhất nhỏ hơn $\frac{x}{y}$, ký hiệu $\frac{a}{b}$ (nếu không tồn tại thì là $\frac{0}{1}$); và phân tử nhỏ nhất lớn hơn $\frac{x}{y}$, ký hiệu $\frac{c}{d}$ (nếu không tồn tại thì là $\frac{1}{0}$). Khi đó con trái của nút này đại diện cho

$$\frac{a+x}{b+y},$$

còn con phải đại diện cho

$$\frac{c+x}{d+y}.$$



Stern–Brocot Tree có các tính chất sau:

Tính chất 4.3.1. Stern–Brocot Tree là một cây nhị phân đầy đủ vô hạn.

Tính chất 4.3.2. Các số do các nút trên Stern–Brocot Tree đại diện đều là phân số tối giản, và tương ứng một-một với các số hữu tỉ.

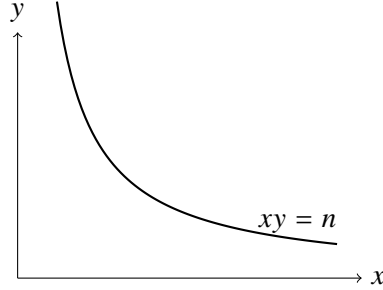
Tính chất 4.3.3. Stern–Brocot Tree có tính chất của cây tìm kiếm: mọi số do cây con trái đại diện đều nhỏ hơn số ở gốc, còn mọi số do cây con phải đại diện đều lớn hơn số ở gốc.

Tính chất 4.3.4. Vẫn dùng định nghĩa trong cách dựng Stern–Brocot Tree, phân tử $\frac{x}{y}$ và phân tử $\frac{a}{b}$, cũng như phân tử $\frac{x}{y}$ và phân tử $\frac{c}{d}$, đều là Farey neighbour.

Các nội dung trên có thể tra cứu trong tài liệu tham khảo 5.

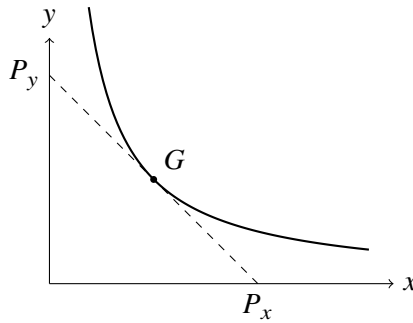
4.4 Ý tưởng ban đầu

Thống kê trực tiếp hiển nhiên không dễ xử lý, nên ta chuyển thành thống kê số điểm nguyên dưới đường cong $xy = n$.



4.5 Dùng Stern–Brocot Tree để cắt đường cong

Xét một cách cắt đường cong như sau. Hiện ta cần thống kê mọi điểm nguyên nằm dưới toàn bộ đường cong. Tìm điểm G trên đồ thị hàm số mà tiếp tuyến có hệ số góc -1 , rồi dựng đường thẳng tiếp xúc với hàm số tại đó. Đường thẳng này cắt trục x và trục y lần lượt tại P_x và P_y .



Thống kê số điểm dưới tiếp tuyến này là dễ: giả sử tọa độ của G là (x_1, y_1) , còn O là $(0, 0)$, thì số điểm nguyên thỏa điều kiện chính là số điểm thỏa $x + y < x_1 + y_1$. Một ý tưởng tự nhiên là sau khi thống kê xong phần này, tiếp tục lặp lại để tính số điểm nguyên ở hai bên.

Định nghĩa 4.4. Giả sử x_0, y_0, a, b, c, d là các số nguyên không âm. Khi $\frac{a}{b}$ và $\frac{c}{d}$ là Farey neighbour, định nghĩa $S(x_0, y_0, a, b, c, d)$ như sau: gọi $P = (x_0, y_0)$; từ P kẻ hai đường thẳng có hệ số góc lần lượt là $-a/b$ và $-c/d$, cắt $xy = n$ tại Q và R . Số điểm nguyên trong tam giác cong PQR là $S(x_0, y_0, a, b, c, d)$. Khi $\frac{a}{b}$ và $\frac{c}{d}$ không phải Farey neighbour, định nghĩa $S(x_0, y_0, a, b, c, d) = 0$.

Vì trục y có thể xem là một tia có hệ số góc $-1/0$, thứ ta cần tính là

$$S(0, 0, 1, 0, 0, 1).$$

4.5.1 Chuyển đổi giữa hệ tọa độ $v - u$ và hệ tọa độ $y - x$. Xét cách tính $S(x_0, y_0, a, b, c, d)$. Vì miền nằm phía trên PR và PQ , ta lấy P làm gốc tọa độ và lấy PR, PQ làm trục để dựng hệ tọa độ mới.

Đặt PR là trục u , PQ là trục v . Tọa độ trên một hướng tăng thêm một đơn vị nghĩa là đi theo hướng đó tới điểm nguyên kế tiếp chạm được. Theo định nghĩa trước đó, a, b nguyên tố cùng nhau, và c, d cũng nguyên tố cùng nhau, nên

$$x = x_0 + ud - vb, \quad y = y_0 - uc + va.$$

Đồng thời, nếu biết các số nguyên x, y , ta cũng có thể tìm được một cặp số nguyên u, v tương ứng. Nhân hai đẳng thức lần lượt với a và b rồi cộng lại:

$$ax + by = ax_0 + by_0 + (ad - bc)u.$$

Theo Tính chất 4.2, $ad - bc = 1$, nên u là số nguyên; tương tự, v cũng là số nguyên. Do đó các điểm trong hệ tọa độ $y - x$ và hệ tọa độ $v - u$ tương ứng một-một.

Xét đường cong tương ứng với $xy = n$ trong hệ tọa độ $v - u$. Đặt

$$w_1 = ax_0 + by_0, \quad w_2 = cx_0 + dy_0.$$

Giải ra được

$$x_0 = dw_1 - bw_2, \quad y_0 = aw_2 - cw_1.$$

Do đó

$$x = d(u + w_1) - b(v + w_2), \quad y = a(v + w_2) - c(u + w_1).$$

Trong hệ tọa độ $v - u$, đường cong $xy = n$ trở thành

$$(d(u + w_1) - b(v + w_2))(a(v + w_2) - c(u + w_1)) = n,$$

tức

$$(u + w_1)(v + w_2) - cd(u + w_1)^2 - ab(v + w_2)^2 = n.$$

Gọi đường cong này là $H(u, v)$.

Vì $w_1, w_2, a/b, c/d \geq 0$, với mọi $u > 0$ tồn tại duy nhất v tương ứng, và với mọi $v > 0$ tồn tại duy nhất u tương ứng. Đặt $v = V(u)$, $u = U(v)$. Khi đó U và V đều là hàm lõm trong góc phần tư thứ nhất, và có tính chất tương tự $xy = n$. Kết luận này có thể chứng minh bằng cách tính trực tiếp biểu thức giải tích hoặc dùng vi phân; ở đây lược bỏ.

Giả sử tọa độ trục v của Q là h , tọa độ trục u của R là w . Thứ ta thực chất cần thống kê là số điểm nguyên trong phần tương ứng dưới đường cong H của hệ tọa độ mới.

4.5.2 Dùng chuyển hệ tọa độ để thống kê số điểm nguyên. Quay lại ý tưởng ban đầu: trong quá trình đếm điểm nguyên, ta muốn dùng phương pháp đã nói, tức cắt đường cong tại điểm có tiếp tuyến hệ số góc -1 , rồi chia để trị xuống dưới. Với bài toán sau khi chuyển đổi ở trên, khi $\min(w, h)$ rất nhỏ thì ta có thể liệt kê thô tọa độ u hoặc v , nên tạm thời không xét phần đó.

Trước hết tìm điểm G có tiếp tuyến hệ số góc -1 . Chú ý rằng trong hệ tọa độ $v - u$, đường thẳng có hệ số góc -1 thực chất có hệ số góc

$$-\frac{b+d}{a+c}$$

trong hệ tọa độ $y - x$. Vì vậy G tồn tại trong hệ $y - x$, và cũng tồn tại trong hệ $v - u$.

Gọi G_u, G_v là tọa độ u, v của G , và G_x, G_y là tọa độ x, y của G . Tìm hai điểm nguyên S, T thỏa

$$S_u \leq G_u < S_u + 1 = T_u, \quad S_v = \lfloor V(S_u) \rfloor, \quad T_v = \lfloor V(T_u) \rfloor.$$

Vì tiếp tuyến tại G có hệ số góc -1 , hai điểm S, T nhất định tồn tại, đồng thời có thể bảo đảm S và T không trùng nhau. Qua S, T , kẻ các đường thẳng song song với tiếp tuyến tại G .

Ta có thể thống kê trước số điểm nguyên trong phần hình có biên đều đặn giữa hai đường song song này, bao gồm cả biên; phần này dễ xử lý. Vì đường thẳng có hệ số góc -1 trong hệ $v - u$ tương ứng với hệ số góc $-(b+d)/(a+c)$ trong hệ $y - x$, còn tính chất của Stern–Brocot Tree bảo đảm

$$\frac{b+d}{a+c}$$

đồng thời là Farey neighbour với $\frac{a}{c}$ và $\frac{d}{b}$, nên các phần điểm nguyên còn lại có thể được tính lặp lại.

Nối P với G . Ta lần lượt thống kê những điểm nguyên chưa được tính ở miền bóng phía trên và phía bên phải PG . Dưới đây chỉ xét miền phía trên.

Gọi A là giao điểm của trục v với đường hệ số góc -1 đi qua S , và M là điểm nhận được bằng cách tịnh tiến A lên trên một đơn vị.

Định lý 4.5. Qua M , kẻ một đường thẳng có hệ số góc -1 cắt đường cong tại N . Tổng số điểm nguyên trong miền bóng chưa được thống kê phía trên PG chính là tổng số điểm nguyên trong miền bóng phía trên MN và PG , kể cả biên, tức

$$S(B_x, B_y, a, b, a + c, b + d).$$

Chứng minh. Dễ thấy N nằm phía trên PG ; nếu không, điều đó mâu thuẫn với cách chọn $S_v = \lfloor V(S_u) \rfloor$. Vì vậy mọi điểm nguyên trong phần bóng phía trên MN và PG đều chưa được thống kê.

Đồng thời, do $S_u < G_u < S_u + 1 = T_u$, mọi điểm nằm dưới MN và phía trên PG chắc chắn đều thuộc miền đều đặn đã thống kê. Kết luận được chứng minh. ■

Miền bên phải có thể xử lý tương tự. Giả sử C là điểm nhận được khi tịnh tiến B sang phải một đơn vị, thì số điểm nguyên phần này là

$$S(C_x, C_y, a + c, b + d, c, d).$$

Tài liệu tham khảo 6 có chi tiết hiện thực của thuật toán này.

4.6 Chứng minh độ phức tạp thời gian

Toàn bộ thuật toán gồm hai phần: liệt kê $O(n^{1/3})$ cột đầu, và tính $O(n^{1/2})$ cột phía sau. Phần liệt kê phía trước cho thấy độ phức tạp thời gian của thuật toán ít nhất là $O(n^{1/3})$; tiếp theo xét độ phức tạp của phần tính phía sau.

Trong quá trình lặp, khi $w < 1$ hoặc $h < 1$ thì ta trực tiếp trả lời, nên chỉ cần tính số lần lặp không trả lời trực tiếp. Nếu một lần lặp $S(x_0, y_0, a, b, c, d)$ không được trả lời trực tiếp, điều đó nghĩa là $w \geq 1$ và $h \geq 1$, cũng tức trong hệ tọa độ $v - u$ đồng thời tồn tại hai điểm $(1, 0)$ và $(0, 1)$. Theo công thức

$$x = d(u + w_1) - b(v + w_2),$$

hiệu hoành độ x giữa điểm có hoành độ lớn nhất và điểm có hoành độ nhỏ nhất trong hệ $v - u$ hiện tại ít nhất là $b + d$. Hiệu này hiển nhiên không vượt quá hiệu hoành độ giữa hai điểm trên đường cong có tiếp tuyến hệ số góc $-c/d$ và $-a/b$.

Từ đó có thể lập điều kiện

$$O\left(\sum_{ad-bc=1} \left[\sqrt{\frac{n}{d}} - \sqrt{\frac{n}{b}} \geq b + d\right]\right).$$

Biến đổi tương đương,

$$\begin{aligned} & O\left(\sum_{ad-bc=1} \left[\sqrt{\frac{d}{c}} - \sqrt{\frac{b}{a}} \geq \frac{b+d}{\sqrt{n}}\right]\right) \\ &= O\left(\sum_{ad-bc=1} \left[\frac{\sqrt{ad-bc}}{\sqrt{ac}} \cdot \frac{b+d}{\sqrt{n}} \geq 1\right]\right) \\ &= O\left(\sum_{ad-bc=1} \left[\frac{\sqrt{bc+1} - \sqrt{bc}}{\sqrt{ac}} \cdot \frac{b+d}{\sqrt{n}} \geq 1\right]\right). \end{aligned}$$

Bổ đề 4.6.

$$\sqrt{w+1} - \sqrt{w} = O\left(\frac{1}{\sqrt{w}}\right).$$

Chứng minh. Lấy $f(w) = \sqrt{w+1} - \sqrt{w}$, rồi khai triển Taylor tại $w = \infty$ là nhận được kết luận. ■

Theo bổ đề này,

$$\sqrt{bc+1} - \sqrt{bc} = O\left(\frac{1}{\sqrt{bc}}\right),$$

nên số lần lặp được chặn bởi

$$O\left(\sum_{ad-bc=1} [abc^2(b+d)^2 \leq n]\right).$$

Liệt kê $bc = x$, ta được

$$\begin{aligned} O\left(\sum_{ad-bc=1} [abc^2(b+d)^2 \leq n]\right) &= O\left(\sum_x \sum_{b|x} \sum_{a|x+1} \left[abx + \frac{x^4}{a^3b} + x \leq n\right]\right) \\ &= O\left(\sum_{x=1}^{n^{1/3}} \sigma(x)\sigma(x+1)\right). \end{aligned}$$

Theo $xy \leq (x+y)^2/2$, suy ra

$$\sum_{x=1}^{n^{1/3}} \sigma(x)\sigma(x+1) \leq \sum_{x=1}^{n^{1/3}} \frac{\sigma^2(x) + \sigma^2(x+1)}{2} = O\left(\sum_{x=1}^{n^{1/3}} \sigma^2(x)\right).$$

Định lý 4.7.

$$\sum_{i=1}^n \sigma^2(i) = \Theta(n \log^3 n).$$

Dãy số này chính là dãy A061502 trên OEIS. Vì vậy ta có chứng minh rằng phần này có độ phức tạp không vượt quá

$$O(n^{1/3} \log^3 n).$$

Hiển nhiên ta không cần tính số điểm nguyên dưới toàn bộ hình, mà có thể chỉ thống kê số điểm nguyên dưới một phần đường cong. Tuy nhiên, phân tích độ phức tạp ở trên chỉ xét trường hợp $\frac{a}{b}, \frac{c}{d}$ là các số hữu tỉ dương. Ta chưa tính trường hợp $a = 1, b = 0$ hoặc $c = 0, d = 1$, trong khi số lần xuất hiện của riêng trường hợp này đã là $O(\sqrt{n})$.

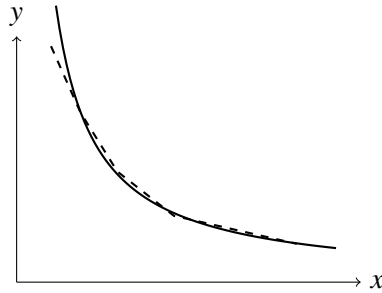
Cách xử lý rất đơn giản: tính thô số điểm nguyên trong $O(B)$ cột đầu. Khi $a = 1, b = 0$, nhất định có $d = 1$ và c là số nguyên; c hiển nhiên không vượt quá trị tuyệt đối hệ số góc tiếp tuyến tại $x = B$. Vì vậy số lần xuất hiện của trường hợp này là $O(\sqrt{n/B})$. Lấy $B = n^{1/3}$, số trường hợp phần này là $O(n^{1/3})$.

Chứng minh trên không chặt. Nghe nói độ phức tạp thời gian của thuật toán này có thể chứng minh tới $O(n^{1/3} \log n)$, nhưng tác giả chưa tìm được chứng minh liên quan, nên chỉ đưa ra một chặn trên để chứng minh độ phức tạp thời gian là

$$\tilde{O}(n^{1/3}).$$

4.7 Tối ưu

Thực ra, ta không cần cắt đường cong lặp đi lặp lại như trên. Có thể trực tiếp dùng Stern–Brocot Tree để xấp xỉ đường cong ban đầu: ta chỉ thống kê các điểm nguyên nằm nghiêm ngặt phía dưới đường gấp khúc xấp xỉ.



Trong hiện thực cụ thể, giả sử điểm đang xấp xỉ là (x, y) . Mỗi lần xấp xỉ ta chọn, trong mọi điểm (p, q) nằm bên phải điểm hiện tại và không nằm trong đường cong, điểm có

$$\frac{q - y}{p - x}$$

lớn nhất, và nếu hòa thì chọn p nhỏ nhất; sau đó thêm đoạn gấp khúc này để bảo đảm mọi điểm nằm dưới đường gấp khúc đều nằm trong đường cong. Dựa vào hệ số góc của đường dùng để xấp xỉ ở lần trước, ta chỉ cần chặt nhị phân trên Stern–Brocot Tree để tìm hệ số góc $-a/b$ của đoạn gấp khúc lần này, rồi di chuyển từ (x, y) tới $(x + a, y - b)$, và thống kê số điểm nguyên có tung độ nằm trong $[y - b, y)$.

Như vậy phương án xấp xỉ bằng đường gấp khúc nhất định là duy nhất và tồn tại, vì điểm $(1, n + 1)$ chắc chắn nằm trên đường gấp khúc xấp xỉ.

Tiếp theo xét độ phức tạp thời gian. Ta có thể tịnh tiến đường gấp khúc xuống dưới một đơn vị. Để thấy khoảng cách từ mỗi đoạn của đường gấp khúc tới đường cong đều không vượt quá 1, và giữa đường gấp khúc với đường cong không có bất kỳ điểm nào. Vì vậy có thể tịnh tiến từng đoạn của đường gấp khúc tới vị trí tiếp xúc với đường cong. Theo ý tưởng của cách làm trước, mỗi lần tương đương với việc lấy một đường thẳng hệ số góc -1 trong hệ tọa độ $v - u$ cho tới khi $w, h \leq 1$. Vì vậy độ phức tạp thời gian của thuật toán này không tệ hơn thuật toán trước.

4.8 Mở rộng

Hiển nhiên, thuật toán tối ưu dùng Stern–Brocot Tree để trực tiếp xấp xỉ đường cong ở trên không chỉ giải được bài toán tính số điểm nguyên dưới hyperbol $xy = n$. Ngay cả khi điểm nguyên có trọng số, cách làm cũng không bị ảnh hưởng đáng kể. Đồng thời, khi cần tính số lượng hoặc tổng trọng số của các điểm nguyên nằm trong miền do các đường cong có tính lồi/lõm khác với các trục tọa độ bao lại, thuật toán này cũng thường có biểu hiện tốt hơn rất nhiều so với thuật toán thô, ví dụ bài toán đếm điểm nguyên trong hình tròn. Có thể nói, thuật toán này có triển vọng mở rộng rất lớn trong lớp bài toán đếm như vậy.

5 Tổng kết

Bài viết này chủ yếu giới thiệu ba bài toán tổng hàm số học đặc biệt, đồng thời đưa ra ba hướng giải quyết khác nhau. Khi giải bài toán thứ nhất, ta dùng một tư tưởng thường gặp trong tính hàm nhân tính: liệt kê thừa số nguyên tố. Khi giải bài toán thứ hai, ta dùng chia khối để giảm độ phức tạp của cách làm ban đầu. Khi giải bài toán thứ ba, ta dùng phương pháp kết hợp số học với hình học và biến đổi hệ tọa độ để nhận được một cách đếm điểm, rồi dùng chặt nhị phân trên Stern–Brocot Tree để có một hiện thực tốt hơn. Tuy ba bài toán này đã được giải quyết, tác giả hy vọng các tư tưởng dùng trong quá trình giải có thể gợi mở cho mọi người, và cũng hy vọng các bạn có hứng thú tiếp tục nghiên cứu, thảo luận thêm.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng giao lưu và học tập.

Cảm ơn thầy Ye Guoping đã giúp đỡ tôi trong học tập và cuộc sống.

Cảm ơn bạn Liu Chengao và đàn anh Tang Jingzhe đã giúp đỡ cho bài viết này.

Cảm ơn các bạn Chen Jianglun, Liu Chengao, Zhong Ziqian, các đàn anh Zhang Qingchuan và Wu Hang đã đọc duyệt bài viết này.

Cảm ơn cha mẹ đã thấu hiểu và quan tâm tới tôi.

Tài liệu tham khảo

1. Pan Chengbiao, *Từ Chebyshev tới Erdos: chứng minh sơ cấp của định lý số nguyên tố*.
2. Ren Zhizhou, *Một vài phương pháp tính tổng hàm nhân tính*, tập luận văn đội tuyển quốc gia năm 2016.
3. <http://mathworld.wolfram.com/DickmanFunction.html>.
4. Zhang Bohang, <https://zhuanlan.zhihu.com/p/33544708>.
5. Wikipedia, *Stern–Brocot tree*, https://en.wikipedia.org/wiki/Stern-Brocot_tree.
6. Richard Sladkey, *A Successive Approximation Algorithm for Computing the Divisor Summatory Function* (draft).

12 Bài 10: Bàn sơ lược về ứng dụng DFT trong thi tin học

Liu Chengao, THPT trực thuộc Đại học Công nghiệp Tây Bắc

Tóm tắt

Bài viết này thảo luận ứng dụng của biến đổi Fourier rời rạc trong thi tin học. Xoay quanh đối tượng nghiên cứu là ánh xạ từ tập hợp tới vành, bài viết trước hết dẫn nhập công cụ DFT từ góc nhìn đa thức, sau đó thảo luận các mở rộng trên vành giao hoán không có ước của không và trên nửa nhóm giao hoán hữu hạn. Cuối cùng, bài viết dùng công cụ này để tối ưu việc tính tích chập tổng quát, thu được phương pháp hiệu quả hơn thuật toán thô.

1 Tổng quan

1.1 Dẫn nhập

Biến đổi Fourier rời rạc (Discrete Fourier transform, DFT) là một phương pháp đại số rất quan trọng và có nhiều ứng dụng. Trong thi tin học, nó có thể được dùng cho suy diễn toán học, hoặc để tính nhanh tích chập và một số biến đổi tuyến tính, chẳng hạn nhân đa thức.

Những năm gần đây trong giới thi đấu, nhiều thuật toán đại số tuyến tính dựa trên DFT đã trở nên phổ biến. DFT có thể dẫn ra các thuật toán tích chập nhanh; muộn nhất tới năm 2014, các thuật toán hàm sơ cấp nhanh của đa thức dựa trên FFT và lặp Newton đã được đưa vào [5]; về sau nhiều thuật toán khác như Berlekamp–Massey cũng được đưa vào. Các thuật toán đại số tuyến tính này đem lại rất nhiều thuận lợi cho sự phát triển nội dung thi đấu.

Tuy nhiên, các ứng dụng đa dạng ấy đều không rời khỏi một phép toán cơ bản: DFT. Vì vậy, bài viết này quay lại bản thân DFT, thảo luận ứng dụng và mở rộng của nó trong thi đấu.

Mục 1 giới thiệu mô hình và định nghĩa tổng quát; Mục 2 dẫn nhập DFT từ góc nhìn đa thức; Mục 3 thảo luận phép hợp thành DFT và mở rộng khi chỉ số là tập lũy thừa; Mục 4 tiếp tục mở rộng, thảo luận

DFT trên nửa nhóm giao hoán hữu hạn và vành giao hoán không có ước của không; Mục 5 giới thiệu thuật toán tính DFT; Mục 6 thảo luận sơ lược thuật toán hiệu quả cho tích chập tổng quát hơn.

1.2 Dẫn vào

Trong tính toán, ta thường gặp dạng bài như sau: nhân tương ứng giữa các dãy, rồi cộng dồn theo một phép toán nào đó trên chỉ số. Khi phép toán trên chỉ số là cộng hoặc trừ, ta thường gọi đó là tích chập; nhưng trong nhiều trường hợp hơn, chỉ số không vận hành bằng phép cộng, chẳng hạn các phép toán bit nhị phân. Để mở rộng kết luận, ta cần rút ra điểm chung của các đối tượng này, tức “tích” và “cộng dồn”, rồi khái quát thành một khái niệm tích chập trừu tượng hơn.

Ta vừa nhắc tới hai khái niệm “chỉ số” và “giá trị”. Để mô tả chúng tổng quát và rõ ràng hơn, ta xem một “dãy” là một ánh xạ từ chỉ số tới giá trị.

1.3 Định nghĩa và mô hình

Ban đầu, ta cho một giả vành⁹ $(R, +, \cdot)$ định nghĩa trên tập R (cũng ký hiệu là R), và một magma¹⁰ $(G, \circ)$ định nghĩa trên tập G (cũng ký hiệu là G).

Đặt đơn vị nhân của R là 1_R . Khi không gây nhầm lẫn, ta dùng số nguyên z để chỉ tổng của z bản sao 1_R . Ta có các định nghĩa sau.

Định nghĩa 1.1. Ký hiệu tập các ánh xạ từ G tới R là R^G . Gọi G là miền xác định của R^G , và R là miền giá trị của R^G .

Định nghĩa 1.2 (Phép cộng). Với $f : G \rightarrow R$ và $g : G \rightarrow R$, định nghĩa $f + g$ là ánh xạ $h : G \rightarrow R$ thỏa $\forall x \in G, h(x) = f(x) + g(x)$.

Định nghĩa 1.3 (Tích điểm). Với $f : G \rightarrow R$ và $g : G \rightarrow R$, định nghĩa $f \cdot g$ là ánh xạ $h : G \rightarrow R$ thỏa $\forall x \in G, h(x) = f(x) \cdot g(x)$.

Định nghĩa 1.4 (Tích chập). Với $f : G \rightarrow R$ và $g : G \rightarrow R$, định nghĩa $f * g$ là ánh xạ $h : G \rightarrow R$ thỏa

$$\forall x \in G, \quad h(x) = \sum_{\substack{y, z \in G \\ y \circ z = x}} f(y) \cdot g(z).$$

Ở trường hợp vô hạn, $h(x)$ có thể không xác định.

Khi ngữ cảnh chưa chỉ rõ G , các ký hiệu trên được viết kèm chỉ số thành $+_G, \cdot_G, *_G$.

Bổ đề 1.5 (Phần tử không). Ánh xạ $f : G \rightarrow R, f(x) = 0$ là đơn vị của R^G đối với $+$, đồng thời là phần tử không đối với $\cdot$ và $*$. Ký hiệu ánh xạ này là 0 .

Bổ đề 1.6 (Đơn vị tích điểm). Nếu R là vành, thì $f : G \rightarrow R, f(x) = 1$ là đơn vị của R^G đối với $\cdot$. Ký hiệu ánh xạ này là 1 .

⁹Tức một vành không nhất thiết có đơn vị nhân, tiếng Anh là *rng*.

¹⁰Tức hệ đại số có một phép toán hai ngôi đóng; xem [https://en.wikipedia.org/wiki/Magma_\(algebra\)](https://en.wikipedia.org/wiki/Magma_(algebra)).

Bổ đề 1.7 (Đơn vị tích chập). Nếu R là vành và G có đơn vị e , tức $\forall a \in G, a \circ e = e \circ a = a$, thì

$$f : G \rightarrow R, \quad f(x) = \begin{cases} 1, & x = e, \\ 0, & \text{otherwise} \end{cases}$$

là đơn vị của R^G đối với $*$. Ký hiệu ánh xạ này là e .

Bổ đề 1.8 (Tính kết hợp của tích chập). Nếu G là nửa nhóm, thì $*$ trên R^G thỏa tính kết hợp, tức $\forall A, B, C \in R^G, (A * B) * C = A * (B * C)$.

Bổ đề 1.9 (Tính giao hoán của tích chập). Nếu G thỏa tính giao hoán và R là vành giao hoán, thì $*$ trên R^G thỏa tính giao hoán, tức $\forall A, B \in R^G, A * B = B * A$.

Bổ đề 1.10 (Tính phân phối của tích chập đối với phép cộng). $*$ trên R^G phân phối đối với $+$, tức

$$A * (B + C) = A * B + A * C, \quad (B + C) * A = B * A + C * A.$$

Từ đó có các kết luận sau.

Bổ đề 1.11. $(R^G, +, \cdot)$ là một giả vành; nếu R là vành thì nó là một vành.

Bổ đề 1.12. Nếu phép lấy tổng là đóng và G là nửa nhóm, thì $(R^G, +, *)$ là một giả vành; nếu R là vành và G là monoid thì nó là một vành.

Ta đặt thêm ký hiệu:

Ký hiệu 1.13 (Vành hoặc giả vành tích điểm). Ký hiệu R^G là $(R^G, +, \cdot)$.

Ký hiệu 1.14 (Vành hoặc giả vành tích chập). Ký hiệu R_*^G là $(R^G, +, *)$; khi không gây nhầm lẫn cũng viết là R^G .

2 Chuỗi lũy thừa hình thức và DFT cổ điển

2.1 Định nghĩa

Trong mục này, ta đặt $G = \mathbb{N}$, khi đó R^G chính là vành chuỗi lũy thừa hình thức $R[[x]]$ trên R .¹¹

Ta dùng

$$A(x) = \sum_{i \in \mathbb{N}} a_i x^i \in R[[x]]$$

để biểu thị một chuỗi lũy thừa hình thức trên R . Khi không gây nhầm lẫn, đôi khi ta dùng dãy $\{a_i\}$ để biểu thị chuỗi $\sum_{i \in \mathbb{N}} a_i x^i$.

2.2 Đa thức

Trước hết ta thảo luận trường hợp đơn giản hơn: đa thức trên trường. Trong mục này, ta yêu cầu R là trường số phức $\mathbb{C}$. Khi đó mọi tính chất ở Mục 1 đều đúng.

¹¹Trong bài này, $\mathbb{N}$ có chứa phần tử 0.

2.2.1 Biểu diễn hệ số và biểu diễn điểm trị của đa thức. Hình thức của tích chập tương đối phức tạp, nên ta thử biểu diễn nó bằng dạng đơn giản.

Ta biết một đa thức bậc n có thể được xác định hoàn toàn bởi $n + 1$ hệ số, hoặc bởi $n + 1$ giá trị x cùng các $f(x)$ tương ứng. Ta xem một dãy n phần tử như vậy $\{a_i\}$, với $i \in [0, n) \cap \mathbb{N}$, là một phần tử của $\mathbb{C}^{\mathbb{Z}_n}$.

Định nghĩa 2.1 (Biểu diễn hệ số). Gọi dãy $\{a_i\} \in \mathbb{C}^{\mathbb{Z}_n}$ là biểu diễn hệ số của đa thức

$$f(x) = \sum_{i=0}^{n-1} a_i x^i$$

có bậc không quá $n - 1$, ký hiệu là $[x]f$.

Định nghĩa 2.2 (Biểu diễn điểm trị). Với dãy $\{x_i\} \in \mathbb{C}^{\mathbb{Z}_n}$ gồm các phần tử đôi một khác nhau, gọi dãy $\{y_i : y_i = f(x_i)\} \in \mathbb{C}^{\mathbb{Z}_n}$ là biểu diễn điểm trị của đa thức $f(x)$ có bậc không quá $n - 1$, ký hiệu là $F_{\{x_i\}}(f)$. Biến đổi ngược được ký hiệu là $f = F_{\{x_i\}}^{-1}(\{y_i\})$.

Bổ đề 2.3 (Nội suy Lagrange). Với dãy $\{x_i\}$, $i \in [0, n) \cap \mathbb{N}$, gồm các phần tử đôi một khác nhau và biểu diễn điểm trị $\{y_i\} = F_{\{x_i\}}(f)$, đa thức duy nhất thỏa điều kiện là

$$f(x) = \sum_{i=0}^{n-1} y_i \prod_{\substack{j=0 \\ j \neq i}}^{n-1} \frac{x - x_j}{x_i - x_j}.$$

Ngoài ra, phép toán tuyến tính trên điểm trị tương đương với phép toán tuyến tính trên đa thức, tức

$$\forall x, y \in \mathbb{C}, f, g \in \mathbb{C}[x], \quad (f + g)(x) = f(x) + g(x), \quad (yf)(x) = y \cdot f(x).$$

Tích chập đa thức là phép nhân; ta có $(fg)(x) = f(x) \cdot g(x)$. Nhưng số lượng điểm trị và bậc của đa thức không giống nhau. Làm thế nào dùng điểm trị để biểu diễn tích chập? Ta có:

Bổ đề 2.4 (Định lý phần dư đa thức).

$$\forall a \in \mathbb{C}, f \in \mathbb{C}[x], \quad f(a) = f \bmod (x - a).$$

Theo nghĩa này, nội suy Lagrange là một trường hợp đặc biệt của định lý phần dư Trung Hoa.

Định lý 2.5. Với mọi $x = \{x_i\}$ và $a, b \in \mathbb{C}^{\mathbb{Z}_n}$, nếu các x_i đôi một khác nhau, thì

$$F_x^{-1}(a \cdot b) = F_x^{-1}(a) F_x^{-1}(b) \bmod \prod_{i=0}^{n-1} (x - x_i).$$

2.2.2 Tích chập. Xét đa thức ở dạng điểm trị. Khai triển ra ta được

$$fg \bmod \prod_{i=0}^{n-1} (x - x_i) = \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} ([x]f)_j ([x]g)_k \left(x^{j+k} \bmod \prod_{i=0}^{n-1} (x - x_i) \right).$$

Quay lại điểm xuất phát, ta muốn dùng phép nhân điểm trị để dựng một tích chập. Để đơn giản, trước hết xét trường hợp hệ số tạo thành tích chập, tức yêu cầu

$$\forall j \in \mathbb{N}, \quad x^j \bmod \prod_{i=0}^{n-1} (x - x_i) = x^k, \quad k \in \mathbb{N}.$$

Điều này có nghĩa $\prod_{i=0}^{n-1} (x - x_i)$ chỉ có hai hệ số khác 0, và hai hệ số này lần lượt là ± 1 . Khi $n > 1$, đa thức này có ít nhất hai hệ số khác 0, nên chỉ có hai nghiệm dạng:

$$x^n - 1, \quad (1)$$

$$x^n - x. \quad (2)$$

Với (1), các nghiệm là các căn đơn vị bậc n $\{\omega_n^i : i \in \mathbb{Z}_n\}$, dưới đây ký hiệu là T_n . Ta có

$$[x]F_{T_n}^{-1}(a \cdot b) = [x]F_{T_n}^{-1}(a) *_{\mathbb{Z}_n} [x]F_{T_n}^{-1}(b).$$

Nói cách khác, phép nhân đa thức theo nghĩa mod $\prod_{i=0}^{n-1} (x - x_i)$ tương đương với tích chập của dãy hệ số trên $\mathbb{Z}_n$.

Với (2), các nghiệm là $\{0\} \cup T_{n-1}$. Ta có

$$[x]F_{T_n}^{-1}(a \cdot b) = [x]F_{T_n}^{-1}(a) *_{\mathbb{Z}'_0} [x]F_{T_n}^{-1}(b),$$

trong đó $\mathbb{Z}'_0 = (\mathbb{Z}_n, \circ)$ và

$$\forall a, b \in \mathbb{Z}_n, \quad a \circ b = \begin{cases} a + b, & a = 0 \wedge b = 0, \\ (a + b - 1) \bmod (n - 1) + 1, & a \neq 0 \vee b \neq 0. \end{cases}$$

Đây là một hệ tạo bởi $\mathbb{Z}_{n-1}$ và nhóm tầm thường¹². Nó rất giống (1), đồng thời là trường hợp đặc biệt của nội dung phía sau, nên ta không thảo luận riêng nữa.

2.3 DFT cổ điển

Từ biểu diễn điểm trị của đa thức, ta nhận được một phương pháp chuyển đổi giữa tích chập và tích điểm. Bây giờ bỏ ràng buộc đa thức, phân tích từ góc nhìn ánh xạ ở Mục 1. Thực chất ta đã thu được một biến đổi.

Định nghĩa 2.6 (DFT cổ điển). Định nghĩa biến đổi $F : \mathbb{C}_*^{\mathbb{Z}_n} \rightarrow \mathbb{C}_*^{\mathbb{Z}_n}$ bởi

$$F(f) = \left(f' : x \mapsto \sum_{i \in \mathbb{Z}_n} \omega_n^{ix} f(i) \right).$$

Khi đó F là DFT trên $\mathbb{C}_*^{\mathbb{Z}_n}$. Ở đây ω_n là một căn đơn vị nguyên thủy bậc n , chẳng hạn $e^{2\pi i/n}$.

Rõ ràng đây là một biến đổi tuyến tính. Ngoài ra nó còn thỏa:

Định lý 2.7 (Định lý tích chập).

$$F(f * g) = F(f) \cdot F(g).$$

Điều này cho ta một đồng cấu từ $\mathbb{C}_*^{\mathbb{Z}_n}$ tới $\mathbb{C}_*^{\mathbb{Z}_n}$. Thực ra ta còn có thể thấy:

Định lý 2.8 (IDFT cổ điển). Định nghĩa biến đổi $F^{-1} : \mathbb{C}_*^{\mathbb{Z}_n} \rightarrow \mathbb{C}_*^{\mathbb{Z}_n}$ bởi

$$F^{-1}(f') = \left(f : x \mapsto n^{-1} \sum_{i \in \mathbb{Z}_n} \omega_n^{-ix} f'(i) \right).$$

Khi đó F^{-1} là IDFT trên $\mathbb{C}_*^{\mathbb{Z}_n}$, và F cùng F^{-1} là hai biến đổi nghịch đảo.

Vậy DFT là đẳng cấu từ $\mathbb{C}_*^{\mathbb{Z}_n}$ tới $\mathbb{C}_*^{\mathbb{Z}_n}$. Hơn nữa, có thể chứng minh:

¹²Tức nhóm chỉ gồm một phần tử.

Định lý 2.9. DFT là đẳng cấu duy nhất từ $\mathbb{C}_*^{\mathbb{Z}_n}$ tới $\mathbb{C}^{\mathbb{Z}_n}$.

Do đó, khi tính các phép toán trên hai vành này, ta có thể dùng DFT để đơn giản hóa và tăng tốc. Các thuật toán như FFT cơ sở 2 và thuật toán Bluestein có thể tính nhanh DFT với $O(n \log n)$ phép nhân căn đơn vị và $O(n \log n)$ phép cộng. Những thuật toán này đã khá phổ biến trong thi đấu, nên bài viết không nhắc lại.

2.4 Điều kiện của DFT

Mục trước đã thảo luận DFT áp dụng cho $\mathbb{C}^{\mathbb{Z}_n}$. Ta xét việc mở rộng R .

Định lý 2.10. Với vành R , nếu R thỏa:

1. R là vành không có ước của không;
2. R có căn đơn vị nguyên thủy bậc n là ω_n , tức tồn tại phần tử ω_n sao cho $\omega_n^i, i \in \mathbb{Z}_n$, đôi một khác nhau và $\omega_n^n = 1$;
3. nghịch đảo nhân của n tồn tại trong R , tức tồn tại x sao cho $xn = nx = 1$, ký hiệu là n^{-1} ;

thì biến đổi $F : R_*^{\mathbb{Z}_n} \rightarrow R^{\mathbb{Z}_n}$ và biến đổi ngược của nó

$$F(f) = \left(f' : x \mapsto \sum_{i \in \mathbb{Z}_n} \omega_n^{ix} f(i) \right),$$

$$F^{-1}(f') = \left(f : x \mapsto n^{-1} \sum_{i \in \mathbb{Z}_n} \omega_n^{-ix} f'(i) \right)$$

là một cặp ánh xạ đẳng cấu nghịch đảo giữa $R_*^{\mathbb{Z}_n}$ và $R^{\mathbb{Z}_n}$.

Chứng minh giống DFT trên $\mathbb{C}^{\mathbb{Z}_n}$, nên lược bỏ. Tuy vậy cần lưu ý rằng đây là điều kiện đủ, không phải điều kiện cần và đủ.

Trong thi tin học, các vành thường gặp nhất thỏa điều kiện trên là $\mathbb{C}$ và $\mathbb{Z}_p$, với $n \mid p-1$ và p là số nguyên tố. DFT trên $\mathbb{Z}_p$ còn gọi là biến đổi số học (NTT).

Đôi khi có thể mở rộng trường để thu được một vành hợp lệ, chẳng hạn trường bậc hai dùng trong bài tính tổng cây khung.¹³

Để tiện cho phần sau, ta bổ sung một định nghĩa.

Định nghĩa 2.11. Nếu F là đẳng cấu từ R_*^G tới R^G , gọi F là một biến đổi có tính tích chập trên R^G .

3 Mở rộng đơn giản

Mục trước đã dùng chuỗi lũy thừa hình thức để dẫn ra và giới thiệu phương pháp DFT. Mục này thảo luận một số mở rộng đơn giản.

3.1 Biến đổi hợp thành

Bổ đề 3.1. $(A^B)^C$ đẳng cấu với $A^{B \otimes C}$.¹⁴

¹³<https://loj.ac/problem/6271>

¹⁴Tích trực tiếp của G và G' , ký hiệu là $G \otimes G'$, là hệ gồm mọi cặp (a, b) với $a \in G, b \in G'$; phép toán thực hiện theo từng thành phần: $(a, b) \circ (c, d) = (a \circ c, b \circ d)$. Tích trực tiếp thỏa tính kết hợp và giao hoán.

Chứng minh. Với $f \in (A^B)^C$ và $g \in A^{B \otimes C}$, đặt $f \cong g$ khi và chỉ khi $\forall x \in C, y \in B, (f(x))(y) = g((x, y))$. ■

Rõ ràng quan hệ tương đương này thỏa tính giao hoán và kết hợp.

Bổ đề 3.2 (Hợp thành biến đổi có tính tích chập). Nếu G, G' là nửa nhóm, và F, F' lần lượt là các biến đổi có tính tích chập trên $R^G, R^{G'}$, thì định nghĩa biến đổi H từ $(R^G)^{G'}$ tới $(R^{G \otimes G'})^{G'}$ bởi

$$H(f) = (x \mapsto (y \mapsto F'(z \mapsto F(f(z))(y))(x))),$$

khi đó H là biến đổi có tính tích chập trên $R^{G \otimes G'}$.

Điều này tương đương với việc sau biến đổi thứ nhất, ta độc lập thực hiện thêm một biến đổi trên các “điểm trị” thu được.

Ký hiệu 3.3. Ký hiệu H ở trên là $F' \circ F$, gọi là hợp thành của F' và F .

Như vậy, Bổ đề 3.2 có thể phát biểu: hợp thành của hai biến đổi có tính tích chập cùng miền giá trị là biến đổi có tính tích chập từ tích trực tiếp của các miền xác định tới miền giá trị.

Áp dụng điều này cho DFT cổ điển sẽ thu được DFT nhiều chiều. Thực ra, miền xác định và miền giá trị của DFT nhiều chiều tương đương với đa thức nhiều biến.

3.2 Chuỗi lũy thừa tập hợp

“Chuỗi lũy thừa tập hợp”¹⁵ là trường hợp chỉ số G là tập lũy thừa của một tập hữu hạn. Ký hiệu tập vũ trụ là U , tập lũy thừa của U là 2^U . Khi đó chuỗi lũy thừa tập hợp trên U là $R^{(2^U)}$.

Có thể thấy với các phép toán tập hợp thỏa tính độc lập từng phần tử, chẳng hạn giao, hợp và hiệu đối xứng, 2^U có thể phân rã thành tích trực tiếp của phép toán trên từng phần tử.

Trong đó, 2^U với phép hiệu đối xứng đẳng cấu với $\mathbb{Z}_2^{|U|}$, tức xor nhị phân. Ta có thể dùng DFT nhiều chiều ở trên.

Biến đổi trên mỗi chiều của giao và hợp, nếu dùng vector $\begin{bmatrix} f(0) \\ f(1) \end{bmatrix}$ để biểu thị f , lần lượt là:

$$\text{giao: } F : a \mapsto \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} a, \quad F^{-1} : a \mapsto \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix} a,$$

$$\text{hợp: } F : a \mapsto \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} a, \quad F^{-1} : a \mapsto \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} a.$$

Ở phần sau ta sẽ thấy đây là hai nghiệm trong trường hợp bậc hai; ở đây không đi sâu thảo luận. Cuối cùng, chỉ cần áp dụng phép hợp thành biến đổi trên từng chiều.

4 DFT trên nửa nhóm giao hoán hữu hạn

Mục này tập trung thảo luận mở rộng liên quan tới G .

¹⁵Cần lưu ý đây không phải thuật ngữ được thừa nhận rộng rãi trong toán học. Cụm này là cách gọi trong thi đấu, có thể xuất hiện sớm nhất trong bài *Tính chất và thuật toán nhanh của chuỗi lũy thừa tập hợp* của Lu Kaifeng [6].

4.1 Dẫn nhập

Mục trước đã thảo luận việc mở rộng DFT trên R và trên phép hợp thành, nhưng các mở rộng đó vẫn chưa đủ mạnh. Có thể mở rộng DFT lên G tổng quát hơn hay không?

Do độ khó thảo luận, trình độ của tác giả và hạn chế thực tế trong tính toán, bài viết không xét trường hợp G vô hạn hoặc không có tính giao hoán, mà chỉ thảo luận nửa nhóm giao hoán hữu hạn.

4.2 Điều kiện và chuẩn bị

Ta trực tiếp đưa ra các điều kiện cho phần thảo luận sau. Ở các mục sau sẽ thấy những điều kiện này là cần thiết đối với phương pháp thảo luận của chúng ta:

1. G là nửa nhóm giao hoán hữu hạn;
2. R là vành giao hoán khác không¹⁶ và không có ước của không;
3. biến đổi có tính tích chập F đang xét là ánh xạ tuyến tính.

Vì đã yêu cầu F là ánh xạ tuyến tính, riêng phần phép toán tuyến tính đã là đẳng cấu từ R_*^G tới R_*^G . Do đó ta tập trung vào định lý tích chập:

$$F(a * b) = F(a) \cdot F(b).$$

Vì G hữu hạn, đặt $|G| = n$, đánh số các phần tử là $0, 1, \dots, n-1$, và quy ước dùng vector cột

$$\begin{bmatrix} f(0) \\ f(1) \\ \vdots \\ f(n-1) \end{bmatrix}$$

để biểu thị một $f \in R^G$.

Có thể thấy R^G có cơ sở tuyến tính $\{f(x) = [x = i] : i \in G\}$; vector cột trên chính là tọa độ theo cơ sở này. Vì có cơ sở, ánh xạ tuyến tính bất yếu có thể biểu diễn bằng ma trận theo cơ sở.

Ký hiệu 4.1 (Ma trận biến đổi). Ký hiệu $T = T(F)$ là ma trận biến đổi tương ứng của F theo cơ sở $\{f(x) = [x = i] : i \in G\}$, tức $\forall a \in R^G, F(a) = Ta$.

Khai triển công thức định lý tích chập, có thể thấy mỗi chiều của a , tương ứng với một hàng của T , thực ra độc lập với nhau. Xét chiều thứ i , ta có:

$$\sum_{j=0}^{n-1} T_{i,j} \sum_{\substack{k,l \\ k \circ l = j}} a_k b_l = \left(\sum_{j=0}^{n-1} T_{i,j} a_j \right) \left(\sum_{j=0}^{n-1} T_{i,j} b_j \right),$$

tức

$$\sum_{k,l} T_{i,k \circ l} a_k b_l = \sum_{k,l} T_{i,k} a_k T_{i,l} b_l = \sum_{k,l} T_{i,k} T_{i,l} a_k b_l. \quad (15)$$

Ở đây dùng tới tính giao hoán.¹⁷

¹⁶Tức không phải vành không: vành chỉ có một phần tử.

¹⁷Thực ra điều kiện vành giao hoán có thể yếu hơn: chỉ cần các phần tử trong T giao hoán với phần tử của R , tức $\forall x \in R, i, j, T_{i,j}x = xT_{i,j}$, thì lập luận trong mục này vẫn đúng. Điều kiện này có thể áp dụng cho ma trận trên vành thỏa điều kiện.

Ta cần đây là hằng đẳng thức trên R . Rõ ràng nếu hệ số tương ứng hai vế bằng nhau, tức $\forall k, l, T_{i,k \circ l} = T_{i,k}T_{i,l}$, thì hằng đẳng thức chắc chắn đúng. Ngược lại, lần lượt đặt $a_i = [x = i]$, $b_j = [y = j]$ với mọi $x, y \in G$, cũng nhận được $\forall k, l, T_{i,k \circ l} = T_{i,k}T_{i,l}$. Vì vậy đây là điều kiện cần và đủ.

Đặt vector hàng $x = \{x_i\} = T_i$, thực chất ta nhận được hệ phương trình

$$\forall i, j, \quad x_{i \circ j} = x_i x_j.$$

Tức x ánh xạ phép toán $\circ$ trong G sang phép nhân trong R , hay nói cách khác, G là nửa nhóm con của nửa nhóm nhân của R .

Để F , tức T , khả nghịch, ta ít nhất cần n nghiệm độc lập tuyến tính của hệ phương trình. Ta gọi $x = 0$ là nghiệm tầm thường và không xét nữa. Hai mục tiếp theo thảo luận nội dung này.

4.3 Tính cần thiết

Định nghĩa 4.2 (Lũy thừa). Gọi

$$\underbrace{i \circ i \circ \dots \circ i}_{j \text{ lần}}$$

là lũy thừa bậc j của i , ký hiệu là i^j .

Định nghĩa 4.3 (Tính tuần hoàn). Gọi magma G thỏa tính tuần hoàn khi và chỉ khi

$$\forall i \in G, \exists j \in \mathbb{N}^+ \quad i^{j+1} = i.$$

Bổ đề 4.4. Việc G thỏa tính tuần hoàn là điều kiện cần để tồn tại biến đổi có tính tích chập trên R^G .

Chứng minh. Giả sử với một $i \in G$, không tồn tại $j \in \mathbb{N}^+$ sao cho $i^{j+1} = i$. Do G hữu hạn, tất yếu tồn tại $x, y \in \mathbb{N}^+$ sao cho $x \neq y$ và $i^x = i^y$. Xét cặp (x, y) nhỏ nhất theo thứ tự từ điển.

Từ giả thiết có $y > x > 1$ và $i^{x-1} \neq i^{y-1}$. Mặt khác, bởi tính kết hợp và giao hoán,

$$\forall a \geq x, \quad i^a = i^{x+((a-x) \bmod (y-x))}.$$

Do đó

$$i^{x-1} \circ i^{y-1} = i^{x+y-2} = i^{x-1} \circ i^{x-1} = i^{2x-2} = i^{y-1} \circ i^{y-1} = i^{2y-2}.$$

Vì R khác không, nên $-1, 1 \neq 0$. Xét ánh xạ

$$f(t) = \begin{cases} -1, & t = i^{x-1} \vee t = i^{y-1}, \\ 0, & \text{otherwise,} \end{cases}$$

có thể thấy $f * f = 0$. Khi đó $F(f) = 0 = F(0)$, mâu thuẫn với việc F là đơn ánh. ■

4.4 Tính đủ và cách dựng

Tiếp theo ta sẽ thấy tính tuần hoàn là điều kiện then chốt để mở rộng DFT lên G . Mục này thảo luận dưới tiền đề luật tuần hoàn.

Định nghĩa 4.5 (Bậc). Với mọi $i \in G$, gọi chu kỳ dương nhỏ nhất của dãy các lũy thừa dương của i là bậc của i , ký hiệu $\text{ord}(i)$. Định nghĩa $i^0 = i^{\text{ord}(i)}$, và với mọi $x \in \mathbb{Z}$,

$$i^x = i^{x \bmod \text{ord}(i)}.$$

Ký hiệu nhóm cyclic sinh bởi các lũy thừa của i là $G_i = \langle i, \circ \rangle$.

Bổ đề 4.6. Với mọi $i \in G$, i^0 là phần tử lũy đẳng duy nhất trong G_i .

Chứng minh. Nếu i^j là lũy đẳng, thì $j = 2j$, suy ra $j = 0 \pmod{\text{ord}(i)}$. ■

Bổ đề 4.7.

$$\forall i, j \in G, \quad (\exists x, y \text{ sao cho } i^x = j^y) \Rightarrow i^0 = j^0.$$

Chứng minh.

$$i^x = j^y \Rightarrow (i^x)^{\text{ord}(i) \text{ord}(j)} = (j^y)^{\text{ord}(i) \text{ord}(j)} \Rightarrow i^0 = j^0.$$

Từ Bổ đề 4.6 và 4.7, ta có thể chia G theo i^0 .

Định nghĩa 4.8. Đặt C_x là lớp tương đương gồm mọi i thỏa $i^0 = x$.¹⁸ Ký hiệu quan hệ tương đương là $i \sim j$, khi và chỉ khi $i^0 = j^0$.

Bổ đề 4.9. $\sim$ bảo toàn phép toán $\circ$, tức

$$\forall a, b, c, d \in G, \quad (a \sim b \wedge c \sim d) \Rightarrow ((a \circ c) \sim (b \circ d)).$$

Từ 4.3 và 4.4, ta định nghĩa phép toán giữa các lớp tương đương:

Định nghĩa 4.10 (Phép toán lớp tương đương). Định nghĩa $\circ$ bởi $C_x \circ C_y = C_{x \circ y}$. Các lớp tương đương lập thành một nửa nhóm giao hoán theo $\circ$, ký hiệu là G_C .¹⁹

Định lý 4.11. Mọi phần tử của G_C đều là phần tử lũy đẳng; G_C là một nửa dàn (*semilattice*).²⁰

Bây giờ xét ảnh hưởng lên hệ phương trình.

Bổ đề 4.12. Với một nghiệm x của hệ phương trình, ta có

$$\forall i, j \in G, \quad (i \sim j) \Leftrightarrow ([x_i = 0] = [x_j = 0]).$$

Tức trong cùng một lớp tương đương, các phần tử tương ứng của x hoặc đều bằng 0, hoặc đều khác 0.

Chứng minh.

$$x_i^{\text{ord}(i)} = x_{i^0} = x_{i^0} \Rightarrow ([x_i = 0] = [x_{i^0} = 0]).$$

Với một lớp tương đương i , đặt

$$S_i = \{j : j \in G_C, j \circ i = i\}.$$

Bổ đề 4.13. Các S_i đôi một khác nhau, tức $\forall i \neq j, i, j \in G_C, S_i \neq S_j$.

¹⁸Thực ra đây là trường hợp đặc biệt của phân rã nửa dàn dưới luật tuần hoàn.

¹⁹Ở đây G_C là một ký hiệu nguyên khối cho nửa nhóm giao hoán này; chữ C không có ý nghĩa riêng, đừng nhầm với định nghĩa ở Mục 1.

²⁰<https://en.wikipedia.org/wiki/Semilattice>

Chứng minh. Giả sử $S_i = S_j$. Khi đó $i \in S_i = S_j$ và $j \in S_j = S_i$, suy ra $i \circ j = i = j$, mâu thuẫn. ■

Bổ đề 4.14. Trong một nghiệm không tầm thường x của hệ phương trình, tập các lớp tương đương ứng với phần tử khác 0 tất yếu bằng một S_i nào đó.

Chứng minh. Gọi S là tập các lớp tương đương ứng với mọi phần tử khác 0 trong x . Đặt $a = \bigcirc_{i \in S} i$. Vì R không có ước của không, từ hệ phương trình suy ra $\forall i \in a, x_i \neq 0$.

Do lũy đẳng trên lớp tương đương,

$$\forall i \in S, \quad a \circ i = (\bigcirc_{j \in S} j) \circ i = \bigcirc_{j \in S} j = a,$$

nên $S \subseteq S_a$. Nếu tồn tại $i \in S_a$ nhưng $i \notin S$, thì từ hệ phương trình suy ra $\forall i \in a, x_i = 0$, mâu thuẫn. Vì vậy $S_a \subseteq S$, suy ra $S_a = S$. ■

Bổ đề 4.15. Với mọi x , C_x là nhóm giao hoán.

Chứng minh. Lấy đơn vị của C_x là x , nghịch đảo của i là $i^{\text{ord}(i)-1}$; kiểm tra trực tiếp là được. ■

Bổ đề 4.16 (Định lý cơ bản của nhóm giao hoán hữu hạn). Nhóm giao hoán hữu hạn G luôn đẳng cấu với tổng trực tiếp²¹ của một số nhóm cyclic có bậc là lũy thừa nguyên tố, tức tồn tại duy nhất các dãy $\{p_i\}, \{c_i\}$ sao cho

$$G \cong \bigoplus_{i \in I} \mathbb{Z}/p_i^{c_i}.$$

Ở đây I là tập chỉ số. Chứng minh xem [4], lược bỏ vì giới hạn dung lượng.

Theo kết luận hợp thành ở Mục 3, ta xét riêng biến đổi có tính tích chập và x tương ứng trên từng chiều $\mathbb{Z}/p_i^{c_i}$. Với các nhóm cyclic này, dùng kết luận DFT cổ điển của Định lý 2.7. Với một lớp tương đương C_a , DFT có thể cho đúng $|C_a|$ nghiệm x , tức từng hàng của ma trận Vandermonde căn đơn vị bậc $|C_a|$.²²

Sau khi có nghiệm x_{C_a} trong lớp tương đương, ta thử dựng một nghiệm đầy đủ x . Từ hệ phương trình tất yếu có $x_a = 1$. Khi đó với mọi k thỏa $C_{k \circ a} \in S_{C_a}$, có

$$x_k = x_k x_a = x_{k \circ a}.$$

Vì $k \circ a \in C_a$, giá trị $x_{k \circ a}$ đã được DFT cho trước, nên giá trị trên các lớp tương đương còn lại là duy nhất.

Bây giờ chứng minh $\{x_k = x_{k \circ a}\}$ là nghiệm hợp lệ. Với mọi i, j , chia trường hợp:

1. Nếu $i, j \in C_a$, điều này hiển nhiên đúng do DFT.
2. Nếu $C_{i \circ 0}, C_{j \circ 0} \in S_{C_a}$ và không thuộc trường hợp 1, thì

$$x_{i \circ j} = x_{(i \circ j) \circ a} = x_{i \circ j \circ (a \circ a)} = x_{(i \circ a) \circ (j \circ a)} = x_{i \circ a} x_{j \circ a} = x_i x_j.$$

3. Nếu $C_{i \circ 0} \notin S_{C_a}$ hoặc $C_{j \circ 0} \notin S_{C_a}$, giả sử không mất tổng quát $C_{i \circ 0} \notin S_{C_a}$, thì tất yếu $C_{(i \circ j) \circ 0} \notin S_{C_a}$. Phương trình tương ứng là $0x_j = 0$, nên chắc chắn đúng. Nếu không, giả sử $C_{(i \circ j) \circ 0} \in S_{C_a}$, ta có $C_{i \circ 0} \circ C_{j \circ 0} \circ C_a = C_a$, từ đó

$$C_{i \circ 0} \circ C_a = C_{i \circ 0} \circ (C_{i \circ 0} \circ C_{j \circ 0} \circ C_a) = (C_{i \circ 0} \circ C_{i \circ 0}) \circ C_{j \circ 0} \circ C_a = C_{i \circ 0} \circ C_{j \circ 0} \circ C_a = C_a,$$

mâu thuẫn.

²¹Với nhóm giao hoán, tổng trực tiếp là tích trực tiếp của hữu hạn nhóm giao hoán.

²²Chính là ma trận T tương ứng với DFT.

Như vậy, với mỗi lớp tương đương ta làm một lần và nhận được $\sum_{C \in G_C} |C| = n$ nghiệm đôi một khác nhau.

Ta còn cần chứng minh biến đổi tạo bởi n nghiệm này khả nghịch. Lần lượt lấy mọi lớp tương đương theo một thứ tự topo nửa dần nào đó của G_C ,²³ rồi sắp x theo thứ tự đó, ta được ma trận biến đổi có dạng

$$T = \begin{bmatrix} T_1 & \cdots & \cdots & \cdots \\ 0 & T_2 & \cdots & \cdots \\ 0 & 0 & T_3 & \cdots \\ \vdots & \vdots & \vdots & \ddots \\ 0 & 0 & 0 & T_{|G_C|} \end{bmatrix},$$

trong đó T_i là ma trận biến đổi DFT của lớp tương đương thứ i . Theo tính chất DFT, mọi T_i đều khả nghịch, nên T chắc chắn khả nghịch. Từ đó ta thu được một biến đổi tuyến tính có tính tích chập.

4.5 Kết luận

Định lý 4.17. Dưới các tiền đề ở Mục 4.2, các điều kiện sau là cần và đủ để tồn tại một biến đổi có tính tích chập trên R^G : G thỏa tính tuần hoàn, và với mỗi nhóm cyclic C thu được từ phân rã trong Mục 4.4, R^C đều tồn tại DFT.

5 Thuật toán

5.1 Thuật toán chính

5.1.1 Quy trình thuật toán.

Thuật toán 1: Construct DFT. Yêu cầu: R, G thỏa Định lý 4.17. Kết quả: ma trận biến đổi T và ma trận biến đổi ngược T^{-1} .

- 1: DFT(R, G).
- 2: $T \leftarrow 0$ bậc $|G|$, $T^{-1} \leftarrow E$ bậc $|G|$; $G_C, C_i \leftarrow \emptyset$, $it \leftarrow 0$.
- 3: Với mọi $i \in G$:
- 4: chèn i vào C_{i0} .
- 5: nếu $C_{i0} \notin G_C$, chèn C_{i0} vào G_C .
- 6: $G_C \leftarrow \text{Toposort}(G_C)$.
- 7: Với mọi $C_x \in G_C$:
- 8: $B \leftarrow \text{FiniteAbelGroupBase}(C_x)$, $S \leftarrow \{x\}$.
- 9: $T_{C_x} \leftarrow 0$ bậc $|C_x|$, $T_{C_x}^{-1} \leftarrow 0$ bậc $|C_x|$, $T_{C_x, x, x} \leftarrow 1$, $T_{C_x, x, x}^{-1} \leftarrow |C_x|^{-1}$.
- 10: Với mọi $i \in B$:

²³Ở trường hợp hữu hạn, có thể hiểu như sau: ta sắp xếp G_C bằng cách mỗi lần lấy ra một lớp tương đương C thỏa $S_C = C$, rồi xóa C và lặp lại tới khi hoàn tất. Thứ tự như vậy luôn tồn tại. Một mặt, theo định nghĩa S_C , sau khi xóa C , phép toán vẫn đóng, nên vẫn thu được nửa dần hữu hạn. Mặt khác, khi không rỗng thì một C như vậy luôn tồn tại: nếu không, mỗi C đều có ít nhất một $X \neq C$ thỏa $X \circ C = C$. Xét mô hình đồ thị, nối cạnh có hướng từ X tới C ; số cạnh bằng số đỉnh và đồ thị hữu hạn nên phải có chu trình. Vì không lớp tương đương nào nối cạnh tới chính nó, chu trình có ít nhất hai phần tử khác nhau. Nhưng với một chu trình, xét kết quả phép $\circ$ của mọi phần tử trên chu trình, theo định nghĩa cạnh ta được kết quả này bằng từng phần tử trên chu trình, mâu thuẫn.

11: Với $j, k = 0, \dots, \text{ord}(i) - 1$ và $c, d \in S$:

$$T_{C_x, i^j \circ a, i^k \circ b} \leftarrow \omega_{\text{ord}(i)}^{jk} T_{C_x, a, b},$$

$$T_{C_x, i^j \circ a, i^k \circ b}^{-1} \leftarrow \omega_{\text{ord}(i)}^{-jk} T_{C_x, a, b}^{-1}.$$

12: Kết thúc vòng lặp.

13: $S \leftarrow \{a \circ i^j : a \in S, j = 0, \dots, \text{ord}(i) - 1\}$.

14: Với mọi $i \in C_x$, đặt $T_{it++} \leftarrow r$, trong đó $r_j = T_{C_x, i, j \circ x}$.

15: Với $C_x \in G_C$ theo thứ tự ngược:

16: $T_{C_x} \leftarrow T_{C_x}^{-1} T_{C_x}, T_{C_x}^{-1} \leftarrow T_{C_x}^{-1} T_{C_x}^{-1}$.

17: Với mọi $C_y \in G_C, C_y \neq C_x$:

$$T_{C_y} \leftarrow T_{C_y} - T_{C_y, C_x} T_{C_x}, \quad T_{C_y}^{-1} \leftarrow T_{C_y}^{-1} - T_{C_y, C_x}^{-1} T_{C_x}^{-1}.$$

18: Trả về T, T^{-1} .

5.1.2 Hiệu quả thời gian và không gian. Số lần tìm căn đơn vị nguyên thủy bậc n của R là $O(|G|)$; dưới đây không xét phần ảnh hưởng này tới độ phức tạp thời gian nữa.

Trong phần sau, số lần gọi phép cộng và phép nhân trong R có cùng bậc với độ phức tạp thời gian. Ta xem chi phí tính phép $\circ$ trong G là hằng số.

Dòng 11 gọi thuật toán phân rã nhóm giao hoán ở phần sau; thuật toán đó có độ phức tạp thời gian và không gian đều là $\Theta(|C_x|^2)$. Vì $|G| = \sum |C_x|$, tổng độ phức tạp thời gian và không gian đều không vượt quá $O(|G|^2)$.

Trong các dòng 13 tới 18, chú ý rằng mỗi lần kích thước $|S|$ bằng $\text{ord}(i)$ lần kích thước trước đó. Do $\text{ord}(i) \geq 2$, có

$$T(n) = O(T(n/2)) + \Theta(n^2), \quad T(n) = \Theta(n^2).$$

Vì vậy phần này có độ phức tạp thời gian và không gian đều là $\Theta(|G|^2)$.

Các dòng 23 tới 28 xem T như ma trận khối gồm các T_i và thực hiện thuật toán nghịch đảo ma trận. Nếu dùng khử Gauss và nhân ma trận thô, thời gian là $\Theta(|G|^3)$, không gian là $\Theta(|G|^2)$. Nếu dùng thuật toán ma trận khác, thời gian là $\Theta(|G|^\omega)$, trong đó ω là độ phức tạp thời gian của nhân ma trận; thuật toán tốt nhất đã biết có $\omega = 2.3728639$ [2].

Các phần còn lại hiển nhiên có độ phức tạp thời gian và không gian $\Theta(|G|^2)$. Tóm lại, độ phức tạp thời gian là $\Theta(|G|^\omega)$, độ phức tạp không gian là $\Theta(|G|^2)$, và độ phức tạp không gian đạt cận dưới theo kích thước đầu vào.

5.2 Phân rã nhóm giao hoán hữu hạn

Định nghĩa 5.1. Với nhóm giao hoán hữu hạn G , gọi $B = \{b_i\}$ là một cơ sở của G^{24} khi và chỉ khi

$$G = \bigoplus_{i \in I} \langle b_i \rangle,$$

trong đó $\langle b_i \rangle$ là nhóm con sinh bởi b_i .

Sau đây là thuật toán tìm một cơ sở của nhóm giao hoán hữu hạn G .

²⁴Cơ sở ở đây không yêu cầu không thể phân rã thêm, nên bậc cũng có thể không phải lũy thừa nguyên tố. Cách nói này chỉ dùng trong bài này.

5.2.1 Mô tả thuật toán.

Bổ đề 5.2. Nếu một tập phần tử $B = \{b_i\}$ thỏa:

1. $\{b_i\}$ độc lập tuyến tính:

$$\forall i \in I, \quad \langle b_i \rangle \cap \langle b_1, \dots, b_{i-1} \rangle = \{0\};$$

2. $\forall g \in G, g \in \bigoplus_{i \in I} \langle b_i \rangle$,

thì B là một cơ sở.

Chứng minh. Từ điều kiện 1, các phần tử của $\bigoplus_{i \in I} \langle b_i \rangle$ đôi một khác nhau và đều thuộc G , nên $\bigoplus_{i \in I} \langle b_i \rangle \subseteq G$. Từ điều kiện 2, $G \subseteq \bigoplus_{i \in I} \langle b_i \rangle$. Do đó hai tập bằng nhau. ■

Vậy ta bắt đầu từ tập rỗng. Mỗi lần lấy phần tử có bậc lớn nhất trong các phần tử độc lập tuyến tính với cơ sở hiện tại, cho tới khi không còn phần tử nào có thể lấy. Rõ ràng cơ sở thu được thỏa hai điều kiện trên.

Mỗi lần thêm một phần tử b_i vào cơ sở, chỉ cần gộp mọi phần tử x và $x + b_i$ vào cùng lớp tương đương phụ thuộc tuyến tính theo cơ sở hiện tại để duy trì quan hệ phụ thuộc tuyến tính.

5.2.2 Quy trình thuật toán.

Thuật toán 2: Decompose Finite Abel Group. Yêu cầu: nhóm giao hoán hữu hạn G . Kết quả: một cơ sở B .

- 1: `FiniteAbelGroupBase( $G$ ).`
- 2: $B \leftarrow \emptyset, S \leftarrow G \setminus \{e\}$.
- 3: Trong khi $S \neq \emptyset$:
- 4: $\text{max_ord} \leftarrow 0$.
- 5: Với mọi $i \in S$:
- 6: nếu $\text{max_ord} < \text{ord}(i)$, đặt $b \leftarrow i, \text{max_ord} \leftarrow \text{ord}(i)$.
- 7: Chèn b vào B .
- 8: Với mọi $i \in S$:
- 9: nếu tồn tại j sao cho $i \circ b^j \notin S$, xóa i khỏi S .
- 10: Trả về B .

5.2.3 Hiệu quả thời gian và không gian. Mỗi vòng lặp có độ phức tạp thời gian $\Theta(|G||S|)$. Chú ý mỗi lần kích thước $|S|$ bị chia cho $\text{ord}(b_i)$, và $\text{ord}(b_i) \geq 2$, nên

$$T(n) = O(T(n/2)) + \Theta(n|G|), \quad T(n) = \Theta(n|G|).$$

Vì vậy độ phức tạp thời gian và không gian đều là $\Theta(|G|^2)$.

5.3 Thuật toán đơn giản hóa

Thuật toán trên hiện thực theo quá trình suy luận trong chứng minh; tuy mạch lạc nhưng khá phức tạp. Thực ra ta có thể có vài thuật toán đơn giản hơn với cùng hiệu quả.

Chú ý rằng phần tử của T chỉ có thể nhận giá trị 0 hoặc căn đơn vị bậc là ước của $|G|$; việc một phần tử có bằng 0 hay không có thể được phán đoán qua lớp tương đương. Định nghĩa ký hiệu $\omega_{|G|}$ là căn đơn vị hình thức bậc $|G|$, khi đó phần tử khác 0 của T đều có thể viết thành $\omega_{|G|}^x$, $x \in \mathbb{Z}_{|G|}$.

Như vậy, ta chuyển hệ phương trình

$$\forall i, j \in G, \quad x_{i \circ j} = x_i x_j$$

thành một hệ phương trình tuyến tính và giải nó. Độ phức tạp thời gian là $\Theta(|G|^\omega)$, độ phức tạp không gian là $\Theta(|G|^2)$, giống thuật toán trên.

6 Tích chập tổng quát hơn

Mục này thảo luận một số thuật toán tính tích chập tổng quát hơn. Do hạn chế trình độ của tác giả, phần này chỉ là một số khám phá đơn giản, chưa có hệ thống. Trong mục này ta vẫn thảo luận dưới các tiền đề ở Mục 4.2.

6.1 Dẫn nhập

Trên R^G , các phép toán tuyến tính khác chỉ cần chi phí $\Theta(|G|)$, đây là cận dưới theo kích thước đầu vào. Chỉ có tích chập là phép toán phi tuyến có chi phí thô $\Theta(|G|^2)$. Dùng biến đổi có tính tích chập để ánh xạ tích chập thành tích điểm đơn giản là một cách tốt: với trường hợp thỏa Định lý 4.17, kết hợp FFT và các thuật toán tương tự có thể hoàn tất tính toán trong $O(|G|^2)$ thời gian. Nhưng điều kiện tồn tại của nó khá nghiêm ngặt. Khi không tồn tại biến đổi có tính tích chập, làm thế nào tăng tốc việc tính tích chập?

Dựa trên nội dung nghiên cứu, phần sau chủ yếu thảo luận trường hợp miền xác định hợp thành, hơn nữa là lũy thừa của một G nào đó, tức “phép toán bit”. Đặc trưng nổi bật của trường hợp miền xác định hợp thành là có bài toán con đệ quy và chi phí nhân lớn.

6.2 Mô hình

Trong R^G , dùng các phép toán đã định nghĩa ở Mục 1, tức phép toán tuyến tính và tích chập (phép nhân), rồi hợp thành hữu hạn lần, thực chất ta thu được đa thức $R_*^G[x]$.

Vì trên vành ta không thể làm phép chia tổng quát, mục này không xét phép chia.²⁵

Khi không xét phép chia, vì tích chập là bậc hai, các biến đổi có thể làm trong quá trình tính tích chập không thể vượt quá bậc hai. Vì vậy chỉ có thể trước hết thực hiện một số phép toán tuyến tính, tính tích của các kết quả này, tức nhân đa thức bậc nhất, rồi lại dùng phép toán tuyến tính trên các kết quả đó để tìm lời giải.

Giả sử cần tính tích chập $a * b$ của $a = \{i \mapsto a_i\}$ và $b = \{i \mapsto b_i\}$. Quá trình tính toán có thể được mô tả bởi mô hình sau:

$$a, b \xrightarrow{\text{phép toán tuyến tính}} \{f_i = A_i a + B_i b\} \xrightarrow{\text{tích chập và phép toán tuyến tính}} a * b = \sum_{i,j} K_{i,j} (f_i * f_j).$$

²⁵Dù vậy, đôi khi nhờ phép chia hình thức, hoặc trong trường hợp phần tử cần dùng khả nghịch, thậm chí R là trường, ta có thể dùng phép chia để nhận được thuật toán hiệu quả hơn chỉ dùng phép toán tuyến tính và phép nhân trên R^G , chẳng hạn một số thuật toán dựa trên nghịch đảo đa thức. Do giới hạn trình độ và dung lượng, bài viết không trình bày thêm.

Ở đây $a, b, a * b$ biểu thị các vector cột tương ứng, f là giá trị trung gian, còn A, B, K là các ma trận hệ số. Dưới đây gọi quá trình tuyến tính thứ nhất là “bước một”, quá trình tích chập và tuyến tính phía sau là “bước hai”.

Chú ý rằng để tốt hơn thuật toán thô, số giá trị trung gian không nên vượt quá $|G|^2$. Theo mô hình trên, ta có thể trực tiếp quy việc tìm thuật toán hiệu quả về một bài toán tối ưu tổ hợp.²⁶ Do đó có thể xét dùng các phương pháp tổng quát để giải bài toán tối ưu tổ hợp, như tìm kiếm, tham lam, kết hợp ngẫu nhiên hóa, học máy, v.v.

Tuy vậy cách làm này quá vét cạn. Dưới đây ta thử thảo luận sâu hơn một chút. Đáng tiếc là hiện vẫn chưa có kết luận tổng quát nào thật tốt.

6.3 Một vài thảo luận

6.3.1 Hiệu quả thời gian. Ở bước hai, hiện có hai cách tính tích chập: tính đệ quy, hoặc dùng DFT tăng tốc. Tuy nhiên, tính đệ quy có thể xem là DFT trên nhóm tầm thường, nên phép toán có thể xem như tổ hợp của một số lần DFT. Thuật toán chỉ dùng tính đệ quy được gọi là nhân chia để trị.

Giả sử kích thước miền xác định hiện tại là N , thời gian tính tích chập là $T(N)$. Khi đó thời gian biến đổi DFT là

$$N \leq T_{\text{DFT}}(N) \leq N|G|, \quad T_{\text{DFT}}(N) = \Theta(N),$$

và thời gian cho phép toán điểm trị của một DFT có kích thước miền xác định là n là $nT(N/|G|)$. Gọi kích thước miền xác định của lần DFT thứ i là n_i , ta được độ phức tạp thời gian

$$T(N) = \sum_{i \in I} \left(n_i \cdot T\left(\frac{N}{|G|}\right) + T_{\text{DFT}}\left(\frac{n_i N}{|G|}\right) \right).$$

Có $\sum_i n_i \geq |G|$; nếu không, trong tích chập sẽ có vài hạng tầm thường và ta có thể xóa chúng để giảm G . Giải ra:

$$T(N) = \begin{cases} \Theta\left(N^{\log_{|G|} \sum_i n_i}\right), & \sum_i n_i > |G|, \\ \Theta(N \log N), & \sum_i n_i = |G|. \end{cases}$$

Đặt $S = \sum_i n_i$.

Thuật toán thô tương đương với $|G|^2$ lần DFT kích thước 1, tức nhân chia để trị; từ đó cũng nhận được độ phức tạp $\Theta(N^2)$.

6.3.2 Một tối ưu đơn giản. Ta có thể đặt một giả thiết, hay ràng buộc, rất mạnh cho thuật toán: hai bước biến đổi tuyến tính giữ nguyên các phần tử ban đầu, tương đương nhân với hoán vị, và chỉ có một lần DFT kích thước $|G|$, các lần còn lại đều là nhân chia để trị.

Như vậy, thao tác của ta tương đương với việc bỏ chi phí làm S tăng thêm 1 để sửa một phần tử trong bảng phép toán của G , rồi tính và bù lại ảnh hưởng do sửa đổi gây ra. Dù thuật toán này đơn giản và thô, hiệu quả thực tế vẫn khá tốt: với $|G| = 3, 4$, kiểm chứng được $S \leq 8, 14$, tốt hơn thuật toán thô với $S = 9, 16$. Nghĩa là bằng phương pháp này, khi $|G| = 3, 4$ ta có thể nhận được các cách làm có độ phức tạp lần lượt là

$$O(n^{\log_3 8}) \approx O(n^{1.89279}), \quad O(n^{\log_4 14}) \approx O(n^{1.90368}).$$

6.3.3 Phỏng đoán.

²⁶Thông thường không gian trạng thái là hữu hạn, hoặc có thể dễ dàng tìm một tập hữu hạn chứa nghiệm tối ưu; nhưng điều này chưa chắc đúng trong một số trường hợp $|R|$ vô hạn.

Phỏng đoán 6.1. Tồn tại một thuật toán có độ phức tạp thời gian thấp nhất thỏa

$$\forall i, \quad A_i = 0 \vee B_i = 0,$$

tức mỗi dãy trung gian chỉ chứa hạng của dãy gốc a hoặc b .

Phỏng đoán này rất có thể sai, nhưng hiện tác giả chưa tìm được phản ví dụ.

Nếu có điều kiện này, thì f_i có thể chia thành hai loại theo a và b , và tích chập chỉ được tính giữa hai loại. Điều này rất có lợi cho bài toán tối ưu tổ hợp.

Đáng tiếc là do hạn chế trình độ, tác giả chưa nhận được kết luận tốt hơn và cũng chưa tìm được tài liệu tốt hơn; thảo luận của mục này tạm dừng ở đây.

7 Tổng kết

Tối đây, ta đã giải quyết DFT từ nửa nhóm giao hoán hữu hạn tới vành, thu được thuật toán biến đổi $O(N|G|\log_{|G|}N)$. Nhờ đó, với hệ có thể biến đổi, ta có thể dùng điểm trị để tính phép toán tuyến tính và phép nhân trong thời gian tuyến tính. Ngoài ra, bài viết cũng thu được phương pháp tốt hơn thuật toán thô cho tích chập tổng quát, thậm chí các phép toán bậc hai.

Bài viết này giới thiệu một ứng dụng cụ thể của DFT trong tính toán. Nội dung thảo luận còn khá cơ bản và nông, tác giả mạo muội trình bày với hy vọng có thể gợi mở thêm. Tới nay, giới toán học và khoa học tính toán đã có rất nhiều nghiên cứu; bạn đọc quan tâm có thể tham khảo tài liệu liên quan. Đáng tiếc là với trình độ của mình, tác giả chưa thể học và khám phá nhiều hơn trước hạn nộp bản thảo.

8 Hậu ký

Ý tưởng giới thiệu DFT cho các bạn thi đấu xuất hiện cách đây nửa năm. Khi ấy tôi rất băn khoăn về nguyên lý của các thuật toán FFT và FWT dùng trong thi đấu: những thuật toán này được phát hiện và xây dựng như thế nào? Sau khi tra cứu tài liệu và tự tìm hiểu, tôi nhận được phương pháp của DFT cổ điển và FWT. Tôi thử mở rộng những nội dung này, về sau có được kết luận ở Mục 5. Trong quá trình đó, tài liệu về nửa nhóm tương đối khó tìm; ngược lại tài liệu về nhóm rất nhiều, và phần lớn là các nội dung thuần toán hơn như giải tích điều hòa trừu tượng. Chúng tuy sâu sắc nhưng không thể trực tiếp dùng cho tối ưu tính toán; toán học cụ thể cũng là cần thiết. Vì vậy tôi chỉnh lý nội dung tự tìm hiểu và các tài liệu thu thập được, rồi chia sẻ với mọi người tại trại mùa đông năm 2018. Về sau tôi thử nghiên cứu tích chập và phép toán tuyến tính tổng quát hơn, thu được nội dung Mục 6. Tiếc rằng do hạn chế trình độ toán học, nhất là đại số tuyến tính, tôi chưa phát hiện kết luận thật tốt nào, nhưng vẫn thu được mô hình tối ưu tổ hợp và thuật toán tốt hơn thuật toán thô.

Những lý thuyết này thoạt nhìn có thể không liên quan nhiều tới thi đấu, nhưng sự trù tuợng và khái quát dù sao cũng là ngọn đèn mà kho tên gọi tri thức phức tạp của thi đấu còn thiếu. Với phương pháp phân tích tổng quát như vậy, ta có thể hiểu tốt hơn các mô hình và thuật toán từng học như FFT, FWT, chuỗi lũy thừa tập hợp và biến đổi Mobius. Dẫu vậy, yêu cầu các bạn tiếp nhận những lý thuyết này, thậm chí đưa chúng vào đề thi, có lẽ không phải lựa chọn tốt nhất trong bầu không khí thi tin học trung học Trung Quốc hiện nay, nơi việc học đã rõ ràng nặng hơn việc suy nghĩ.

Dù thế nào, hy vọng nội dung bài viết có thể giúp ích và gợi mở cho bạn đọc.

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn huấn luyện viên Zhang Ruizhe của đội tuyển quốc gia đã chỉ đạo.

Cảm ơn cô Liu Lili của trường tôi và các đàn anh đã giúp đỡ, bồi dưỡng và chỉ dạy tôi.

Cảm ơn các đàn anh Tang Longke, Wu Hongxun, cùng các bạn Liang Yancheng, Wang Chengrui và Wang Zeyu đã giúp đỡ về nội dung.

Cảm ơn các đàn anh Sun Yican, Zhao Yikai, Zhang Qingchuan, Zhang Mingyuan và bạn Ni Jinhan đã đọc kiểm bài viết này.

Tài liệu tham khảo

1. Donald E. Knuth, *The Art of Computer Programming, Volume 2, Seminumerical Algorithms*. Massachusetts: Addison-Wesley, 1997.
2. Francois Le Gall, *Powers of Tensors and Fast Matrix Multiplication*.
3. Jorg Arndt, *Matters Computational*.
4. Wikipedia, *Finitely generated abelian group*, [Wikipedia](https://en.wikipedia.org/wiki/Finitely_generated_abelian_group).
5. Peng Yuxiang, *Picks's Blog*, <http://picks.logdown.com>.
6. Lu Kaifeng, *Tính chất và thuật toán nhanh của chuỗi lũy thừa tập hợp*, tập luận văn ứng viên đội tuyển quốc gia Trung Quốc IOI2015.
7. Mao Xiao, *Tìm hiểu lại biến đổi Fourier nhanh*, tập luận văn ứng viên đội tuyển quốc gia Trung Quốc IOI2016.
8. Liang Yancheng, *Báo cáo ra đề Jellyfish và khảo cứu mở rộng*, tập luận văn ứng viên đội tuyển quốc gia Trung Quốc IOI2018.
9. Liu Chengao, *Bàn sơ lược về ứng dụng và mở rộng của định lý tích chập trong OI*, giao lưu trại viên NOI Winter Camp 2018.

13 Bài 11: Báo cáo ra đề Hàng đợi hoàn hảo

Lin Xuheng, THPT trực thuộc Đại học Sư phạm Phúc Kiến

Tóm tắt

Bài viết này giới thiệu một bài cấu trúc dữ liệu truyền thống do tác giả ra trong kỳ kiểm tra chéo của đội tuyển tập huấn. Đề bài có ý nghĩa đơn giản; thuật toán đúng và phân hiện thực mã đều không phức tạp, nhưng lại kiểm tra rất mạnh năng lực phân tích tính chất bài toán của thí sinh. Bài viết sẽ giới thiệu lời giải và bối cảnh ra đề.

1 Đề bài

1.1 Mô tả bài toán

Có n hàng đợi, đánh số từ 1 đến n . Hàng đợi thứ i có giới hạn dung lượng là a_i ; ban đầu mọi hàng đợi đều rỗng.

Cho m thao tác. Mỗi thao tác được mô tả bởi $l\ r\ x$, nghĩa là chèn phần tử x vào cuối mọi hàng đợi có chỉ số nằm trong $[l, r]$. Nếu sau thao tác chèn, số phần tử của một hàng đợi vượt quá giới hạn dung lượng, hãy xóa phần tử ở đầu hàng đợi đó.

Cần tính sau mỗi thao tác, trong tất cả các hàng đợi có bao nhiêu loại phần tử khác nhau.

1.2 Định dạng nhập

Dòng đầu gồm hai số nguyên dương n, m , lần lượt biểu thị số hàng đợi và số thao tác.

Dòng thứ hai gồm n số nguyên dương; số thứ i biểu thị a_i .

Tiếp theo là m dòng, mỗi dòng gồm ba số nguyên dương l, r, x ; dòng thứ i biểu thị thao tác thứ i .

1.3 Định dạng xuất

Xuất m dòng, mỗi dòng là một số nguyên không âm, biểu thị số loại phần tử khác nhau trong tất cả các hàng đợi sau khi thao tác thứ i kết thúc.

1.4 Phạm vi dữ liệu và quy ước

Với mọi dữ liệu, $n, m, a_i, x \leq 10^5, l \leq r$.

Có 20 test, mỗi test 5 điểm; test thứ k thỏa

$$n, m, a_i, x \leq 5000k.$$

Đặc biệt, một số test sau thỏa các tính chất riêng:

- test 5: $a_i = 1$;
- test 7: $a_i = 2$;
- test 9: $a_i = 10$;
- test 11: $a_i \leq 10$;
- test 13, 15: $\sum a_i \leq 10^6$.

Với mỗi test, bạn cần vượt qua mọi dữ liệu thỏa phạm vi và tính chất của test đó để nhận điểm của test.

2 Thuật toán ngây thơ

2.1 Mô phỏng

Trực tiếp dùng mảng hoặc `std::queue` của C++ để mô phỏng từng thao tác.

Dùng một mảng ghi số lần xuất hiện của mỗi giá trị; khi chen và xóa thì bảo trì mảng này, và khi tính tồn tại của một giá trị thay đổi thì cập nhật đáp án.

Độ phức tạp thời gian là $O(nm)$, độ phức tạp bộ nhớ là $O(\sum a_i)$.

Tùy cách hiện thực, điểm kỳ vọng là 5–15 điểm.

2.2 Phân tích ban đầu

Cách mô phỏng trực tiếp bỏ qua quá nhiều tính chất có thể dùng. Bây giờ ta phân tích sơ bộ bài toán.

Đề bài không yêu cầu xử lý online từng thao tác, nên ta có thể cân nhắc xử lý offline. Trước hết xét ảnh hưởng của mỗi thao tác lên đáp án. Thời gian một thao tác có ảnh hưởng tới đáp án hiển nhiên chỉ nằm sau khi thao tác đó được thực hiện và trước khi mọi phần tử do thao tác đó chen đều bị xóa. Ta gọi thời điểm mọi phần tử do một thao tác chen đều bị xóa là *thời điểm bị xóa* của thao tác đó.

Nếu có thể tìm thời điểm bị xóa của mỗi thao tác, ta dễ dàng tính được đáp án cần tìm. Một cách là trừu tượng hóa thời gian tồn tại của mỗi thao tác thành một đoạn trên trục thời gian. Với các thao tác chèn cùng một phần tử, thời gian phần tử đó đóng góp vào đáp án là hợp của các đoạn thời gian này. Sau khi tìm hợp đoạn cho từng phần tử, ta dùng mảng hiệu để nhanh chóng tính đáp án. Cách khác là chuyển bài toán thành bảo trì một đa tập: mỗi thao tác của bài gốc biến thành thêm vào tập một phần tử mà thao tác đó chèn tại thời điểm thực hiện, và xóa khỏi tập một phần tử đó tại thời điểm thao tác bị xóa. Khi ấy đáp án của bài gốc bằng số phần tử khác nhau trong đa tập này; dùng phương pháp đã nhắc ở phần mô phỏng thì không khó để bảo trì số loại phần tử khác nhau. Cách thứ hai hiện thực hơi đơn giản hơn cách thứ nhất.

Như vậy, phần còn lại của bài toán hoàn toàn chuyển thành tìm thời điểm bị xóa của mỗi thao tác.

2.3 Thuật toán ngây thơ dựa trên phân tích ban đầu

Dễ nghĩ tới rằng thời điểm mọi phần tử do một thao tác chèn đều bị xóa chính là giá trị lớn nhất trong các thời điểm bị xóa của từng phần tử được thao tác đó chèn; nếu một phần tử mãi không bị xóa thì xem thời điểm là dương vô cùng. Ta xét tính riêng thời điểm ra hàng đợi của từng phần tử. Không khó thấy trong hàng đợi thứ i , phần tử được chèn ở lần chèn thứ j sẽ bị xóa ở lần chèn thứ $j + a_i$. Ta có thể với mỗi hàng đợi, tìm riêng các thao tác chèn phần tử vào hàng đợi đó, rồi cập nhật thô bạo thời điểm bị xóa của từng thao tác trong đó.

Độ phức tạp thời gian là $O(nm)$, độ phức tạp bộ nhớ là $O(n + m)$. Điểm kỳ vọng là 20–30 điểm.

3 Thuật toán online

Phần này không liên quan nhiều tới ý tưởng lời giải chuẩn. Các thuật toán trong phần này được đưa vào dưới dạng dữ liệu đặc biệt để kiểm tra liệu thí sinh có thể lấy điểm cao hơn bằng cách khác hay không khi chưa có ý tưởng thật rõ ràng về lời giải chuẩn. Dưới đây giới thiệu thuật toán của phần này qua lời giải cho từng test đặc biệt.

3.1 Test 5: $a_i = 1$

Giới hạn dung lượng của mỗi hàng đợi đúng bằng 1. Bài toán tương đương với bảo trì một dãy, mỗi thao tác sửa mọi phần tử trong một đoạn của dãy thành x , đồng thời tính số loại phần tử khác nhau trong dãy.

Bảo trì dãy này như thế nào? Các thao tác sửa đoạn truyền thống thường dùng cây đoạn, nhưng số loại phần tử khác nhau mà bài này yêu cầu lại khó bảo trì trực tiếp trên cây đoạn.

Chú ý một sự thật: do thao tác sửa của ta sửa toàn bộ đoạn thành x , trong dãy đang bảo trì thường xuất hiện các đoạn dài gồm phần tử trùng nhau. Liệu ta có thể thiết kế một thuật toán nhắm vào tính chất này để bảo trì dãy hay không?

Câu trả lời là có. Ta có thể gộp các phần tử kề nhau và bằng nhau thành một đoạn nguyên vẹn. Khi sửa, không khó để bảo trì các đoạn phần tử này dựa trên quan hệ vị trí giữa từng đoạn cũ và đoạn cần sửa; đáp án cũng có thể tính bằng cách bảo trì số đoạn mà mỗi phần tử xuất hiện. Hiển nhiên, khi các đoạn sửa được chọn ngẫu nhiên đều, số đoạn phần tử khác nhau thường không quá nhiều, nên hiệu quả của cách làm này rất đáng kể.

Tuy nhiên, trong dữ liệu được dựng, số đoạn phần tử khác nhau rất dễ đạt tới cấp $O(m)$; ta có thể giả sử m cùng cấp với n . Khi đó thời gian bảo trì thô bạo các đoạn phần tử là không thể chấp nhận.

Ta có thể dùng cấu trúc dữ liệu để tối ưu quá trình bảo trì này. Trước hết, chỉ những đoạn phần tử giao với đoạn sửa mới bị sửa. Nếu có thể nhanh chóng tìm được các đoạn phần tử đó, ta có thể tối ưu thêm thuật toán.

Để nghĩ tới việc dùng cây cân bằng, hoặc `std::set` của C++, để bảo trì các đoạn phần tử theo thứ tự vị trí trong dãy. Khi đó mỗi lần ta có thể dùng $O(\log m)$ thời gian để nhanh chóng tìm một đoạn phần tử giao với đoạn sửa và xử lý ảnh hưởng của thao tác sửa.

Tuy vậy, số đoạn phần tử giao với một đoạn sửa vẫn có thể đạt $O(m)$, nên có vẻ tối ưu này về bản chất chưa hạ độ phức tạp thời gian. Nhưng ta chú ý rằng nếu một đoạn phần tử bị đoạn sửa chứa, đoạn phần tử này sẽ bị xóa trực tiếp. Hơn nữa, do các đoạn phần tử ta bảo trì không giao nhau, số đoạn phần tử giao với một đoạn sửa nhưng không bị chứa nhiều nhất chỉ là 2. Vậy ngoài nhiều nhất 2 đoạn phần tử đó, mọi đoạn phần tử khác giao với đoạn sửa đều bị ta xóa. Mỗi thao tác cũng sinh thêm nhiều nhất 2 đoạn phần tử: nếu đoạn sửa nằm trong một đoạn phần tử, đoạn phần tử bị tách thành hai đoạn và thêm một đoạn mới, tức tăng 2 đoạn so với ban đầu; nếu không, đoạn bị sửa hoặc bị rút ngắn hoặc bị xóa, và chỉ thêm một đoạn mới. Do đó tổng số đoạn phần tử mà mọi thao tác có thể sinh ra chỉ là $O(m)$, nên số lần xóa đoạn phần tử cũng không vượt quá $O(m)$. Như vậy ta chứng minh được độ phức tạp thời gian của thuật toán này là $O(m \log m)$, đủ để vượt qua dữ liệu của test này.

Khi hiện thực, ta chỉ cần bảo trì đầu trái của các đoạn phần tử. Để tiện, có thể giả sử ban đầu tồn tại một đoạn phần tử rộng phủ toàn bộ dãy, và chỉ gộp các phần tử giống nhau được cùng một thao tác sửa ra thành một đoạn. Hiển nhiên điều này không ảnh hưởng tới độ phức tạp. Khi sửa, đầu trái của đoạn sửa và vị trí sau đầu phải của đoạn sửa chắc chắn trở thành đầu trái của một đoạn phần tử sau khi sửa; chỉ cần thêm trực tiếp chúng vào cây cân bằng. Còn các đầu đoạn cũ nằm giữa hai đầu nút này có thể xóa trực tiếp.

Cách chỉ bảo trì đầu trái giảm được nhiều trường hợp biên không cần thiết, và ý tưởng cũng rõ ràng hơn. Nhìn từ quá trình của cách làm này, mỗi lần chỉ thêm 2 đầu đoạn mới và xóa các đầu đoạn cũ, nên kết luận tổng số lần sửa là $O(m)$ cũng rõ ràng hơn.

3.2 Test 7: $a_i = 2$

Khi giới hạn dung lượng tăng lên 2, bài toán không còn là bảo trì một dãy đơn giản. Nhưng ta vẫn có thể chuyển nó thành bài toán dãy: cần bảo trì hai dãy, lần lượt biểu thị hai phần tử của mỗi hàng đợi. Mỗi thao tác sửa đoạn $[l, r]$ của dãy thứ nhất thành x ; mỗi khi một phần tử bị sửa, vị trí tương ứng trên dãy thứ hai phải được thay bằng phần tử đó.

Tiếp tục xét dùng thuật toán của test 5: bảo trì các đoạn phần tử của dãy thứ nhất. Khi một đoạn phần tử của dãy thứ nhất bị sửa, phần bị rút ngắn hoặc bị xóa sẽ thực hiện thao tác sửa đoạn trên vị trí tương ứng ở dãy thứ hai. Vì khi bảo trì các đoạn phần tử của dãy thứ nhất, tổng số lần sửa đoạn phần tử là $O(m)$, nên số lần sửa đoạn ta thực hiện trên dãy thứ hai cũng là $O(m)$; dùng cùng một thuật toán là có thể bảo trì.

Độ phức tạp thời gian tổng của thuật toán này vẫn là $O(m \log m)$, và có thể vượt qua test này.

3.3 Test 9: $a_i = 10$

Bây giờ xét trường hợp mọi giới hạn dung lượng đều bằng K . Để nghĩ rằng ta có thể làm tương tự trường hợp giới hạn dung lượng đều bằng 2. Ta bảo trì K dãy, trong đó dãy thứ i biểu thị phần tử thứ i trong mỗi hàng đợi. Khi một đoạn phần tử của một dãy bị sửa, đồng thời thực hiện sửa đoạn trên dãy tiếp theo. Nếu số lần sửa của một dãy là $O(m)$, thì số lần sửa của dãy tiếp theo cũng là $O(m)$. Thế là dường như ta đương nhiên thu được một thuật toán $O(Km \log m)$.

Nhưng cần chú ý một điểm: khi tính độ phức tạp bảo trì các đoạn phần tử, $O(m)$ lần sửa mà ta suy ra thật ra đã bỏ qua hằng số. Hằng số này rất có thể liên tục phình ra khi ta chuyển thao tác sửa từ một dãy sang dãy kế tiếp, thậm chí tăng theo cấp số mũ. Chẳng hạn, liệu có khả năng dãy thứ nhất cần $2m$ lần, dãy thứ hai cần $4m$ lần, dãy thứ ba cần $8m$ lần, và dãy thứ K cần $2^K m$ lần hay không?

Ta thử phân tích độ phức tạp của thuật toán này từ một góc nhìn khác. Xét mọi đầu mút đoạn phần tử. Trên thực tế, trong mỗi thao tác, các đầu mút mới sinh ra trên mọi dãy chỉ xuất hiện tại đầu trái và đầu phải của thao tác, tức nhiều nhất tăng $O(K)$ đầu mút. Nhưng điều này không có nghĩa tổng độ phức tạp thời gian là $O(Km \log m)$, vì không phải trong mọi lần sửa ta đều xóa được ít nhất một đầu mút.

Vậy tính độ phức tạp thế nào? Có thể xét như sau: với mỗi vị trí, định nghĩa độ sâu của nó là chỉ số dãy đầu tiên mà nó xuất hiện với vai trò đầu mút đoạn phần tử. Khi mỗi thao tác được thực hiện, độ sâu của hai đầu mút thao tác được cập nhật thành 1, còn mọi vị trí đầu mút khác bị ta ảnh hưởng đều có độ sâu tăng 1; nếu vượt quá K thì bị xóa hoàn toàn. Nói cách khác, mọi đầu mút mới được mỗi thao tác thêm vào nhiều nhất chỉ bị ta xử lý $O(K)$ lần, nên độ phức tạp thời gian thật sự của thuật toán này là $O(K^2 m \log m)$.

Thực ra thuật toán này còn có thể tối ưu thêm. Chú ý rằng mỗi thao tác sửa ta làm tương đương với việc trước hết chèn các đầu mút đoạn phần tử tại đầu trái và đầu phải của đoạn sửa, sau đó xóa các đoạn phần tử trong dãy thứ K nằm trong đoạn sửa, và chuyển các đoạn phần tử trong những dãy khác sang dãy tiếp theo. Ta có thể dùng cây cân bằng hỗ trợ chèn xóa đoạn, chẳng hạn splay, để bảo trì các đầu mút đoạn phần tử; như vậy có thể nhanh chóng chuyển các đoạn phần tử sang dãy tiếp theo.

Sau tối ưu, mỗi đầu mút chỉ bị ta xử lý một lần khi bị xóa khỏi dãy thứ K . Mỗi lần chuyển các đoạn phần tử của từng dãy sang dãy tiếp theo có độ phức tạp $O(K \log m)$, nên tổng độ phức tạp thời gian là $O(Km \log m)$.

3.4 Test 11: $a_i \leq 10$

Khi giới hạn dung lượng không nhất thiết bằng nhau, bài toán trở nên phức tạp hơn. Một ý tưởng là với mỗi loại giới hạn dung lượng, dùng riêng thuật toán đã tối ưu của test 9; độ phức tạp thời gian khi đó là $O((\max a_i)^2 m \log m)$.

Thực ra ta có một cách đơn giản hơn. Xét bảo trì tất cả hàng đợi cùng nhau, bảo trì $\max a_i$ dãy. Khi đó dãy có thể bị “cắt ngang” vì dung lượng của một hàng đợi nào đó không đủ; nhưng nếu ta cũng cắt thao tác sửa trên các dãy này để cưỡng ép bảo trì, độ phức tạp hiển nhiên là sai.

Ta đổi cách nghĩ: bù các vị trí bị cắt trong dãy. Phần tử tại các vị trí được bù không có ý nghĩa thực tế, mà chỉ dùng để hỗ trợ bảo trì dãy. Một đoạn phần tử trong các dãy này có ít nhất một phần tử thật sự tồn tại trong hàng đợi khi và chỉ khi giá trị lớn nhất của a_i trên mọi hàng đợi tương ứng với đoạn phần tử đó không nhỏ hơn chỉ số của dãy chứa đoạn phần tử. Sau tiên xử lý, dùng RMQ không khó để kiểm tra trong $O(1)$ liệu một đoạn phần tử có thật sự tồn tại hay không. Trước hết không xét dùng cây cân bằng cao cấp để tối ưu. Theo phân tích độ phức tạp trước đó, cách làm này cũng có độ phức tạp thời gian $O((\max a_i)^2 m \log m)$.

Nếu dùng cây cân bằng hỗ trợ chèn xóa đoạn để tối ưu, so với cách làm của test 9, lúc này đoạn phần tử có thể mất đóng góp vào đáp án ngay khi được chuyển sang dãy tiếp theo. Với mỗi đoạn phần tử vẫn thật sự tồn tại, ta có thể dùng RMQ để tìm nó sẽ mất đóng góp khi chuyển tới dãy nào, rồi bảo trì giá trị nhỏ nhất này trên cây cân bằng. Như vậy mỗi lần chuyển đoạn phần tử, không khó để tìm các đoạn màu mất đóng góp trên cây cân bằng và xử lý ảnh hưởng lên đáp án.

Có thể chứng minh độ phức tạp thời gian sau tối ưu là $O(\max a_i m \log m)$.

3.5 Test 13, 15: $\sum a_i \leq 10^6$

Khi tổng các a_i bị giới hạn, không dễ xuất hiện quá nhiều hàng đợi có dung lượng lớn.

Ta có thể phân loại theo a_i : các hàng đợi có $a_i \leq K$ được bảo trì cùng nhau bằng cách trước đó, còn

các hàng đợi có $a_i > K$ thì mô phỏng trực tiếp. Tổng độ phức tạp thời gian là

$$O\left(Km \log m + \frac{\sum a_i}{K} m\right).$$

Chọn giá trị K thích hợp, ta thu được một cách làm có độ phức tạp

$$O\left(m \sqrt{\sum a_i \log m}\right),$$

kèm một vài tối ưu hằng số là có thể vượt qua phần dữ liệu này.

4 Phương pháp chia đôi

Dưới đây giới thiệu một lớp lời giải offline của bài này. Lớp lời giải này tìm thời điểm bị xóa của mỗi thao tác bằng chia đôi hoặc một số phương pháp tương tự rồi tính đáp án, và có thể nhận được số điểm đáng kể.

4.1 Kiểm tra tính hợp lệ

Trước đó ta đã nhắc rằng nếu tìm được thời điểm bị xóa của mỗi thao tác thì có thể thu được đáp án. Bây giờ câu hỏi là làm sao nhanh chóng tìm thời điểm bị xóa của từng thao tác.

Rất tự nhiên, ta nghĩ tới việc dùng chia đôi để tìm thời điểm bị xóa cho từng thao tác. Khi đó bài toán được chuyển thành một bài toán quyết định: sau khi thao tác thứ mid kết thúc, các phần tử do thao tác thứ x chèn đã bị xóa hết hay chưa?

Trước hết, ta biết rằng trong hàng đợi thứ i , phần tử được chèn ở thao tác thứ j sẽ bị xóa ở thao tác thứ $j + a_i$, tức ở lần thao tác thứ a_i sau nó trên hàng đợi đó. Nói cách khác, chỉ các thao tác sau thao tác thứ x mới có ảnh hưởng tới nó. Dùng b_i biểu thị phần tử mà thao tác thứ x chèn vào hàng đợi thứ i còn cần bao nhiêu lần chèn vào hàng đợi i nữa mới bị xóa. Khi thao tác thứ x được thực hiện, mọi b_i trong đoạn $[l, r]$ mà nó ảnh hưởng được khởi tạo thành a_i . Sau đó, mỗi thao tác làm mọi b_i trong đoạn tương ứng của thao tác đó giảm 1. Khi có b_i bị giảm về 0, tức là phần tử do thao tác thứ x chèn trong hàng đợi này đã bị xóa. Khi mọi b_i đều bị giảm về 0, mọi phần tử do thao tác thứ x chèn đều đã bị xóa.

Để tiện, ta cho phép b_i nhỏ hơn 0. Khi đó mỗi thao tác tương đương với giảm đoạn 1; khi giá trị lớn nhất của b_i trên đoạn $[l, r]$ mà thao tác x chèn không lớn hơn 0, mọi phần tử do thao tác x chèn đều đã bị xóa.

Giảm đoạn và truy vấn giá trị lớn nhất trên đoạn có thể được bảo trì bằng cây đoạn. Như vậy ta có một phương pháp kiểm tra: dựng cây đoạn với giá trị ban đầu là a_i , với mỗi thao tác có thời gian trong $(x, mid]$, giảm 1 trên đoạn $[l, r]$ tương ứng, rồi kiểm tra xem giá trị lớn nhất trên đoạn $[l, r]$ mà thao tác x chèn có không lớn hơn 0 hay không.

Nếu mỗi lần kiểm tra đều dựng lại cây đoạn từ đầu và thực hiện các thao tác này, tổng độ phức tạp thời gian là $O(n^2 \log^2 n)$; ở đây ta xem n và m cùng cấp, không phân biệt. Cách này thậm chí còn kém thuật toán ngây thơ.

Nhưng ta chú ý rằng nếu đã có cây đoạn sau khi giảm 1 trên mọi đoạn của các thao tác có thời gian trong $[l, r]$, thì chỉ cần một thao tác cộng hoặc trừ 1 trên đoạn với chi phí $O(\log n)$ là có thể thu được cây đoạn tương ứng với một trong bốn khoảng thời gian $[l - 1, r]$, $[l + 1, r]$, $[l, r - 1]$, $[l, r + 1]$.

Hiển nhiên, ta có thể tận dụng tính chất này để thiết kế một vài thuật toán tối ưu độ phức tạp thời gian.

4.2 Chia đôi toàn cục

Ta thiết kế một thuật toán giống chia để trị. Dùng $\text{solve}(S, l, r)$ biểu thị việc ta tìm thời điểm bị xóa trong khoảng thời gian $[l, r]$ cho mỗi thao tác thuộc tập S . Trước hết lấy điểm giữa mid của $[l, r]$, rồi lần lượt kiểm tra mỗi thao tác trong S xem sau thao tác thứ mid nó đã bị xóa hết hay chưa. Dựa vào kết quả, chia S thành hai phần và đệ quy lần lượt vào $\text{solve}(S_1, l, mid)$ và $\text{solve}(S_2, mid + 1, r)$.

Để tối ưu thời gian kiểm tra, ta chỉ bảo trì một cây đoạn và ghi lại khoảng thời gian mà nó tương ứng, tức đầu trái và đầu phải của khoảng các thao tác đã được giảm 1. Khi cần truy vấn, với chi phí $O(\log n)$ mỗi lần, ta dịch đầu trái hoặc đầu phải sang vị trí kề bên, cho tới khi tới vị trí cần thiết. Khi đó độ phức tạp chỉ liên quan tới tổng số lần dịch hai đầu nút. Đặc biệt, lần truy vấn đầu tiên tương đương với $O(n)$ lần dịch đầu nút; phần phân tích dưới đây bỏ qua chi phí này.

Trước hết xét đầu phải. Có thể phân tích như sau: tại $\text{solve}(S, l, r)$, đầu phải ở mid . Khi gọi $\text{solve}(S_1, l, mid)$, ta dịch đầu phải từ mid tới điểm giữa của $[l, mid]$, và dịch trở lại khi lời gọi kết thúc. Khi gọi $\text{solve}(S_2, mid + 1, r)$, ta dịch đầu phải từ mid tới điểm giữa của $[mid + 1, r]$, và cũng dịch trở lại khi lời gọi kết thúc. Thực tế không khó thấy số lần dịch đầu phải còn ít hơn phân tích này, nhưng như vậy đã đủ. Theo đó số lần dịch đầu phải cùng cấp $O(r - l)$, nên tổng số lần dịch đầu phải không vượt quá $O(n \log n)$.

Tuy nhiên với đầu trái, ta không thể bảo đảm tổng số lần dịch nằm trong phạm vi chấp nhận được. Do phân bố thời điểm bị xóa của mọi thao tác không có tính chất đặc biệt, trong mỗi lời gọi hàm solve , số lần dịch đầu trái đều có thể đạt $O(n)$. Ở trường hợp cực đoan, tổng số lần dịch đầu trái có thể đạt $O(n^2)$, khó chấp nhận.

Ta chú ý rằng đầu trái chỉ dịch trong phạm vi từ thao tác có chỉ số nhỏ nhất tới thao tác có chỉ số lớn nhất trong tập S . Có thể chia các thao tác theo chỉ số thành K khối, và gọi một lần $\text{solve}(S_i, 1, m)$ cho mỗi khối, trong đó S_i là tập các thao tác thuộc khối đó. Khi ấy mỗi lần kiểm tra, đầu trái dịch không quá $O(n/K)$ lần. Mỗi thao tác sẽ được đệ quy và kiểm tra $O(\log n)$ lần, nên tổng số lần dịch đầu trái không vượt quá $O(n^2 \log n / K)$. Mặt khác, vì ta gọi tổng cộng K lần $\text{solve}(S_i, 1, m)$, tổng số lần dịch đầu phải là $O(Kn \log n)$. Tính thêm độ phức tạp của cây đoạn, tổng độ phức tạp thời gian của cách làm này là

$$O\left(\left(\frac{n^2}{K} + Kn\right) \log^2 n\right).$$

Khi K lấy $O(\sqrt{n})$, độ phức tạp thời gian là

$$O(n\sqrt{n} \log^2 n).$$

Do trong thực tế độ phức tạp của thuật toán này không đầy, kèm một số tối ưu có thể nhận được số điểm khá đáng kể.

4.3 Thuật toán kiểu Mo

Quay lại cách chia đôi trực tiếp.

Chú ý rằng thao tác dịch hai đầu nút ta thực hiện rất giống thuật toán Mo. Thuật toán Mo thông thường chia các truy vấn thành khối theo đầu trái, trong mỗi khối xử lý lần lượt theo thứ tự đầu phải; đầu trái chỉ dịch trong khối, còn đầu phải liên tục dịch sang phải, nhờ đó bảo đảm độ phức tạp. Ta có thể thiết kế một thuật toán tương tự để đáp ứng nhu cầu kiểm tra tính hợp lệ trong quá trình chia đôi.

Ta chia khối theo đầu phải của khoảng thời gian cần truy vấn khi kiểm tra, tức $(x, mid]$ đã nhắc ở trên. Sau đó với mỗi khối, dựng một cây đoạn và ghi lại hai đầu nút tương ứng của nó. Khi truy vấn, ta tìm khối chứa đầu phải rồi dịch hai đầu nút của cây đoạn tương ứng tới vị trí truy vấn. Nếu ta chia đôi lần lượt từng thao tác theo thứ tự thời gian, không khó thấy đầu trái của mọi cây đoạn chỉ dịch sang

phải. Nếu có tổng cộng K khối, tổng số lần dịch đầu trái là $O(Kn)$. Còn đầu phải chỉ dịch trong khối; vì trong quá trình chia đôi ta thực hiện tổng cộng $O(n \log n)$ lần kiểm tra, nên tổng số lần dịch đầu phải là $O(n^2 \log n/K)$. Tính thêm độ phức tạp của cây đoạn, tổng độ phức tạp thời gian là

$$O\left(\left(\frac{n^2 \log n}{K} + Kn\right) \log n\right).$$

Khi K lấy $O(\sqrt{n \log n})$, độ phức tạp thời gian là

$$O(n\sqrt{n} \log^{1.5} n).$$

4.4 Cải tiến thuật toán kiểu Mo

Trong thuật toán trước, quá trình liên tục dịch đầu trái của mọi cây đoạn sang phải thực ra đã cho ta thông tin với mỗi thao tác làm đầu trái và đầu phải nằm trong từng khối. Cách chia đôi chưa tận dụng hết các thông tin này, nên ta có thể cải tiến.

Ta đặt đều K điểm then chốt trên trục thời gian, tương đương với các khối trước đó. Sau đó lần lượt lấy mỗi điểm then chốt làm đầu phải và tính thông tin cho mọi đầu trái. Trong lúc tính, ta kiểm tra liệu mỗi thao tác đã bị xóa khi tới từng điểm then chốt hay chưa, từ đó xác định thời điểm bị xóa của mỗi thao tác nằm giữa hai điểm then chốt nào. Phần này có độ phức tạp thời gian $O(Kn \log n)$.

Cuối cùng, với mỗi cặp điểm then chốt kề nhau, xét các thao tác có thời điểm bị xóa nằm giữa chúng theo thứ tự, và tìm thời điểm bị xóa bằng cách lần lượt kiểm tra từng thời điểm. Như vậy với mỗi cặp điểm then chốt kề nhau, đầu trái liên tục dịch sang phải, tổng số lần dịch là $O(Kn)$. Với mỗi thao tác, ta kiểm tra thô bạo mọi đầu phải giữa cặp điểm then chốt tương ứng, nên tổng số lần dịch đầu phải là $O(n^2/K)$. Tính thêm độ phức tạp của cây đoạn, tổng độ phức tạp thời gian là

$$O\left(\left(\frac{n^2}{K} + Kn\right) \log n\right).$$

Khi K lấy $O(\sqrt{n})$, thuật toán này có độ phức tạp thời gian là

$$O(n\sqrt{n} \log n).$$

5 Chia khối dãy

Phần này là lời giải chuẩn của bài. So với các thuật toán trước, nó tận dụng tốt hơn tính chất của bài toán.

5.1 Tính đơn điệu

Ta chú ý một sự thật hiển nhiên: trong mỗi hàng đợi, thời điểm bị xóa của từng phần tử tăng đơn điệu theo thời điểm chen. Nhưng với toàn bộ thao tác, thời điểm bị xóa của chúng lại không tăng đơn điệu theo thời gian thao tác. Nguyên nhân là các phần tử được các thao tác khác nhau chen có thể nằm trong các hàng đợi khác nhau, và giữa các hàng đợi khác nhau không có liên hệ.

Từ đây nảy sinh một ý tưởng: nếu có một số thao tác chen vào các đoạn hàng đợi hoàn toàn giống nhau, thì thời điểm bị xóa của chúng có đơn điệu tăng theo thứ tự thời gian hay không?

Câu trả lời là hiển nhiên. Trong mỗi hàng đợi, thời điểm bị xóa của phần tử tăng theo thời gian; vậy với các thao tác chen vào cùng đúng một tập hàng đợi, thời điểm bị xóa của chúng, tức giá trị lớn nhất trong các thời điểm bị xóa của từng phần tử được chen, tự nhiên cũng tăng theo thời gian.

Ta có thể thiết kế thuật toán sau để tính thời điểm bị xóa của các thao tác có đoạn hàng đợi được chèn hoàn toàn giống nhau. Trước hết dựng cây đoạn trên các a_i giống phần trước. Sau đó lấy thao tác đầu tiên cần tìm làm đầu trái, và liên tục đẩy đầu phải sang phải cho tới khi giá trị lớn nhất trên đoạn được chèn của thao tác cần tìm không lớn hơn 0; khi đó ta tìm được thời điểm bị xóa của thao tác đầu tiên. Tiếp theo, ta tính thời điểm bị xóa của từng thao tác cần tìm theo thứ tự thời gian. Do thời điểm bị xóa của chúng tăng đơn điệu, mỗi lần chỉ cần dịch đầu trái tới vị trí tương ứng, còn đầu phải tiếp tục đẩy sang phải. Vì cả hai đầu mút của ta đều không ngừng đẩy sang phải, tổng số lần dịch là $O(n)$. Do đó thời gian tìm thời điểm bị xóa cho mọi thao tác có đoạn hàng đợi được chèn bằng một đoạn nào đó là $O(n \log n)$.

Nhưng hai thao tác khác nhau rất có thể chèn vào các đoạn không giống nhau, vậy làm sao tận dụng tính chất này? Với mỗi thao tác, ta chỉ quan tâm giá trị lớn nhất trong các thời điểm bị xóa của từng phần tử được nó chèn. Ta có thể chia đoạn chèn của mỗi thao tác thành một số đoạn con để biến nó thành một số thao tác con, rồi lần lượt tìm thời điểm bị xóa của các thao tác con này và lấy giá trị lớn nhất trong đó.

Ta nghĩ tới việc chia mọi hàng đợi theo chỉ số thành các khối. Nếu kích thước mỗi khối là K , thì mỗi thao tác có thể được tách thành $O(n/K)$ thao tác trên khối nguyên và $O(1)$ thao tác không nguyên khối. Do tổng cộng chỉ có $O(n/K)$ khối khác nhau, ta có thể dùng thuật toán vừa nêu cho mỗi khối để tìm thời điểm bị xóa của mọi thao tác nguyên khối trên khối đó. Tổng độ phức tạp thời gian của phần này là

$$O\left(\frac{n^2}{K} \log n\right).$$

Đối với thao tác không nguyên khối, tổng cộng chỉ có $O(n)$ thao tác như vậy, tức nhiều nhất chỉ chèn $O(nK)$ phần tử. Ta xét trực tiếp tìm thời điểm bị xóa của từng phần tử trong đó. Do trong hàng đợi thứ i , phần tử được chèn thứ j sẽ bị xóa ở lần chèn thứ $j + a_i$, ta có thể trước hết tìm một thao tác là thao tác thứ mấy trong hàng đợi thứ i , rồi tìm thao tác thứ a_i sau nó trong hàng đợi thứ i là thao tác nào.

Ta có thể theo thứ tự chỉ số hàng đợi để lần lượt tìm tập thao tác tương ứng với mỗi hàng đợi. Với mỗi thao tác, thêm nó vào tập ở đầu trái đoạn chèn và xóa nó khỏi tập ở đầu phải đoạn chèn. Để hỗ trợ truy vấn thông tin, chỉ cần bảo trì một cây Fenwick theo thời gian thao tác, hỗ trợ tính tổng tiền tố và chia đôi trên cây Fenwick. Tổng độ phức tạp thời gian là $O(nK \log n)$.

Vậy khi K lấy $O(\sqrt{n})$, ta thu được một cách làm

$$O(n\sqrt{n} \log n).$$

5.2 Tối ưu thao tác nguyên khối

Khi tính thời điểm bị xóa của các thao tác nguyên khối trên một khối, ta chú ý rằng các a_i thật sự được dùng chỉ là phần nằm trong khối.

Ta có thể chỉ dựng cây đoạn trên các a_i trong khối và bỏ qua trực tiếp ảnh hưởng của mọi thao tác lên ngoài khối. Khi đó ta phát hiện tuyệt đại đa số thao tác cộng 1 và trừ 1 trên đoạn thật ra đều tác động trực tiếp lên cả khối. Trên thực tế, khi chia khối, ảnh hưởng của mỗi thao tác lên mỗi khối cũng được chia thành $O(n/K)$ khối nguyên và $O(1)$ khối không đầy đủ. Nói cách khác, tổng cộng ta cần thực hiện $O(n^2/K)$ lần cộng trừ nguyên khối và $O(n)$ lần cộng trừ đoạn. Đánh dấu lười trực tiếp có thể hoàn thành một lần cộng trừ nguyên khối trong $O(1)$, nên độ phức tạp thời gian giảm xuống

$$O\left(\frac{n^2}{K} + n \log n\right).$$

Thực tế, ta có thể dùng mảng mô phỏng thay cho cây đoạn để hoàn thành cộng trừ đoạn và bảo trì giá trị lớn nhất; cộng trừ nguyên khối vẫn dùng đánh dấu. Khi đó độ phức tạp tuy trở thành

$$O\left(\frac{n^2}{K} + nK\right),$$

nhưng dễ hiện thực hơn và không ảnh hưởng tới độ phức tạp tổng.

Với thao tác không nguyên khối, ta dùng cách làm giống trước đó. Chọn K phù hợp, độ phức tạp thời gian giảm xuống

$$O(n\sqrt{n \log n}),$$

đủ để vượt qua toàn bộ dữ liệu.

5.3 Tối ưu thao tác không nguyên khối

Trong cách dùng cây Fenwick để tính thời điểm bị xóa của thao tác không nguyên khối trước đó, ta chưa tận dụng tính chất thời điểm bị xóa của phần tử trong cùng một hàng đợi tăng đơn điệu. Liệu ta có thể thiết kế cho thao tác không nguyên khối một thuật toán tương tự thao tác nguyên khối hay không?

Ta có thể xét riêng từng khối. Với một khối, gọi A là tập các thao tác nguyên khối của khối này, gọi B là tập các thao tác không nguyên khối trong khối này. Ta cần tìm cho mỗi thao tác trong B thời điểm bị xóa của từng phần tử mà nó chèn trong khối. Với một hàng đợi i trong khối, gọi B_i là tập các thao tác trong B có ảnh hưởng tới hàng đợi này. Với mỗi thao tác x trong B_i , ta tìm thao tác sớm nhất y trong B_i sao cho trước khi y được thực hiện, phần tử mà x chèn vào i đã bị xóa. Theo tính chất thời điểm bị xóa trong cùng một hàng đợi tăng đơn điệu, y hiển nhiên cũng không giảm. Vì vậy ta vẫn có thể dùng phương pháp hai đầu mút liên tục đẩy sang phải trong B_i để tìm. Vấn đề là làm sao kiểm tra phần tử mà x chèn đã bị xóa hay chưa, tức tính giữa hai thao tác có bao nhiêu thao tác ảnh hưởng tới i . Do các thao tác ảnh hưởng tới i chỉ gồm thao tác trong B_i và trong A , ta có thể tiền xử lý với mỗi thao tác trong B , thao tác trước nó thuộc A là thao tác thứ mấy trong A ; như vậy có thể tính trong $O(1)$. Sau khi tìm được y , qua một số phép tính đơn giản, không khó suy ra thời điểm bị xóa thật sự của phần tử mà thao tác x chèn trong i là thao tác thứ mấy trước y trong các thao tác có ảnh hưởng tới i . Dùng thông tin tiền xử lý có thể tìm thao tác đó trong $O(1)$. Như vậy, ngoài thời gian tiền xử lý, ta chỉ dùng $O(|B_i|)$ thời gian để tìm thời điểm bị xóa trong i cho mọi thao tác thuộc B_i .

Không khó chứng minh sau khi dùng thuật toán này tối ưu, tổng độ phức tạp thời gian là

$$O\left(\frac{n^2}{K} + nK\right).$$

Khi K lấy $O(\sqrt{n})$, ta thu được một thuật toán xuất sắc có độ phức tạp thời gian

$$O(n\sqrt{n}).$$

6 Bối cảnh ra đề và tình hình điểm số

Nguyên mẫu ban đầu của bài này là một bài toán như sau: cần bảo trì một dãy, hỗ trợ sửa đoạn thành x và truy vấn số loại giá trị khác nhau trong đoạn. Bài gốc tận dụng phương pháp bảo trì đoạn phần tử đã nhắc trong phần thuật toán online của bài viết này; sau đó với mỗi phần tử, trừu tượng hóa nó thành một điểm trên mặt phẳng hai chiều theo chỉ số của nó và chỉ số của phần tử trước đó có cùng giá trị, rồi chuyển truy vấn thành tổng trên hình chữ nhật để giải quyết. Trong một lần tình cờ, tôi nảy ra ý tưởng suy ra đáp án bằng cách tính thời điểm bị xóa của mỗi lần sửa, và đã thiết kế riêng bài này cho ý tưởng đó. Sau đó, tôi liên tục cải tiến thuật toán của mình, cuối cùng đưa bài này vào kỳ kiểm tra chéo của đội tuyển tập huấn để chia sẻ với mọi người.

Gần đây, khi các tuyển thủ OI nghiên cứu cấu trúc dữ liệu ngày càng sâu, nhiều người ra đề thường dùng mô hình phức tạp, cấu trúc dữ liệu và thuật toán cao cấp để tăng độ khó của bài, từ đó hình thành nhiều “mô típ” thường gặp. Điều này không chỉ khiến việc làm bài cấu trúc dữ liệu thường đòi hỏi nắm rất nhiều thuật toán khó và nhiều điểm kiến thức rời rạc, mà còn đòi hỏi năng lực viết mã rất mạnh, trái với mục tiêu ban đầu là rèn luyện tư duy của thí sinh.

So với vậy, lời giải chuẩn của bài này chỉ dùng tới chia khối và hai con trỏ, tức cách làm mà hai đầu mút liên tục đẩy sang phải. Trong một hiện thực của mã chuẩn cuối cùng, chỉ dùng mảng đơn giản nhất, không dùng bất kỳ cấu trúc dữ liệu tương đối cao cấp nào, kể cả cây đoạn, và độ dài mã chỉ 1.4 kB.

Trên cơ sở đó, bài này vẫn có độ khó tư duy và khả năng phân hóa nhất định. Khi ra đề, tôi dự đoán trong đội dự tuyển sẽ có 1–3 người vượt qua bài này, và khoảng một nửa số người có thể dùng nhiều cách làm khác nhau để lấy điểm cao. Trong kỳ kiểm tra chéo, không có ai trong đội dự tuyển vượt qua bài này, có 3 người đạt hơn 60 điểm, hơi ít hơn dự kiến. Điều đó cho thấy mọi người vẫn chưa đủ thành thạo với kiểu bài này. Ngoài ra, trong contest đồng bộ trên LOJ, đàn anh Ji Ruyi từ Đại học Bắc Kinh đã dùng tại chỗ cách làm $O(n\sqrt{n})$ để vượt qua bài này.

7 Tổng kết

Bài này kiểm tra năng lực phân tích tính chất bài toán của thí sinh và năng lực thiết kế thuật toán dựa trên tính chất. Cách thiết kế điểm từng phần đặc biệt có thể cho thí sinh nhận các mức điểm khác nhau tùy thuật toán và cách hiện thực. Để lấy điểm cao, không chỉ cần phân tích tính chất, mà còn cần hiểu và vận dụng đầy đủ thuật toán chia khối cùng cấu trúc dữ liệu cơ bản.

Hy vọng bài này có thể gợi mở cho mọi người và tăng cường khả năng vận dụng linh hoạt các cấu trúc dữ liệu cơ bản.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng giao lưu và học tập.

Cảm ơn huấn luyện viên Zhang Ruizhe đã chỉ đạo.

Cảm ơn thầy Zhou Cheng đã chỉ đạo và giúp đỡ tôi.

Cảm ơn đàn anh Huang Zhewei đã gợi mở và giúp đỡ tôi trên con đường thi tin học.

Cảm ơn mọi bạn học đã trao đổi, thảo luận bài này với tôi trước và sau kỳ thi; trong đó đặc biệt cảm ơn đàn anh Ji Ruyi đã chia sẻ cách làm của anh.

Cảm ơn bạn Hu Xuejun và bạn Chen Zhehuan đã đọc kiểm bài viết này.

Cảm ơn mọi người đã dành thời gian quý báu để đọc bài viết này.

Cảm ơn tất cả những người từng giúp đỡ tôi.

Tài liệu tham khảo

1. Xu Mingkuan, *Tìm hiểu ban đầu về thuật toán chia khối kích thước phi truyền thống*, tập luận văn đội tuyển quốc gia năm 2017.

14 Bài 12: Bàn sơ lược về một số mở rộng của matroid và ứng dụng

Yang Qianlan, Trường Trung học Hoài Âm, Giang Tô

Tóm tắt

Matroid là một khái niệm quen thuộc trong thi tin học nhưng lâu nay thường bị bỏ qua. Bài viết này giới thiệu một số phép mở rộng quan trọng của matroid, giúp matroid biểu diễn được rất nhiều bài toán tổ hợp mới và trở nên mạnh hơn trong việc giải các bài toán tối ưu tổ hợp. Hy vọng bài viết giúp bạn đọc hiểu sâu hơn về matroid và cảm nhận được vai trò quan trọng của matroid trong tối ưu tổ hợp.

1 Dẫn nhập

Matroid là một công cụ quan trọng để nghiên cứu các bài toán tối ưu tổ hợp. Khái niệm này được Hassler Whitney nghiên cứu đầu tiên và có vị trí quan trọng trong lý thuyết tối ưu tổ hợp.

Trong thi tin học, từ năm 2007 đã có luận văn giới thiệu matroid một cách hệ thống [1]. Đáng tiếc là trong nhiều năm, phần lớn thí sinh mới chỉ dừng ở nhận thức đơn giản rằng matroid có thể dùng để chứng minh tính đúng đắn của một lớp thuật toán tham lam.

Matroid là sự trừu tượng hóa khái niệm “độc lập”. Nó rút ra nhiều tính chất chung xuất hiện rộng rãi trong lý thuyết đồ thị và đại số tuyến tính, rồi tạo thành một khái niệm riêng không phụ thuộc vào các tính chất cụ thể của đồ thị hay đại số tuyến tính. Sau khi có thêm một số phép toán trên matroid, ta còn có thể quy nhiều khái niệm về matroid và dùng matroid để giải nhiều bài toán.

Bài viết bắt đầu từ các định nghĩa cơ bản của matroid, tập trung giới thiệu một số phép toán trên matroid, đồng thời đưa vào vài phương pháp nghiên cứu matroid. Hy vọng qua đó bạn đọc cảm nhận được tầm quan trọng và sức hấp dẫn của matroid.

Mục 2 giới thiệu các định nghĩa cơ sở của matroid. Mục 3 chứng minh tính đúng đắn của bài toán tối ưu trên matroid. Mục 4 và Mục 5 lần lượt đưa vào phép giao và phép hợp của matroid. Mục 7 bổ sung một số tính chất quan trọng của matroid và một vài cách nghiên cứu matroid.

2 Dẫn vào

2.1 Định nghĩa

Định nghĩa 2.1. Ký hiệu $M = (S, \mathcal{I})$ là một matroid định nghĩa trên tập hữu hạn S , trong đó tập các tập độc lập là $\mathcal{I}$. Ở đây $\mathcal{I} \subseteq 2^S$, tức $\mathcal{I}$ là một tập gồm một số tập con của S . Matroid M cần thỏa các tiên đề sau:

1. Nếu $I \in \mathcal{I}$ và $J \subseteq I$, thì $J \in \mathcal{I}$.
2. Nếu $I, J \in \mathcal{I}$ và $|J| > |I|$, thì tồn tại phần tử $z \in J \setminus I$ sao cho $I \cup \{z\} \in \mathcal{I}$.

Ở đây 2^S là tập lũy thừa của S , tức tập tất cả các tập con của S .

Định nghĩa 2.2. Nếu $I \in \mathcal{I}$, ta nói I là độc lập, hay I là một tập độc lập; ngược lại gọi là không độc lập. Thông thường, để thuận tiện, ta xem tập rỗng $\emptyset$ là độc lập.

Nói đơn giản, matroid là một cấu trúc toán học có một tập nền hữu hạn S ; một số tập con của S được xem là độc lập và chúng cùng tạo thành $\mathcal{I}$. Các tiên đề của matroid quy định điều kiện của tập độc lập, và là nền tảng của matroid. Tiên đề thứ nhất thường được gọi là tiên đề di truyền: mọi tập con của một tập độc lập đều độc lập. Tiên đề thứ hai thường được gọi là tiên đề trao đổi hoặc tiên đề mở rộng: nếu tập độc lập A có một tập độc lập B lớn hơn, thì tồn tại một phần tử chỉ thuộc B mà không thuộc A , có thể dùng để mở rộng A thành một tập độc lập lớn hơn.

Hai tiên đề này cũng là một cách định nghĩa matroid. Chúng là các tính chất chung được trừu tượng hóa từ nhiều bài toán. Một khi một bài toán tối ưu tổ hợp thỏa hai tiên đề này, mọi định lý về matroid đều đúng trên bài toán tổ hợp đó.

Để hiểu rõ hơn, ta xét vài ví dụ đơn giản về matroid và về cấu trúc không phải matroid.

Ví dụ 2.1 (matroid đều). Đặt matroid $U_n^k = (S, \mathcal{I})$, trong đó $|S| = n$ và

$$\mathcal{I} = \{I \subseteq S : |I| \leq k\}.$$

Ví dụ này rất dễ chứng minh hai tiên đề, nên bỏ qua.

Ví dụ 2.2 (matroid đồ thị). Cho đồ thị vô hướng $G = (V, E)$, đặt matroid đồ thị $M = (E, \mathcal{I})$, trong đó

$$\mathcal{I} = \{F \subseteq E : F \text{ không chứa chu trình}\}.$$

Đây là matroid đồ thị rất quan trọng trong lý thuyết matroid. Tập nền là tập cạnh, còn tập độc lập là một rừng sinh. Ta có thể chứng minh cấu trúc này là một matroid.

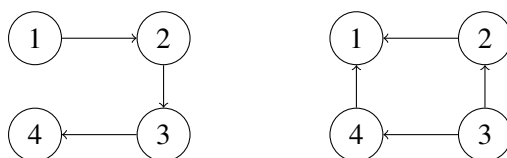
Trước hết, tính di truyền là hiển nhiên: nếu một tập cạnh F không có chu trình thì mọi tập con của F cũng không có chu trình.

Chứng minh tính trao đổi cũng không khó. Giả sử tồn tại hai tập con cạnh A, B thỏa $|A| < |B|$. Khi đó số thành phần liên thông trong đồ thị do A tạo ra chắc chắn nhiều hơn số thành phần liên thông trong đồ thị do B tạo ra. Vì thế chắc chắn tồn tại một thành phần liên thông của đồ thị do B tạo ra nhưng không liên thông trong đồ thị do A tạo ra. Suy ra có một cạnh trong B có thể thêm vào A mà không tạo chu trình.

Ví dụ 2.3 (“matroid” đồ thị có hướng). Cho đồ thị có hướng $G = (V, E)$, đặt $M = (E, \mathcal{I})$, trong đó

$$\mathcal{I} = \{F \subseteq E : F \text{ không chứa chu trình có hướng}\}.$$

Vì cấu trúc cây sinh của đồ thị vô hướng là matroid, ta tự nhiên đoán rằng trên đồ thị có hướng cũng như vậy. Đáng tiếc là ví dụ này sai: cấu trúc tổ hợp tạo bởi đồ thị có hướng không phải là matroid. Có thể chứng minh bằng một phản ví dụ đơn giản.



Chú ý rằng trong hình bên phải, bất kỳ cạnh nào khác với các cạnh trong hình bên trái nếu thêm vào hình bên trái đều sẽ tạo chu trình. Do đó trên cấu trúc này không có tính trao đổi.

Ví dụ 2.4 (matroid ghép cặp). Cho đồ thị vô hướng $G = (V, E)$, đặt matroid ghép cặp $M = (V, \mathcal{I})$, trong đó một tập đỉnh là độc lập khi và chỉ khi tồn tại một matching phủ được mọi đỉnh trong tập đó; matching được phép phủ thêm các đỉnh ngoài tập.

Trong ví dụ này, tính di truyền khá hiển nhiên, còn chứng minh tính trao đổi thì khó hơn. Một phương pháp truyền thống là dùng đường tăng, nhưng điều này không liên quan tới chủ đề chính của bài viết nên bỏ qua. Về sau ta sẽ chứng minh bài toán matching trên đồ thị hai phía là một matroid từ góc nhìn phép hợp matroid.

2.2 Cơ sở và mạch

Định nghĩa 2.3. Với một tập độc lập I trong matroid $M = (S, \mathcal{I})$, nếu thêm bất kỳ phần tử nào thuộc $S \setminus I$ vào I đều khiến nó trở thành không độc lập, thì gọi I là một *cơ sở* của matroid, hay một tập độc lập cực đại.

Định nghĩa 2.4. Với một tập không độc lập I' trong matroid $M = (S, \mathcal{I})$, nếu bỏ đi bất kỳ phần tử nào thuộc I' đều khiến nó trở thành độc lập, thì gọi I' là một *mạch* của matroid, hay một tập không độc lập cực tiểu.

Trong matroid đồ thị, một cơ sở là một rừng sinh cực đại, còn một mạch là một chu trình đơn. Đặc biệt, nếu đồ thị liên thông thì một cơ sở là một cây sinh.

Trong đại số tuyến tính, một cơ sở là một cơ sở của không gian tuyến tính, còn một mạch là một tập phụ thuộc tuyến tính cực tiểu.

Trước đó ta nói tập rỗng $\emptyset$ chắc chắn là độc lập, nên từ đây suy ra $\emptyset$ chắc chắn không phải là mạch.

Ta có một số kết luận đơn giản về cơ sở.

Bổ đề 2.1. Với mọi matroid M , mọi cơ sở đều có cùng kích thước.

Chứng minh. Nếu tồn tại hai cơ sở A, B với $|A| < |B|$, thì theo tính trao đổi, A vẫn có thể mở rộng, mâu thuẫn với tính cực đại. ■

Định lý 2.2 (định lý trao đổi cơ sở). Với mọi matroid M , nếu tồn tại hai cơ sở khác nhau A, B , thì với mọi $z \in A \setminus B$, đều tồn tại $y \in B \setminus A$ sao cho $A - \{z\} + \{y\}$ là một tập độc lập.

Chứng minh. Ta có $|A - \{z\}| < |B|$. Theo tiên đề trao đổi, chắc chắn tồn tại phần tử $y \in B \setminus (A - \{z\}) = B \setminus A$ thỏa mãn yêu cầu. ■

Ký hiệu $\mathcal{C}(M)$ là tập tất cả các mạch của matroid M . Các mạch có những tính chất sau.

Hệ quả 2.3. Nếu tồn tại hai mạch $X, Y \in \mathcal{C}(M)$ và $X \subseteq Y$, thì $X = Y$.

Chứng minh. Suy ra trực tiếp từ định nghĩa. ■

Hệ quả 2.4. Nếu tồn tại hai mạch $X, Y \in \mathcal{C}(M)$, phần tử $e \in X \cap Y$, và $X \neq Y$, thì tồn tại mạch $C \in \mathcal{C}(M)$ sao cho

$$C \subseteq X \cup Y - \{e\}.$$

Chứng minh. Từ Hệ quả 2.3, $X \setminus Y$ không rỗng. Lấy $f \in X \setminus Y$. Giả sử $X \cup Y - \{e\}$ là độc lập. Vì X là mạch, $X - \{f\}$ là độc lập. Gọi Z là tập độc lập lớn nhất trong $X \cup Y$ chứa $X - \{f\}$. Do Y không độc lập nên $Y \not\subseteq Z$. Suy ra

$$|Z| \leq |X \cup Y - \{f\}| - 1 \leq |X \cup Y| - 2 < |X \cup Y - \{e\}|.$$

Vì Z đã là tập độc lập lớn nhất, $X \cup Y - \{e\}$ không thể độc lập. Mệnh đề được chứng minh. ■

Hệ quả 2.5. Gọi I là một cơ sở của matroid $M = (S, \mathcal{I})$, và $e \notin I$. Khi đó $I + e$ chứa đúng một mạch.

Chứng minh. Giả sử $I + e$ có hai mạch khác nhau C_1, C_2 . Vì việc thêm e tạo ra mạch, ta có $e \in C_1 \cap C_2$. Theo Hệ quả 2.4, $C_1 \cup C_2 - \{e\}$ chứa một mạch, mâu thuẫn với việc I độc lập. ■

2.3 Hàm hạng

Định nghĩa 2.5. Với matroid $M = (S, \mathcal{I})$, số phần tử của một cơ sở được gọi là hạng của matroid. Với mỗi tập con U của S , định nghĩa hàm hạng $r(U)$ là kích thước của tập độc lập cực đại trong U .

Hàm hạng có các tính chất sau.

Định lý 2.6 (bị chặn).

$$\forall U \subseteq S, \quad 0 \leq r(U) \leq |U|.$$

Định lý 2.7 (đơn điệu).

$$\forall A \subseteq B \subseteq S, \quad r(A) \leq r(B).$$

Định lý 2.8 (dưới môđun).

$$\forall A, B \subseteq S, \quad r(A \cup B) + r(A \cap B) \leq r(A) + r(B).$$

Định lý 2.6 và 2.7 đều suy ra trực tiếp từ định nghĩa. Ta tập trung chứng minh Định lý 2.8.

Chứng minh. Gọi Z là tập độc lập lớn nhất trong $A \cap B$. Theo tính trao đổi, bắt đầu từ Z , ta có thể mở rộng thành tập độc lập lớn nhất Z_A trong A , tập độc lập lớn nhất Z_B trong B , và tập độc lập lớn nhất Z_{AB} trong $A \cup B$. Ta chia Z_{AB} thành ba phần lần lượt thuộc $A \cap B, A \setminus B, B \setminus A$. Theo tính di truyền, tổ hợp bất kỳ của các phần này đều là độc lập. Gọi phần của Z_{AB} trong $A \setminus B$ là E_A , và phần trong $B \setminus A$ là E_B . Khi đó

$$r(A \cap B) + r(A \cup B) = |E_A| + |E_B| + |Z| + |Z| = (|Z| + |E_A|) + (|Z| + |E_B|) \leq |Z_A| + |Z_B| = r(A) + r(B).$$

■

Ở trên ta đã đưa ra vài tính chất quan trọng của hàm hạng. Thực ra, thông qua các tính chất này, ta có thể trực tiếp định nghĩa một hàm duy nhất $r : 2^S \rightarrow \mathbb{Z}_{\geq 0}$. Nhờ hàm này, ta thậm chí có thể trực tiếp định nghĩa matroid:

$$M = (S, \mathcal{I}), \quad \mathcal{I} = \{I : r(I) = |I|\}.$$

Nếu chứng minh kết luận này đúng, ta có được một công cụ quan trọng để nghiên cứu matroid: đại số hóa. Ta chuyển bài toán tổ hợp trên matroid thành bài toán đại số trên hàm hạng, rồi nghiên cứu tính chất của hàm hạng để suy ra tính chất của matroid. Nếu một hàm do ta định nghĩa thỏa các điều kiện của hàm hạng, bài toán tổ hợp tương ứng sẽ thỏa các điều kiện matroid.

Dưới đây là quá trình suy ra các tiên đề matroid từ các tính chất của hàm hạng.

Chứng minh. Trước hết chứng minh tính di truyền. Gọi B là một tập độc lập bất kỳ, và A là một tập con bất kỳ của B . Theo tính dưới môđun,

$$|B| = r(B) \leq r(A) + r(B - A).$$

Theo tính bị chặn, $r(A) \leq |A|$ và $r(B - A) \leq |B - A|$. Do đó $r(A) = |A|$ và $r(B - A) = |B - A|$, tức A là độc lập.

Chứng minh tính trao đổi phức tạp hơn một chút. Gọi A, B là hai tập độc lập bất kỳ, thỏa $|A| < |B|$. Ta chứng minh bằng phản chứng. Giả sử thêm bất kỳ phần tử nào của B vào A đều không cho tập độc lập. Gọi các phần tử của B là $b_1, b_2, \dots, b_{|B|}$. Ta dùng quy nạp để suy ra mâu thuẫn. Vì thêm bất kỳ phần tử nào vào A đều không độc lập,

$$r(A \cup \{b_1\}) = |A|.$$

Giả sử đã có $r(A \cup \{b_1, b_2, \dots, b_n\}) = |A|$. Theo tính dưới môđun,

$$\begin{aligned} & r((A \cup \{b_1, \dots, b_n\}) \cap (A \cup \{b_{n+1}\})) \\ & + r((A \cup \{b_1, \dots, b_n\}) \cup (A \cup \{b_{n+1}\})) \\ & \leq r(A \cup \{b_1, \dots, b_n\}) + r(A \cup \{b_{n+1}\}). \end{aligned}$$

Vì giao ở dòng đầu là A , suy ra

$$r(A \cup \{b_1, b_2, \dots, b_{n+1}\}) = |A|.$$

Quy nạp lần lượt, ta được $r(A \cup B) = |A|$. Nhưng theo tính đơn điệu,

$$|B| = r(B) \leq r(A \cup B),$$

mâu thuẫn. ■

3 Tối ưu trên matroid

Bài toán tối ưu trên matroid được định nghĩa như sau. Cho matroid $M = (S, \mathcal{I})$ và hàm $\omega : S \rightarrow \mathbb{R}$. Với một tập con của S , định nghĩa trọng số

$$\omega(I) = \sum_{e \in I} \omega(e).$$

Cần tìm giá trị $\omega(I)$ lớn nhất sao cho $I \in \mathcal{I}$.

Đây là ứng dụng đơn giản nhất của matroid. Nếu một bài toán tối ưu tổ hợp có thể chứng minh là phù hợp với mô hình matroid, ta có thể giải bằng tham lam đơn giản. Cụ thể, thuật toán như sau.

1. Sắp xếp các phần tử theo ω theo thứ tự không tăng:

$$\omega(s_1) \geq \omega(s_2) \geq \dots \geq \omega(s_{|S|}).$$

2. Duy trì một tập I , ban đầu rỗng. Lần lượt xét từng s_i theo thứ tự trên; nếu $I \cup \{s_i\}$ là độc lập thì thêm s_i vào I .

Ta chứng minh thuật toán đúng. Chứng minh gồm hai bước: trước hết chứng minh I thu được cuối cùng là một cơ sở của matroid, sau đó chứng minh cơ sở này là nghiệm tối ưu.

Với bước đầu, đặt $U_i = \{s_1, s_2, \dots, s_i\}$. Ta chứng minh bằng quy nạp rằng tại mọi thời điểm i , luôn có $r(U_i) = |I|$. Với $i = 0$ mệnh đề hiển nhiên. Giả sử mệnh đề đúng với i , xét $i + 1$. Có hai trường hợp:

1. $r(U_{i+1}) = r(U_i)$. Khi đó hiển nhiên.
2. $r(U_{i+1}) > r(U_i)$. Nếu $I \cup \{s_{i+1}\}$ là độc lập thì xong. Ngược lại, xét một cơ sở I' hiện tại. Vì $|I'| > |I|$, theo tính trao đổi phải có một phần tử $z \in I' \setminus I$ có thể thêm vào I . Phần tử này không thể là s_{i+1} , nên chắc chắn thuộc U_i . Điều này nói rằng trước đó đã tồn tại một tập độc lập có kích thước vượt quá $r(U_i)$, mâu thuẫn.

Bước thứ hai chứng minh cơ sở này là nghiệm tối ưu. Giả sử lựa chọn đầu tiên mà nghiệm tối ưu thật sự khác với I là s_i , tức nghiệm tối ưu chọn s'_i . Vì $\omega(s'_i) \leq \omega(s_i)$, về sau chắc chắn tồn tại một lựa chọn nào đó thỏa $\omega(s_j) \leq \omega(s'_j)$. Nhưng theo định lý trao đổi cơ sở, $I - \{s_j\} + \{s'_j\}$ cũng nhất định độc lập, nên thuật toán tham lam chắc chắn sẽ chọn s'_j khi gặp nó.

Như vậy ta đã chứng minh bài toán tối ưu trên matroid có thể giải bằng tham lam đơn giản. Do đó, với một lớp bài toán có cấu trúc matroid, ta đều có thể chứng minh tính đúng đắn của tham lam, chẳng hạn cây khung nhỏ nhất hoặc matching hai phía. Với một số bài toán mới, nếu phân tích được sự tồn tại của tính trao đổi còn tính di truyền thường khá hiển nhiên, ta có thể giải bằng tham lam đơn giản, chẳng hạn rừng cactus sinh nhỏ nhất.

4 Một số định nghĩa bổ sung

Để thuận tiện đưa vào phép giao và phép hợp của matroid, trước hết ta cần bổ sung một số phép toán cơ bản trên matroid.

4.1 Matroid đối ngẫu

Định nghĩa 4.1. Với matroid $M = (S, \mathcal{I})$, định nghĩa matroid đối ngẫu của M là $M^* = (S, \mathcal{I}^*)$, trong đó

$$\mathcal{I}^* = \{I : \text{tồn tại một cơ sở } B \text{ của } M \text{ sao cho } B \subseteq S \setminus I\}.$$

Trong định nghĩa trên, ta chưa chứng minh mà đã khẳng định đối ngẫu của một matroid vẫn là matroid. Để nội dung sau nghiêm ngặt hơn, ta cần chứng minh điều này.

Ở đây ta thử dùng một cách chứng minh khác: chứng minh hàm hạng của matroid này hợp lệ. Gọi $r^*(U)$ là hàm hạng của M^* . Trước hết cần tính $r^*(U)$:

$$\begin{aligned} r^*(U) &= \max_{I \subseteq U, I \in \mathcal{I}^*} |I| \\ &= \max_{B \text{ là cơ sở của } M} |U \setminus B| \\ &= |U| - \min_{B \text{ là cơ sở của } M} |U \cap B| \\ &= |U| - r(S) + \max_{B \text{ là cơ sở của } M} |B \cap (S \setminus U)| \\ &= |U| - r(S) + r(S \setminus U). \end{aligned}$$

Bây giờ ta chứng minh hàm hạng này thỏa các tiên đề của hàm hạng:

1. Tính bị chặn: vì $r(S \setminus U) \leq r(S)$, nên $r^*(U) \leq |U|$.
2. Tính đơn điệu: gọi $T \subseteq U \subseteq S$. Khi đó

$$\begin{aligned} r^*(T) &= |T| - r(S) + r(S \setminus T) \\ &\leq |T| - r(S) + r(S \setminus U) + r(U \setminus T) \\ &\leq |T| - r(S) + r(S \setminus U) + (|U| - |T|) \\ &= |U| - r(S) + r(S \setminus U) = r^*(U). \end{aligned}$$

3. Tính dưới môđun: gọi $A, B \subseteq S$, $T = S \setminus A$ và $U = S \setminus B$. Theo tính dưới môđun của r ,

$$r(T \cup U) + r(T \cap U) = r(S \setminus (A \cap B)) + r(S \setminus (A \cup B)) \leq r(S \setminus A) + r(S \setminus B).$$

Mặt khác $|A \cup B| + |A \cap B| = |A| + |B|$. Vì các hạng tử còn lại đều giống nhau, suy ra

$$r^*(A \cup B) + r^*(A \cap B) \leq r^*(A) + r^*(B).$$

4.2 Xóa và co

Trong mục này ta định nghĩa hai phép toán trên matroid: xóa và co.

Định nghĩa 4.2. Với matroid $M = (S, \mathcal{I})$ và $Z \subseteq S$, định nghĩa matroid thu được bằng cách xóa tập con Z khỏi M là $(S \setminus Z, \mathcal{I}')$, trong đó

$$\mathcal{I}' = \{I : I \subseteq S \setminus Z, I \in \mathcal{I}\}.$$

Ký hiệu matroid này là $M \setminus Z$.

Định nghĩa 4.3. Với matroid $M = (S, \mathcal{I})$ và $Z \subseteq S$, định nghĩa matroid thu được bằng cách co tập con Z của M là $(M^* \setminus Z)^*$, ký hiệu M/Z .

Phép xóa khá dễ hiểu: nó tương đương với việc không xét tập con Z nữa, và hàm hạng có thể dùng trực tiếp hàm hạng của matroid ban đầu.

Phép co khó hiểu hơn. Ta thử suy ra hàm hạng của matroid mới để hỗ trợ trực giác:

$$\begin{aligned} r_{M/Z}(U) &= |U| - r_{M^* \setminus Z}(S \setminus Z) + r_{M^* \setminus Z}((S \setminus Z) \setminus U) \\ &= |U| - r_{M^*}(S \setminus Z) + r_{M^*}((S \setminus Z) \setminus U) \\ &= |U| - (|S \setminus Z| - r_M(S) + r_M(Z)) \\ &\quad + (|(S \setminus Z) \setminus U| - r_M(S) + r_M(Z \cup U)) \\ &= r_M(Z \cup U) - r_M(Z). \end{aligned}$$

Qua hàm hạng này, có thể hiểu bản chất của phép co là bắt buộc chọn một cơ sở trong Z . Điều kiện để một tập là độc lập trong M/Z là hợp của nó với một tập độc lập trong Z vẫn là một tập độc lập của M . Trên thực tế, tên gọi “co” tương ứng với thao tác co cạnh trong matroid đồ thị.

Áp dụng bất kỳ phép toán nào ở trên lên một matroid đều thu được một matroid; điều này có thể chứng minh bằng hàm hạng, ở đây không trình bày thêm.

4.3 Phần tử cực tiểu

Định nghĩa 4.4. Với matroid M , bất kỳ matroid M' nào thu được sau một dãy phép xóa và co được gọi là một phần tử cực tiểu của M .

Nhiều tính chất của matroid có thể biểu hiện qua các phần tử cực tiểu của nó. Nói không hoàn toàn nghiêm ngặt, phần tử cực tiểu của matroid có thể xem là một đặc trưng cục bộ của matroid ban đầu. Bằng cách nghiên cứu các phần tử cực tiểu, ta có thể suy ra tính chất của matroid tổng quát hơn; phần sau sẽ dùng công cụ này.

5 Giao của matroid

Trước mục này, số bài toán mà matroid có thể khắc họa còn ít, yêu cầu khá chặt, và số bài toán giải được cũng không nhiều. Vì vậy trong thi tin học, nó chưa được coi trọng; phần lớn thí sinh chỉ có ấn tượng rằng matroid dùng để chứng minh tính đúng đắn của một lớp tham lam. Từ mục này trở đi, ta đưa vào phép giao và phép hợp của matroid. Nhờ hai phép toán này, matroid có thể khắc họa nhiều bài toán tối ưu tổ hợp hơn, và ta cũng thấy nhiều bài toán tổ hợp có thể được giải bằng matroid.

Định nghĩa 5.1. Cho hai matroid định nghĩa trên cùng tập nền $M_1 = (S, \mathcal{I}_1)$, $M_2 = (S, \mathcal{I}_2)$. Định nghĩa giao của M_1 và M_2 là tất cả các tập I đồng thời độc lập trong cả hai matroid.

Bài toán giao matroid yêu cầu tìm tập độc lập lớn nhất trong số các tập độc lập đồng thời thuộc hai matroid. Nếu giao của hai matroid vẫn là một matroid thì ta có thể dùng tham lam đơn giản để giải. Đáng tiếc điều này không đúng; phản ví dụ xuất hiện rất phổ biến. Ta cần dùng công cụ khác để giải bài toán giao matroid. Định lý sau là chìa khóa.

Định lý 5.1 (định lý min–max).

$$\max_{I \in \mathcal{I}_1 \cap \mathcal{I}_2} |I| = \min_{U \subseteq S} (r_1(U) + r_2(S \setminus U)).$$

Định lý này sẽ là mấu chốt để giải bài toán giao matroid. Trước hết ta chứng minh vế đơn giản hơn, tức $\max \leq \min$. Với mọi I và U ,

$$|I| = |I \cap U| + |I \cap (S \setminus U)| \leq r_1(U) + r_2(S \setminus U).$$

Vì vậy, chỉ cần tìm được một cặp I, U thỏa đẳng thức này, ta sẽ giải xong bài toán giao matroid. Dưới đây ta đưa ra một thuật toán và dùng nó để chứng minh mang tính xây dựng cho định lý. Trước đó cần thêm một số chuẩn bị.

5.1 Định lý trao đổi cơ sở mạnh

Trước hết ta định nghĩa một tập hữu ích cho chứng minh.

Định nghĩa 5.2. Với matroid $M = (S, \mathcal{I})$ và $A \subseteq S$, định nghĩa toán tử bao đóng của A là

$$\text{cl}(A) = \{e \in S : r(A \cup \{e\}) = r(A)\}.$$

Toán tử bao đóng có các tính chất sau.

Bổ đề 5.2. Với matroid $M = (S, \mathcal{I})$, nếu $A \subseteq B$, thì $\text{cl}(A) \subseteq \text{cl}(B)$.

Chứng minh. Giả sử $e \in \text{cl}(A)$. Theo tính dưới môđun,

$$r(A \cup \{e\}) + r(B) \geq r((A \cup \{e\}) \cap B) + r(B \cup \{e\}) \geq r(A) + r(B \cup \{e\}).$$

Khử $r(A \cup \{e\}) = r(A)$ ở hai vế, ta được $r(B) \geq r(B \cup \{e\})$, tức $r(B) = r(B \cup \{e\})$, nên $e \in \text{cl}(B)$. Vì mọi phần tử của $\text{cl}(A)$ đều thuộc $\text{cl}(B)$, mệnh đề được chứng minh. ■

Bổ đề 5.3. Với matroid $M = (S, \mathcal{I})$ và $A \subseteq S$, nếu $e \in \text{cl}(A)$, thì

$$\text{cl}(A) = \text{cl}(A \cup \{e\}).$$

Chứng minh. Từ Bổ đề 5.2, $\text{cl}(A) \subseteq \text{cl}(A \cup \{e\})$. Ta chỉ cần chứng minh chiều ngược lại. Giả sử $f \in \text{cl}(A \cup \{e\})$. Dùng tính dưới môđun,

$$r(A \cup \{e\}) + r(A \cup \{f\}) \geq r(A \cup \{e, f\}) + r(A \cup (\{e\} \cap \{f\})) \geq r(A \cup \{e, f\}) + r(A).$$

Khử các phần bằng nhau, ta được

$$r(A \cup \{f\}) \geq r(A \cup \{e, f\}).$$

Vì $f \in \text{cl}(A \cup \{e\})$,

$$r(A \cup \{e, f\}) = r(A \cup \{e\}).$$

Suy ra $r(A \cup \{f\}) = r(A)$, tức $f \in \text{cl}(A)$. ■

Bổ đề 5.4. Với matroid $M = (S, \mathcal{I})$ và $A \subseteq S$,

$$\text{cl}(A) = \text{cl}(\text{cl}(A)).$$

Chứng minh. Dùng Bổ đề 5.3 và lần lượt thêm các phần tử của $\text{cl}(A) \setminus A$. ■

Định lý 5.5 (định lý trao đổi cơ sở mạnh). Với matroid M , giả sử tồn tại hai cơ sở khác nhau A, B . Khi đó với mọi phần tử $x \in A \setminus B$, đều tồn tại phần tử $y \in B \setminus A$ sao cho cả $A - \{x\} + \{y\}$ và $B - \{y\} + \{x\}$ đều là cơ sở của matroid.

Chứng minh. Vì B là cơ sở, $B + \{x\}$ chứa đúng một mạch C , và $x \in C$. Ta có

$$x \in \text{cl}(C - \{x\}).$$

Dùng Bổ đề 5.2, suy ra

$$x \in \text{cl}((A \cup C) - \{x\}).$$

Theo Bổ đề 5.3,

$$\text{cl}((A \cup C) - \{x\}) = \text{cl}(A \cup C).$$

Vì A là cơ sở, $S = \text{cl}(A) \subseteq \text{cl}(A \cup C)$. Do đó $(A \cup C) - \{x\}$ chứa một cơ sở, gọi là A' . Vì $A - \{x\}$ và A' đều độc lập, và $|A'| > |A - \{x\}|$, chắc chắn tồn tại

$$y \in A' \setminus (A - \{x\})$$

sao cho $A - \{x\} + \{y\}$ là cơ sở. Mặt khác

$$A' \setminus (A - \{x\}) \subseteq ((A \cup C) - \{x\}) \setminus (A - \{x\}) \subseteq C - \{x\}.$$

Vì vậy tồn tại $y \in C - \{x\}$ thỏa $A - \{x\} + \{y\}$ là cơ sở. Do C là mạch, $B - \{y\} + \{x\}$ cũng là cơ sở. ■

5.2 Thuật toán

Trong mục này ta đưa ra một thuật toán. Nó bắt đầu từ tập độc lập rỗng và dần mở rộng. Khi thuật toán dừng, ta thu được tập độc lập lớn nhất I . Đồng thời ta xây dựng được U , từ đó chứng minh đây chính là tập độc lập lớn nhất cần tìm. Toàn bộ thuật toán được định nghĩa trên một loại đồ thị hai phía gọi là đồ thị trao đổi.

Định nghĩa 5.3. Với matroid $M = (S, \mathcal{I})$ và một tập độc lập $I \in \mathcal{I}$, định nghĩa đồ thị trao đổi $D_M(I)$ của I trong M như sau. Hai phía của đồ thị lần lượt là I và $S \setminus I$. Có một cạnh nối $y \in I$ và $x \in S \setminus I$ khi và chỉ khi

$$I - \{y\} + \{x\} \in \mathcal{I}.$$

Bổ đề 5.6. Gọi I và J là hai tập độc lập của matroid M , với $|I| = |J|$. Khi đó trong $D_M(I)$ tồn tại một matching hoàn hảo giữa $I \setminus J$ và $J \setminus I$.

Chứng minh. Để tiện dùng định lý trao đổi cơ sở mạnh, xét matroid mới M' có các tập độc lập là

$$\{I' : I' \in \mathcal{I}, |I'| \leq |I|\}.$$

Khi đó I và J đều là cơ sở của matroid mới. Lấy tùy ý một phần tử $x \in J \setminus I$. Theo định lý trao đổi cơ sở mạnh, chắc chắn tồn tại $y \in I \setminus J$ sao cho $I - \{y\} + \{x\}$ và $J - \{x\} + \{y\}$ đều độc lập. Theo định nghĩa, cạnh (y, x) chắc chắn tồn tại. Ta chọn cạnh này vào matching và thay J bằng $J - \{x\} + \{y\}$. Đệ quy như vậy sẽ chứng minh được mệnh đề. ■

Định lý 5.7. Gọi I là một tập độc lập của matroid M , và J là một tập có cùng kích thước với I . Nếu trong đồ thị hai phía $D_M(I)$ tồn tại đúng một matching hoàn hảo từ $I \setminus J$ tới $J \setminus I$, thì J cũng độc lập.

Chứng minh. Gọi N là matching duy nhất. Định hướng các cạnh trong N từ $S \setminus I$ về I , và định hướng các cạnh còn lại từ I sang $S \setminus I$. Tính duy nhất của matching cho thấy đồ thị không có chu trình có hướng. Vì vậy đây là một đồ thị có thứ tự tôpô; ta có thể gán số thứ tự cho mỗi đỉnh sao cho mọi cạnh đều đi từ số nhỏ tới số lớn.

Đánh số lại các điểm trong N :

$$N = \{(y_1, x_1), (y_2, x_2), \dots, (y_t, x_t)\},$$

trong đó $x_i \in J \setminus I$. Không mất tính tổng quát, quy ước nếu $i < j$ thì không tồn tại cạnh (y_i, x_j) . Phản chứng rằng J không độc lập. Khi đó trong J chắc chắn tồn tại một mạch C . Lấy i nhỏ nhất sao cho $x_i \in C$. Do cách xây dựng ở trên, với mọi $x \in C - \{x_i\}$, cạnh (y_i, x) không tồn tại, tức

$$C - \{x_i\} \subseteq \text{cl}(I - \{y_i\}).$$

Vì vậy

$$\text{cl}(C - \{x_i\}) \subseteq \text{cl}(\text{cl}(I - \{y_i\})) = \text{cl}(I - \{y_i\}).$$

Do C là mạch, $x_i \in \text{cl}(C - \{x_i\})$. Suy ra $x_i \in \text{cl}(I - \{y_i\})$, mâu thuẫn với việc (y_i, x_i) là cạnh của đồ thị trao đổi. Vậy J chắc chắn độc lập. ■

Định nghĩa 5.4. Cho hai matroid M_1, M_2 và một tập $I \in \mathcal{I}_1 \cap \mathcal{I}_2$. Định nghĩa đồ thị trao đổi $D_{M_1, M_2}(I)$ của I như sau. Hai phía của đồ thị lần lượt là I và $S \setminus I$. Có một cạnh có hướng từ $y \in I$ tới $x \in S \setminus I$ khi và chỉ khi

$$I - \{y\} + \{x\} \in \mathcal{I}_1;$$

có một cạnh có hướng từ $x \in S \setminus I$ tới $y \in I$ khi và chỉ khi

$$I - \{y\} + \{x\} \in \mathcal{I}_2.$$

Mọi chuẩn bị đã xong, ta có thể mô tả thuật toán.

Ban đầu $I = \emptyset$. Định nghĩa

$$X_1 = \{x \notin I : I + \{x\} \in \mathcal{I}_1\}, \quad X_2 = \{x \notin I : I + \{x\} \in \mathcal{I}_2\}.$$

Mỗi lần, thuật toán tìm trong đồ thị trao đổi hiện tại D_{M_1, M_2} một đường đi từ X_1 tới X_2 sao cho không có cạnh nào có thể rút ngắn độ dài của đường đi này; gọi đường đi đó là P . Sau đó đặt $I \leftarrow I \Delta P$, trong đó Δ là hiệu đối xứng. Ta gọi quá trình này là một lần tăng cường. Lặp lại tăng cường cho tới khi không còn đường đi, ta thu được I lớn nhất và tìm được

$$U = \{z \in S : z \text{ có thể đi tới } X_2\}.$$

Nếu $X_1 \cap X_2$ không rỗng, thuật toán sẽ trực tiếp mở rộng bằng một phần tử thuộc giao này.

Một lần tăng cường có dạng xen kẽ: mọi cặp (y_i, x_i) biểu thị $I - \{y_i\} + \{x_i\} \in \mathcal{I}_1$, còn mọi cặp (x_i, y_{i+1}) biểu thị $I - \{y_{i+1}\} + \{x_i\} \in \mathcal{I}_2$.

Thuật toán có thể viết dưới dạng giả mã:

1. $I \leftarrow \emptyset$.
2. Lặp:

- (a) Dựng $D_{M_1, M_2}(I)$.
- (b) $X_1 \leftarrow \{z \in S \setminus I : I + \{z\} \in \mathcal{T}_1\}$.
- (c) $X_2 \leftarrow \{z \in S \setminus I : I + \{z\} \in \mathcal{T}_2\}$.
- (d) Tìm đường đi ngắn nhất P từ X_1 tới X_2 trong $D_{M_1, M_2}(I)$.
- (e) Nếu không có P , dừng.
- (f) $I \leftarrow I \Delta P$.

3. Trả về I .

Để chứng minh thuật toán đúng, cần chứng minh hai điểm. Một là sau khi thuật toán kết thúc, $|I| = r_1(U) + r_2(S \setminus U)$. Hai là tại mọi thời điểm trong thuật toán, $I \Delta P$ đều độc lập.

Trước hết chứng minh $|I| = r_1(U) + r_2(S \setminus U)$. Ta chứng minh

$$r_1(U) \leq |I \cap U|.$$

Nếu $r_1(U) > |I \cap U|$, thì tồn tại phần tử $x \in U \setminus I$ sao cho $(I \cap U) + \{x\} \in \mathcal{T}_1$. Nếu $I + \{x\}$ cũng là tập độc lập trong M_1 , thì $x \in X_1$, tức tồn tại đường đi từ X_1 tới X_2 . Vì vậy $I + \{x\}$ không độc lập trong M_1 . Kết hợp hai điều trên, tồn tại một phần tử $y \in I \setminus U$ sao cho $I - \{y\} + \{x\} \in \mathcal{T}_1$. Nhưng khi đó y cũng có thể đi tới X_2 , tức y cũng phải thuộc U , mâu thuẫn. Mệnh đề được chứng minh.

Chứng minh phía còn lại,

$$r_2(S \setminus U) = |I \cap (S \setminus U)|,$$

tương tự nên không lặp lại.

Cuối cùng xét vì sao ở mỗi bước $I \Delta P$ đều độc lập. Gọi đường đi ngắn nhất là

$$P = (x_0, y_1, x_1, y_2, \dots, x_t, y_t),$$

và đặt

$$J = \{x_1, x_2, \dots, x_t\} \cup (I \setminus \{y_1, y_2, \dots, y_t\}).$$

Khi đó $|J| = |I|$, và $I \setminus J$ với $J \setminus I$ có đúng một matching hoàn hảo. Theo Định lý 5.7, J là tập độc lập trong M_1 . Tiếp theo xét việc thêm x_0 . Vì P là đường đi ngắn nhất, $x_1, x_2, \dots, x_t$ đều không thuộc X_1 , nên

$$r_1(I \cup J) = r_1(I) = r_1(J).$$

Vì $I + \{x_0\}$ là độc lập, dùng lại cách chứng minh trong bài toán tối ưu matroid, $J + \{x_0\}$ cũng độc lập. Mệnh đề được chứng minh.

Ta đã chứng minh tính đúng đắn của thuật toán trên, đồng thời chứng minh định lý min–max và đưa ra một thuật toán giải giao matroid.

5.3 Độ phức tạp

Gọi

$$r = \max(r_1(S), r_2(S)).$$

Mỗi đồ thị được dựng có n đỉnh và rn cạnh. Tìm đường đi ngắn nhất trên đồ thị không trọng số có độ phức tạp $O(n + m)$, nên độ phức tạp của một lần tăng cường là $O(rn)$. Vì sau mỗi lần tăng cường, tập độc lập chắc chắn tăng thêm một phần tử, số lần tăng cường không vượt quá r . Do đó độ phức tạp của thuật toán giao matroid là $O(r^2n)$.

5.4 Giao matroid có trọng số

Bài toán giao matroid có trọng số tương tự bài toán tối ưu trên matroid. Cho hàm trọng số $\omega : S \rightarrow \mathbb{R}$, cần tìm

$$\max_{I \in \mathcal{I}_1 \cap \mathcal{I}_2} \sum_{e \in I} \omega(e).$$

Cách giải bài toán có trọng số tương tự bài toán thường, thông qua định lý min–max mở rộng:

$$\max_{I \in \mathcal{I}_1 \cap \mathcal{I}_2} \sum_{e \in I} \omega(e) = \min_{\omega_1, \omega_2: \forall x, \omega_1(x) + \omega_2(x) = \omega(x)} \left(\max_{I \in \mathcal{I}_1} \omega_1(I) \right) + \left(\max_{I \in \mathcal{I}_2} \omega_2(I) \right).$$

Trong thuật toán thực tế, ta chỉ cần định nghĩa trọng số đỉnh trên đồ thị trao đổi:

$$f(x) = \begin{cases} \omega(x), & x \in S \setminus I, \\ -\omega(x), & x \in I. \end{cases}$$

Quá trình tìm đường đi ngắn nhất được thay bằng tìm một đường từ X_1 tới X_2 , khóa chính là tổng trọng số đỉnh nhỏ nhất, khóa phụ là số cạnh đi qua ít nhất. Có thể chứng minh trên đồ thị trao đổi không tồn tại chu trình âm, nên thuật toán này luôn có nghiệm.

5.5 Giao của nhiều matroid

Tương tự giao của hai matroid, ta có thể định nghĩa giao của nhiều matroid, tức tìm tập lớn nhất đồng thời độc lập trong tất cả các matroid. Đáng tiếc là giao từ ba matroid trở lên là NP-hard. Có thể chứng minh bằng quy giảm từ bài toán đường Hamilton.

Xét đồ thị có hướng $G = (V, E)$, cần tìm đường Hamilton từ s tới t . Ta xây dựng ba matroid. Matroid thứ nhất $M_1 = (E, \mathcal{I}_1)$ yêu cầu sau khi xem các cạnh có hướng là cạnh vô hướng, toàn đồ thị không có chu trình; đây là matroid đồ thị kinh điển. Matroid thứ hai $M_2 = (E, \mathcal{I}_2)$ yêu cầu với mỗi đỉnh không phải s , bậc vào không vượt quá 1, còn với s , bậc vào không vượt quá 0. Đây là một matroid rất đơn giản và dễ chứng minh. Matroid thứ ba $M_3 = (E, \mathcal{I}_3)$ yêu cầu với mỗi đỉnh không phải t , bậc ra không vượt quá 1, còn với t , bậc ra không vượt quá 0. Nếu giao của ba matroid có tập I với $|I| = n - 1$, ta tìm được một đường Hamilton; nếu không thì đường Hamilton không tồn tại.

5.6 Ứng dụng

Sau khi có giao matroid, ta có thể dùng nó để giải nhiều mô hình tổ hợp. Vì phép giao tương ứng với việc đồng thời thỏa nhiều điều kiện logic, với nhiều bài toán tổ hợp, ta có thể tách các điều kiện thành từng nhóm sao cho nếu chỉ nhìn một nhóm riêng lẻ thì điều kiện đó thỏa các tiên đề matroid. Sau đó lấy giao các matroid này để giải bài toán gốc.

Ta xem vài ví dụ.

Ví dụ 5.1 (matching hai phía). Cho đồ thị hai phía $G = (V, E)$, trong đó $V = V_1 + V_2$ lần lượt là phía trái và phía phải. Cần tìm một matching lớn nhất.

Nếu xem tập đỉnh là tập nền, bài toán matching hai phía bản thân là một bài toán matroid. Tuy vậy, câu hỏi liệu sau khi thêm một đỉnh thì tập vẫn độc lập hay không không dễ kiểm tra trực tiếp; cách thông thường là dùng thuật toán Hungary. Nhưng nếu định nghĩa hai matroid như sau: $M_1 = (E, \mathcal{I}_1)$, trong đó

$$\mathcal{I}_1 = \{I : \text{mỗi đỉnh trong } V_1 \text{ có bậc không quá } 1\},$$

và matroid thứ hai tương tự với điều kiện mỗi đỉnh trong V_2 có bậc không quá 1, thì matching hai phía có thể xem là giao của hai matroid. Hơn nữa, việc kiểm tra độc lập trong hai matroid này đều có thể làm đơn giản trong $O(1)$.

Thực ra, phân tích kỹ sẽ thấy đường tăng mà thuật toán Hungary tìm được trùng với đường tăng mà giao matroid tìm được.

Ví dụ 5.2 (cây phân nhánh nhỏ nhất). Cho đồ thị có hướng có trọng số $G = (V, E)$, cần tìm cây phân nhánh nhỏ nhất của G gốc x .

Ta chỉ cần lấy hai matroid: matroid thứ nhất hạn chế rằng khi xem cạnh là vô hướng thì không có chu trình, matroid thứ hai hạn chế rằng ngoài đỉnh x , mỗi đỉnh có bậc vào không quá 1. Sau đó thực hiện giao matroid có trọng số.

Ví dụ 5.3 (Colourful tree). Cho đồ thị vô hướng có trọng số $G = (V, E)$, mỗi cạnh có một màu trong 1 tới $n - 1$. Cần tìm cây sinh có trọng số lớn nhất sao cho mỗi màu xuất hiện đúng một lần.

Đặt $M_1 = (E, \mathcal{I}_1)$, trong đó $\mathcal{I}_1 = \{I : I \text{ không chứa chu trình}\}$; đặt $M_2 = (E, \mathcal{I}_2)$, trong đó

$$\mathcal{I}_2 = \{I : \text{mỗi màu trong } I \text{ xuất hiện không quá một lần}\}.$$

Chỉ cần tìm giao của M_1 và M_2 .

Ví dụ 5.4 (Rainbow Graph). Cho đồ thị vô hướng có trọng số $G = (V, E)$, mỗi cạnh có một trong ba màu đỏ, lục, lam. Với mỗi k từ 1 tới $|E|$, cần tìm trọng số nhỏ nhất khi chọn k cạnh sao cho đồ thị không xét cạnh đỏ và đồ thị không xét cạnh lam đều liên thông; nếu không tồn tại thì xuất -1 .

$$|V|, |E| \leq 100.$$

Điều kiện chọn một số cạnh để đồ thị liên thông không phải là một điều kiện matroid hợp lệ, nên ta chuyển sang xét các cạnh bị xóa. Vì sau khi xóa cạnh mà đồ thị vẫn liên thông tương đương với điều kiện trong matroid đối ngẫu của matroid đồ thị, đây là một điều kiện matroid hợp lệ. Bài toán vì thế rất đơn giản: với mỗi k , chỉ cần xét phương án xóa $|E| - k$ cạnh sao cho cả hai đồ thị đều liên thông. Đây hiển nhiên là giao của hai matroid. Còn yêu cầu đáp án cho mọi k cũng tự nhiên, vì thuật toán giao matroid bắt đầu từ tập rỗng và mỗi bước mở rộng thêm một phần tử, nên ta có thể lấy được đáp án cho từng k .

6 Hợp của matroid

6.1 Định nghĩa

Định nghĩa 6.1. Với k matroid $M_i = (S_i, \mathcal{I}_i)$, $1 \leq i \leq k$, định nghĩa hợp của chúng là matroid $M = (S, \mathcal{I})$, trong đó

$$S = \bigcup_{i=1}^k S_i, \quad \mathcal{I} = \left\{ \bigcup_{i=1}^k I_i : I_i \in \mathcal{I}_i \right\}.$$

Trong định nghĩa trên, ta chưa chứng minh mà đã khẳng định hợp của một số matroid là matroid; phần sau sẽ chứng minh kết luận này. Trước hết xét hợp của matroid là khái niệm như thế nào. Nếu tập nền của k matroid đôi một không giao nhau thì định nghĩa khá hiển nhiên: tập nền mới là hợp của các tập nền cũ, và một tập là độc lập khi và chỉ khi phần của nó trong mỗi matroid đều độc lập. Trường hợp này cũng khá dễ chứng minh là matroid. Nhưng nếu các tập nền có giao nhau thì tình hình không hiển nhiên như vậy. Giải thích trực tiếp theo nghĩa tổ hợp, một tập là độc lập khi và chỉ khi tồn tại cách tách

nó thành k tập con, sao cho k phần này lần lượt được định nghĩa và độc lập trong các matroid tương ứng. Ta thử chứng minh nó là một matroid bằng cách suy ra hàm hạng.

Trước hết, chưa chứng minh, ta đưa ra định nghĩa của hàm hạng.

Định lý 6.1. Với hợp matroid M của k matroid M_i , hàm hạng của M là

$$r_M(U) = \min_{T \subseteq U} \left(|U \setminus T| + \sum_{i=1}^k r_i(T \cap S_i) \right).$$

Để chứng minh tính đúng đắn của hàm hạng này, trước hết cần chứng minh một bổ đề.

Bổ đề 6.2. Gọi $\hat{M} = (\hat{S}, \hat{\mathcal{T}})$ là một matroid bất kỳ, đồng thời là hợp của k matroid $\hat{M}_i = (\hat{S}_i, \hat{\mathcal{T}}_i)$ có tập nền đôi một không giao nhau. Với một hàm bất kỳ $f : \hat{S} \rightarrow S$, định nghĩa matroid $M = (S, \mathcal{T})$, trong đó

$$\mathcal{T} = \{f(\hat{I}) : \hat{I} \in \hat{\mathcal{T}}\}.$$

Ta có hàm hạng của M :

$$r_M(U) = \min_{T \subseteq U} (r_{\hat{M}}(f^{-1}(T)) + |U \setminus T|).$$

Ở đây $f(\hat{I}) = \{f(\hat{x}) : \hat{x} \in \hat{I}\}$, còn $f^{-1}(e)$ là tập tất cả phần tử x thỏa $f(x) = e$.

Chứng minh. Trước hết chứng minh M là matroid. Xét một tập độc lập bất kỳ I . Chắc chắn tồn tại $\hat{I} \in \hat{\mathcal{T}}$ sao cho $I = f(\hat{I})$ và $|I| = |\hat{I}|$. Vì mỗi tập con của I đều là ảnh qua f của một số tập con của $\hat{I}$, mọi tập con của I chắc chắn độc lập.

Xét một tập độc lập $J = f(\hat{J}) \in \mathcal{T}$. Tồn tại $\hat{J} \in \hat{\mathcal{T}}$ thỏa $|\hat{I}| < |\hat{J}| = |J|$. Điều này cho thấy tồn tại $e \in \hat{J} \setminus \hat{I}$ sao cho $\hat{I} + \{e\} \in \hat{\mathcal{T}}$. Giả sử $\hat{I}$ được chọn trong số tất cả các tập độc lập thỏa $f(\hat{I}) = I$, $|\hat{I}| = |\hat{J}|$, sao cho $|\hat{I} \cap \hat{J}|$ lớn nhất. Ta sẽ chứng minh $f(e) \notin I$, từ đó suy ra tính trao đổi. Nếu $f(e) \in I \cap J$, thì tồn tại $e' \in \hat{I} \setminus \hat{J}$ thỏa $f(e') = f(e)$. Khi đó

$$\hat{I}' = \hat{I} + \{e\} - \{e'\}$$

cũng là độc lập, và làm tăng $|\hat{I} \cap \hat{J}|$, mâu thuẫn với cách chọn.

Để chứng minh hàm hạng, với một tập con U của S , ta định nghĩa một matroid phân hoạch trên $\hat{S}$: $(\hat{S}, \mathcal{T}_p)$, trong đó

$$\mathcal{T}_p = \{I \subseteq \hat{S} : |f^{-1}(e) \cap I| \leq 1 \ \forall e \in U, \ |f^{-1}(e) \cap I| = 0 \ \forall e \notin U\}.$$

Hàm hạng của matroid này là

$$r_{M_p}(T) = |\{e \in U : f^{-1}(e) \cap T \neq \emptyset\}|.$$

Khi đó một tập con của U trong M là độc lập khi và chỉ khi tồn tại một tập con $\hat{I} \subseteq \hat{U} = f^{-1}(U)$ đồng thời độc lập trong M_p và $\hat{M}$. Vì vậy

$$\max_{I \subseteq U, I \in \mathcal{T}} |I| = \max_{\hat{I} \subseteq \hat{U}, \hat{I} \in \hat{\mathcal{T}} \cap \mathcal{T}_p} |\hat{I}|.$$

Theo định lý min-max, bổ đề được chứng minh. ■

Từ bổ đề tới định lý không khó. Với mỗi phần tử e trong S_i , biến nó thành cặp (i, e) làm phần tử của $\hat{S}_i$, và lấy ánh xạ $f((i, e)) = e$. Vì các $\hat{S}_i$ mới dựng đôi một không giao nhau, áp dụng Bổ đề 6.2 là chứng minh được Định lý 6.1.

6.2 Bài toán kiểm tra tập độc lập

Dù ta đã chứng minh hợp của các matroid vẫn là một matroid, việc kiểm tra một tập con có độc lập hay không không phải là chuyện dễ, ngay cả khi kiểm tra độc lập trong từng matroid riêng lẻ đều rất dễ.

Có hai cách thường dùng để kiểm tra một phần tử có tạo được tập độc lập hay không.

6.2.1 Giao matroid

Cách thứ nhất dùng thuật toán giao matroid. Xây dựng $\hat{S}_i = \{(e, i) : e \in S_i\}$, và xây dựng matroid $(\bigcup \hat{S}_i, \hat{\mathcal{T}})$, trong đó một tập con U là độc lập khi và chỉ khi với mỗi matroid M_i , tập

$$f(\{e : (e, i) \in U\})$$

là độc lập trong M_i . Xây dựng matroid thứ hai

$$M_p = (\hat{S}, \mathcal{T}_p),$$

trong đó

$$\mathcal{T}_p = \{I \subseteq \hat{S} : \forall e \in S, |I \cap \{(e, i) : e \in S_i\}| \leq 1\}.$$

Matroid thứ hai hạn chế rằng cùng một phần tử chỉ xuất hiện một lần, còn matroid thứ nhất hạn chế rằng trong mỗi matroid riêng lẻ đều độc lập. Chỉ cần xét tập con cần kiểm tra có phải là giao của hai matroid này hay không là có thể phán đoán tính độc lập.

6.2.2 Phân hoạch matroid

Ta xét cách giải trực tiếp hơn. Giả sử tồn tại một thuật toán phân hoạch nhận vào một tập độc lập I của

$$M_1 \cup M_2 \cup \dots \cup M_k,$$

một phân hoạch $I = I_1 \cup I_2 \cup \dots \cup I_k$ của I , trong đó $I_i \in \mathcal{T}_i$, và một phần tử $s \notin I$. Thuật toán quyết định liệu $I + \{s\}$ còn độc lập hay không, đồng thời đưa ra một phân hoạch mới. Khi đó ta có thể lần lượt thêm phần tử để kiểm tra một tập con có độc lập hay không.

Thuật toán này hơi giống thuật toán giao matroid. Ta định nghĩa đồ thị trao đổi $D_{M_i}(I_i)$ của tập độc lập I_i trên matroid M_i là một đồ thị hai phía có hướng: phía trái là I_i , phía phải là $S_i \setminus I_i$, và có cạnh có hướng từ $y \in I_i$ tới $x \in S_i \setminus I_i$ khi và chỉ khi

$$I_i - \{y\} + \{x\}$$

là độc lập trong M_i .

Định nghĩa đồ thị trao đổi hiện tại D là hợp của tất cả $D_{M_i}(I_i)$. Với mỗi matroid, đặt

$$F_i = \{x \in S_i \setminus I_i : I_i + \{x\} \in \mathcal{T}_i\},$$

và $F = \bigcup F_i$.

Dưới đây là định lý phán đoán liệu $I + \{s\}$ có độc lập hay không; chứng minh cũng đưa ra quy trình thuật toán.

Định lý 6.3. Với mọi phần tử $s \in S \setminus I$, $I + \{s\} \in \mathcal{T}$ khi và chỉ khi tồn tại một đường đi có hướng từ F tới s .

Chứng minh. Trước hết chứng minh tính đủ. Giả sử tồn tại một đường đi có hướng từ F tới s , gọi P là đường ngắn nhất trong số đó. Giả sử

$$P = \{s_0, s_1, \dots, s_p\}.$$

Không mất tính tổng quát, cho $s_0 \in F_1$. Với mỗi j từ 1 tới k , đặt

$$S_j = \{s_i, s_{i+1} : (s_i, s_{i+1}) \in D_{M_j}(I_j)\}.$$

Định nghĩa

$$I'_1 = (I_1 \triangle S_1) \cup \{s_0\}, \quad I'_j = I_j \triangle S_j \quad (j > 1).$$

Vì ngoài hai đầu mút, các điểm còn lại đều xuất hiện hai lần, phân hoạch mới chính là phân hoạch có thêm điểm s . Ta chỉ cần chứng minh mỗi I'_j vẫn độc lập. Tương tự chứng minh trong giao matroid, do ta chọn đường đi ngắn nhất, với mỗi $j > 1$, trong $D_{M_j}(I_j)$ tồn tại đúng một matching hoàn hảo giữa các điểm bị xóa và các điểm mới thêm, nên $I'_j \in \mathcal{I}_j$. Cũng tương tự giao matroid, vì $s_0 \in F_1$, ta chứng minh được I'_1 cũng độc lập.

Bây giờ chứng minh tính cần. Gọi T là tập tất cả các điểm có thể đi tới s . Phản chứng rằng không tồn tại đường đi, tức $F \cap T = \emptyset$. Trước hết cần chứng minh

$$|I_i \cap T| = r_i(S_i \cap T).$$

Giả sử tồn tại i không thỏa yêu cầu, tức

$$|I_i \cap T| < r_i(S_i \cap T).$$

Khi đó tồn tại $x \in T \cap (S_i \setminus I_i)$ sao cho $(I_i \cap T) + \{x\}$ là độc lập. Nhưng do $T \cap F = \emptyset$, nên $x \notin F$. Từ đó tất yếu tồn tại $y \in I_i \setminus T$ sao cho

$$I_i - \{y\} + \{x\}$$

là độc lập, suy ra $y \in T$, mâu thuẫn với cách chọn y . Vì vậy $|I_i \cap T| = r_i(T \cap S_i)$, tức I_i đã chứa tập độc lập lớn nhất trong T .

Chú ý $s \in T$, nên

$$(I + \{s\}) \cap T = \left(\bigcup I_i \cap T\right) + \{s\}.$$

Theo hàm hạng của hợp matroid,

$$\begin{aligned} r_M(I + \{s\}) &\leq |(I + \{s\}) \setminus T| + \sum_{i=1}^k r_i(T \cap S_i) \\ &\leq |(I + \{s\}) \setminus T| + \sum_{i=1}^k r_i(T \cap I_i) \\ &= |I \setminus T| + \sum_{i=1}^k |I_i \cap T| = |I|. \end{aligned}$$

Suy ra $I + \{s\}$ không độc lập. Mệnh đề được chứng minh. ■

6.3 Ứng dụng

Nếu đặc biệt hóa phép hợp matroid, đặt

$$M^k = M \cup M \cup \dots \cup M,$$

tức hợp của k bản sao của M , thì ta có thể kiểm tra một tập con có thể được chia thành không quá k tập độc lập hay không.

Xét hàm hạng của M^k :

$$r_{M^k}(U) = \min_{T \subseteq U} (|U \setminus T| + kr_M(T)).$$

Chú ý giá trị nhỏ nhất của biểu thức trên chỉ đạt tại các T thỏa $\text{cl}(T) = T$; nếu không, ta có thể mở rộng T để giảm đáp án.

Thông qua hàm hạng, ta có thể thu được vài kết luận đơn giản.

Định lý 6.4 (định lý phủ bằng cơ sở của matroid). Một matroid M có thể được phủ bởi không quá k cơ sở khi và chỉ khi

$$\forall T \subseteq S, \quad |T| \leq kr_M(T).$$

Chứng minh. Thay trực tiếp vào hàm hạng là được kết luận này. ■

Định lý 6.5 (định lý chứa cơ sở của matroid). Một matroid M có thể chứa ít nhất k cơ sở đôi một không giao nhau khi và chỉ khi

$$\forall T \subseteq S, \quad |S \setminus T| \geq k(r_M(S) - r_M(T)).$$

Chứng minh. Chỉ cần chứng minh hạng của matroid sau khi hợp ít nhất bằng k lần hạng của matroid ban đầu. ■

Thông qua hai định lý trên, ta có thể giải nhiều hệ quả. Ở đây chỉ nêu hai ví dụ.

Định lý 6.6. Một đồ thị vô hướng $G = (V, E)$ có thể được phủ bởi không quá k cây sinh khi và chỉ khi

$$\forall U \subseteq V, \quad |E(U)| \leq k(|U| - 1).$$

Chứng minh. Xét chiều thuận. Dùng trực tiếp Định lý 6.4 sẽ yêu cầu điều kiện với mọi tập cạnh. Chia tập cạnh theo tính liên thông thành nhiều thành phần liên thông; nếu không thỏa điều kiện, chắc chắn tồn tại một thành phần liên thông không thỏa, và ta chỉ cần xét thành phần này. Khi chỉ xét một thành phần liên thông, hạng của tập con chắc chắn là số đỉnh trừ 1, và trường hợp xấu nhất là thêm toàn bộ các cạnh không làm thay đổi hạng, tức $E(U)$. Vì vậy chỉ cần bảo đảm

$$|E(U)| \leq k(|U| - 1).$$

Chiều ngược lại chứng minh tương tự. ■

Định lý 6.7. Một đồ thị có hướng chứa ít nhất k cây sinh đôi một không giao nhau khi và chỉ khi với mọi phân hoạch tập đỉnh của đồ thị gốc $(V_1, V_2, \dots, V_p)$, đều có

$$\delta(V_1, V_2, \dots, V_p) \geq k(p - 1),$$

trong đó $\delta(V_1, V_2, \dots, V_p)$ là số cạnh băng qua phân hoạch.

Chứng minh. Chứng minh tương tự chứng minh trên. ■

Ví dụ 6.1 (matching hai phía). Cho đồ thị hai phía vô hướng $G(V, E)$, trong đó $V = V_1 + V_2$, lần lượt là phía trái và phía phải. Cần tìm matching lớn nhất.

Trước đó ta đã dùng mô hình giao matroid để giải thích matching hai phía. Ở đây ta thấy matching hai phía cũng có thể được giải thích bằng hợp matroid. Gọi số đỉnh phía trái là n , số đỉnh phía phải là m . Ta xây dựng m matroid, mỗi matroid có tập nền là V_1 . Điều kiện độc lập của matroid thứ i là tập chỉ có một đỉnh và đỉnh đó kề với đỉnh thứ i ở phía phải. Khi đó đáp án của matching hai phía chính là hạng của hợp matroid.

Quan sát kỹ sẽ thấy thuật toán Hungary giải matching hai phía cũng tương tự thuật toán hợp matroid.

Ví dụ 6.2 (Ông già năm môi và rừng rậm). Cho đồ thị có hướng G gồm n đỉnh và m cạnh. Hãy xác định có tồn tại một phương án chọn cạnh sao cho dù giữ các cạnh đã chọn hay giữ các cạnh không được chọn thì đồ thị đều không có chu trình hay không.

$$n \leq 2000, \quad m \leq 4000.$$

Chú ý điều kiện của bài tương đương với việc toàn đồ thị có thể được phủ bởi không quá hai cây sinh, nên ta chỉ cần kiểm tra với mọi tập đỉnh U , có

$$E(U) \leq 2|U| - 2$$

hay không. Đây là một bài toán mạng luồng kinh điển, có thể giải bằng đồ thị con đóng trọng số lớn nhất. Do bài này có thể có đáp án âm, cần bắt buộc chọn một đỉnh và dùng tối ưu đẩy ngược luồng để cải thiện độ phức tạp, nhưng đó không phải trọng tâm của bài viết nên không trình bày thêm.

7 Tính biểu diễn được của matroid

Matroid còn có rất nhiều nội dung quan trọng. Do trình độ tác giả có hạn, ở đây chỉ trình bày một nội dung.

7.1 Định nghĩa

Định nghĩa 7.1. Với matroid $M = (S, \mathcal{I})$, nói M biểu diễn được trên trường F khi và chỉ khi tồn tại một matroid tuyến tính định nghĩa trên trường F đẳng cấu với M .

Matroid biểu diễn được tương đương với việc tồn tại một ánh xạ

$$f : S \rightarrow F^k$$

sao cho một tập độc lập khi và chỉ khi các vector tương ứng với nó độc lập tuyến tính.

Ta xét một ví dụ đơn giản.

Ví dụ 7.1. Matroid đều U_4^2 không biểu diễn được trên trường $GF(2)$.

Ta cần dựng bốn vector trên $GF(2)$ sao cho từng cặp độc lập tuyến tính, còn bất kỳ ba vector nào cũng phụ thuộc tuyến tính. Vì hạng của matroid là 2, ma trận tạo bởi matroid dựng ra cũng phải có chiều 2. Nhưng trên $GF(2)$ chỉ có bốn vector hai chiều, và vector $(0, 0)$ chắc chắn phụ thuộc tuyến tính, nên không tồn tại bốn vector như vậy. Do đó matroid này không biểu diễn được.

7.2 Matroid chính quy

Định nghĩa 7.2. Một matroid biểu diễn được trên $GF(2)$ được gọi là matroid nhị phân. Một matroid biểu diễn được trên mọi trường được gọi là matroid chính quy.

Định lý 7.1. Matroid đồ thị là matroid chính quy.

Chứng minh. Với đồ thị $G = (V, E)$, xét xây dựng ma trận A kích thước $|E| \times |V|$, trong đó mỗi hàng biểu thị một cạnh, đồng thời biểu thị một vector. Giả sử cạnh thứ i là (u, v) , đặt

$$A_{i,u} = 1, \quad A_{i,v} = -1.$$

Sau khi xây dựng như vậy, nếu tồn tại một chu trình, ta có thể điều chỉnh dấu dương âm sao cho tại mỗi đỉnh đúng một giá trị là 1 và một giá trị là -1 , vì vậy chắc chắn phụ thuộc tuyến tính.

Cách dựng ở đây xét trong $\mathbb{Z}$, nhưng với một trường bất kỳ, ta chỉ cần lấy đơn vị nhân của trường và phần tử đối cộng của đơn vị nhân để lần lượt tương ứng với 1 và -1 . Trong trường có đặc số 2, 1 và -1 là cùng một phần tử, nhưng điều này không ảnh hưởng tới cách dựng.

Nghiên cứu tính biểu diễn được của matroid cung cấp một cách phân loại matroid. Matroid chính quy là một lớp matroid có tính chất rất mạnh; matroid đồ thị là một phần của matroid chính quy. Điều này cho thấy ngoài tính tổng quát của matroid, matroid đồ thị còn có tính đặc thù mạnh hơn.

Trong quá trình nghiên cứu, tác giả phát hiện rằng số cơ sở của mọi matroid chính quy đều có thể tính bằng định thức của một ma trận liên thuộc, và định lý ma trận cây là một trường hợp riêng. Tuy nhiên tác giả chưa tìm được tài liệu phù hợp; nếu bạn đọc có kết luận nào, rất hoan nghênh trao đổi với tác giả. ■

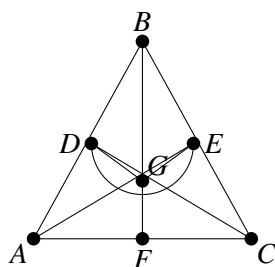
7.3 Một số kết luận

Phần tử cực tiểu là cấu trúc quan trọng biểu thị đặc trưng của một matroid. Trong tính biểu diễn được, nhiều matroid có biểu diễn được hay không có thể được phán đoán bằng phần tử cực tiểu. Trong quá trình nghiên cứu, tôi tìm thấy một số kết luận khá quan trọng.

Định lý 7.2. Một matroid là matroid nhị phân khi và chỉ khi không tồn tại phần tử cực tiểu là U_4^2 .

Định lý 7.3. Một matroid biểu diễn được trên $GF(3)$ khi và chỉ khi không tồn tại phần tử cực tiểu là U_5^2 , U_5^3 , F_7 và F_7^* . Ở đây F_7 là matroid Fano bậc ba có kích thước 7.

Matroid Fano là đồ thị dưới đây; các tập độc lập của nó là mọi tập 3 điểm không thẳng hàng.



8 Tổng kết

Matroid là một công cụ mạnh trong tối ưu tổ hợp. Nó trừu tượng hóa tính chất chung của một lớp bài toán tối ưu tổ hợp, và nghiên cứu nhiều tính chất suy ra từ khái niệm “độc lập”. Sau khi đưa vào giao matroid và hợp matroid, rất nhiều bài toán tối ưu tổ hợp kinh điển có thể được giải bằng matroid.

Lý thuyết matroid rất rộng và sâu, nhưng trình độ tác giả vô cùng hạn chế. Vì vậy hy vọng bài viết có thể đóng vai trò gợi mở, giúp nhiều bạn hơn cảm nhận được sức hấp dẫn của matroid, và hy vọng những độc giả có hứng thú sẽ tiếp tục nghiên cứu sâu hơn.

9 Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng giao lưu và học tập.

Cảm ơn huấn luyện viên đội tuyển quốc gia Zhang Ruizhe đã chỉ đạo.

Cảm ơn thầy Xue Zhijian và các đàn anh đã nhiều năm chỉ đạo và giúp đỡ tôi.

Cảm ơn bạn Wang Xiuhan đã gợi mở và giúp đỡ tôi về bài viết này.

Cảm ơn bạn Wang Xiuhan, bạn Zhong Ziqian, đàn anh Wu Hang và đàn anh Zhang Qingchuan đã đọc kiểm bài viết này.

Tài liệu tham khảo

1. Liu Yuchen, *Nghiên cứu bước đầu về matroid*, tập luận văn ứng viên đội tuyển quốc gia Trung Quốc IOI2007.
2. W. T. Tutte. A homotopy theorem for matroids, i, ii. Trans. Amer. Math Soc., 88:144–174, 1958.
3. J. G. Oxley. *Matroid Theory*. Oxford University Press, 1992.
4. H. Whitney. On the abstract properties of linear dependence. Amer. J. Math., 57:509–533, 1935.
5. P. Seymour. Matroid representation over $GF(3)$. J. Combin. Theory Ser. B, 26:159–173, 1979.
6. A. Schrijver. *Combinatorial Optimization: Polyhedra and Efficiency*, volume B. Springer, 2003.

15 Bài 13: Bàn sơ lược về tính chất của Splay và Treap cùng ứng dụng

Dong Weijun, Trường Trung học số 6 Quảng Châu

Tóm tắt

Splay và Treap là hai loại cây cân bằng thường dùng. Bài viết trước hết giới thiệu ngắn gọn các thao tác cơ sở trên hai loại cây cân bằng này, sau đó tập trung phân tích *finger search* trên Splay và Treap, trường hợp cây cân bằng có trọng số, và trình bày một số ứng dụng thực tế của chúng.

Dẫn nhập

Cây cân bằng đã được đưa vào giới OI gần hai mươi năm. Trong thời gian đó, nhiều loại cây cân bằng đã được ứng dụng rộng rãi. Trong số này, Splay và Treap đều có tính tổng dụng tốt và độ khó viết mã tương đối nhỏ, nên không ít thí sinh đã nắm được các thao tác cơ bản của chúng.

Thực ra Splay và Treap còn có một số tính chất đặc biệt, giúp chúng có chức năng mạnh hơn cây cân bằng thông thường. Chẳng hạn, gộp theo heuristic trên Splay và Treap đều có thể đạt một logarit về độ phức tạp thời gian; cả hai đều có thể hỗ trợ trường hợp có trọng số, v.v. Hiện nay trong giới OI chưa có nhiều tài liệu giới thiệu phần này. Một số thí sinh có hiểu biết nhất định, nhưng thường không biết cách chứng minh độ phức tạp thời gian. Vì vậy bài viết này sẽ giới thiệu chi tiết các tính chất đó của Splay và Treap, đồng thời đưa ra chứng minh cho phần lớn kết luận, với hy vọng bù đắp phần trống này và giới thiệu hai loại cây cân bằng xuất sắc này tới nhiều thí sinh hơn.

Hai mục đầu của bài viết trước hết giới thiệu các thao tác cơ sở của Splay và Treap, làm nền cho các phần sau. Sau đó, trong mục 3, bài viết giới thiệu cách hiện thực *finger search* trên Splay và Treap, đồng thời trình bày một số ứng dụng cụ thể. Cuối cùng, trong mục 4, bài viết thảo luận trường hợp Splay và Treap có trọng số, rồi xét ứng dụng của chúng vào một lớp bài toán nhiều tầng cây lồng nhau.

1 Splay

Splay, hay cây vườn, là một loại cây cân bằng thực dụng.

Trong cây tìm kiếm nhị phân thông thường, thời gian của thao tác liên quan tới độ sâu của nút. Nếu thực hiện rất nhiều thao tác trên một nút có độ sâu lớn, chi phí thời gian sẽ trở nên rất lớn. Một ý tưởng tự nhiên là sau mỗi lần thao tác trên một nút, ta xoay nó lên gốc, để lần truy cập nút đó sau này rất rẻ. Tuy nhiên, xoay trực tiếp nút đó từ dưới lên gốc không khả thi, vì rất dễ dựng dữ liệu làm cách đó mất cân bằng. Splay dùng một chiến lược xoay đặc biệt để bảo đảm độ phức tạp thời gian.

1.1 Thao tác splay

Khi truy cập một nút x trong Splay, ta cần thực hiện một thao tác splay trên x , đưa nó lên nút gốc. Trong quá trình splay có ba trường hợp:

1. Khi cha của x là gốc, chỉ cần xoay x lên một lần.
2. Khi x và cha của nó đều là con trái, hoặc đều là con phải, thực hiện thao tác Zig–Zig hoặc Zag–Zag: trước hết xoay cha của x lên, sau đó xoay x lên.
3. Khi trong hai nút x và cha của nó, một nút là con trái còn nút kia là con phải, thực hiện thao tác Zig–Zag hoặc Zag–Zig: trực tiếp xoay nút x lên hai lần.

Có thể thấy, sau một thao tác splay, độ sâu của các nút trên đường splay đều giảm khoảng một nửa. Điều này đem lại cho Splay tính chất tự điều chỉnh tốt. Sau trực giác đó, ta chứng minh chặt chẽ độ phức tạp thời gian của thao tác splay.

Định lý 1.1. Trong một Splay có n nút, thực hiện thao tác splay trên một nút có độ phức tạp thời gian khấu hao $O(\log n)$.

Chứng minh. Gọi $\text{size}(x)$ là kích thước cây con của nút x . Xét phân tích thế năng, định nghĩa thế năng của nút x là

$$r(x) = \log \text{size}(x),$$

và thế năng của toàn cây là tổng thế năng của mọi nút. Gọi y là cha của x , và z là cha của y , nếu tồn tại.

1. Khi cha của x là gốc, ta thực hiện một phép xoay lên. Độ tăng thế năng là

$$r'(x) + r'(y) - r(x) - r(y) = r'(x) - r(x).$$

2. Với trường hợp Zig–Zig hoặc Zag–Zag, độ tăng thế năng là

$$\begin{aligned} & r'(x) + r'(y) + r'(z) - r(x) - r(y) - r(z) \\ &= r'(y) + r'(z) - r(x) - r(y) \\ &\leq r'(x) + r'(z) - 2r(x) \\ &\leq 3(r'(x) - r(x)) - 2. \end{aligned}$$

Ở bước cuối, ta cộng vào biểu thức $2r'(x) - r(x) - r'(z) - 2$. Chú ý rằng

$$\text{size}'(x) = \text{size}(x) + \text{size}'(z) + 1,$$

nên $2r'(x) - r(x) - r'(z) \geq 2$.

3. Với trường hợp Zig–Zag hoặc Zag–Zig, có thể chứng minh tương tự rằng độ tăng thế năng không vượt quá $2(r'(x) - r(x)) - 2$.

Tổng hợp lại, sau một thao tác splay gồm k lần xoay, độ tăng thế năng không vượt quá $3(r'(x) - r(x)) - k + 1$. Thế năng ban đầu của cả cây là $O(n \log n)$, và thế năng luôn không âm tại mọi thời điểm, vì vậy thực hiện m thao tác splay trên một Splay có n nút tốn thời gian $O((n + m) \log n)$. Nói cách khác, thao tác splay có độ phức tạp thời gian khấu hao $O(\log n)$. ■

1.2 Các thao tác

Ta có thể dùng thao tác splay để hoàn thành chèn và xóa. Có nhiều cách hiện thực cụ thể; ở đây chỉ đưa ra một cách viết tham khảo. Khi chèn một phần tử, trước hết chèn nó vào một lá, sau đó splay nó lên gốc. Khi xóa một phần tử, trước hết splay nó lên gốc rồi xóa, sau đó dùng cách nối hai cây Splay sẽ giới thiệu dưới đây để ghép hai cây con trái và phải lại với nhau.

Splay cũng có thể dùng trong bài toán duy trì dãy. Trong loại bài này, mỗi cây Splay biểu diễn dãy tương ứng với thứ tự duyệt giữa của nó, và thường cần dùng hai thao tác sau:

- Nối hai cây Splay T_1, T_2 thành một cây Splay T . Giả sử dãy do T_1, T_2 biểu diễn lần lượt là $a_1, a_2, \dots, a_n$ và $b_1, b_2, \dots, b_m$, thì T phải biểu diễn dãy $a_1, a_2, \dots, a_n, b_1, b_2, \dots, b_m$.
- Tách một cây Splay T thành hai cây Splay T_1, T_2 . Giả sử T biểu diễn dãy $a_1, a_2, \dots, a_n$, thì T_1, T_2 lần lượt biểu diễn $a_1, a_2, \dots, a_k$ và $a_{k+1}, a_{k+2}, \dots, a_n$.

Hai thao tác này cũng có thể hoàn thành bằng splay. Khi nối hai cây Splay, trước hết splay phần tử cuối cùng của T_1 lên gốc, sau đó đặt T_2 làm cây con phải của nó. Khi tách, splay nút thứ k lên gốc, rồi cắt cạnh nối nó với con phải là thu được hai cây cần thiết.

Trong quá trình các thao tác này, ngoài thao tác splay, các bước khác chỉ gây ra lượng thay đổi thế năng $O(\log n)$, vì vậy các thao tác trên đều có độ phức tạp thời gian khấu hao $O(\log n)$.

Cần chú ý rằng, chỉ cần đã duyệt đường đi từ gốc của Splay tới một nút nào đó, ta bắt buộc phải thực hiện splay trên nút đó; nếu không, sẽ không thể bảo đảm độ phức tạp thời gian.

2 Treap

Treap là một cây tìm kiếm nhị phân ngẫu nhiên thường dùng. Đúng như tên gọi, Treap là tree cộng heap. Mỗi nút trong Treap được gán một độ ưu tiên ngẫu nhiên; cây vừa thỏa tính chất cây tìm kiếm nhị phân, vừa thỏa tính chất heap theo độ ưu tiên của các nút. Trong phần dưới, ta mặc định độ ưu tiên của các nút trong Treap thỏa tính chất heap lớn.

2.1 Độ sâu của nút

Sau khi đưa vào độ ưu tiên ngẫu nhiên, Treap tương đương với cây tìm kiếm nhị phân thu được bằng cách chèn theo thứ tự ngẫu nhiên, nên hình dạng của nó có xu hướng cân bằng. Dưới đây ta chứng minh chặt chẽ cận tiệm cận của độ sâu nút.

Định lý 2.1. Trong một Treap có n nút, độ sâu của nút là kỳ vọng $O(\log n)$.

Chứng minh. Xét khi nào mỗi nút i trở thành tổ tiên của nút x . Khi i là tổ tiên của x , độ ưu tiên của i lớn hơn độ ưu tiên của mọi nút nằm giữa i và x . Vì vậy độ sâu kỳ vọng của nút x là

$$1 + \sum_{i=1}^{x-1} \frac{1}{x-i} + \sum_{i=x+1}^n \frac{1}{i-x} = 1 + \sum_{i=1}^{x-1} \frac{1}{i} + \sum_{i=1}^{n-x} \frac{1}{i} = O(\log n).$$

Chú ý rằng trong chứng minh này và các chứng minh sau, ta mặc định độ ưu tiên của các nút đôi một khác nhau. ■

2.2 Chèn và xóa

Khi chèn một nút, ta có thể trước hết chèn nút đó vào một lá, rồi xoay nó lên một số lần để duy trì tính chất heap.

Xóa là thao tác ngược của chèn, nên có thể trực tiếp xoay nút cần xóa xuống vị trí lá, sau đó xóa nó.

Thời gian của chèn và xóa đều không vượt quá độ sâu của nút, tức kỳ vọng $O(\log n)$.

2.3 Nối và tách

Treap cũng có thể dùng trong bài toán duy trì dãy. Khi đó ta có thể cần nối hai cây Treap thành một cây Treap, hoặc tách một cây Treap thành hai cây Treap. Dưới đây là mã giả của thao tác nối và tách.

Thuật toán 1. Treap-Join.

```
function Join(T1, T2)
  if T1 = NULL then
    return T2
  end if
  if T2 = NULL then
    return T1
  end if
  if T1.priority > T2.priority then
    T1.rchild <- Join(T1.rchild, T2)
    return T1
  else
    T2.lchild <- Join(T1, T2.lchild)
    return T2
  end if
end function
```

Thuật toán 2. Treap-Split.

```
procedure Split(T, k, &T1, &T2)
  if T.maxkey <= k then
    [T1, T2] <- [T, NULL]
    return
  end if
  if T.minkey > k then
```

```

    [T1, T2] <- [NULL, T]
    return
end if
if T.key <= k then
    T1 <- T
    Split(T.rchild, k, T1.rchild, T2)
else
    T2 <- T
    Split(T.lchild, k, T1, T2.lchild)
end if
end procedure

```

Có thể thấy thời gian của thao tác nối/tách không vượt quá tổng độ sâu của hai cây. Giả sử kích thước của hai cây Treap trước khi nối hoặc sau khi tách lần lượt là n, m , thì thao tác này có độ phức tạp thời gian kỳ vọng $O(\log n + \log m)$.

Thực ra ta cũng có thể chỉ dùng thao tác xoay để hiện thực nối hoặc tách. Khi nối hai cây Treap, trước hết tạo một nút phụ, đặt hai cây Treap làm cây con trái và phải của nó, sau đó xoay nút phụ xuống lá rồi xóa nó. Khi tách hai cây Treap, trước hết chèn một nút phụ tại điểm tách, sau đó xoay nó lên gốc; khi đó cây con trái và phải của nó chính là hai cây Treap sau khi tách. Có thể thấy cách hiện thực này về bản chất giống cách trước, nên độ phức tạp thời gian cũng là kỳ vọng $O(\log n + \log m)$.

2.4 Cân bằng theo trọng lượng

Khi Treap chèn hoặc xóa một phần tử, hình dạng tổng thể của cây thay đổi rất ít: số lần xoay là kỳ vọng $O(1)$, và kích thước cây con bị xoay là kỳ vọng $O(\log n)$.

Đàn anh Chen Lijie đã giới thiệu chi tiết phần này trong luận văn đội tuyển tập huấn của anh ấy,²⁷ nên ở đây không nhắc lại.

2.5 Khả trì hóa

Độ phức tạp thời gian của Treap là kỳ vọng, nên nó có thể được khả trì hóa trực tiếp.

Cần chú ý rằng nếu quá trình khả trì hóa có liên quan tới việc sao chép một cây Treap, cây Treap được sao chép ra sẽ có độ ưu tiên ngẫu nhiên hoàn toàn giống cây Treap ban đầu; điều này có thể khiến các thao tác sau đó trên Treap không còn cân bằng.

Cách xử lý thường dùng hiện nay trong giới OI là không lưu độ ưu tiên ngẫu nhiên của nút, mà khi cần so sánh độ ưu tiên thì trả về kết quả ngẫu nhiên dựa trên kích thước cây con của nút. Ví dụ, khi nối hai cây Treap T_1, T_2 , xác suất độ ưu tiên của T_1 lớn hơn độ ưu tiên của T_2 là

$$\frac{\text{size}(T_1)}{\text{size}(T_1) + \text{size}(T_2)}.$$

Khi đó chỉ cần trả về kết quả so sánh độ ưu tiên theo đúng xác suất này.

3 Finger Search

Thông thường, độ phức tạp thời gian để tìm một phần tử trong cấu trúc dữ liệu được biểu diễn như một hàm theo n , tức số phần tử trong cấu trúc dữ liệu. Gọi $d(x, y)$ là chênh lệch hạng của hai phần tử x, y

²⁷Xem tài liệu tham khảo [5].

trong tập. Trong một số cấu trúc dữ liệu, nếu khi tìm phần tử y ta bắt đầu tìm từ nút x , thì độ phức tạp thời gian có thể được tối ưu thành hàm theo $d(x, y)$. *Finger search* chính là việc bắt đầu tìm các phần tử khác từ một số nút đặc biệt, tức các “finger”, trong cấu trúc dữ liệu. Nếu phần tử cần tìm có khoảng cách hạng rất gần với “finger”, hiệu quả truy vấn sẽ tăng đáng kể.

Ví dụ, trong một mảng có thứ tự, ta có thể dùng nhân đôi để hiện thực *finger search*, với độ phức tạp thời gian

$$O(\log(d(x, y) + 1)).$$

Ở đây và ở một số chỗ bên dưới, ta dùng $d(x, y) + 1$ để tránh trường hợp $\log 0$.

3.1 Finger Search trên Splay

Splay là một cấu trúc dữ liệu hỗ trợ *finger search* một cách tự nhiên. Ta có thể xem nút gốc là một “finger”, khi đó mọi thao tác trên Splay thực chất đều được hoàn thành bằng *finger search*.

Dưới đây là độ phức tạp thời gian của *finger search* trên Splay.

Định lý 3.1 (Dynamic Finger Theorem). Trên một Splay có n nút, thực hiện m thao tác chèn, xóa hoặc tìm kiếm có tổng thời gian

$$O\left(n + m + \sum_{i=1}^m \log(d_i + 1)\right).$$

Trong đó d_i là chênh lệch hạng, tại thời điểm thao tác, giữa phần tử của thao tác thứ i và phần tử của thao tác thứ $i - 1$. Phần tử của thao tác thứ 0 có thể xem là nút gốc ban đầu.

Chứng minh của định lý này khá phức tạp. Do trình độ có hạn, người viết hiện vẫn chưa đọc hiểu được chứng minh đó. Độc giả có hứng thú có thể tự tra cứu tài liệu tham khảo [1].

3.2 Finger Search trên Treap

Định lý 3.2. Độ dài đường đi giữa hai phần tử x, y trên Treap là kỳ vọng $O(\log(d(x, y) + 1))$.

Chứng minh. Trước hết giả sử $x < y$. Xét xác suất để mỗi nút i , với $i \neq x, y$, trở thành tổ tiên của x nhưng không là tổ tiên của y ; hiển nhiên khi đó $i < y$.

- Nếu $i < x$, thì độ ưu tiên của nút i chắc chắn lớn hơn các nút $i + 1, \dots, x$, đồng thời trong các nút $x + 1, \dots, y$ có ít nhất một nút có độ ưu tiên lớn hơn nút i .
- Nếu $x < i < y$, thì độ ưu tiên của nút i chắc chắn lớn hơn các nút $x, \dots, i - 1$, đồng thời trong các nút $i + 1, \dots, y$ có ít nhất một nút có độ ưu tiên lớn hơn nút i .

Vì vậy độ dài kỳ vọng của đường đi từ nút x tới tổ tiên chung gần nhất của x, y là

$$\begin{aligned} & O(1) + \sum_{i=1}^{x-1} \left(\frac{1}{x-i} - \frac{1}{y-i} \right) + \sum_{i=x+1}^{y-1} \left(\frac{1}{i-x} - \frac{1}{y-x} \right) \\ &= O(1) + \sum_{i=1}^{x-1} \frac{1}{i} - \left(\sum_{i=1}^{y-1} \frac{1}{i} - \sum_{i=1}^{y-x} \frac{1}{i} \right) + \sum_{i=1}^{y-x-1} \frac{1}{i} - \frac{y-x-1}{y-x} \\ &\leq O(1) + \sum_{i=1}^{y-x} \frac{1}{i} + \sum_{i=1}^{y-x-1} \frac{1}{i} \\ &= O(\log(y-x)). \end{aligned}$$

Độ dài kỳ vọng của đường đi từ nút y tới tổ tiên chung gần nhất của x, y được chứng minh tương tự. Do đó định lý được chứng minh. ■

Vì vậy, khi hiện thực *finger search* trên Treap, chỉ cần đi từ nút cho trước x lên tới tổ tiên chung gần nhất của x, y , rồi đi xuống tìm nút y . Trong quá trình này cần phán đoán nút hiện tại có phải tổ tiên chung gần nhất của x, y hay không; có thể duy trì khoảng mà mỗi nút biểu diễn giống cây đoạn, và nếu khoảng do nút hiện tại biểu diễn đã chứa y , ta không cần đi lên nữa.

3.2.1 Nối nhanh Treap. Mục 2 đã giới thiệu cách nối hoặc tách Treap trong thời gian kỳ vọng $O(\log n + \log m)$. Thực ra ta có thể dùng tư tưởng *finger search* để tối ưu thêm độ phức tạp thành kỳ vọng $O(\log \min(n, m))$.

Có thể thấy, khi nối hai cây Treap T_1, T_2 , thực chất ta đang gộp từ trên xuống chuỗi phải của T_1 và chuỗi trái của T_2 theo độ ưu tiên. Tuy nhiên trong trường hợp cực đoan, ví dụ T_1 là một Treap rất lớn còn T_2 chỉ có một nút, nút đó của T_2 nhiều khả năng sẽ được đặt gần tầng đáy, nhưng ta lại phải duyệt gần như toàn bộ chuỗi phải của T_1 ; điều này tạo ra nhiều chi phí dư thừa.

Ta chuyển sang gộp chuỗi phải của T_1 và chuỗi trái của T_2 từ dưới lên, và dừng quá trình khi đã gộp tới gốc của một trong hai cây. Giả sử ta gộp tới gốc của T_1 trước, thì quá trình này tương đương với việc duyệt đường đi trong cây mới từ phần tử cuối cùng của T_1 hoặc phần tử đầu tiên của T_2 tới gốc của T_1 . Vì vậy độ phức tạp khi nối hai cây Treap theo cách này là kỳ vọng $O(\log \min(n, m))$.

3.2.2 Tách nhanh Treap. Có thể thấy, quá trình tách một Treap thành T_1, T_2 thực chất là duyệt đường đi từ gốc tới nút y , phần tử cuối cùng của T_1 , rồi chia các nút trên đường đi vào T_1, T_2 . Khi một trong hai cây T_1, T_2 chứa rất nhiều phần tử còn cây kia chứa rất ít phần tử, một đoạn rất dài ở phía trên của đường đi này sẽ bị chia vào cây Treap lớn hơn. Vì vậy ta cần tránh duyệt đoạn đường này nhiều nhất có thể.

Giả sử số phần tử trong T_1 nhỏ hơn T_2 ; trường hợp còn lại tương tự. Xét bắt đầu từ nút đầu tiên x của T_1 và thực hiện một lần *finger search*, tìm tổ tiên chung gần nhất z của nó và nút y . Hiển nhiên các nút nằm ngoài cây con của z đều sẽ được chia vào T_2 . Khi đó chỉ cần bắt đầu từ nút z và thực hiện một lần tách thông thường. Toàn bộ quá trình chỉ duyệt đường đi từ nút x tới nút y , nên độ phức tạp thời gian được tối ưu thành kỳ vọng $O(\log \min(n, m))$.

Đôi khi trước khi tách Treap ta có thể chưa biết kích thước của T_1 và T_2 . Ta có thể đồng thời bắt đầu *finger search* từ nút đầu tiên và nút cuối cùng, cho tới khi một phía tìm được tổ tiên chung gần nhất với nút y . Độ phức tạp thời gian vẫn không đổi.

3.3 Ứng dụng

3.3.1 Gộp theo heuristic. Khi gộp cây cân bằng, người ta thường dùng chiến lược gộp theo heuristic, tức chèn thô từng phần tử của cây cân bằng nhỏ hơn vào cây cân bằng lớn hơn. Mỗi khi một phần tử được chèn vào một cây cân bằng khác, kích thước cây chứa nó tăng ít nhất gấp đôi, nên mỗi phần tử nhiều nhất chỉ bị chèn $O(\log n)$ lần. Độ phức tạp chèn một phần tử vào cây cân bằng thường là $O(\log n)$, vì vậy độ phức tạp thời gian của gộp heuristic bằng cây cân bằng thông thường là $O(n \log^2 n)$.

Xét dùng *finger search* để tối ưu gộp heuristic.

Với Splay, chỉ cần chèn các phần tử trong cây Splay nhỏ hơn vào cây Splay kia theo thứ tự tăng hoặc giảm.

Với Treap, cũng chèn các phần tử trong cây Treap nhỏ hơn vào cây Treap kia theo thứ tự tăng hoặc giảm. Mỗi lần chèn, bắt đầu một lần *finger search* từ vị trí vừa chèn trước đó, tìm vị trí mà phần tử mới cần được chèn. Khi hiện thực cụ thể, có thể hiện thực từ trên xuống, như vậy không cần ghi con trỏ cha của mỗi nút. Tài liệu [7] còn giới thiệu một cách gộp Treap khác; độc giả có hứng thú có thể tự tra cứu.

Giả sử kích thước của hai cây cân bằng cần gộp lần lượt là n, m , với $n \geq m$. Khi đó, ngoài lần chen đầu tiên tốn $O(\log n)$, tổng thời gian của các lần chen còn lại là

$$O\left(\sum_{i=2}^m \log(d_i + 1)\right),$$

và thỏa $\sum_{i=2}^m d_i \leq n$. Do log là hàm lõm, trường hợp xấu nhất là mọi d_i bằng nhau, tức $d_i = \frac{n}{m}$. Vì vậy độ phức tạp của một lần gộp là

$$O\left(m \log \frac{n}{m}\right).$$

Thực ra điều này cũng đạt cận dưới lý thuyết, vì khi gộp hai tập hợp cần phân biệt $\binom{n+m}{n}$ trường hợp, nên ít nhất cần $\log \binom{n+m}{n}$ phép so sánh.

Tiếp theo xét tổng độ phức tạp thời gian của các lần gộp.

Định lý 3.3. Giả sử dùng *finger search* để gộp một số cây Splay hoặc Treap, theo thứ tự tùy ý, thành một cây chứa n phần tử. Tổng thời gian gộp là $O(n \log n)$.

Chứng minh. Xét riêng đóng góp của mỗi phần tử. Giả sử trong quá trình gộp, kích thước tập hợp chứa nó lần lượt là $s_1, s_2, s_3, \dots, s_k$, thì đóng góp của nó là

$$O\left(\sum_{i=2}^k \log \frac{s_i}{s_{i-1}}\right) = O(\log n).$$

Theo Định lý 3.1, độ phức tạp gộp bằng Splay còn có một hạng n khởi tạo. Tuy nhiên ta có thể xem mọi thao tác chen trong mỗi Splay là một quá trình liên tục, nên hạng này không ảnh hưởng tới độ phức tạp cuối cùng. ■

Ví dụ 1. Tree. Cho một cây có n nút và độ dài của mỗi cạnh. Hỏi có bao nhiêu cặp điểm có khoảng cách không vượt quá k .

$$n \leq 10000,$$

trọng số cạnh là số nguyên dương không vượt quá 1000.

Một lời giải kinh điển của bài này là phân trị trọng tâm, nhưng không liên quan tới chủ đề bài viết nên không nhắc lại.

Xét chọn tùy ý một nút làm gốc để bắt đầu DFS, và với mỗi nút duy trì một cây cân bằng biểu diễn khoảng cách từ nó tới mỗi nút trong cây con. Khi đó trước mỗi lần gộp hai cây cân bằng, cần tính giữa chúng có bao nhiêu cặp phần tử có tổng nhỏ hơn k ; việc này cũng có thể hoàn thành bằng *finger search*. Độ phức tạp thời gian là $O(n \log n)$.

3.3.2 Duy trì dãy. Trong các bài toán duy trì dãy thông thường, dùng nối nhanh và tách nhanh của Treap không mang lại hiệu quả tối ưu rõ rệt. Tuy nhiên trong một số trường hợp đặc biệt, nó có thể tối ưu độ phức tạp của bài toán.

Ví dụ 2. tree. Thông qua tương tác, yêu cầu dùng cây nhị phân để duy trì một số dãy.

Ban đầu có n dãy, mỗi dãy chỉ có một phần tử. Sau đó xảy ra m sự kiện, gồm ba loại:

- `join`: ghép hai dãy thành một dãy.
- `split`: tách một dãy thành hai dãy.

- *visit*: hỏi tổng thông tin của một dãy.

Thư viện tương tác cung cấp cho mỗi cây hai con trỏ, lần lượt trỏ tới một nút nào đó trên cây. Ta cần thao tác các con trỏ để duy trì những sự kiện này. Được phép sửa một con trỏ để nó trỏ tới con của nút hiện tại, cập nhật tổng thông tin cây con của nút mà con trỏ trỏ tới, và di chuyển một con trỏ tới nút kế. Thư viện tương tác đóng gói ba thao tác này trong hàm *move*, đồng thời giới hạn tổng số lần gọi hàm này không vượt quá $2 \cdot 10^6$; trong mỗi sự kiện, số lần gọi hàm này không được vượt quá *maxcnt*. Ngoài ra, độ sâu lớn nhất của cây nhị phân được truy vấn không được vượt quá *maxdep*.

$$1 \leq n, m \leq 2 \cdot 10^5, \quad \text{maxdep} \geq 60, \quad \text{maxcnt} \geq 250, \quad k \leq 20000,$$

trong đó k là tổng số sự kiện *split* và *visit*.

Xét dùng Treap để duy trì các dãy này. Chú ý rằng số sự kiện *join* lớn hơn rất nhiều so với các sự kiện khác, nên có thể dùng nổi nhanh Treap đã giới thiệu ở trên để tối ưu hiệu quả của phần này. Tuy nhiên để giữ nguyên độ phức tạp, sau khi gộp không thể đi ngược lên để cập nhật tổng thông tin cây con của các nút nữa. Cách giải quyết cũng rất đơn giản: hiển nhiên các nút chưa được cập nhật trong mỗi cây nhất định đều nằm trên một đoạn ở đỉnh chuỗi trái hoặc chuỗi phải, nên ta trực tiếp trì hoãn việc cập nhật tới lần *split* hoặc *visit* tiếp theo.

Ta phân tích độ phức tạp của toàn bộ cách làm. Định nghĩa hàm thế năng của một dãy độ dài l là $\log l$. Khi ghép hai dãy độ dài lần lượt là l_1, l_2 , độ tăng thế năng là

$$\begin{aligned} & \log(l_1 + l_2) - \log l_1 - \log l_2 \\ &= \log(l_1 + l_2) - \log \max(l_1, l_2) - \log \min(l_1, l_2) \\ &\leq 1 - \log \min(l_1, l_2). \end{aligned}$$

Ban đầu tổng thế năng bằng 0, và cuối cùng tổng thế năng không âm, nên nếu chỉ có sự kiện *join*, độ phức tạp là $O(n)$. Một sự kiện *split* chỉ tăng thêm $O(\log n)$ thế năng. Vì vậy độ phức tạp thời gian của thuật toán là

$$O(n + m + k \log n).$$

Tương tự, nếu bài này đổi thành chỉ có sự kiện *split*, ta cũng có thể dùng tách nhanh Treap để đạt độ phức tạp tuyến tính.

Ngoài ra, khi không giới hạn *maxdep* và *maxcnt*, bài này cũng có thể hoàn thành bằng Splay. Gọi kích thước của hai cây Splay là a, b . Nếu dùng trực tiếp hàm thế năng thông thường của Splay, phân tích sẽ cho độ phức tạp của thao tác nổi là $O(\log(a + b))$. Xét sửa đổi nhẹ: định nghĩa hàm thế năng của Splay là tổng $\log \text{size}(x)$ trên mọi nút x ngoài gốc. Khi nối hai cây Splay, trước hết xoay đầu mút của cây Splay nhỏ hơn lên gốc; bước này làm thế năng tăng $O(\log \min(a, b))$. Sau đó nối cây Splay kia xuống dưới nút này, độ tăng thế năng cũng là $O(\log \min(a, b))$. Như vậy ta chứng minh được dùng Splay cũng có độ phức tạp thời gian $O(n + m + k \log n)$.

4 Cây tìm kiếm nhị phân có trọng số

Trong một số tình huống, ta gán cho mỗi nút của cây tìm kiếm nhị phân một trọng số w_i , cũng có thể xem là tần suất truy cập. Khi đó cần giảm chi phí truy cập các nút có trọng số lớn càng nhiều càng tốt.

4.1 Trường hợp tĩnh

Trước hết xét trường hợp tĩnh. Gọi W là tổng trọng số của mọi phần tử. Khi dựng cây, ta tìm k nhỏ nhất sao cho

$$\sum_{i=1}^k w_i > \frac{W}{2},$$

đặt nút k làm gốc, rồi đệ quy dựng cây ở hai phía.

Định lý 4.1. Trong cây do thuật toán trên dựng ra, độ sâu của nút x là

$$O\left(\log \frac{W}{w_x}\right).$$

Chứng minh. Hiển nhiên khi đi từ một nút xuống một con bất kỳ, tổng trọng số trong cây con giảm ít nhất một nửa. Vì vậy từ gốc đi xuống nhiều nhất

$$\left\lceil \log \frac{W}{w_x} \right\rceil$$

bước là có thể tới nút x . ■

Như vậy độ phức tạp thời gian dựng cây là $O(n \log W)$.

4.2 Splay có trọng số

Tính tự điều chỉnh xuất sắc của Splay giúp nó trực tiếp hỗ trợ trường hợp có trọng số, thậm chí ta không cần biết trọng số tương ứng với mỗi nút. Trong Splay có trọng số, độ phức tạp thời gian của thao tác splay trên một nút cũng liên quan tới trọng số của nó.

Định lý 4.2. Trong một Splay có tổng trọng số W , thực hiện thao tác splay trên nút x có độ phức tạp thời gian khẩu hao

$$O\left(\log \frac{W}{w_x}\right).$$

Chứng minh. Định nghĩa lại $\text{size}(x)$ là tổng trọng số của mọi nút trong cây con của nút x , rồi áp dụng chứng minh của Định lý 1.1. ■

4.3 Treap có trọng số

Ta có thể sửa đổi Treap để nó áp dụng được cho trường hợp có trọng số. Ta muốn nút có trọng số lớn nằm gần gốc hơn, nên cần gán cho nút có trọng số lớn một độ ưu tiên ngẫu nhiên lớn hơn.

4.3.1 Phân phối độ ưu tiên ngẫu nhiên cho nút. Với một nút có trọng số w , ta có thể sinh ngẫu nhiên w số, rồi lấy giá trị lớn nhất trong w số đó làm độ ưu tiên của nút. Trọng số của nút có thể thường xuyên thay đổi, nên trực tiếp tính độ ưu tiên trong $O(w)$ hiển nhiên là không chấp nhận được; ta cần tìm một cách tính nhanh hơn.

Giả sử số ngẫu nhiên được sinh đều trong $[0, 1]$. Xét tìm một hàm f . Mỗi khi cần tính độ ưu tiên, ta chọn đều một số ngẫu nhiên x trong $[0, 1]$, rồi lấy $f(x)$ làm độ ưu tiên của nút. Trong cách ban đầu, xác suất để giá trị lớn nhất của w số ngẫu nhiên nhỏ hơn k là k^w . Giả sử $f(x)$ đơn điệu tăng, thì k^w đồng thời là xác suất của

$$x < f^{-1}(k),$$

trong đó f^{-1} là hàm ngược của f . Vì vậy ta có thể chọn

$$f(x) = x^{1/w}.$$

Do độ ưu tiên chỉ cần so sánh lớn nhỏ, ta cũng có thể trực tiếp ghi

$$\frac{\log x}{w}.$$

4.3.2 Độ sâu của nút. Độ sâu của nút trong Treap cũng có tính chất tương tự.

Định lý 4.3. Trong một Treap có tổng trọng số W , độ sâu kỳ vọng của nút x là

$$O\left(\log \frac{W}{w_x}\right).$$

Chứng minh. Xét xác suất để mỗi nút i , với $i \neq x$, trở thành tổ tiên của nút x . Ký hiệu

$$\text{sum}(l, r) = \sum_{i=l}^r w_i.$$

Khi đó độ sâu kỳ vọng của nút x là

$$\begin{aligned} 1 + \sum_{i=1}^{x-1} \frac{w_i}{\text{sum}(i, x)} + \sum_{i=x+1}^n \frac{w_i}{\text{sum}(x, i)} &\leq 1 + \sum_{i=1}^{x-1} \sum_{j=1}^{w_i} \frac{1}{\text{sum}(i+1, x) + j} \\ &\quad + \sum_{i=x+1}^n \sum_{j=1}^{w_i} \frac{1}{\text{sum}(x, i-1) + j} \\ &= 1 + \sum_{i=w_x+1}^{\text{sum}(1, x)} \frac{1}{i} + \sum_{i=w_x+1}^{\text{sum}(x, n)} \frac{1}{i} \\ &= O\left(\log \frac{W}{w_x}\right). \end{aligned}$$

Các thao tác của Treap đều có thể biểu diễn độ phức tạp thời gian bằng độ sâu của một số nút, nên ở đây không phân tích lần lượt nữa. ■

4.3.3 Sửa trọng số. Khi trọng số của một nút bị sửa, ta có thể phân phối lại cho nó một độ ưu tiên ngẫu nhiên. Khi đó độ ưu tiên có thể không thỏa tính chất heap, nên cần thực hiện một số thao tác xoay lên hoặc xoay xuống trên nút này. Có thể thấy số lần xoay thực chất là chênh lệch độ sâu của nút đó trước và sau khi sửa. Tuy nhiên trước đó ta tính cận trên của độ sâu nút, không thể trực tiếp từ đó suy ra cận trên của chênh lệch độ sâu. Vì vậy cần tính riêng độ phức tạp thời gian của phần này.

Định lý 4.4. Nếu trọng số của nút x được sửa từ w_x thành w'_x , thì chênh lệch độ sâu của nút này trước và sau khi sửa là

$$O\left(\log \frac{\max(w_x, w'_x)}{\min(w_x, w'_x)}\right).$$

Chứng minh. Đặt $\Delta = w'_x - w_x$. Ở đây chỉ chứng minh trường hợp $\Delta > 0$; trường hợp $\Delta < 0$ tương tự. Ký hiệu

$$\text{sum}(l, r) = \sum_{i=l}^r w_i.$$

Khi đó kỳ vọng của chênh lệch độ sâu là

$$\sum_{i=1}^{x-1} \left(\frac{w_i}{\text{sum}(i, x)} - \frac{w_i}{\text{sum}(i, x) + \Delta} \right) + \sum_{i=x+1}^n \left(\frac{w_i}{\text{sum}(x, i)} - \frac{w_i}{\text{sum}(x, i) + \Delta} \right).$$

Xét cận trên của nửa trái:

$$\begin{aligned}
& \sum_{i=1}^{x-1} \left(\frac{w_i}{\text{sum}(i, x)} - \frac{w_i}{\text{sum}(i, x) + \Delta} \right) \\
& \leq \sum_{i=1}^{x-1} \sum_{j=1}^{w_i} \left(\frac{1}{\text{sum}(i+1, x) + j} - \frac{1}{\text{sum}(i+1, x) + \Delta + j} \right) \\
& = \sum_{i=w_x+1}^{\text{sum}(1, x)} \frac{1}{i} - \sum_{i=w_x+\Delta+1}^{\text{sum}(1, x)+\Delta} \frac{1}{i} \\
& = O\left(\log \frac{w'_x}{w_x}\right).
\end{aligned}$$

Tương tự, nửa phải cũng là $O(\log(w'_x/w_x))$. Định lý được chứng minh. ■

4.4 Một lớp bài toán nhiều tầng cây lồng nhau

Trong các bài toán phân rã cây theo chuỗi nặng thường gặp, ta thường cần dùng cây đoạn hoặc cấu trúc dữ liệu khác để duy trì thông tin của mỗi chuỗi nặng. Nếu xem cây đoạn của mỗi chuỗi nặng là được lồng dưới một nút cây đoạn tương ứng với cha của đỉnh đầu chuỗi nặng đó, thì phân rã cây theo chuỗi nặng cộng cây đoạn có thể xem là một cấu trúc cây lồng cây lồng cây, v.v., gồm $O(\log n)$ tầng. Tương tự, Link-Cut Tree và phân trị trọng tâm cộng cây đoạn cũng có thể xem là một lớp bài toán nhiều tầng cây lồng nhau như vậy.

Trong loại bài toán này, khi cần truy cập một nút, ta thường cần duyệt đường đi từ gốc tầng trên cùng tới nút đó. Nếu dùng cây đoạn hoặc cây cân bằng để duy trì riêng từng cây, giả sử có tổng cộng k tầng, thì độ phức tạp thời gian để truy cập một nút sẽ là $O(k \log n)$. Dù cách làm này bảo đảm mỗi cây nội bộ đều cân bằng, nó không tận dụng tính chất của cấu trúc tổng thể, nên không thu được độ phức tạp tốt. Ta thử cải tiến để đạt hiệu quả cân bằng toàn cục.

4.4.1 Trường hợp tĩnh. Xét định nghĩa trọng số của một nút là tổng kích thước các cây lồng dưới nó cộng 1, rồi dùng phương pháp trong mục 4.1 để dựng mỗi cây thành một cây nhị phân có trọng số.

Định lý 4.5. Giả sử nút x nằm ở tầng thứ k . Nếu dùng thuật toán trên để dựng cây, thì độ sâu của nút x trong cấu trúc tổng thể là $O(k + \log n)$.

Chứng minh. Giả sử nút x được lồng dưới một nút có trọng số s_i ở tầng thứ i , trong đó s_k chính là w_x . Khi đó độ sâu là

$$\begin{aligned}
1 + O\left(\log \frac{n}{s_1}\right) + \sum_{i=2}^k \left(1 + O\left(\log \frac{s_{i-1}}{s_i}\right)\right) &= k + O(\log n - \log w_x) \\
&= O(k + \log n).
\end{aligned}$$

■

Phần lớn các bài phân rã cây theo chuỗi nặng hoặc phân trị trọng tâm cộng cây đoạn đều có thể dùng phương pháp này để tối ưu xuống một logarit.

4.4.2 Trường hợp động. Khi cây trở thành động, ta có thể dùng Splay hoặc Treap để duy trì mỗi cây, và cũng có thể truy cập một nút trong thời gian $O(k + \log n)$.

Ví dụ 3. Ác chiến biểu thức. Đây là một bài tương tác.

Ta cần xử lý một lớp biểu thức gồm n phần tử và $n - 1$ toán tử; giữa hai phần tử kề nhau có đúng một toán tử, và trong biểu thức không có dấu ngoặc. Có tổng cộng k loại toán tử, mỗi loại có độ ưu tiên khác nhau. Để thuận tiện, ta dùng các số nguyên từ 1 tới k để biểu diễn các toán tử này, và số càng lớn thì độ ưu tiên càng cao. Các toán tử đều thỏa luật giao hoán và luật kết hợp.

Gọi biểu thức ban đầu là phiên bản thứ 0. Có m thao tác, mỗi thao tác thuộc một trong các loại sau:

- Sửa phần tử: với một phiên bản nào đó, sửa phần tử ở một vị trí.
- Sửa toán tử: với một phiên bản nào đó, sửa toán tử nằm giữa hai phần tử nào đó.
- Lật: với một phiên bản nào đó, lật mọi phần tử và toán tử từ phần tử thứ l tới phần tử thứ r , bao gồm cả hai phần tử thứ l và thứ r .

Ta gọi biểu thức thu được sau thao tác thứ i là phiên bản thứ i . Sau mỗi thao tác, cần tính giá trị của biểu thức hiện tại.

Ta chỉ có thể gọi một hàm $F(a, b, op)$ để nhận kết quả của $a op b$, và số lần gọi hàm này không được vượt quá 10^7 .

$$n, m \leq 20000, \quad k \leq 100.$$

Xét dùng cây cân bằng để duy trì biểu thức. Để bảo đảm tính đúng đắn của phép tính, cần làm cho độ ưu tiên của toán tử do mỗi nút đại diện lớn hơn độ ưu tiên của các tổ tiên của nó. Có thể thấy như vậy sẽ tạo thành một cấu trúc nhiều tầng cây lồng nhau. Do cần hỗ trợ khả tri hóa, ta dùng Treap để duy trì mỗi cây.

Mọi thao tác của bài này đều có thể hoàn thành bằng thao tác tách và nối Treap. Ta phân tích kỹ độ phức tạp thời gian của thao tác tách. Giả sử tổng trọng số của các nút trong một Treap là W , và ta tách nó giữa nút thứ p và nút thứ $p + 1$. Khi đó độ phức tạp thời gian của lần tách này là

$$O\left(\log \frac{W}{\max(w_p, w_{p+1})}\right),$$

và tổng thời gian không vượt quá độ sâu của điểm tách thấp nhất. Cần chú ý rằng một số hiện thực kém sẽ khiến độ phức tạp thời gian của thao tác tách trở thành

$$O\left(\log \frac{W}{\min(w_p, w_{p+1})}\right),$$

dẫn tới suy giảm độ phức tạp. Sau khi tách xong, trọng số của một số nút trên đường đi sẽ giảm, khi đó ta cần xoay chúng xuống một số lần. Độ phức tạp thời gian của phần này cũng không vượt quá độ sâu cuối cùng của điểm tách thấp nhất, tức $O(k + \log n)$.

Thao tác nối là thao tác ngược của tách, nên độ phức tạp thời gian cũng như vậy. Do đó độ phức tạp thời gian và bộ nhớ của toàn bộ thuật toán là $O(k + \log n)$ cho mỗi thao tác theo phân tích trên.

4.4.3 Link-Cut Tree. Thông thường ta dùng Splay để duy trì mỗi đường thực của Link-Cut Tree; chứng minh độ phức tạp có thể tìm thấy trong nhiều tài liệu,²⁸ nên ở đây không nhắc lại. Vậy ta có thể dùng Treap để thay thế Splay trong Link-Cut Tree hay không?

Xét quá trình chuyển đổi cạnh ảo và cạnh thực trong Link-Cut Tree. Trước hết ta cần tách phần nằm bên phải nút x ra để trở thành cây con ảo, sau đó ghép một cây con ảo ban đầu nào đó vào bên phải x , cuối cùng còn cần điều chỉnh trọng số của nút x . Độ phức tạp của hai bước đầu đều bị chặn bởi độ sâu

²⁸Phân tích độ phức tạp thời gian của Link-Cut Tree có thể tham khảo tài liệu [4].

ban đầu của nút x . Tuy nhiên ở bước thứ ba, nút x có thể phải xoay xuống do trọng số giảm. Vì sau thao tác, nút được access ban đầu đã không còn nằm trong cây con ảo của x , tổng độ phức tạp của các lần xoay xuống không thể bị chặn bởi độ sâu của một nút như trước. Vì vậy Link-Cut Tree hiện thực theo cách này vẫn có độ phức tạp hai logarit. Có thể thấy, Treap vẫn có hạn chế nhất định trong một số bài toán nhiều tầng cây lồng nhau mà hình dạng cây thay đổi phức tạp.

Ở đây người viết chưa nghĩ ra cách tốt để giải quyết vấn đề này; nếu độc giả có ý tưởng hay, rất hoan nghênh trao đổi.

5 Tổng kết

Hai loại cây cân bằng được giới thiệu trong bài viết đều dựa vào chiến lược cân bằng ngắn gọn và đẹp để đạt độ phức tạp thời gian rất tốt trong nhiều bài toán. Tính tự điều chỉnh của Splay thực chất là một tính chất rất mạnh. Hai người phát minh ra nó, Sleator và Tarjan, còn nêu một giả thuyết gọi là Dynamic Optimality Conjecture: giả sử A là một thuật toán cây tìm kiếm nhị phân nào đó, trong đó khi truy cập nút x , ta cần trả chi phí $\text{depth}(x) + 1$ để duyệt đường đi từ gốc tới x , và giữa hai lần truy cập có thể thực hiện một số thao tác xoay với chi phí 1 mỗi lần. Xét một dãy truy cập S , thời gian để dùng thuật toán này lần lượt truy cập các phần tử trong S là $A(S)$. Khi đó, dùng Splay để truy cập S có độ phức tạp thời gian $O(n + A(S))$. Giả thuyết này tới nay vẫn chưa được chứng minh hoặc bác bỏ, điều này cũng có nghĩa là Splay vẫn còn tiềm năng rất lớn chờ được khai thác. Mặc dù Treap trong một số bài toán có thể không biểu hiện tốt bằng Splay, ưu thế của nó cũng rất rõ ràng: khi chèn xóa phần tử, hình dạng tổng thể của cây thay đổi ít, đồng thời có thể hỗ trợ khả trì hóa một cách thuận tiện.

Thực ra giới học thuật từ lâu đã nghiên cứu sâu nhiều loại cây cân bằng khác nhau và thu được nhiều kết quả thú vị. Nhưng làm thế nào để kết hợp hữu cơ các kết quả này với nội dung trong giới OI vẫn là vấn đề cần suy nghĩ. Hy vọng bài viết có thể đóng vai trò gợi mở, thu hút nhiều độc giả hơn nghiên cứu lớp vấn đề này.

Lời cảm ơn

- Cảm ơn China Computer Federation đã cung cấp nền tảng giao lưu và học tập.
- Cảm ơn cha mẹ đã quan tâm và chăm sóc tôi.
- Cảm ơn thầy Yan Kaoming đã nhiều năm quan tâm và chỉ dẫn.
- Cảm ơn đàn anh Wang Yisong đã chia sẻ với tôi một số thuật toán liên quan tới bài viết này, đồng thời giúp đỡ việc viết bài.
- Cảm ơn các bạn Wu Hongxun, Wu Jinzhao, Wang Xiuhuan và Mai Jun đã kiểm bài viết này.
- Cảm ơn mọi độc giả đã dành thời gian đọc bài viết.

Tài liệu tham khảo

1. R. Cole. On the dynamic finger conjecture for Splay trees. part II: The proof. *SIAM Journal of Computing*, 30(1):44–85, 2000.
2. D. D. Sleator and R. E. Tarjan. Self-adjusting binary search trees. *JACM*, 32(3):652–686, 1985.
3. R. Seidel and C. R. Aragon. Randomized search trees. *Algorithmica*, 16(4/5):464–497, 1996.
4. Yang Zhe, *Một số nghiên cứu về lời giải SPOJ375 QTREE*, bài tập đội tuyển tập huấn quốc gia năm 2007.

5. Chen Lijie, *Ứng dụng cây cân bằng theo trọng lượng và cây cân bằng hậu tố trong Olympic Tin học*, tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2013.
6. Chen Lijie, *Nghiên cứu cấu trúc dữ liệu khả tri*, bài tập đội tuyển tập huấn quốc gia năm 2012.
7. Guy E. Blelloch and Margaret Reid-Miller. Fast set operations using treaps. In *10th Annual ACM Symposium on Parallel Algorithms and Architectures*, Puerto Vallarta, Mexico, June–July 1998. ACM.
8. Wang Yisong, slide bài giảng bình luận *Ác chiến biểu thức*, Trại đông toàn quốc năm 2016.

16 Chủ đề chính

Từ mục lục và phần mở đầu của các bài, tập năm 2018 bao phủ các nhóm chủ đề sau:

- hàm sinh, xác suất, kỳ vọng và các bài toán gieo xúc xắc;
- cấu trúc dữ liệu chuỗi, cây hậu tố, suffix automaton, KMP và tính chu kỳ của chuỗi;
- hồi quy bảo toàn thứ tự, chia đôi toàn cục và tối ưu trên quan hệ thứ tự bộ phận;
- quy hoạch động và cấu trúc dữ liệu cho khối liên thông trên cây;
- tích chập, DFT, FFT, FWT, FMT và các bài toán đa thức;
- cây nhị phân, *Leafy Tree*, Splay, Treap, tìm kiếm theo ngón tay và cây cân bằng có trọng số;
- tổng hàm số học, hàm nhân tính, tổng trên số nguyên tố và các biến thể số học đặc biệt;
- matroid, tối ưu tổ hợp, cây khung phương sai nhỏ nhất và đồ thị Euler;
- các báo cáo ra đề cho *Số nút của cây hậu tố*, *Fim 4*, *Jellyfish*, *Tiểu H thích tô màu*, *Hàng đợi hoàn hảo* và *Cây khung phương sai nhỏ nhất*.

Ghi chú về nguồn

Tập PDF có 210 trang. pdftotext trích xuất được Unicode đúng cho phần đầu sách, mục lục và các trang thân bài đã kiểm tra, nên không cần render mục lục bằng ảnh để đọc lại. Bản dịch đã bổ sung toàn văn bài đầu tiên của Yang Maolong, tương ứng trang in 1–10 và trang PDF 3–12; bài thứ hai của Chen Jianglun, tương ứng trang in 11–22 và trang PDF 13–24; bài thứ ba của Gao Ruiquan, tương ứng trang in 23–33 và trang PDF 25–35; bài thứ tư của Wu Jinzhao, tương ứng trang in 34–44 và trang PDF 36–46; bài thứ năm của Ren Xuandi, tương ứng trang in 45–57 và trang PDF 47–59; bài thứ sáu của Liang Yancheng, tương ứng trang in 58–74 và trang PDF 60–76; cùng bài thứ bảy của Wang Siqi, tương ứng trang in 75–86 và trang PDF 77–88; cùng bài thứ tám của Chen Jiale, tương ứng trang in 87–91 và trang PDF 89–93; bài thứ chín của Zhu Zhenting, tương ứng trang in 92–112 và trang PDF 94–114; cùng bài thứ mười của Liu Chengao, tương ứng trang in 113–131 và trang PDF 115–133; cùng bài thứ mười một của Lin Xuheng, tương ứng trang in 132–142 và trang PDF 134–144. Một số công thức dài, ví dụ mẫu và hình minh họa được đối chiếu thêm bằng ảnh render từ PDF vì bố cục công thức và chú thích hình có chỗ bị méo trong văn bản trích xuất. Hình đường gấp khúc trong bài thứ ba được dựng lại bằng TikZ từ trang PDF 33. Bài thứ tư không có hình; ở bài thứ năm, các hình cây của cấu trúc LCM và ví dụ đường kính được kiểm tra bằng render trang PDF 53–55 và được diễn giải trong văn bản vì chúng chỉ minh họa cấu trúc, không chứa dữ liệu đề bài riêng. Bài thứ sáu không có bảng, hình hay mẫu code; các công thức dài về $\oplus$, Karatsuba, xor_K và phép toán $*$ được đối chiếu bằng ảnh render các trang PDF

60–76. Bài thứ bảy có hai sơ đồ xoay cây được dựng lại bằng TikZ từ trang PDF 80; biểu đồ tốc độ chạy trên trang PDF 84 được kiểm tra bằng render và diễn giải trong văn bản, vì dữ liệu chính của biểu đồ là so sánh định tính giữa các cây cân bằng. Bài thứ tám không có hình, bảng hay mẫu code; các công thức về nghịch đảo nhị thức, số Stirling và chập NTT được đối chiếu bằng ảnh render trang PDF 89–93. Bài thứ chín không có bảng hay mẫu code; các công thức về sàng Min 25, Meissel–Lehmer, tổng lũy thừa trên số nguyên tố và phân tích đường cong $xy = n$ được đối chiếu bằng văn bản trích xuất và ảnh render trang PDF 94–114. Các hình hình học trong mục 4 được dựng lại ở mức sơ đồ bằng TikZ hoặc diễn giải trong văn bản vì chúng chỉ minh họa miền cắt và đường gấp khúc, không chứa dữ liệu số riêng. Bài thứ mười không có hình hay mẫu code; hai đoạn giả mã được dịch thành các bước đánh số, còn các công thức ma trận, hệ phương trình và độ phức tạp được đối chiếu với ảnh render trang PDF 115–133. Bài thứ mười một không có hình, bảng hay mẫu code; các công thức độ phức tạp và các mốc chia khối/chia đôi được đối chiếu bằng văn bản trích xuất và ảnh render trang PDF 134–144. Bài thứ mười hai có sơ đồ matroid Fano được dựng lại bằng TikZ; các định nghĩa và định lý về giao, hợp và tính biểu diễn được của matroid được đối chiếu bằng văn bản trích xuất và ảnh render trang PDF 145–165. Bài thứ mười ba, tương ứng trang in 164–179 và trang PDF 166–181, có một sơ đồ minh họa thao tác `splay`; nội dung sơ đồ được diễn giải bằng mô tả ba trường hợp xoay, còn các công thức thể năng, *finger search*, Treap có trọng số và ví dụ ứng dụng được đối chiếu bằng văn bản trích xuất.